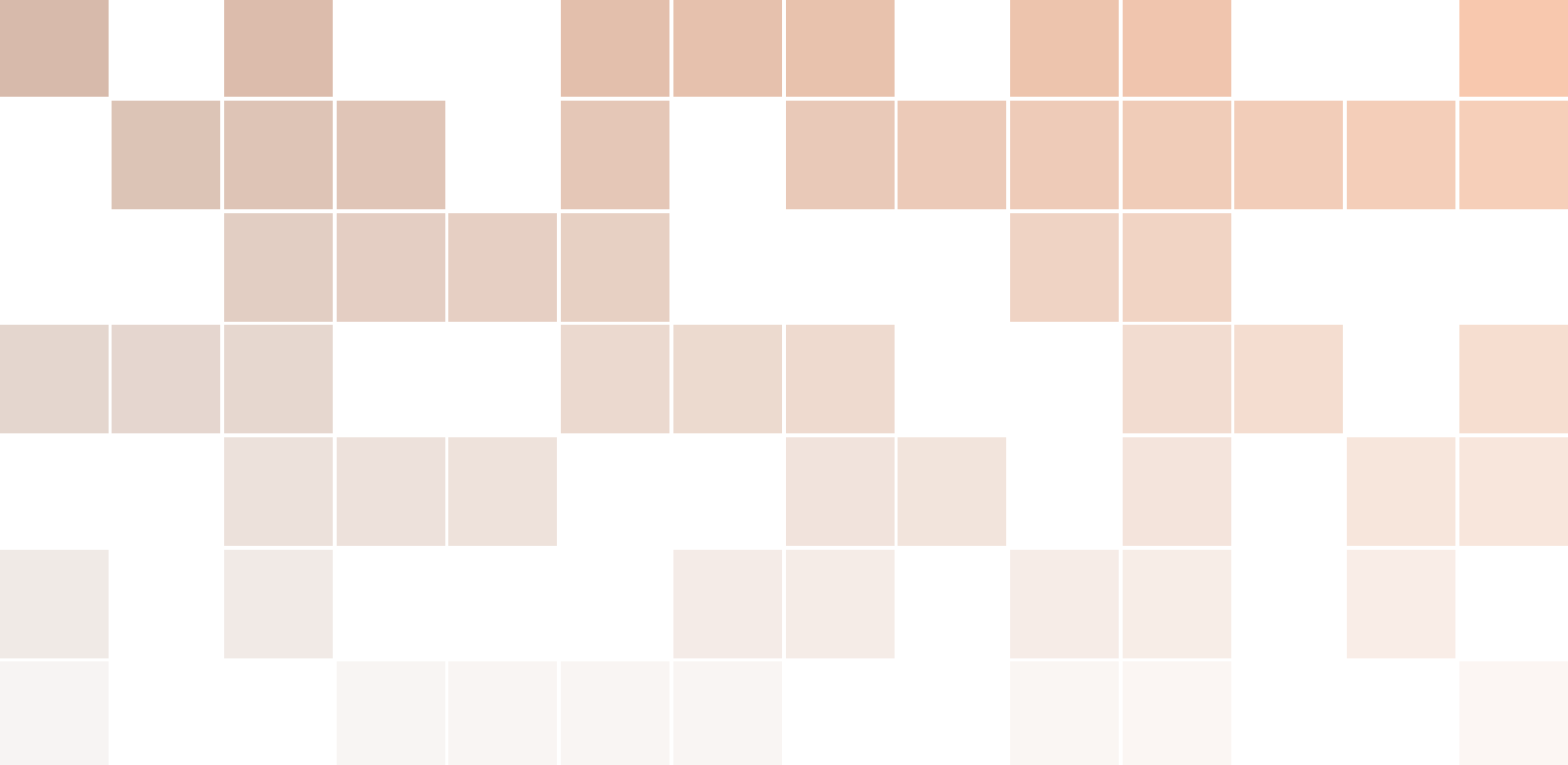
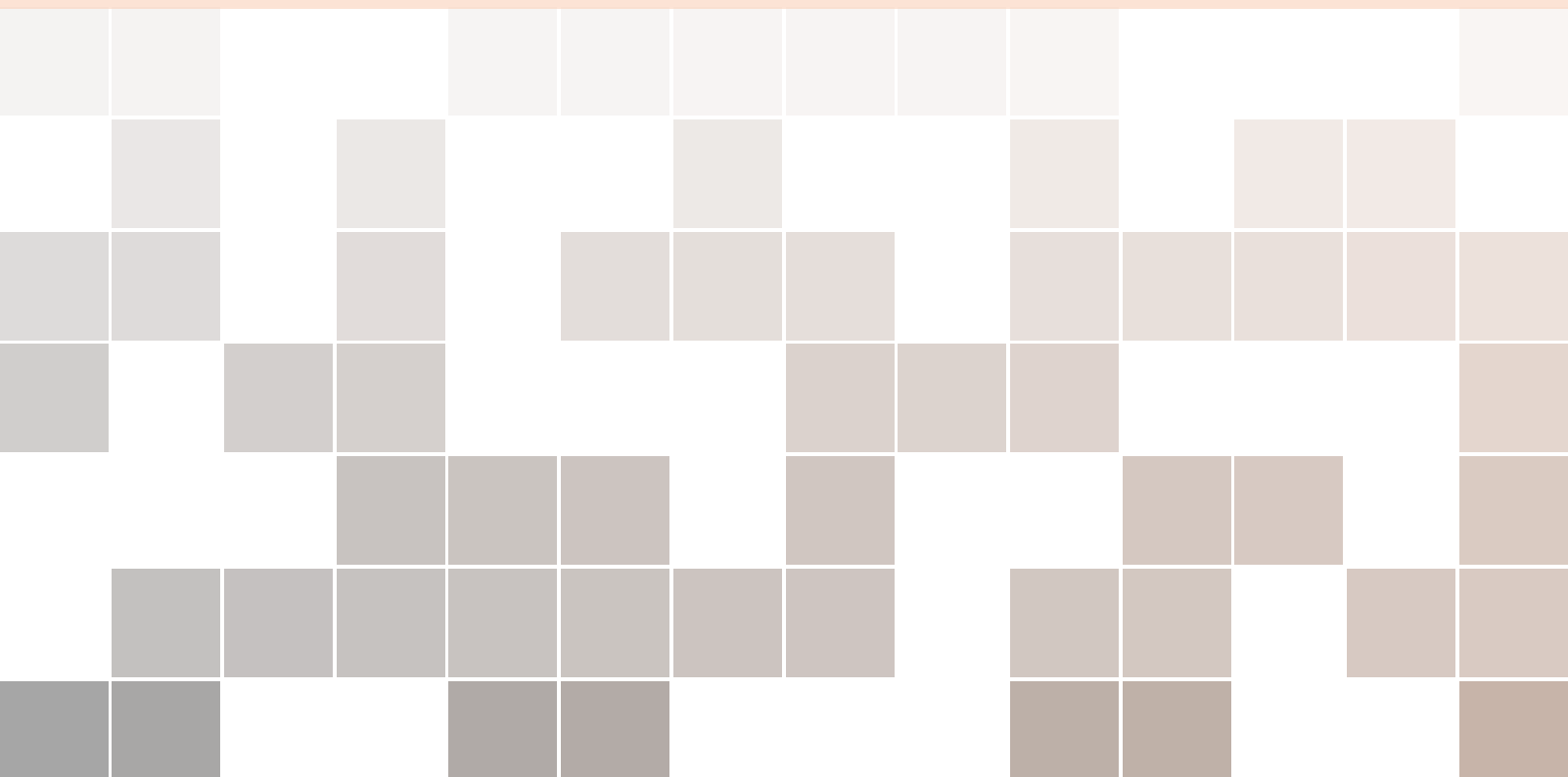




NHẬP MÔN KHOA HỌC DỮ LIỆU

Nguyễn Kiều Linh, Vũ Hoài Nam,
Đinh Xuân Trường



This page is intentionally left blank



Mục lục

Mở đầu	13
1 Kiến thức cơ sở	15
1.1 Ôn tập về đại số tuyến tính	15
1.2 Ôn tập về xác suất	20
1.2.1 Độc lập xác suất và phụ thuộc xác suất	20
1.2.2 Xác suất có điều kiện và quy tắc Bayes	20
1.2.3 Biến ngẫu nhiên	24
1.2.4 Phân phối liên tục	24
1.2.5 Phân phối chuẩn	25
1.3 Giả thuyết	28
1.3.1 Khoảng tin cậy	28
1.3.2 Kiểm định giả thuyết thống kê	32
1.3.3 Ví dụ	34
Bài tập Chương 1	36
2 Chuẩn bị dữ liệu	41
2.1 Thu thập dữ liệu	41
2.1.1 Thuật toán thu thập cơ bản	44
2.1.2 Thu thập ưu tiên	46
2.1.3 Đa luồng	47
2.1.4 Tránh các bẫy Spider	47
2.1.5 Phát hiện sự trùng lặp	47
2.1.6 Áp dụng thu thập dữ liệu - Web Scraping	48
2.2 Làm sạch dữ liệu	58
2.2.1 Xử lý giá trị bị thiếu	59

2.2.2	Xử lý giá trị sai hoặc không nhất quán	60
2.3	Co giãn và chuẩn hóa dữ liệu	62
2.4	Giảm chiều và biến đổi dữ liệu	63
2.4.1	Lấy mẫu	64
2.4.2	Lựa chọn đặc trưng	67
2.4.3	Giảm chiều dữ liệu	67
2.4.4	Biến đổi dữ liệu	77
	Bài tập Chương 2	77
3	Trực quan hóa dữ liệu	81
3.1	Cơ bản về Matplotlib	81
3.2	Đồ thị dạng đường	87
3.3	Đồ thị điểm rời rạc	92
3.4	Trực quan hóa lỗi	95
3.5	Đồ thị đường viền	99
3.6	Histograms và mật độ	102
3.7	Đồ thị ba chiều	107
3.8	Dữ liệu địa lý	113
3.8.1	Trực quan dữ liệu trên bản đồ	121
3.9	Văn bản và chú giải	122
3.9.1	Tùy chỉnh mũi tên chú giải	124
3.9.2	Tùy chỉnh cách hiển thị của trục biểu đồ	127
	Bài tập Chương 3	130
4	Học máy	133
4.1	Các khái niệm cơ bản	133
4.1.1	Học và suy diễn	134
4.1.2	Đánh giá mô hình	135
4.1.3	Overfitting và Underfitting	138
4.1.4	Bias - Sai số định kiến và Variance - Phương sai	140
4.2	Lựa chọn và trích chọn đặc trưng	142
4.2.1	Đặc trưng phân loại	142
4.2.2	Đặc trưng văn bản	144
4.2.3	Đặc trưng ảnh	146
4.3	Học có giám sát	150
4.4	Học không giám sát	153

4.5	Học bán giám sát	154
4.6	Học tăng cường	156
	Bài tập Chương 4	158
5	Cơ sở dữ liệu và SQL	161
5.1	Cơ sở dữ liệu cơ bản	161
5.1.1	CREATE TABLE và INSERT	161
5.1.2	UPDATE	163
5.1.3	DELETE	164
5.1.4	SELECT	164
5.1.5	GROUP BY	165
5.1.6	ORDER BY	166
5.1.7	JOIN	166
5.2	Cơ sở dữ liệu nâng cao	168
5.2.1	Truy vấn con	168
5.2.2	Tối ưu truy vấn	168
5.2.3	NoSQL	170
	Bài tập Chương 5	171
6	Hệ khuyến nghị	177
6.1	Giới thiệu chung	177
6.1.1	Hệ khuyến nghị là gì	177
6.1.2	Hệ khuyến nghị hoạt động thế nào	178
6.2	Lọc dựa trên nội dung	179
6.2.1	Utility matrix	180
6.2.2	Hồ sơ của sản phẩm (item profiles)	181
6.2.3	Xây dựng hàm mất mát	182
6.2.4	Ví dụ hàm mất mát cho user P2	183
6.3	Lọc cộng tác	184
6.3.1	Lọc cộng tác dựa trên người dùng (user-user collaborative filtering)	184
6.3.2	Lọc cộng tác dựa trên sản phẩm (item-item collaborative filtering)	188
6.4	Lọc cộng tác dựa trên phân rã ma trận (matrix factorization)	190
6.4.1	Giới thiệu	190
	Bài tập Chương 6	191
	Tài liệu tham khảo	197



Danh sách hình vẽ

1.1	Minh hoạ đồ thị của một số hàm mật độ của một số phân phối chuẩn.	26
1.2	Output của Ví dụ 1.11.	28
2.1	Tìm nhiều bằng phương pháp tập trung vào dữ liệu	61
2.2	Biểu đồ hộp.	62
2.3	Dữ liệu có độ tương quan cao được biểu thị bằng ba chiều trong một hệ trục xoay thích hợp	68
2.4	Phương sai ứng với số lượng giá trị riêng tăng dần của tập dữ liệu Arrythmia . .	71
2.5	Bổ sung cơ sở cho ma trận phân tích trong SVD	75
3.1	Ví dụ cơ bản về trục quan hoá.	83
3.2	Kết xuất PNG của hình ảnh cơ bản.	84
3.3	Sử dụng <i>subplot</i> để vẽ nhiều biểu đồ trong một <i>Figure</i>	85
3.4	Sử dụng <i>subplot</i> để vẽ 2 biểu đồ trong một <i>Figure</i>	86
3.5	Sử dụng <i>subplot</i> để vẽ 4 biểu đồ trong một <i>Figure</i>	87
3.6	Đồ thị đường hình <i>sin</i>	88
3.7	Thay đổi màu sắc sử dụng hàm <i>plot</i>	89
3.8	Các kiểu dạng của đồ thị đường.	89
3.9	Điều chỉnh màu sắc và kiểu đồ thị bằng cách rút gọn.	90
3.10	Thay đổi biên của các trục toạ độ.	91
3.11	Thay đổi biên của các trục toạ độ bằng <i>plt.axis()</i>	92
3.12	Thay đổi biên của các trục toạ độ bằng <i>tight</i>	93
3.13	Thay đổi biên của các trục toạ độ bằng <i>equal</i>	93
3.14	Ví dụ về đặt tên đồ thị và nhãn trục.	94
3.15	Ví dụ chú giải nhiều đường.	94
3.16	Đồ thị điểm rời rạc.	95
3.17	Đồ thị điểm rời rạc.	95
3.18	Đồ thị kết hợp điểm và đường.	96

3.19	Đồ thị điểm rời rạc sử dụng <i>plt.scatter</i>	96
3.20	Thay đổi kích thước, màu sắc và độ trong suốt ở các điểm phân tán.	97
3.21	Trục quan bằng thanh lỗi.	97
3.22	Trục quan bằng thanh lỗi.	98
3.23	Trục quan miền lỗi liên tục.	98
3.24	Trục quan hóa dữ liệu ba chiều với các đường viền.	100
3.25	Trục quan hóa dữ liệu ba chiều với các đường viền nhiều màu và nhiều mức.	100
3.26	Trục quan hóa dữ liệu các đường viền chuyển màu liên tục.	101
3.27	Trục quan hóa dữ liệu các đường viền liên tục.	102
3.28	Trục quan hóa dữ liệu các đường viền liên tục và đường đồng mức.	103
3.29	Một đồ thị <i>Histogram</i> đơn giản.	103
3.30	Một đồ thị <i>Histogram</i> tùy chỉnh.	104
3.31	Một đồ thị <i>Histogram</i> kết hợp.	104
3.32	Đồ thị <i>histogram</i> hai chiều.	105
3.33	Đồ thị <i>histogram</i> hai chiều.	106
3.34	Đồ thị <i>histogram</i> hai chiều.	106
3.35	Khởi tạo trục đồ thị 3D.	107
3.36	Đồ thị điểm và đường ba chiều.	108
3.37	Đồ thị đường viền ba chiều.	109
3.38	Đồ thị đường viền ba chiều tùy chỉnh góc mặc định.	110
3.39	Đồ thị khung lưới <i>wireframe</i>	110
3.40	Đồ thị bề mặt <i>surface</i>	111
3.41	Đồ thị bề mặt cực - <i>polar surface</i>	112
3.42	Trục quan hóa dải Mobius.	113
3.43	Phép chiếu " <i>bluemarble</i> " của Trái đất.	114
3.44	Vẽ dữ liệu và nhãn trên bản đồ.	114
3.45	Phép chiếu hình trụ.	116
3.46	Phép chiếu Mollweide.	117
3.47	Phép chiếu trục giao.	117
3.48	Phép chiếu Alberse.	118
3.49	Bản đồ ranh giới ở độ phân giải thấp và cao.	120
3.50	Biểu đồ phân tán trên nền bản đồ.	122
3.51	So sánh các hệ tọa độ của Matplotlib.	124
3.52	So sánh các hệ tọa độ của Matplotlib.	125
3.53	Ví dụ về tùy chỉnh mũi tên chú giải.	126
3.54	Tùy chỉnh các loại mũi tên chú giải.	127
3.55	Ví dụ về thang đo logarit và nhãn.	128
3.56	Vẽ biểu đồ ẩn các nhãn đánh dấu trục x và trục y.	129
3.57	Ẩn đánh dấu các trục tọa độ trong biểu đồ hiển thị các hình ảnh.	130
3.58	Một ô mặc định có nhiều dấu tích.	130
4.1	Confusion Matrix (Ma trận nhầm lẫn).	137

4.2	Overfitting và underfitting.	139
4.3	Overfitting và underfitting.	141
4.4	Sử dụng one-hot vector.	144
4.5	Ý nghĩa tên của từng cột.	144



Danh sách bảng

6.1	Ví dụ Utility matrix	180
6.2	Xây dựng item profile	182
6.3	Utility matrix cho users từ u_0 đến u_5 đánh giá items từ i_0 tới i_5	185
6.4	Utility matrix chuẩn hóa cho users từ u_0 đến u_5 đánh giá items từ i_0 tới i_5	186
6.5	Ma trận độ tương đồng của các users	187



Mở đầu

Trong thời đại số hóa ngày càng phát triển mạnh mẽ, dữ liệu đã trở thành một tài nguyên vô cùng quý giá và quyết định đến sức mạnh của nhiều lĩnh vực. Khám phá và phân tích dữ liệu không chỉ là một xu hướng mà còn là trọng tâm của sự tiến bộ, đem lại cái nhìn sâu rộng và thông tin quý báu cho các tổ chức và cộng đồng. Bởi vậy, Khoa học Dữ liệu ra đời để khám phá, phân tích và hiểu sâu hơn về dữ liệu khổng lồ này, từ đó cung cấp thông tin hữu ích, đưa ra quyết định thông minh và tạo ra giá trị từ những dữ liệu này. Các nhà nghiên cứu, doanh nghiệp và cả xã hội đang nhận ra tiềm năng lớn của việc sử dụng dữ liệu một cách thông minh và hiệu quả, từ đó đã thúc đẩy sự phát triển của lĩnh vực Khoa học Dữ liệu.

Khoa học Dữ liệu không chỉ là một lĩnh vực mới mẻ mà còn là trung tâm của sự biến đổi kỹ thuật số trong thế giới ngày nay. Với sự bùng nổ của dữ liệu và công nghệ, khả năng nắm bắt, phân tích và hiểu biết sâu rộng về dữ liệu đang trở thành một trong những kỹ năng quan trọng nhất của con người.

Bài giảng Nhập môn Khoa học Dữ liệu được thiết kế để giúp sinh viên làm quen và bước đầu học tập, nghiên cứu với Khoa học Dữ liệu. Bài giảng được viết với các mục tiêu:

1. Giới thiệu cơ bản và nền tảng: Tóm tắt cho sinh viên kiến thức cơ bản về nền tảng Toán học cần dùng trong Khoa học Dữ liệu như Đại số, xác suất và thống kê.
2. Hướng dẫn về các công cụ và phương pháp: Bài giảng này giúp sinh viên hiểu về các công cụ và phương pháp quan trọng trong việc xử lý và phân tích dữ liệu. Nó cung cấp cái nhìn tổng quan về các công nghệ, ngôn ngữ lập trình, công cụ phân tích và cơ sở dữ liệu quan trọng mà sinh viên cần biết.

3. Thực hành và dự án: Bài giảng Nhập môn thường kết hợp giữa lý thuyết và thực hành. Điều này giúp sinh viên áp dụng kiến thức vào các dự án thực tế, từ đó cải thiện kỹ năng thực tế và giải quyết vấn đề.

4. Hiểu về ứng dụng thực tế: Bài giảng giúp sinh viên hiểu rõ hơn về cách áp dụng Khoa học Dữ liệu vào thực tế, từ việc phân tích dữ liệu đến việc đưa ra quyết định thông minh trong các lĩnh vực như kinh doanh, y tế, marketing, khoa học xã hội, và nhiều lĩnh vực khác.

Để đạt được các mục tiêu đó, Bài giảng Nhập môn Khoa học dữ liệu cung cấp kiến thức cơ bản, kỹ năng thực hành ban đầu và nhận thức về tiềm năng ứng dụng trong thực tế của lĩnh vực này, giúp sinh viên chuẩn bị tốt cho việc học tập và nghiên cứu về lĩnh vực Khoa học Dữ liệu tiếp theo. Cụ thể, nội dung Bài giảng sẽ trang bị cho sinh viên các kiến thức cơ bản về khoa học dữ liệu (dữ liệu, các kiểu dữ liệu, giả thuyết thống kê và kiểm định giả thuyết thống kê), các phương pháp xử lý và phân tích dữ liệu bao gồm chuẩn bị dữ liệu (thu thập dữ liệu, làm sạch dữ liệu, chuẩn hóa dữ liệu, giảm chiều dữ liệu, biến đổi dữ liệu), trực quan hóa dữ liệu, và các mô hình cho phép dự đoán, phân loại, phân cụm, tư vấn dựa trên dữ liệu. Ngoài ra sinh viên cũng được cung cấp các kiến thức liên quan tới đánh giá các mô hình phân tích dữ liệu.

1. Kiến thức cơ sở

Toán học (Đại số, Xác suất, Thống kê) trong ngành khoa học dữ liệu được sử dụng để xác định các mẫu dữ liệu, sinh ra và áp dụng các thuật toán, thực hiện tính toán, xây dựng các mô hình tính toán, tiền xử lý dữ liệu và hiểu dữ liệu ở cấp độ cao. Chính vì vậy Chương 1 trình bày những nội dung cơ bản về Đại số, xác suất và Thống kê cần thiết ban đầu làm cơ sở cho việc học tập và nghiên cứu về Khoa học dữ liệu.

1.1 Ôn tập về đại số tuyến tính

Định nghĩa 1.1 Ma trận cỡ $m \times n$ là một bảng số hình chữ nhật có m hàng và n cột, được ký hiệu là

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix},$$

trong đó a_{ij} là phần tử nằm trên hàng thứ i , cột thứ j của ma trận A .

- Ma trận A còn được ký hiệu gọn là $A = (a_{ij})_{m \times n}$.
- Nếu $m = n$ thì A được gọi là ma trận vuông cấp n . Khi đó $A = (a_{ij})_{n \times n}$.
- Nếu $n = 1$ thì A được gọi là ma trận cột, hay một vectơ.

Định nghĩa 1.2 Cho ma trận vuông $A = (a_{ij})_{m \times n}$. Đường chéo $a_{11}, a_{22}, \dots, a_{nn}$ được gọi là đường chéo chính. Tổng các phần tử trên đường chéo chính được gọi là vết của A , ký hiệu là $\text{trace}(A)$.

Định nghĩa 1.3 • Ma trận vuông mà mọi phần tử nằm phía dưới đường chéo chính đều bằng 0 (tức là $a_{ij} = 0, \forall i > j$) gọi là ma trận tam giác trên.

- Ma trận vuông mà mọi phần tử nằm phía trên đường chéo chính đều bằng 0 (tức là $a_{ij} = 0, \forall i < j$) gọi là ma trận tam giác dưới.
- Ma trận vuông mà mọi phần tử không nằm trên đường chéo chính đều bằng 0

(tức là $a_{ij} = 0, \forall i \neq j$) gọi là *ma trận đường chéo*, ký hiệu $\text{diag}(a_{11}, a_{22}, \dots, a_{nn})$.

- Ma trận đường chéo mà mọi phần tử của đường chéo chính đều bằng 1 được gọi là *ma trận đơn vị*, được ký hiệu là I .

Định nghĩa 1.4 — Phép cộng hai ma trận.

Cho hai ma trận cùng cỡ $A = (a_{ij})_{m \times n}, B = (b_{ij})_{m \times n}$. Tổng của hai ma trận A và B , ký hiệu $A + B$, là ma trận xác định bởi

$$A + B = (a_{ij} + b_{ij})_{m \times n}.$$

Định nghĩa 1.5 — Phép nhân ma trận với một số.

Cho ma trận $A = (a_{ij})_{m \times n}$ và số thực k . Tích của ma trận A với số k , ký hiệu kA , là ma trận xác định bởi

$$kA = (ka_{ij})_{m \times n}.$$

Định nghĩa 1.6 — Phép nhân hai ma trận.

Cho ma trận $A = (a_{ij})_{m \times p}, B = (b_{ij})_{p \times n}$. Tích của ma trận A và B , ký hiệu AB , là ma trận $AB = (c_{ij})_{m \times n}$ xác định bởi

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{ip}b_{pj} = \sum_{k=1}^p a_{ik}b_{kj}.$$

- **Ví dụ 1.1** Cho $A = \begin{bmatrix} 1 & 1 & 3 \\ 2 & -1 & -2 \end{bmatrix}, B = \begin{bmatrix} 1 & -2 \\ -2 & 1 \\ 1 & 2 \end{bmatrix}$. Tính AB, BA . ■

Lời giải. Ta có

$$AB = \begin{bmatrix} 1 & 1 & 3 \\ 2 & -1 & -2 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ -2 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 5 \\ 2 & -9 \end{bmatrix}$$

$$BA = \begin{bmatrix} 1 & -2 \\ -2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 & 3 \\ 2 & -1 & -2 \end{bmatrix} = \begin{bmatrix} -3 & 3 & 7 \\ 0 & -3 & -8 \\ 5 & -1 & -1 \end{bmatrix}.$$

■

Định nghĩa 1.7 Ma trận thu được từ ma trận A bằng cách chuyển các hàng thành các cột (các cột chuyển thành các hàng) được gọi là *ma trận chuyển vị* của ma trận A , ký hiệu là A^T .

Định nghĩa 1.8 Ma trận vuông A được gọi là *ma trận đối xứng* nếu $A^T = A$.

Định nghĩa 1.9 Xét ma trận $A = (a_{ij})_{n \times n}$. Ma trận vuông cấp $n - 1$ thu được từ ma trận A bằng cách bỏ đi hàng thứ i và cột thứ j được gọi là *ma trận con ứng với phần tử a_{ij}* được ký hiệu là M_{ij} .

Định nghĩa 1.10 *Định thức* của ma trận A , ký hiệu $\det(A)$ hoặc $|A|$, được định nghĩa theo quy nạp như sau:

- A là ma trận cấp 1: $A = (a_{11})$ thì $\det(A) = a_{11}$.
- A là ma trận cấp n thì

$$\det(A) = a_{11} \det(M_{11}) - a_{12} \det(M_{12}) + \cdots + (-1)^{n+1} a_{1n} \det(M_{1n}).$$

Định thức của ma trận cấp n còn gọi là định thức cấp n .

■ **Ví dụ 1.2** Nếu A là ma trận cấp 2, $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ thì

$$\det(A) = a_{11}a_{22} - a_{12}a_{21}.$$

■

Tính chất 1.1 (i) $\det(A) = \det(A^T)$.

(ii) Nếu ma trận B thu được từ ma trận A bởi phép đổi chỗ hai hàng cho nhau thì $\det(B) = -\det(A)$.

(iii) Một định thức có hai hàng (hoặc hai cột) như nhau thì bằng 0.

(iv) Khai triển định thức theo hàng i :

$$\det(A) = (-1)^{i+1} a_{i1} \det(M_{i1}) + (-1)^{i+2} a_{i2} \det(M_{i2}) + \cdots + (-1)^{i+n} a_{in} \det(M_{in}).$$

Khai triển định thức theo cột j :

$$\det(A) = (-1)^{1+j} a_{1j} \det(M_{1j}) + (-1)^{2+j} a_{2j} \det(M_{2j}) + \cdots + (-1)^{n+j} a_{nj} \det(M_{nj}).$$

(v) Một định thức có một hàng (hay một cột) toàn là số không thì bằng 0.

(vi) Khi nhân các phần tử của một hàng (hay một cột) với cùng một số k thì được một định thức mới bằng định thức cũ nhân với k .

(vii) Một định thức có hai hàng (hay hai cột) tỷ lệ thì bằng không.

(viii) Khi tất cả các phần tử của một hàng (hay một cột) có dạng tổng của hai số hạng thì định thức có thể phân thành tổng của hai định thức, chẳng hạn:

$$\begin{vmatrix} a_{11} & a_{12} + a'_{12} \\ a_{21} & a_{22} + a'_{22} \end{vmatrix} = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} + \begin{vmatrix} a_{11} & a'_{12} \\ a_{21} & a'_{22} \end{vmatrix}.$$

(ix) Nếu định thức có một hàng (hay một cột) là tổ hợp tuyến tính của các hàng khác (hay các cột khác) thì định thức ấy bằng không.

(x) Khi cộng bội k của một hàng vào một hàng khác (hay bội k của một cột vào một cột khác) thì được một định thức mới bằng định thức cũ.

■ **Ví dụ 1.3** Tính định thức của ma trận $A = \begin{bmatrix} 1 & 1 & 3 \\ 1 & 0 & 5 \\ 3 & 1 & 2 \end{bmatrix}$. ■

Lời giải. Khai triển theo hàng thứ nhất ta được

$$A = 1 \cdot |M_{11}| - 1 \cdot |M_{12}| + 3 \cdot |M_{13}|,$$

trong đó

$$|M_{11}| = \begin{vmatrix} 0 & 5 \\ 1 & 2 \end{vmatrix} = -5, |M_{12}| = \begin{vmatrix} 1 & 5 \\ 3 & 2 \end{vmatrix} = -13, |M_{13}| = \begin{vmatrix} 1 & 0 \\ 3 & 1 \end{vmatrix} = 1.$$

Vậy $|A| = -5 + 13 + 3 = 11$. ■

Định nghĩa 1.11 Giả sử A là ma trận vuông cấp n , I là ma trận đơn vị cùng cấp. Ma trận A được gọi là *khả nghịch* nếu tồn tại ma trận vuông B sao cho $AB = BA = I$. Khi đó B được gọi là *ma trận nghịch đảo* của ma trận A , ký hiệu là A^{-1} .

Xét ma trận $A = (a_{ij})_{n \times n}$. Đặt $A_{ij} = (-1)^{i+j} \det(M_{ij})$, trong đó M_{ij} là ma trận con ứng với phần tử a_{ij} . Khi đó $P_A = (A_{ij})_{n \times n}$ được gọi là ma trận phụ hợp của ma trận A .

Định lý 1.1 Ma trận vuông A có ma trận nghịch đảo A^{-1} khi và chỉ khi $\det(A) \neq 0$. Khi đó

$$A^{-1} = \frac{1}{\det(A)} P_A^T = \frac{1}{\det(A)} \begin{bmatrix} A_{11} & A_{21} & \dots & A_{n1} \\ A_{12} & A_{22} & \dots & A_{n2} \\ \dots & \dots & \dots & \dots \\ A_{1n} & A_{2n} & \dots & A_{nn} \end{bmatrix}.$$

■ **Ví dụ 1.4** Tìm ma trận nghịch đảo của $A = \begin{bmatrix} 1 & 1 & 3 \\ 1 & 0 & 5 \\ 3 & 1 & 2 \end{bmatrix}$. ■

Lời giải. Theo ví dụ trên, ta có $\det(A) = 11 \neq 0$ nên tồn tại A^{-1} . Ta có

$$\begin{aligned} A_{11} &= (-1)^{1+1} |M_{11}| = -5, A_{12} = (-1)^{1+2} |M_{12}| = 13, A_{13} = (-1)^{1+3} |M_{13}| = 1, \\ A_{21} &= (-1)^{2+1} |M_{21}| = 1, A_{22} = (-1)^{2+2} |M_{22}| = -4, A_{23} = (-1)^{2+3} |M_{23}| = 2, \\ A_{31} &= (-1)^{3+1} |M_{31}| = 5, A_{32} = (-1)^{3+2} |M_{32}| = -2, A_{33} = (-1)^{3+3} |M_{33}| = -1. \end{aligned}$$

Vậy

$$P_A = \begin{bmatrix} -5 & 13 & 1 \\ 1 & -4 & 2 \\ 5 & -2 & -1 \end{bmatrix} \Rightarrow A^{-1} = \frac{1}{\det(A)} P_A^T = \frac{1}{11} \begin{bmatrix} -5 & 1 & 5 \\ 13 & -4 & -2 \\ 1 & 2 & -1 \end{bmatrix}.$$

■

Các phép toán ma trận trong Python

Trong Python, ma trận có thể được triển khai dưới dạng Danh sách 2D hoặc Mảng 2D và cung cấp đầy đủ các phép toán ma trận. Các hàm này được định nghĩa trong mô-đun “numpy”.

Dưới đây là một số hàm thực hiện các phép toán ma trận.

- *add()*: Phép cộng ma trận.
- *subtract()*: Phép trừ ma trận.
- *Divide()*: Phép chia các phần tử của ma trận.
- *multiple()*: Phép nhân các phần tử của ma trận.
- *dot()*: Phép nhân ma trận.
- “T” : Đổi số này dùng cho ma trận chuyển vị.
- *numpy.linalg.inv()*: Tính ma trận nghịch đảo.
- *numpy.linalg.det()*: Tính định thức của một ma trận.

```
# importing numpy for matrix operations
import numpy

# initializing matrices
x = numpy.array([[1, 2], [4, 5]])
y = numpy.array([[7, 8], [9, 10]])

# print out the results of matrix operations

# using add() to add matrices
print (numpy.add(x,y))

# using subtract() to subtract matrices
print (numpy.subtract(x,y))

# using divide() to divide matrices
print (numpy.divide(x,y))

# using multiply() to multiply matrices element wise
print (numpy.multiply(x,y))

# using dot() to multiply matrices
print (numpy.dot(x,y))

# using "T" to transpose the matrix
print (x.T)

# Calculating the inverse of the matrix
print(np.linalg.inv(x))
```

```
# calculating the determinant of matrix
print(np.linalg.det(n_array))
```

1.2 Ôn tập về xác suất

1.2.1 Độc lập xác suất và phụ thuộc xác suất

Định nghĩa 1.12 (i) *Phép thử* là việc thực hiện một thí nghiệm hoặc quan sát theo dõi một hiện tượng thực tế trong từng trường hợp cụ thể với những điều kiện cho trước.

(ii) Hiện tượng có thể xảy ra hoặc không xảy ra trong kết quả của phép thử được gọi là *sự kiện* hay *biến cố*.

(iii) *Sự kiện ngẫu nhiên* là sự kiện có thể xảy ra hoặc không xảy ra khi phép thử được thực hiện và ký hiệu là $A, B, C, \dots$

(iv) Trong một phép thử, tập hợp tất cả các kết quả có thể xảy ra được gọi là *không gian mẫu* và được ký hiệu là Ω .

Định nghĩa 1.13 — Quan hệ giữa các sự kiện.

(i) Hai sự kiện A và B được gọi *xung khắc* với nhau nếu chúng không đồng thời xảy ra trong cùng một phép thử, ký hiệu $AB = \emptyset$.

(ii) Họ các sự kiện $A_1, A_2, \dots, A_n$ được gọi là *xung khắc từng đôi* nếu một sự kiện bất kỳ trong họ xảy ra khi các sự kiện còn lại không xảy ra, tức là $A_i \cap A_j = \emptyset, \forall i \neq j$.

(iii) Hai sự kiện A và B được gọi là *độc lập* với nhau nếu sự kiện này xảy ra hay không xảy ra không làm ảnh hưởng tới khả năng xảy ra của sự kiện kia. Ngược lại thì chúng *phụ thuộc*.

(vi) Họ các sự kiện $A_1, A_2, \dots, A_n$ được gọi là *độc lập từng đôi* nếu A_i, A_j độc lập với mọi $i \neq j$.

(vii) Nhóm n sự kiện $A_1, A_2, \dots, A_n$ được gọi là *nhóm đầy đủ các sự kiện* nếu nhất định phải xảy ra một và chỉ một sự kiện trong các sự kiện ấy sau phép thử. Tức là $A_i A_j = \emptyset$ với $\forall i \neq j$ và $A_1 \cup A_2 \cup \dots \cup A_n = \Omega$.

1.2.2 Xác suất có điều kiện và quy tắc Bayes

Định nghĩa 1.14 *Xác suất* của sự kiện A trong điều kiện biết sự kiện B đã xảy ra được gọi là xác suất của A với điều kiện B . Ký hiệu là $P(A|B)$. Khi đó

$$P(A|B) = \frac{P(AB)}{P(B)}, \text{ với } P(B) > 0.$$

■ **Ví dụ 1.5** Một ngân hàng đề thi có cấu trúc như sau:

	Dễ	Khó
Lý thuyết	20	10
Bài tập	30	40

Bốc ngẫu nhiên một câu hỏi. Tính xác suất bốc trúng câu dễ. Nếu biết bốc được câu bài tập, tính xác suất bốc được câu dễ. ■

Lời giải. Gọi A : bốc được câu dễ

B : bốc được câu bài tập.

Khi đó

$$P(A) = \frac{50}{100} = 0.5, \quad P(A|B) = \frac{P(AB)}{P(B)} = \frac{\frac{30}{100}}{\frac{70}{100}} = \frac{3}{7}.$$

■

Định lý 1.2 — Định lý nhân xác suất.

Cho A và B là hai sự kiện bất kỳ. Khi đó

$$P(AB) = P(A)P(B|A) = P(B)P(B|A).$$

Chú ý 1.1 • Nếu A và B độc lập thì

$$P(AB) = P(A)P(B).$$

• Nếu A và B xung khắc thì

$$P(AB) = 0.$$

• Mở rộng cho tích n sự kiện bất kỳ $A_1, A_2, \dots, A_n$:

$$P(A_1 A_2 \cdots A_n) = P(A_1)P(A_2|A_1)P(A_3|A_1 A_2) \cdots P(A_n|A_1 A_2 \cdots A_{n-1}).$$

■ **Ví dụ 1.6** Có 4 que thăm, trong đó có 3 que thăm dài bằng nhau và 1 que thăm ngắn hơn. Bốn người lần lượt lên rút ngẫu nhiên một que thăm. Tính xác suất người thứ i rút được thăm ngắn ($i = 1, 2, 3, 4$). ■

Lời giải. Gọi A_i là sự kiện “người thứ i rút được que thăm ngắn”, $i = 1, 2, 3, 4$. Khi đó

$$P(A_1) = \frac{1}{4},$$

$$P(A_2) = P(\bar{A}_1 A_2) = P(\bar{A}_1)P(A_2|\bar{A}_1) = \frac{3}{4} \times \frac{1}{3} = \frac{1}{4},$$

$$P(A_3) = P(\bar{A}_1 \bar{A}_2 A_3) = P(\bar{A}_1)P(\bar{A}_2|\bar{A}_1)P(A_3|\bar{A}_1 \bar{A}_2) = \frac{3}{4} \times \frac{2}{3} \times \frac{1}{2} = \frac{1}{4},$$

$$P(A_4) = P(\bar{A}_1 \bar{A}_2 \bar{A}_3 A_4) = P(\bar{A}_1)P(\bar{A}_2|\bar{A}_1)P(\bar{A}_3|\bar{A}_1 \bar{A}_2)P(A_4|\bar{A}_1 \bar{A}_2 \bar{A}_3) = \frac{1}{4}.$$

Như vậy khả năng rút được thăm ngắn của 4 người là như nhau và bằng $\frac{1}{4}$. ■

Định lý 1.3 — Công thức cộng xác suất.

Nếu A và B là hai sự kiện bất kỳ thì

$$P(A + B) = P(A) + P(B) - P(AB).$$

Chú ý 1.2 • Nếu A và B là hai sự kiện xung khắc thì

$$P(A + B) = P(A) + P(B).$$

• Nếu A, B, C là ba sự kiện bất kỳ thì

$$P(A + B + C) = P(A) + P(B) + P(C) - P(AB) - P(AC) - P(BC) + P(ABC).$$

• Nếu $\bar{A}$ là sự kiện đối của A thì $P(\bar{A}) = 1 - P(A)$.

■ **Ví dụ 1.7** Một lớp có 100 sinh viên, trong đó có 30 sinh viên giỏi tiếng Anh, 40 sinh viên giỏi xác suất, 10 sinh viên giỏi cả tiếng Anh lẫn xác suất. Chọn ngẫu nhiên một sinh viên trong lớp. Tìm xác suất để sinh viên đó giỏi ít nhất 1 trong 2 môn trên. ■

Lời giải. Gọi A là sự kiện “sinh viên đó giỏi ít nhất 1 trong 2 môn”, B là sự kiện “sinh viên đó giỏi tiếng Anh”; C là sự kiện “sinh viên đó giỏi xác suất”. Khi đó, $A = B + C$. Suy ra

$$\begin{aligned} P(A) &= P(B + C) = P(B) + P(C) - P(BC) \\ &= \frac{30}{100} + \frac{40}{100} - \frac{10}{100} = \frac{60}{100} = 0.6. \end{aligned}$$

Định lý 1.4 — Công thức xác suất đầy đủ. Cho các sự kiện $A_1, A_2, \dots, A_n$ là một nhóm đầy đủ các sự kiện của một phép thử ngẫu nhiên, B là một sự kiện ngẫu nhiên bất kỳ. Khi đó

$$P(B) = \sum_{i=1}^n P(A_i B) = \sum_{i=1}^n P(A_i) P(B|A_i).$$

Công thức trên được gọi là *công thức xác suất đầy đủ*, hay *công thức xác suất toàn phần*.

■ **Ví dụ 1.8** Có 3 hộp đựng bi. Hộp I đựng 4 bi trắng và 6 bi đen, hộp II đựng 3 bi trắng và 6 bi đen, hộp III đựng 5 bi trắng và 3 bi đen. Lấy ngẫu nhiên một hộp, rồi từ hộp đó lấy ngẫu nhiên một viên bi. Tìm xác suất để được bi trắng. ■

Lời giải. Gọi A_i là sự kiện “lấy được hộp thứ i ”, $i = 1, 2, 3$, B là sự kiện “lấy được bi trắng”. Theo giả thiết ta có

$$P(A_1) = P(A_2) = P(A_3) = \frac{1}{3},$$

$$P(B|A_1) = \frac{4}{10}, P(B|A_2) = \frac{3}{9}, P(B|A_3) = \frac{5}{8}.$$

Áp dụng công thức xác suất đầy đủ ta có

$$P(B) = \sum_{i=1}^3 P(A_i)P(B|A_i) = \frac{1}{3} \frac{4}{10} + \frac{1}{3} \frac{3}{9} + \frac{1}{3} \frac{5}{8} = \frac{163}{360}.$$

■

Định lý 1.5 — Công thức Bayes. Cho các sự kiện $A_1, A_2, \dots, A_n$ là một nhóm đầy đủ các sự kiện và B là một sự kiện bất kỳ nào đó. Khi đó xác suất có điều kiện của sự kiện $A_k, k = 1, 2, \dots, n$ với điều kiện B được xác định bởi

$$P(A_k|B) = \frac{P(A_k)P(B|A_k)}{\sum_{i=1}^n P(A_i)P(B|A_i)}.$$

■ **Ví dụ 1.9** Hai xạ thủ bắn mỗi người một phát đạn vào mục tiêu. Xác suất bắn trúng của từng người lần lượt là 0.6 và 0.7. Xác suất mục tiêu bị tiêu diệt khi trúng 1 phát đạn là 0.5 và khi trúng 2 phát là 0.8.

a) Tính xác suất mục tiêu bị tiêu diệt.

b) Giả sử mục tiêu bị diệt. Tính xác suất chỉ có một xạ thủ bắn trúng. ■

Lời giải. Gọi A là sự kiện “xạ thủ thứ nhất bắn trúng”, B là sự kiện “xạ thủ thứ hai bắn trúng”, C_i là sự kiện có i phát đạn trúng mục tiêu, $i = 0, 1, 2$, D là sự kiện “mục tiêu bị tiêu diệt”.

a) Theo giả thiết ta có

$$P(A) = 0.6, P(B) = 0.7, P(D|C_0) = 0, P(D|C_1) = 0.5, P(D|C_2) = 0.8,$$

$$P(C_0) = P(\bar{A}\bar{B}) = P(\bar{A})P(\bar{B}) = 0.4 \cdot 0.3 = 0.12,$$

$$P(C_1) = P(A\bar{B}) + P(\bar{A}B) = 0.6 \cdot 0.3 + 0.4 \cdot 0.7 = 0.46,$$

$$P(C_2) = P(AB) = P(A)P(B) = 0.6 \cdot 0.7 = 0.42.$$

Các sự kiện C_0, C_1, C_2 tạo thành nhóm đầy đủ các sự kiện. Áp dụng công thức xác suất đầy đủ ta tính được

$$P(D) = P(C_0)P(D|C_0) + P(C_1)P(D|C_1) + P(C_2)P(D|C_2) = 0.566.$$

b) Xác suất cần tính là

$$P(C_1|D) = \frac{P(C_1D)}{P(D)} = \frac{P(C_1)P(D|C_1)}{P(D)} = \frac{0.46 \cdot 0.5}{0.566} \approx 0.406.$$

■

1.2.3 Biến ngẫu nhiên

Định nghĩa 1.15 *Biến ngẫu nhiên* là biến nhận giá trị là các số thực phụ thuộc vào kết cục của một phép thử ngẫu nhiên.

Ta thường dùng các chữ cái hoa $X, Y, Z, \dots$ để chỉ các biến ngẫu nhiên và các chữ cái thường $x, y, z, \dots$ để chỉ các giá trị cụ thể mà biến ngẫu nhiên đó nhận.

Biến ngẫu nhiên được phân làm hai loại: biến ngẫu nhiên rời rạc và biến ngẫu nhiên liên tục.

(a) **Biến ngẫu nhiên rời rạc:** X là biến ngẫu nhiên rời rạc nếu tập giá trị của nó là tập hợp hữu hạn hoặc vô hạn đếm được phân tử. Nói cách khác, ta có thể liệt kê tất cả các giá trị của biến ngẫu nhiên đó.

(b) **Biến ngẫu nhiên liên tục:** X là biến ngẫu nhiên liên tục nếu tập giá trị của nó lấp đầy một khoảng trên trục số.

■ **Ví dụ 1.10** (a) Gọi X là tổng số chấm xuất hiện khi tung đồng thời hai con xúc xắc thì X là một biến ngẫu nhiên rời rạc có thể nhận các giá trị từ 2 đến 12.

(b) Tại một bến xe buýt cứ 20 phút lại có một chuyến xe. Một người tới bến xe tại một thời điểm nào đó. Gọi Z là thời gian người đó phải chờ xe. Khi đó Z là một biến ngẫu nhiên liên tục có thể nhận các giá trị thuộc $[0, 20)$.

■

1.2.4 Phân phối liên tục

Định nghĩa 1.16 Biến ngẫu nhiên X được gọi là tuân theo luật *phân phối đều* trên đoạn $[a, b]$ nếu có hàm mật độ

$$f(x) = \begin{cases} 0 & \text{nếu } x \notin [a, b] \\ \frac{1}{b-a} & \text{nếu } x \in [a, b]. \end{cases}$$

Định nghĩa 1.17 Biến ngẫu nhiên X được gọi là tuân theo luật *phân phối mũ* nếu hàm mật độ của nó có dạng

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{khi } x > 0 \\ 0 & \text{khi } x \leq 0. \end{cases}$$

Định nghĩa 1.18 Biến ngẫu nhiên liên tục X được gọi là tuân theo luật phân phối khi bình phương với n bậc tự do, ký hiệu là $X \sim \chi_n^2$, nếu hàm mật độ xác suất của X có dạng

$$f(x) = \frac{1}{2} e^{-\frac{x}{2}} \frac{\left(\frac{x}{2}\right)^{\frac{n}{2}-1}}{\Gamma\left(\frac{n}{2}\right)}, x > 0,$$

ở đây $\Gamma(x) = \int_0^{+\infty} t^{x-1} e^{-t} dt, x > 0$ là hàm Gamma.

Định nghĩa 1.19 Biến ngẫu nhiên liên tục X được gọi là tuân theo luật phân phối Student với n bậc tự do, ký hiệu là $X \sim T^n$, nếu hàm mật độ xác suất của X có dạng

$$f(x) = \left(1 + \frac{x^2}{n}\right)^{-\frac{n+1}{2}} \frac{\Gamma\left(\frac{n+1}{2}\right)}{\sqrt{n\pi}\Gamma\left(\frac{n}{2}\right)}, -\infty < x < +\infty.$$

1.2.5 Phân phối chuẩn

Định nghĩa 1.20 Biến ngẫu nhiên X được gọi là tuân theo *phân phối chuẩn* với tham số μ, σ^2 , ký hiệu là $X \sim N(\mu, \sigma^2)$, nếu hàm mật độ xác suất của X có dạng

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \forall x \in \mathbb{R}.$$

Định lý 1.6 Kỳ vọng và phương sai của biến ngẫu nhiên X tuân theo luật phân phối chuẩn là

$$E(X) = \mu, \quad D(X) = \sigma^2,$$

độ lệch chuẩn là $\sigma(X) = \sqrt{D(X)} = \sigma$.

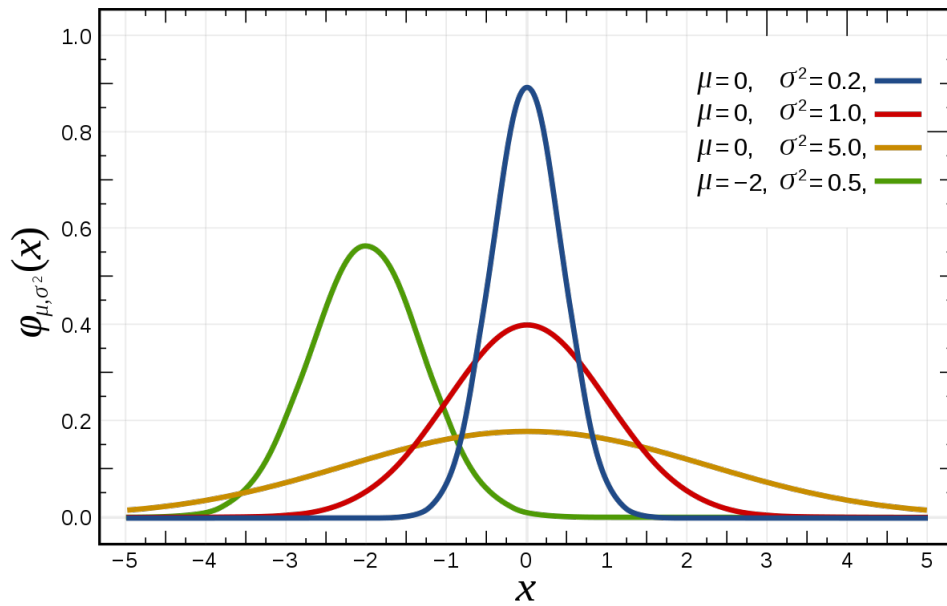
Định nghĩa 1.21 Phân phối chuẩn $N(\mu, \sigma^2)$ với $\mu = 0$ và $\sigma = 1$ gọi là *phân phối chuẩn tắc* $N(0, 1)$.

Hàm mật độ xác suất của biến ngẫu nhiên U có phân phối chuẩn tắc là

$$\varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}, \quad x \in \mathbb{R}.$$

Hàm phân phối xác suất của biến ngẫu nhiên U có phân phối chuẩn tắc là

$$\Phi(x) = \int_{-\infty}^x \varphi(t) dt = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{t^2}{2}} dt, \quad x \in \mathbb{R}.$$



Hình 1.1: Minh họa đồ thị của một số hàm mật độ của một số phân phối chuẩn.

Định lý 1.7 Nếu X là biến ngẫu nhiên có phân phối chuẩn $N(\mu, \sigma^2)$ thì

$$Z = \frac{X - \mu}{\sigma}$$

là biến ngẫu nhiên có phân phối chuẩn tắc $N(0, 1)$.

Quy tắc tính xác suất:

- Nếu $Z \sim N(0, 1)$ thì

$$P(a \leq Z < b) = \Phi(b) - \Phi(a).$$

- Nếu $X \sim N(\mu, \sigma^2)$ thì

$$P(a \leq X < b) = F(b) - F(a) = \Phi\left(\frac{b - \mu}{\sigma}\right) - \Phi\left(\frac{a - \mu}{\sigma}\right).$$

- Hàm $\varphi(x)$ đối xứng qua đường thẳng $x = 0$ nên

$$\Phi(-x) = 1 - \Phi(x).$$

Cách vẽ phân phối chuẩn trong Python

Cần sử dụng các mô-đun cần thiết sau:

- *Matplotlib* là thư viện trực quan hóa dữ liệu của Python được sử dụng rộng rãi cho mục đích trực quan hóa dữ liệu (sẽ được giới thiệu kỹ hơn trong Chương 3).

- *Numpy* cung cấp một đối tượng mảng đa chiều hiệu suất cao và các công cụ để làm việc với các mảng này. Đây là gói cơ bản cho tính toán khoa học với Python.
- *Scipy* là một thư viện python rất hữu ích trong việc giải nhiều phương trình và thuật toán Toán học.
- Mô-đun *Statistics* cung cấp các chức năng tính toán thống kê toán học của dữ liệu số.

Các hàm được sử dụng:

- Hàm tính giá trị trung bình của dữ liệu là `mean(data)`.
- Hàm tính độ lệch chuẩn của dữ liệu là `stdev(data)`.
- Hàm tính mật độ xác suất của phân phối chuẩn `norm.pdf(Data, loc, scale)`.

Các bước vẽ đồ thị:

- Nhập mô-đun
- Tạo dữ liệu
- Tính giá trị trung bình và độ lệch
- Tính mật độ xác suất của phân phối chuẩn
- Vẽ đồ thị sử dụng các giá trị tính toán ở trên
- Hiển thị đồ thị

■ **Ví dụ 1.11** Tạo một dãy dữ liệu cách đều nhau trong phạm vi từ -20 đến 20 và minh họa đồ thị của phân phối chuẩn tương ứng. ■

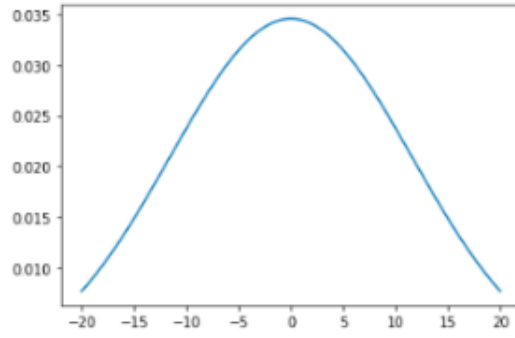
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
import statistics

# Plot between -20 and 20 with .001 steps.
x_axis = np.arange(-20, 20, 0.01)

# Calculating mean and standard deviation
mean = statistics.mean(x_axis)
sd = statistics.stdev(x_axis)

plt.plot(x_axis, norm.pdf(x_axis, mean, sd))
plt.show()
```

Kết quả nhận được (Output):



Hình 1.2: Output của Ví dụ 1.11.

1.3 Giả thuyết

1.3.1 Khoảng tin cậy

Định nghĩa 1.22 Giả sử chưa biết đặc trưng θ nào đó của biến ngẫu nhiên X . Ước lượng khoảng của θ là chỉ ra một khoảng số (g_1, g_2) nào đó chứa θ , tức là có thể ước lượng $g_1 < \theta < g_2$.

Định nghĩa 1.23 Với $\alpha > 0$ khá bé, ta tìm được $P(g_1 < \theta < g_2) = 1 - \alpha =: \beta$ thì ta kết luận: với độ tin cậy $1 - \alpha = \beta$, tham số θ nằm trong khoảng (g_1, g_2) . Khi đó

- (a) (g_1, g_2) được gọi là khoảng tin cậy của θ với độ tin cậy $\beta = 1 - \alpha$.
- (b) $1 - \alpha = \beta$ được gọi là độ tin cậy của ước lượng.
- (c) $I = g_2 - g_1$ được gọi là độ dài khoảng tin cậy.

Bài toán 1.1 Giả sử biến ngẫu nhiên X tuân theo luật phân phối chuẩn $N(\mu, \sigma^2)$ với kỳ vọng $E(X) = \mu$ chưa biết. Hãy ước lượng $E(X)$. ■

(i) Trường hợp đã biết phương sai $V(X) = \sigma^2$

(a) Khoảng tin cậy đối xứng của $E(X)$ là

$$\left(\bar{x} - z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{n}}, \bar{x} + z_{\frac{\alpha}{2}} \frac{\sigma}{\sqrt{n}} \right),$$

ở đây

- $\bar{x}$ là trung bình mẫu, $\bar{x} = \frac{1}{n}(x_1 + x_2 + \cdots + x_n)$,
- $z_{\frac{\alpha}{2}}$ được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_{\frac{\alpha}{2}}) = 1 - \frac{\alpha}{2}.$$

(b) Khoảng tin cậy trái của $E(X)$ là

$$\left(-\infty; \bar{x} + z_{\alpha} \frac{\sigma}{\sqrt{n}} \right),$$

trong đó z_α được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_\alpha) = 1 - \alpha.$$

(c) Khoảng tin cậy phải của $E(X)$ là

$$\left(\bar{x} - z_\alpha \frac{\sigma}{\sqrt{n}}, +\infty \right),$$

trong đó z_α được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_\alpha) = 1 - \alpha.$$

(ii) Trường hợp chưa biết phương sai, cỡ mẫu $n \geq 30$

(a) Khoảng tin cậy đối xứng của $E(X)$ là

$$\left(\bar{x} - z_{\frac{\alpha}{2}} \frac{s}{\sqrt{n}}, \bar{x} + z_{\frac{\alpha}{2}} \frac{s}{\sqrt{n}} \right),$$

ở đây

- $\bar{x}$ là trung bình mẫu, $s = \sqrt{s^2}$, s^2 là phương sai mẫu hiệu chỉnh

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2,$$

- $z_{\frac{\alpha}{2}}$ được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_{\frac{\alpha}{2}}) = 1 - \frac{\alpha}{2}.$$

(b) Khoảng tin cậy trái của $E(X)$ là

$$\left(-\infty; \bar{x} + z_\alpha \frac{s}{\sqrt{n}} \right),$$

trong đó z_α được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_\alpha) = 1 - \alpha.$$

(c) Khoảng tin cậy phải của $E(X)$ là

$$\left(\bar{x} - z_\alpha \frac{s}{\sqrt{n}}, +\infty \right),$$

trong đó z_α được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_\alpha) = 1 - \alpha.$$

(iii) Trường hợp chưa biết phương sai, cỡ mẫu $n < 30$

(a) Khoảng tin cậy đối xứng của $E(X)$ là

$$\left(\bar{x} - t_{\frac{\alpha}{2}}^{(n-1)} \frac{s}{\sqrt{n}}, \bar{x} + t_{\frac{\alpha}{2}}^{(n-1)} \frac{s}{\sqrt{n}} \right),$$

ở đây

- $\bar{x}$ là trung bình mẫu, $s = \sqrt{s^2}$, s^2 là phương sai mẫu hiệu chỉnh,
- $t_{\frac{\alpha}{2}}^{(n-1)}$ được xác định từ bảng phân phối Student với $n - 1$ bậc tự do.

(b) Khoảng tin cậy trái của $E(X)$ là

$$\left(-\infty; \bar{x} + t_{\alpha}^{(n-1)} \frac{s}{\sqrt{n}} \right),$$

trong đó $t_{\alpha}^{(n-1)}$ được xác định từ bảng phân phối Student với $n - 1$ bậc tự do.

(c) Khoảng tin cậy phải của $E(X)$ là

$$\left(\bar{x} - t_{\alpha}^{(n-1)} \frac{s}{\sqrt{n}}, +\infty \right),$$

trong đó $t_{\alpha}^{(n-1)}$ được xác định từ bảng phân phối Student với $n - 1$ bậc tự do.

Bài toán 1.2 Xác suất xảy ra sự kiện A là p . Do không biết p nên người ta thực hiện n phép thử độc lập, cùng điều kiện, trong đó có m phép thử xảy ra A . Khi đó tần suất xuất hiện A là $f = \frac{m}{n}$ là ước lượng điểm không chệch cho p . Với độ tin cậy $\gamma = 1 - \alpha$ hãy ước lượng khoảng cho p . ■

(a) Khoảng tin cậy đối xứng của p là

$$\left(f - z_{\frac{\alpha}{2}} \sqrt{\frac{f(1-f)}{n}}, f + z_{\frac{\alpha}{2}} \sqrt{\frac{f(1-f)}{n}} \right),$$

ở đây $z_{\frac{\alpha}{2}}$ được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_{\frac{\alpha}{2}}) = 1 - \frac{\alpha}{2}.$$

(b) Khoảng tin cậy trái của p là

$$\left(-\infty; f + z_{\alpha} \sqrt{\frac{f(1-f)}{n}} \right),$$

trong đó z_{α} được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_{\alpha}) = 1 - \alpha.$$

(c) Khoảng tin cậy phải của p là

$$\left(f + z_{\alpha} \sqrt{\frac{f(1-f)}{n}}, +\infty \right),$$

trong đó z_α được xác định từ bảng giá trị hàm phân phối chuẩn tắc từ hệ thức

$$\Phi(z_\alpha) = 1 - \alpha.$$

Chú ý 1.3 (a) Do tỷ lệ chỉ nhận giá trị từ 0 đến 1 nên ta có thể thay giá trị $-\infty$ bằng 0 và $+\infty$ bằng 1 trong khoảng tin cậy trái (phải).

(b) Các khoảng tin cậy trên được xây dựng khi kích thước mẫu n đủ lớn thỏa mãn $nf \geq 10$ và $n(1-f) \geq 10$.

Cách xác định khoảng tin cậy trong Python

- Với kích thước dữ liệu đầu vào cần xác định sử dụng hàm của phân phối chuẩn (norm.interval() function) hay phân phối t (t.interval() function) tương ứng.
- Xác định kích thước mẫu, độ tin cậy.
- Sử dụng thư viện scipy.stats trong Python.

■ **Ví dụ 1.12** Ước lượng bằng khoảng tin cậy 95% chiều cao trung bình của một tổng thể cây dựa vào mẫu quan sát gồm độ cao của 22 cây sau:

11, 12, 12.5, 13, 13, 15.5, 16, 17, 22, 23, 25, 26, 27, 28, 28, 29, 30, 32,
33, 33, 34, 34.

■

Trong ví dụ này, cỡ mẫu là $n = 22 < 30$ và chưa biết phương sai nên ta sử dụng phân phối student (t.interval() function) để xác định khoảng tin cậy.

```
import numpy as np
import scipy.stats as st

#defined sample data
data = [11, 12, 12.5, 13, 13, 15.5, 16, 17, 22, 23,
        25, 26, 27, 28, 28, 29, 30, 32, 33, 33, 34, 34]

# create 95% confidence interval
st.t.interval(alpha=0.95,
              df=len(gfg_data)-1,
              loc=np.mean(gfg_data),
              scale=st.sem(gfg_data))
```

■ **Ví dụ 1.13** Ước lượng bằng khoảng tin cậy 95% chiều cao trung bình của một tổng thể cây dựa vào mẫu quan sát gồm độ cao của 33 cây sau:

11, 11, 11.5, 11.5, 11.5, 12, 12, 12, 12.5, 13, 13, 15.5, 16, 17, 22,

23, 25, 26, 27, 28, 28, 29, 30, 32, 33, 33, 34, 34, 35, 35, 35.5, 35.5, 36.

■

Trong ví dụ này, cỡ mẫu là $n = 33 > 30$ và chưa biết phương sai nên ta sử dụng phân phối chuẩn (norm.interval() function) để xác định khoảng tin cậy.

```
import numpy as np
import scipy.stats as st

#defined sample data
data = [11, 11, 11.5, 11.5, 11.5, 12, 12, 12, 12.5, 13, 13,
        15.5, 16, 17, 22, 23, 25, 26, 27, 28, 28, 29, 30, 32, 33, 33, 34, 34,
        35, 35, 35.5, 35.5, 36.]

# create 95% confidence interval
st.norm.interval(alpha=0.95,
                 df=len(gfg_data)-1,
                 loc=np.mean(gfg_data),
                 scale=st.sem(gfg_data))
```

1.3.2 Kiểm định giả thuyết thống kê

Định nghĩa 1.24 — Giả thuyết cơ bản, giả thuyết đối.

- (i) Giả sử cần nghiên cứu tham số θ của biến ngẫu nhiên X và có cơ sở nào đó để nêu lên giả thuyết $\theta = \theta_0$. Giả thuyết này ký hiệu là H_0 , còn gọi là *giả thuyết cần kiểm định* hay *giả thuyết cơ bản* hay *giả thuyết không* (null hypothesis).
- (ii) Mệnh đề đối lập với giả thuyết H_0 ký hiệu là H_1 , còn gọi là *đối thuyết* (alternative hypothesis). Dạng tổng quát nhất của H_1 là $\theta \neq \theta_0$. Trong nhiều trường hợp giả thuyết đối được phát biểu cụ thể là $H_1 : \theta > \theta_0$ hoặc $H_1 : \theta < \theta_0$.

Bài toán 1.3 Giả sử biến ngẫu nhiên gốc X trong tổng thể có phân phối chuẩn $N(\mu, \sigma^2)$, trong đó $E(X) = \mu$ chưa biết nhưng có cơ sở để nêu lên giả thuyết $H_0 : \mu = \mu_0$ với μ_0 là tham số đã biết. Với mức ý nghĩa α hãy kiểm định giả thuyết này với các đối thuyết $H_1 : \mu \neq \mu_0$ hoặc $\mu > \mu_0$ hoặc $\mu < \mu_0$. ■

(i) Trường hợp đã biết phương sai $V(X) = \sigma^2$

Bước 1. Đặt giả thuyết H_0 và đối thuyết H_1 .

Bước 2. Xác định miền bác bỏ H_0 là W_α theo bảng sau

H_0	H_1	Miền bác bỏ W_α
$\mu = \mu_0$	$\mu \neq \mu_0$	$(-\infty, -z_{\frac{\alpha}{2}}) \cup (z_{\frac{\alpha}{2}}, +\infty)$
$\mu = \mu_0$	$\mu > \mu_0$	$(z_\alpha, +\infty)$
$\mu = \mu_0$	$\mu < \mu_0$	$(-\infty, -z_\alpha)$

trong đó $z_\alpha, z_{\frac{\alpha}{2}}$ được xác định từ bảng giá trị hàm phân phối chuẩn tắc $\Phi(x)$.

Bước 3. Tính giá trị quan sát

$$z_{qs} = \frac{\bar{x} - \mu_0}{\sigma} \sqrt{n}.$$

Bước 4. Xem z_{qs} có thuộc W_α hay không để kết luận.

- Nếu $z_{qs} \in W_\alpha$ thì bác bỏ giả thuyết H_0 .
- Nếu $z_{qs} \notin W_\alpha$ thì chưa có cơ sở để bác bỏ giả thuyết H_0 .

(ii) Trường hợp chưa biết phương sai, cỡ mẫu $n \geq 30$

Bước 1. Đặt giả thuyết H_0 và đối thuyết H_1 .

Bước 2. Xác định miền bác bỏ H_0 là W_α theo bảng sau trong đó $z_\alpha, z_{\frac{\alpha}{2}}$ được xác

H_0	H_1	Miền bác bỏ W_α
$\mu = \mu_0$	$\mu \neq \mu_0$	$(-\infty, -z_{\frac{\alpha}{2}}) \cup (z_{\frac{\alpha}{2}}, +\infty)$
$\mu = \mu_0$	$\mu > \mu_0$	$(z_\alpha, +\infty)$
$\mu = \mu_0$	$\mu < \mu_0$	$(-\infty, -z_\alpha)$

định từ bảng giá trị hàm phân phối chuẩn tắc $\Phi(x)$.

Bước 3. Tính giá trị quan sát

$$z_{qs} = \frac{\bar{x} - \mu_0}{s} \sqrt{n}.$$

Bước 4. Xem z_{qs} có thuộc W_α hay không để kết luận.

- Nếu $z_{qs} \in W_\alpha$ thì bác bỏ giả thuyết H_0 .
- Nếu $z_{qs} \notin W_\alpha$ thì chưa có cơ sở để bác bỏ giả thuyết H_0 .

(iii) Trường hợp chưa biết phương sai, cỡ mẫu $n < 30$

Bước 1. Đặt giả thuyết H_0 và đối thuyết H_1 .

Bước 2. Xác định miền bác bỏ H_0 là W_α theo bảng sau trong đó $t_\alpha^{(n-1)}, t_{\frac{\alpha}{2}}^{(n-1)}$ được xác định từ bảng phân phối Student.

Bước 3. Tính giá trị quan sát

$$t_{qs} = \frac{\bar{x} - \mu_0}{s} \sqrt{n}.$$

H_0	H_1	Miền bác bỏ W_α
$\mu = \mu_0$	$\mu \neq \mu_0$	$(-\infty, -t_{\frac{\alpha}{2}}^{(n-1)}) \cup (t_{\frac{\alpha}{2}}^{(n-1)}, +\infty)$
$\mu = \mu_0$	$\mu > \mu_0$	$(t_{\alpha}^{(n-1)}, +\infty)$
$\mu = \mu_0$	$\mu < \mu_0$	$(-\infty, -t_{\alpha}^{(n-1)})$

Bước 4. Xem t_{qs} có thuộc W_α hay không để kết luận.

- Nếu $t_{qs} \in W_\alpha$ thì bác bỏ giả thuyết H_0 .
- Nếu $t_{qs} \notin W_\alpha$ thì chưa có cơ sở để bác bỏ giả thuyết H_0 .

1.3.3 Ví dụ

■ **Ví dụ 1.14** Một hãng bảo hiểm thông báo rằng số tiền trung bình hãng chi trả cho khách hàng bị tai nạn ô tô là 8500 USD. Để kiểm tra lại, người ta kiểm tra ngẫu nhiên hồ sơ chi trả của 25 khách hàng thì thấy số tiền trung bình chi trả là 8900 USD. Giả sử số tiền chi trả tuân theo luật phân phối chuẩn với độ lệch chuẩn là 2600 USD. Hãy kiểm định lại thông báo của hãng bảo hiểm trên với mức ý nghĩa 5%. ■

Lời giải. Gọi X là số tiền hãng bảo hiểm chi trả cho khách hàng. $X \sim N(\mu, \sigma^2)$ với $\sigma = 2600$. Số tiền trung bình hãng chi trả cho khách hàng là $E(X) = \mu$ chưa biết. Đây là bài toán kiểm định giả thuyết về kỳ vọng của biến ngẫu nhiên phân phối chuẩn trường hợp đã biết phương sai.

Bước 1. Đặt giả thuyết $H_0 : \mu = \mu_0$, đối thuyết $H_1 : \mu \neq \mu_0$ với $\mu_0 = 8500$.

Bước 2. Với $\alpha = 0.05, z_{\frac{\alpha}{2}} = z_{0.025} = 1,96$. Miền bác bỏ giả thuyết H_0 là

$$W_\alpha = (-\infty, -z_{\frac{\alpha}{2}}) \cup (z_{\frac{\alpha}{2}}, +\infty) = (-\infty, -1.96) \cup (1.96, +\infty).$$

Bước 3. Từ số liệu của bài toán ta có $n = 25, \mu_0 = 8500, \bar{x} = 8900, \sigma = 2600$ suy ra giá trị quan sát

$$z_{qs} = \frac{\bar{x} - \mu_0}{\sigma} \sqrt{n} = \frac{8900 - 8500}{2600} \sqrt{25} \approx 0.77.$$

Bước 4. Vì $z_{qs} = 0.77 \notin W_\alpha$ nên chưa có cơ sở để bác bỏ giả thuyết H_0 . Tức là chưa có cơ sở để bác bỏ thông báo của hãng bảo hiểm với mức ý nghĩa 5%. ■

■ **Ví dụ 1.15** Nếu máy móc hoạt động bình thường thì trọng lượng sản phẩm là biến ngẫu nhiên có phân phối chuẩn $N(\mu, \sigma^2)$ với trọng lượng trung bình $\mu_0 = 100$ gam, độ lệch chuẩn $\sigma = 2$ gam. Qua một thời gian sản xuất người ta nghi ngờ trọng lượng

sản phẩm có xu hướng tăng lên, cân thử 100 sản phẩm thì trọng lượng trung bình của chúng là 100.4 gam. Với mức ý nghĩa $\alpha = 5\%$ hãy kết luận về điều nghi ngờ trên. ■

Lời giải. Gọi X là trọng lượng sản phẩm thì $X \sim N(\mu, \sigma^2)$ với $\sigma = 2$. Đây là bài toán kiểm định giả thuyết về kỳ vọng của biến ngẫu nhiên phân phối chuẩn trường hợp đã biết phương sai.

Bước 1. Đặt giả thuyết $H_0 : \mu = \mu_0$, đối thuyết $H_1 : \mu > \mu_0$ với $\mu_0 = 100$.

Bước 2. Với $\alpha = 0.05, z_\alpha = z_{0.05} = 1.65$. Miền bác bỏ giả thuyết H_0 là

$$W_\alpha = (z_\alpha, +\infty) = (1.65, +\infty).$$

Bước 3. Từ số liệu của bài toán ta có $n = 100, \mu_0 = 100, \bar{x} = 100.4, \sigma = 2$ suy ra giá trị quan sát

$$z_{qs} = \frac{\bar{x} - \mu_0}{\sigma} \sqrt{n} = \frac{100.4 - 100}{2} \sqrt{100} = 2.$$

Bước 4. Vì $z_{qs} = 2 \in W_\alpha$ nên bác bỏ giả thuyết H_0 . Tức là điều nghi ngờ nói trên là có sở sở với mức ý nghĩa 5%. ■

■ **Ví dụ 1.16** Một công ty có một hệ thống máy tính có thể xử lý 1200 hóa đơn trong một giờ. Công ty mới nhập một hệ thống máy tính mới. Hệ thống này khi chạy kiểm tra trong 40 giờ cho thấy số hóa đơn được xử lý trung bình trong một giờ là 1260 với độ lệch chuẩn hiệu chỉnh 215. Với mức ý nghĩa 5% hãy nhận định xem hệ thống mới có tốt hơn hệ thống cũ hay không? ■

Lời giải. Gọi X là số hóa đơn mà hệ thống máy tính mới xử lý được trong vòng một giờ. Ta thấy $E(X) = \mu$ là số hóa đơn trung bình mà hệ thống máy tính mới xử lý được trong một giờ chưa biết. Đây là bài toán kiểm định giả thuyết về kỳ vọng của biến ngẫu nhiên phân phối chuẩn trường hợp chưa biết phương sai và mẫu cỡ $n = 40 > 30$.

Bước 1. Đặt giả thuyết $H_0 : \mu = \mu_0$, đối thuyết $H_1 : \mu > \mu_0$ với $\mu_0 = 1200$.

Bước 2. Với $\alpha = 0.05, z_\alpha = z_{0.05} = 1.65$. Miền bác bỏ giả thuyết H_0 là

$$W_\alpha = (z_\alpha, +\infty) = (1.65, +\infty).$$

Bước 3. Từ số liệu của bài toán ta có $n = 40, \mu_0 = 1200, \bar{x} = 1250, \sigma = 215$ suy ra giá trị quan sát

$$z_{qs} = \frac{\bar{x} - \mu_0}{s} \sqrt{n} = \frac{1260 - 1200}{215} \sqrt{40} = 1.76.$$

Bước 4. Vì $z_{qs} = 1.76 \in W_\alpha$ nên bác bỏ giả thuyết H_0 , nghĩa là với số liệu này có thể coi hệ thống máy mới tốt hơn hệ thống máy cũ với mức ý nghĩa 5%. ■

Cách kiểm định giả thuyết thống kê trong python

■ **Ví dụ 1.17** Giả sử có một mẫu dữ liệu gồm khối lượng (gam) của 10 quả táo như sau:

98, 102, 95, 105, 99, 101, 97, 103, 100, 98.

Hãy kiểm tra xem khối lượng trung bình của những quả táo có sai khác 100 gam hay không? ■

Với ví dụ này, kích thước mẫu $n = 10 < 30$ và trường hợp chưa biết phương sai nên ta sử dụng phân phối student để thực hiện kiểm định (ttest_1samp).

```
import numpy as np
from scipy.stats import ttest_1samp

# Load the data
data = np.array([98, 102, 95, 105, 99, 101, 97, 103, 100, 98])

# Define the hypothesized value
hypothesized_value = 100

# Perform the One Sample T-Test
t_statistic, p_value = ttest_1samp(data, hypothesized_value)

# Interpret the results
if p_value < 0.05:
    print('Reject Null Hypothesis')
else:
    print('Fail to Reject Null Hypothesis')
```

Bài tập Chương 1

Bài tập 1.1 Thực hiện nhân hai ma trận

$$\text{a) } \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 1 & 4 \end{bmatrix}$$

$$\text{b) } \begin{bmatrix} 1 & 2 \\ 1 & 1 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix};$$

Bài tập 1.2 Tính các định thức cấp 3

$$a) \begin{vmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ -1 & -1 & 0 \end{vmatrix}$$

$$b) \begin{vmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{vmatrix};$$

Bài tập 1.3 Tìm các ma trận nghịch đảo

$$a) \begin{vmatrix} 2 & 1 & -1 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{vmatrix}$$

$$b) \begin{vmatrix} 1 & -1 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{vmatrix}$$

Bài tập 1.4 Viết chương trình python thực hiện các yêu cầu dưới đây:

- (a) Kiểm tra một ma trận bất kỳ có phải là ma trận tam giác trên, dưới, đường chéo không?
- (b) Tính tổng hai ma trận, nhân ma trận với một số, nhân hai ma trận?
- (c) Tính định thức của một ma trận, tính ma trận nghịch đảo của một ma trận?

Chú ý: Mỗi yêu cầu thực hiện theo hai cách:

- ★ Cách 1: code trực tiếp theo định nghĩa, định lý, cách xây dựng,...
- ★ Cách 2: sử dụng thư viện từ python.

Bài tập 1.5 Một hệ thống bơm trong sản xuất nông nghiệp sẽ ngừng hoạt động khi máy bơm bị hỏng hoặc các chỗ nối bị rò rỉ. Xác suất hỏng bơm là 0,07, xác suất rò rỉ là 0,1, xác suất hỏng bơm và rò rỉ là 0,03. Tính xác suất hệ thống bơm ngừng hoạt động.

Bài tập 1.6 Cho ba biến cố A, B và C trong cùng một không gian xác suất, có xác suất tương ứng: $P(A) = 0,7, P(B) = 0,6, P(C) = 0,5, P(AB) = 0,4, P(BC) = 0,2, P(AC) = 0,3$ và $P(ABC) = 0,1$. Tính xác suất để có đúng 2 trong 3 biến cố A, B và C xảy ra.

Bài tập 1.7 Một cuộc điều tra cho thấy, ở một thành phố, có 20,7% dân số dùng loại sản phẩm X , 50% dùng loại sản phẩm Y và trong số những người dùng Y , có 36,5% dùng X . Phỏng vấn ngẫu nhiên một người dân trong thành phố đó, tính xác suất để người ấy dùng cả X và Y .

Bài tập 1.8 Khi làm một xét nghiệm X , người bị HIV có kết quả dương tính với xác suất 95%, người không bị HIV có kết quả dương tính với xác suất 1%. Làm xét nghiệm X cho một người đến từ một địa phương có tỉ lệ nhiễm HIV là 1%. Tính xác suất kết quả là dương tính.

Bài tập 1.9 Ba lô thuốc A, B và C chứa rất nhiều chai thuốc. Tỷ lệ chai thuốc hỏng ở mỗi lô, theo thứ tự, là 0.1, 0.08 và 0.05. Chọn ngẫu nhiên một trong 3 lô rồi từ đó lấy ra 3 chai. Tính xác suất để được 2 chai tốt và 1 chai hỏng.

Bài tập 1.10 Có hai lô sản phẩm. Lô thứ nhất có tỷ lệ sản phẩm loại 1 là 90%, lô thứ hai có tỷ lệ sản phẩm loại 1 là 70%. Chọn ngẫu nhiên một lô rồi từ lô đó lấy ngẫu nhiên ra 1 sản phẩm thì được sản phẩm loại 1. Tính xác suất để sản phẩm đó thuộc lô thứ nhất.

Bài tập 1.11 Khi làm một xét nghiệm X , người bị HIV có kết quả dương tính với xác suất 95%, người không bị HIV có kết quả dương tính với xác suất 1%. Làm xét nghiệm X cho một người đến từ một địa phương có tỷ lệ nhiễm HIV là 1% thì được kết quả dương tính. Tính xác suất để người này thực sự nhiễm HIV.

Bài tập 1.12 Tuổi thọ của một loại sản phẩm do một nhà máy sản xuất là biến ngẫu nhiên X (giờ). Biết $X \sim N(1000, 100)$. Nếu thời gian bảo hành là 980 giờ thì tỷ lệ sản phẩm phải bảo hành là bao nhiêu.

Bài tập 1.13 Chiều dài của chi tiết được gia công trên máy tự động là biến ngẫu nhiên tuân theo quy luật phân phối chuẩn với độ lệch chuẩn là 0,01 mm. Chi tiết được coi là đạt tiêu chuẩn nếu các kích thước thực tế của nó sai lệch so với kích thước trung bình không vượt quá 0,02 mm. Tính tỷ lệ chi tiết không đạt tiêu chuẩn.

Bài tập 1.14 Thời gian cho đến khi sạc lại pin cho máy tính xách tay trong các điều kiện thông thường (thời gian sử dụng pin) có phân phối chuẩn với giá trị trung bình là 260 phút và độ lệch chuẩn là 50 phút.

- (a) Tính xác suất để thời gian sử dụng pin là hơn bốn giờ?
- (b) Thời gian sử dụng pin tối thiểu bằng bao nhiêu để có xác suất là 95%?
- (c) Vẽ đồ thị của phân phối chuẩn đã cho sử dụng thư viện của python?

Bài tập 1.15 Thời gian để sản xuất một sản phẩm loại A là một đại lượng ngẫu nhiên tuân theo luật phân phối chuẩn với các tham số $\mu = 10$ và $\sigma = 1$ (đơn vị là phút). Tính xác suất để một sản phẩm loại A nào đó được sản xuất trong khoảng thời gian từ 9 phút đến 12 phút.

Bài tập 1.16 Giả sử rằng tuổi thọ của một loại bóng đèn hình TV có độ lệch chuẩn bằng 500, nhưng chưa biết trung bình. Ngoài ra, tuổi thọ của loại bóng đèn đó tuân theo luật phân phối chuẩn. Khảo sát trên một mẫu ngẫu nhiên gồm 15 bóng loại trên, người ta tính được tuổi thọ trung bình là 8900 giờ. Hãy tìm khoảng tin cậy

95% cho tuổi thọ trung bình của loại bóng đèn hình nói trên. Kiểm nghiệm kết quả tính được bằng chương trình Python?

Bài tập 1.17 Một tổng thể X có phân phối chuẩn. Quan sát một mẫu ngẫu nhiên kích thước 25 người ta tính được trung bình là 15 và độ lệch chuẩn là 3. Hãy ước lượng kỳ vọng của X bằng khoảng tin cậy 95%. Kiểm nghiệm kết quả tính được bằng chương trình Python?

Bài tập 1.18 Kiểm tra tuổi thọ của một loại bóng đèn hình TV trên một mẫu ngẫu nhiên gồm 100 bóng đèn tính được giá trị trung bình mẫu là 8900 giờ và độ lệch chuẩn mẫu bằng 500 giờ. Hãy tìm khoảng tin cậy 95% cho trung bình tổng thể. Kiểm nghiệm kết quả tính được bằng chương trình Python?

Bài tập 1.19 Trọng lượng của một sản phẩm theo quy định là 6kg. Sau một thời gian sản xuất, người ta nghi ngờ trọng lượng của sản phẩm giảm đi. Bởi vậy, người ta tiến hành kiểm tra 121 sản phẩm và tính được trung bình mẫu là 5.975kg và phương sai mẫu hiệu chỉnh là 5.7596kg^2 . Với mức ý nghĩa 5%, hãy kết luận về nghi ngờ nói trên. Kiểm nghiệm kết quả tính được bằng chương trình Python?

Bài tập 1.20 Một công ty sản xuất hạt giống tuyên bố rằng một loại giống mới của họ có năng suất trung bình là 21.5 tạ/ha. Gieo thử hạt giống mới này tại 16 vườn thí nghiệm và thu được kết quả:

19.2, 18.7, 22.4, 20.3, 16.8, 25.1, 17.0, 15.8, 21.0, 18.6, 23.7, 24.1, 23.4, 19.8, 21.7, 18.9.

Dựa vào kết quả này hãy xác nhận xem quảng cáo của công ty có đúng không với mức ý nghĩa $\alpha = 5\%$. Biết rằng năng suất giống cây trồng là một biến ngẫu nhiên tuân theo luật phân phối chuẩn. Kiểm nghiệm kết quả tính được bằng chương trình Python?

Bài tập 1.21 Trọng lượng của một loại gà công nghiệp ở một trại chăn nuôi có phân phối chuẩn. Trọng lượng trung bình khi xuất chuồng năm trước là 2.8kg/con. Năm nay, người ta sử dụng một loại thức ăn mới. Cân thử 25 con khi xuất chuồng người ta tính được trung bình mẫu là 3.2kg và phương sai mẫu hiệu chỉnh là 0.25kg^2 . Với mức ý nghĩa 5% hãy kết luận xem loại thức ăn mới có thực sự làm tăng trọng lượng trung bình của đàn gà hay không? Kiểm nghiệm kết quả tính được bằng chương trình Python?

Bài tập 1.22 Trọng lượng của một loại gà công nghiệp ở một trại chăn nuôi có phân phối chuẩn. Trọng lượng trung bình khi xuất chuồng năm trước là 2.8kg/con. Năm

nay, người ta sử dụng một loại thức ăn mới. Cân thử 25 con khi xuất chuồng người ta tính được trung bình mẫu là 3.2kg và phương sai mẫu hiệu chỉnh là 0.25kg^2 . Với mức ý nghĩa 5% hãy kết luận xem loại thức ăn mới có thực sự làm tăng trọng lượng trung bình của đàn gà hay không? Kiểm nghiệm kết quả tính được bằng chương trình Python?



2. Chuẩn bị dữ liệu

“Dữ liệu là một thứ quý giá và sẽ tồn tại lâu hơn các hệ thống.”—Tim Berners-Lee

Chương 2 trình bày những phương pháp cơ bản trong khoa học dữ liệu bao gồm việc thu thập dữ liệu và xử lý dữ liệu như làm sạch, co giãn, chuẩn hóa, giảm chiều và biến đổi dữ liệu.

2.1 Thu thập dữ liệu

Thu thập dữ liệu là một quá trình tổng hợp tất cả các thông tin từ nhiều nguồn khác nhau và lưu trữ dữ liệu trong một hệ thống đã được thiết lập sẵn, sau đó cho phép trả lời câu hỏi có liên quan đến dữ liệu và đánh giá kết quả. Việc thu thập dữ liệu phục vụ cho việc phân tích, nghiên cứu và quản lý hoặc đưa ra quyết định liên quan đến các lĩnh vực như khoa học, xã hội và trong kinh doanh.

Các nguồn dữ liệu có thể bao gồm các hình ảnh, văn bản, video, âm thanh, dữ liệu từ mạng xã hội, website trực tuyến hay các nguồn dữ liệu khác. Quá trình thu thập dữ liệu có thể được thực hiện thông qua nhiều phương pháp khác nhau ví dụ như khảo sát, phân tích dữ liệu, thăm dò ý kiến, từ dữ liệu lớn và các công cụ thu thập dữ liệu từ nhiều nguồn trực tuyến. Mặc dù, có nhiều phương pháp thu thập dữ liệu khác nhau tùy theo ngành và lĩnh vực, nhưng mục tiêu chính là đảm bảo thu thập dữ liệu chính xác, trung thực và đáng tin cậy để phục vụ các mục đích đề ra.

Để dễ dàng cho việc áp dụng các phương pháp và kỹ thuật, học phần Nhập môn khoa học dữ liệu chủ yếu tập trung vào việc thu thập dữ liệu từ các nguồn website trực tuyến (*Web crawler*). Thu thập dữ liệu web có liên quan tới các thuật ngữ như nhện (*Spiders*), trình cào dữ liệu (*Crawler*) hay Bot (*Robots*). Lý do lớn nhất khi tập trung vào thu thập dữ liệu trên các website trực tuyến là do các tài nguyên trên các website có sự phổ biến rộng rãi và luôn sẵn sàng trên mạng Internet. Đồng thời, nó là một kho tàng phong phú gồm nhiều loại dữ liệu khác nhau như nguồn kiến thức

từ nhiều chủ đề hoặc thông tin về sản phẩm và người dùng. Trong nhiều ứng dụng, chẳng hạn như tìm kiếm và khám phá tri thức, cần phải tải xuống tất cả các trang có liên quan tại cơ sở dữ liệu trung tâm để các thuật toán máy học sử dụng các nguồn này để phân tích một cách có hiệu quả.

Ngoài các nội dung có sẵn trên website trực tuyến, việc sử dụng web dẫn đến một lượng dữ liệu đáng kể dưới dạng nhật ký người dùng (*user logs*) hoặc giao dịch web (*web transactions*). Có hai loại dữ liệu chính có sẵn trên web được sử dụng bởi các thuật toán khai phá dữ liệu:

1. *Web content information - Thông tin nội dung web*: Thông tin này tương ứng với các tài liệu, văn bản và liên kết Web do người dùng tạo ra. Các tài liệu, văn bản này được liên kết với nhau bằng các liên kết siêu văn bản (*hypertext links*). Do đó, thông tin nội dung chứa hai thành phần có thể được khai thác cùng nhau hoặc độc lập.
 - *Document data - Dữ liệu tài liệu*: được trích xuất từ các trang trên World Wide Web.
 - *Linkage data - Dữ liệu liên kết*: Web có thể được xem như một đồ thị lớn, trong đó các trang web tương ứng với các nút và các liên kết tương ứng với các cạnh giữa các nút. Thông tin liên kết này có thể được sử dụng theo nhiều cách, chẳng hạn như tìm kiếm trên Web hoặc xác định sự giống nhau giữa các nút.
2. *Web usage data - Dữ liệu sử dụng web*: Dữ liệu này tương ứng với các mô thức hoạt động của người dùng được kích hoạt bởi các ứng dụng Web. Những mẫu này có thể thuộc nhiều loại khác nhau:
 - *Web transactions, ratings & user feedback - Giao dịch web, đánh giá & phản hồi người dùng*: Người dùng web thường xuyên mua các loại mặt hàng khác nhau trên Web hoặc thể hiện sự yêu thích của họ đối với các sản phẩm cụ thể dưới dạng xếp hạng. Trong những trường hợp như vậy, hành vi mua hàng hoặc xếp hạng có thể được tận dụng để suy luận về sở thích của những người dùng khác nhau. Trong một số trường hợp, phản hồi của người dùng được cung cấp dưới dạng văn bản đánh giá của người dùng đều là các thông tin cần thiết để khai thác.
 - *Web logs - Nhật ký web*: Hành vi thực hiện trên trình duyệt web của người dùng được ghi lại dưới dạng nhật ký web thường được duy trì tại hầu hết các trang web. Thông tin thực hiện trên trình duyệt web này có thể được tận dụng để suy luận về hoạt động của người dùng.

Sự đa dạng các kiểu dữ liệu sẽ xác định các kiểu ứng dụng phổ biến trên web, việc phối hợp với các loại dữ liệu khác nhau dẫn đến sự ra đời của các ứng dụng tập trung vào nội dung (*content-centric*) hoặc tập trung vào việc sử dụng (*usage-centric*).

1. *Content-centric applications* : Các tài liệu và liên kết trên Web được sử dụng trong các ứng dụng khác nhau như tìm kiếm, phân cụm và phân loại. Một số ví dụ về các ứng dụng như sau:

- *Data mining applications - Ứng dụng khai phá dữ liệu*: Các tài liệu, văn bản web được sử dụng cùng với các loại ứng dụng khai thác dữ liệu khác nhau như phân cụm và phân loại. Các ứng dụng như vậy được các cổng thông tin web (*web portals*) sử dụng thường xuyên để tổ chức các trang web này.
- *Web crawling & resource discovery - Thu thập dữ liệu web & khám phá tài nguyên*: Web là một nguồn dữ liệu khổng lồ về các chủ đề và lĩnh vực khác nhau. Tuy nhiên, tài nguyên này được phân phối rộng rãi trên Internet và nó cần được thu thập, lưu trữ tại một cơ sở dữ liệu duy nhất để đưa ra các phương pháp suy luận hợp lý.
- *Web search - Tìm kiếm web*: Mục tiêu của việc tìm kiếm thông tin trên web là khai phá các dữ liệu và thông tin có liên quan, đảm bảo chất lượng để đáp ứng một bộ từ khóa do người dùng truy vấn. Như vậy, cần có các yêu cầu về chất lượng và mức độ phù hợp với các liên kết và cấu trúc của nội dung các dữ liệu.
- *Web linkage mining - Khai phá liên kết web*: Trong các ứng dụng khai phá liên kết web, các biểu diễn trong thực tế hoặc logic của cấu trúc liên kết trên web được khai thác để có những hiểu biết hữu ích. Ví dụ về các biểu diễn logic của cấu trúc web bao gồm mạng xã hội và thông tin các dữ liệu thì tồn tại mạng liên kết là mối quan hệ giữa các người dùng, trong khi mạng thông tin là mạng liên kết của người dùng và thực thể khác.

2. *Usage-centric applications* : Các hành vi của người dùng trên web sẽ được khai thác để tạo ra suy diễn về mặt tri thức. Có hai phương pháp để khai thác hành vi của người dùng như sau:

- *Recommender systems - Hệ thống khuyến nghị*: thông tin ưu tiên được tổng hợp dưới dạng xếp hạng các sản phẩm hoặc hành vi mua sản phẩm, chúng được sử dụng để đưa ra đề xuất cho những người dùng khác có những hành vi tương tự.
- *Web log analysis - Phân tích nhật ký website*: Nhật ký web là một tài

nguyên hữu ích cho chủ sở hữu trang web để xác định các hình thức tham gia trình duyệt web của người dùng. Các nhóm hành vi có thể tận dụng để đưa ra các suy luận như tìm các hành vi bất thường, sở thích của người dùng và tối ưu việc thiết kế trang web.

Trình thu thập dữ liệu web (*Web crawlers*) có rất nhiều ứng dụng và đặc biệt trong các ứng dụng tìm kiếm như Google và Bing, các trang website được tải xuống và lưu trữ, đồng thời được lập chỉ mục để cung cấp kết quả cho các truy vấn bằng từ khóa của người dùng. Để cập nhật dữ liệu Web crawlers làm mới định kỳ các nguồn website đã tải xuống tại cơ sở dữ liệu máy chủ của họ. Những trình thu thập dữ liệu như vậy được gọi là trình thu thập dữ liệu phổ quát (*universal crawlers*) vì chúng thu thập dữ liệu tất cả các trang trên các website bất kể chủ đề hay vị trí. Trình thu thập dữ liệu web cũng được sử dụng cho các nghiệp vụ thông minh (BI), trong đó các trang web liên quan đến cùng một chủ đề sẽ được thu thập thông tin hoặc các trang web của đối các thủ cạnh tranh được theo dõi, giám sát và thu thập dữ liệu sẽ được cập nhật và tăng dần khi có sự thay đổi và cập nhật dữ liệu mới. Những trình thu thập như vậy còn được gọi là trình thu thập dữ liệu ưu tiên (*preferential crawlers*) vì có sự phân biệt giữa mức độ liên quan của các trang web khác nhau đối với ứng dụng hiện tại.

2.1.1 Thuật toán thu thập cơ bản

Thiết kế của một trình thu thập dữ liệu khá phức tạp, với kiến trúc phân tán và nhiều quy trình hoặc luồng. Phần này sẽ mô tả trình thu thập dữ liệu một cách tuần tự và tổng quát từ đơn giản nhất để có thể nắm bắt được bản chất của cách trình thu thập dữ liệu được xây dựng.

Thuật toán trình thu thập dữ liệu cơ bản, được mô tả theo cách tổng quan nhất, sử dụng đầu vào bao gồm một tập hợp các URL (*Universal Resource Locators*) ký hiệu là S và thuật toán lựa chọn A . Thuật toán A quyết định dữ liệu nào sẽ thu thập tiếp theo từ danh sách URL hiện tại (*frontier list*). Danh sách này thể hiện các URL được trích xuất từ các trang web mà trình thu thập tìm nạp để thu thập dữ liệu. Thuật toán lựa chọn A rất quan trọng vì nó quy định chiến lược cơ bản được trình thu thập dữ liệu sử dụng để khai phá các tài nguyên. Ví dụ: nếu các URL mới được thêm vào cuối danh sách và thuật toán A chọn các nguồn trang web thu thập từ đầu danh sách, thì điều này tương ứng với thuật toán sử dụng là thuật toán tìm kiếm theo chiều rộng.

Thuật toán trình thu thập thông tin cơ bản được mô tả trong thuật toán 2.1.1 dưới

đây:

Algorithm 1 Thuật toán thu thập cơ bản: *BasicCrawler*(URLs: S , Thuật toán lựa chọn: A)

```

1: begin
2:  $FrontierList = S$ ;
3: repeat
4:   Sử dụng thuật toán  $A$  để lựa chọn URL từ  $X \in FrontierList$ ;
5:    $FrontierList = FrontierList - X$ ;
6:   Tìm nạp dữ liệu từ URL  $X$  và thêm cơ sở dữ liệu lưu trữ;
7:   Thêm tất cả các URL có liên quan trong  $X$  và nạp vào cuối  $FrontierList$ ;
8: until Điều kiện kết thúc
9: end

```

Thuật toán trình thu thập dữ liệu cơ bản tiến hành như sau:

- Tập hợp URL gốc được thêm vào danh sách - *frontier list*. Trong mỗi lần lặp lại, thuật toán lựa chọn A chọn một trong các URL từ *frontier list*. URL này bị xóa khỏi *frontier list* và sau đó được tìm nạp bằng giao thức HTTP. Đây là cơ chế tương tự được các trình duyệt sử dụng để tìm nạp các trang Web.
- Trang web đã tìm nạp được lưu trữ trong cơ sở dữ liệu cục bộ cùng với các URL trong trang được trích xuất đó. Các URL này sau đó được thêm vào danh sách *frontier list*, với điều kiện là chúng chưa được truy cập tới. Do đó, một cấu trúc dữ liệu riêng, dưới dạng bảng băm, được duy trì để lưu trữ tất cả các URL đã truy cập. Trong triển khai thực tế của trình thu thập, không phải tất cả các URL chưa được truy cập đều được thêm vào *frontier list* do *Web spam*, *bay spider*, chủ đề lựa chọn hoặc đơn giản là giới hạn về kích thước của *frontier list*.
- Sau khi các URL đã được thêm vào *frontier list*, lần lặp lại tiếp theo sẽ lặp lại quy trình với URL tiếp theo trong danh sách này. Quá trình kết thúc khi *frontier list* trống. Nếu *frontier list* trống, điều đó không có nghĩa toàn bộ thông tin của web đã được thu thập dữ liệu.

Trình thu thập thông tin sử dụng một thuật toán tìm kiếm trên đồ thị với các liên kết giữa các URL bằng cách phân tích cú pháp các trang Web và trích xuất các URL. Việc lựa chọn thuật toán A đôi khi dẫn đến thu thập dữ liệu không đầy đủ, đặc biệt là trong trường hợp không thể thu thập dữ liệu tất cả các trang có liên quan do hạn chế về tài nguyên. Trình thu thập sử dụng nhiều chiến lược lựa chọn khác được xác định bởi thuật toán A như sau:

- *Chiến lược dựa theo sự cập nhật dữ liệu*: Các trình thu thập phổ biến đều là

các trình thu thập gia tăng dữ liệu với mục đích làm mới các dữ liệu đã được thu thập từ trước đó, nên cập nhật dữ liệu từ các trang thường xuyên thay đổi là điều vô cùng cần thiết. Tần suất thay đổi có thể được ước tính từ các lần thu thập lặp đi lặp lại trước đó của cùng một trang, ví dụ một số nguồn dữ liệu như cổng thông tin luôn được cập nhật dữ liệu mới. Do đó, các nguồn trang thường xuyên cập nhật dữ liệu sẽ được lựa chọn thu thập hơn từ thuật toán lựa chọn A.

- *Chính sách ưu tiên*: Thuật toán lựa chọn A có thể chọn cụ thể các trang Web có *PageRank* cao từ danh sách *frontier list*.

2.1.2 Thu thập ưu tiên

Trong một trình thu thập ưu tiên, chỉ những trang web đáp ứng tiêu chí do người dùng xác định mới được thu thập dữ liệu. Các tiêu chí này có thể là từ khóa có trong trang, chủ đề văn bản (xác định bằng thuật toán học máy), tiêu chí địa lý về vị trí trang hoặc kết hợp các tiêu chí khác nhau. Các tiêu chí tùy thuộc vào yêu cầu thu thập để tạo cơ sở cho việc thu thập, sự khác biệt này thể hiện cách tiếp cận khác nhau để cập nhật danh sách *frontier list* trong quá trình thu thập dữ liệu với các lưu ý dưới đây:

- Trang web thu thập cần đáp ứng tiêu chí do người dùng chỉ định để các URL được trích xuất từ trang web đó được thêm vào *frontier list*.
- Đối với trình thu thập chú trọng ngữ cảnh (*context-focused*), trình thu thập được đào tạo để học khả năng liên quan của các trang nằm trong khoảng cách ngắn của trang web, ngay cả khi trang Web không liên quan trực tiếp đến tiêu chí do người dùng đặt ra. Ví dụ: một trang Web về “*khai phá dữ liệu*” có nhiều khả năng trở đến một trang web về “*truy xuất thông tin*”, mặc dù trang khai phá dữ liệu có thể không liên quan đến các truy vấn về “*truy xuất thông tin*”. URL từ các trang web như vậy có thể được thêm vào *frontier list*. Do đó, các phương pháp dự đoán cần được thiết kế để hiểu mức độ phù hợp với các ngữ cảnh cụ thể.

Các thay đổi trong trình thu thập ưu tiên được thực hiện với thuật toán A. Ví dụ: các URL có văn bản neo (*anchor text*) hoặc có các token liên quan trong địa chỉ web, có thể được ưu tiên chọn trước bằng thuật toán A. Một URL chẳng hạn như *http://www.golf.com*, với từ “*golf*” trong địa chỉ Web có thể phù hợp hơn với chủ đề “*golf*” hơn là một URL không có từ đó.

2.1.3 Đa luồng

Khi trình thu thập đưa ra yêu cầu về một URL và đợi yêu cầu đó, lúc đó hệ thống sẽ không hoạt động, không có công việc nào được thực hiện đến khi trình thu thập kết thúc. Điều này dẫn đến một sự lãng phí tài nguyên, do đó, đa luồng là một phương pháp để tăng tốc độ thu thập thông tin. Ý tưởng là sử dụng nhiều luồng của trình thu thập để cập nhật cấu trúc dữ liệu dùng chung cho các URL đã truy cập và kho lưu trữ dữ liệu của trang. Trong triển khai thực tế của các công cụ tìm kiếm lớn, trình thu thập được phân phối theo vùng địa lý với mỗi "*sub-crawl*" để thu thập các trang trong khoảng cách địa lý gần.

2.1.4 Tránh các bẫy Spider

Lý do chính khiến thuật toán thu thập luôn truy cập các trang Web khác nhau vì nó duy trì một danh sách các URL đã truy cập trước đó cho mục đích so sánh. Tuy nhiên, một số trang web mua sắm tạo các URL động trong đó trang cuối cùng đã truy cập được thêm vào cuối để cho phép máy chủ ghi lại chuỗi hành động của người dùng trong URL để phân tích trong tương lai. Ví dụ: khi người dùng nhấp vào liên kết cho trang 2 từ <http://www.examplesite.com/page1>, URL mới được tạo động sẽ là <http://www.examplesite.com/page1/page2>. Các trang được truy cập thêm sẽ tiếp tục được thêm vào cuối URL, ngay cả khi các trang này đã được truy cập trước đó. Một cách tự nhiên để chống lại điều này là giới hạn kích thước tối đa của URL. Ngoài ra, giới hạn tối đa cũng có thể được đặt cho số lượng URL được thu thập thông tin từ một trang web cụ thể.

2.1.5 Phát hiện sự trùng lặp

Một trong những vấn đề chính với các trang Web được thu thập nhiều lần. Do đó, trình thu thập phải có khả năng phát hiện các bản sao trùng lặp. Một cách tiếp cận được gọi là *shingling* thường được sử dụng để giải quyết vấn đề này.

Một *K-shingle* từ một tài liệu văn bản là một chuỗi gồm k từ xuất hiện liên tiếp trong văn bản. Một *shingling* cũng có thể được xem giống như một *k-gram*. Ví dụ: xem xét một văn bản bao gồm câu sau: *Lan có một chú cún con, lông của nó trắng như tuyết*.

Bộ *K-shingle* được trích ra từ câu này là “*Lan có*”, “*có một*”, “*một chú*”, “*chú cún*”, “*cún con*”, “*con lông*”, “*lông của*”, “*của nó*”, “*nó trắng*”, “*trắng như*” và “*như tuyết*”. Gọi $S1$ và $S2$ là *k-shingles* được trích xuất từ hai tài liệu $D1$ và $D2$. Sự tương đồng dựa trên *shingling* giữa $D1$ và $D2$ được tính bằng hệ số Jaccard giữa $S1$

và $S2$:

$$J(S1, S2) = \frac{|S1 \cap S2|}{|S1 \cup S2|}$$

Thông thường, giá trị k nằm trong khoảng từ 5 đến 10 từ, tùy thuộc vào kích thước của tập văn bản và lĩnh vực áp dụng.

2.1.6 Áp dụng thu thập dữ liệu - Web Scraping

Bước đầu của việc lấy dữ liệu từ các trang web bao gồm tìm nạp các trang web và nhận các thông tin có cấu trúc có ý nghĩa.

Phân tích cú pháp HTML

Các trang web được viết bằng HTML, trong đó phần văn bản được tạo thành từ các phần tử và thuộc tính của nó:

```
<html>
<head>
<title>A web page</title>
</head>
<body>
<p id="author">Joel Grus</p>
<p id="subject">Data Science</p>
</body>
</html>
```

Trong các trang web được tạo theo ngữ nghĩa dữ liệu có thể trích xuất dữ liệu bằng cách sử dụng các quy tắc “tìm phần tử trong $\langle p \rangle$ có *id* là *subject* và trả về văn bản trong đó.”. Thực tế, HTML thường không có một định dạng tốt, điều này có nghĩa cần hiểu ý nghĩa các thành phần của các thẻ HTML.

Để lấy dữ liệu từ HTML, có thể sử dụng thư viện *Beautiful Soup* là một thư viện Python được sử dụng để phân tích cú pháp HTML và XML, giúp cho việc trích xuất thông tin từ các trang web trở nên dễ dàng hơn. Trong giáo trình này, phiên bản được sử dụng là Beautiful Soup 4.6.0. Bên cạnh đó là thư viện *Requests*, để gửi tất cả các loại yêu cầu HTTP, là thư viện thực hiện nhiệm vụ này hơn so với các thư viện khác được tích hợp trong Python.

Trình phân tích cú pháp HTML tích hợp của Python không phải lúc cũng xử lý tốt với các trang không có một định dạng HTML tốt. Do đó, để khắc phục điều này chương trình thu thập cần cài đặt thêm một thư viện phân tích cú pháp là *html5lib*. Trong môi trường lập trình ảo của Python - *Python Virtual Environment*, cài đặt các thư viện bằng câu lệnh dưới đây:

```
python -m pip install beautifulsoup4 requests html5lib
```

Để sử dụng *BeautifulSoup*, chuyển một chuỗi chứa HTML vào hàm *BeautifulSoup*. Trong ví dụ dưới đây là kết quả của một lệnh gọi tới *request.get*, sau đó, có thể phát triển việc thu thập bằng một vài phương pháp đơn giản.

```
from bs4 import BeautifulSoup
import requests

# I put the relevant HTML file on GitHub. In order to fit
# the URL in the book I had to split it across two lines.
# Recall that whitespace-separated strings get concatenated.

url = ("https://raw.githubusercontent.com/"
"joelgrus/data/master/getting-data.html")
html = requests.get(url).text
soup = BeautifulSoup(html, 'html5lib')
```

Thông thường, việc thu thập sẽ làm việc với các đối tượng tương ứng với các thẻ (*Tag objects*), chúng biểu thị cấu trúc của một trang HTML. Ví dụ: để tìm thẻ `<p>` đầu tiên (và nội dung của thẻ), sử dụng đoạn lệnh sau:

```
first_paragraph = soup.find('p')    # hoac chi can su dung soup.p
```

Nội dung văn bản của một thẻ - *Tag* sẽ được lấy bằng cách sử dụng thuộc tính của nó như sau:

```
first_paragraph_text = soup.p.text
first_paragraph_words = soup.p.text.split()
```

Các thuộc tính của thẻ - *Tag* có thể được trích xuất bằng cách xử lý nó như một từ điển *dict*:

```
first_paragraph_id = soup.p['id']      # Bao loi KeyError neu khong co 'id'
first_paragraph_id2 = soup.p.get('id') # Tra ve None neu khong co 'id'
```

Nhiều thẻ được lấy một lần bằng cách sử dụng các câu lệnh dưới đây:

```
all_paragraphs = soup.find_all['p'] # Hoac chi can soup('p')
paragraphs_with_ids = [p for p in soup('p') if p.get('id')]
```

Thông thường, để tìm các thẻ với một lớp (*class*) cụ thể:

```
important_paragraphs = soup('p', {'class': 'important'})
important_paragraphs2 = soup('p', 'important')
important_paragraphs3 = [ p for p in soup('p')
if 'important' in p.get('class', [])]
```

Từ đó có thể kết hợp các phương thức này để triển khai thu thập với cấu trúc logic phức tạp hơn. Ví dụ: nếu muốn tìm mọi phần tử `<span>` có trong phần tử `<div>`, sẽ tiến hành như sau:

```
# Warning: chương trình sẽ tra về thẻ <span> nhiều lần
# nếu nó nằm trong nhiều thẻ <div>.
spans_inside_divs = [span
for div in soup('div') # cho mọi thẻ <div> trong trang web
for span in div('span')] # tìm mọi thẻ <span> trong <div>
```

Đối với những tác vụ phức tạp hơn có thể tìm đọc tại tài liệu *BeautifulSoup* ¹.

Ví dụ: Theo dõi đại biểu Quốc hội Mỹ

Bài toán theo dõi đại biểu Quốc hội Mỹ yêu cầu định lượng những thông tin về Quốc hội, đặc biệt, cần tìm tất cả những người đại diện có thông cáo báo chí các thông tin về Quốc hội. Một trang có liên kết đến tất cả các trang web của các đại diện, đại biểu Quốc hội Mỹ tại <https://www.house.gov/representatives> và tất cả các liên kết đến các trang web này khi xem nguồn html sẽ có dạng:

```
<td>
<a href=https://jayapal.house.gov>Jayapay, Pramila</a>
</td>
```

Bước đầu tiên là việc thu thập toàn bộ các link URL từ trang này như sau:

```
from bs4 import BeautifulSoup
import requests

url = "https://www.house.gov/representatives"
text = requests.get(url).text
soup = BeautifulSoup(text, "html5lib")

all_urls = [a['href']]
for a in soup('a')
if a.has_attr('href')]

print(len(all_urls)) # Kết quả trả về 965
```

Dữ liệu trả về rất nhiều các URL, chúng thường bắt đầu bởi `http://` hoặc `https://` và kết thúc bởi `.house.gov` hoặc `.house.gov/`. Sử dụng biểu thức chính quy để loại bỏ những URL không thỏa mãn:

```
Import re
```

¹<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>


```
# URL phải bắt đầu bởi http:// hoặc https://
# URL kết thúc bởi .house.gov hoặc .house.gov/.
regex = r"^https?:/*.*\.house\.gov/?$"

# Thu kiểm tra một vài ví dụ sau
assert re.match(regex, "http://joel.house.gov")
assert re.match(regex, "https://joel.house.gov")
assert re.match(regex, "http://joel.house.gov/")
assert re.match(regex, "https://joel.house.gov/")
assert not re.match(regex, "joel.house.gov")
assert not re.match(regex, "http://joel.house.gov")
assert not re.match(regex, "https://joel.house.gov/biography")

# Áp dụng cho toàn bộ URL
good_urls = [url for url in all_urls if re.match(regex, url)]
print(len(good_urls)) # Kết quả trả về 862
```

Trong danh sách URL trả về vẫn có nhiều URL bị trùng lặp do đó cần loại bỏ trùng lặp bằng câu lệnh sau:

```
good_urls = list(set(good_urls))
print(len(good_urls)) # Kết quả chỉ còn 431
```

Trong thực tế, sẽ luôn luôn có một số ghế trống ở Quốc hội hoặc có đại biểu không có website thông tin hoặc không có link đính kèm. Ví dụ như dưới đây:

```
html = requests.get('https://jayapal.house.gov').text
soup = BeautifulSoup(html, 'html5lib')

# Sử dụng kiểu dữ liệu set vì một link có thể xuất hiện nhiều lần
links = {a['href'] for a in soup('a') if 'press releases' in a.text.lower()}
print(links) # {'/media/press-releases'}
```

Chú ý đây chỉ là các đường link tương đối, cần gắn với link của trang web gốc. Tiếp theo, thực hiện các thao tác thu thập như sau:

```
from typing import Dict, Set

press_releases: Dict[str, Set[str]] = {}

for house_url in good_urls:
    html = requests.get(house_url).text
    soup = BeautifulSoup(html, 'html5lib')
    pr_links = {a['href'] for a in soup('a') if 'press releases'
                in a.text.lower()}
    print(f"{house_url}:{pr_links}")
    press_releases[house_url] = pr_links
```

Chú ý 2.1 Thông thường, việc thu thập dữ liệu của một trang web như này cần phải lưu ý đến các chính sách của trang web đó. Hầu hết các trang web sẽ có tệp *robots.txt* cho biết tần suất phép thu thập trong trang web (và những đường dẫn nào không được phép thu thập), nhưng trong ví dụ là thông tin công khai từ Quốc hội Mỹ nên được phép thu thập.

Khi cuộn lần lượt trang web này, sẽ thấy rất nhiều */media/press-releases* và *media-center/press-releases*, cũng như nhiều địa chỉ khác. Ví dụ một trong những URL này là *https://jayapal.house.gov/media/press-releases*. Mục tiêu của việc thu thập dữ liệu này là tìm ra những đại biểu Quốc hội nào có thông cáo báo chí đề cập đến “dữ liệu”. Cần viết một hàm kiểm tra xem một trang thông cáo báo chí có đề cập đến một thuật ngữ nhất định nào không như *facebook*, *twitter* hay *data*. Trong nguồn HTML của trang web thì nội dung của các đoạn văn bản thường nằm bên trong các thẻ *<p>*, ta sẽ tìm các từ khoá trong các đoạn văn bản đó như sau:

```
def paragraph_mentions(text: str, keyword: str) -> bool:
    """
    Tra về True nếu văn bản trong thẻ <p> chứa từ khóa trong {keyword}
    """
    soup = BeautifulSoup(text, 'html5lib')
    paragraphs = [p.get_text() for p in soup('p')]

    return any(keyword.lower() in paragraph.lower()
               for paragraph in paragraphs)
```

Kiểm tra kết quả vừa thu thập bằng các câu lệnh dưới đây:

```
text = """<body><h1>Facebook</h1><p>Twitter</p>"""
assert paragraph_mentions(text, "twitter") # nam trong the <p>
assert not paragraph_mentions(text, "facebook") # không nam trong the <p>
```

Cuối cùng, tìm những đại biểu liên quan và đưa ra tên của họ mà trong trang của họ có xuất hiện từ khoá *data*:

```
for house_url, pr_link in press_releases.items():
    for pr_link in pr_links:
        url = f"{house_url}/{pr_link}"
        text = requests.get(url).text

        if paragraph_mentions(text, 'data'):
            print(f"{house_url}")
            break
```

Sử dụng APIs

Nhiều trang web và dịch vụ web cung cấp Giao diện lập trình ứng dụng (*API*), cho phép yêu cầu rõ ràng dữ liệu có cấu trúc. Điều này giúp tránh được những rắc rối khi phải thu thập chúng bằng các thẻ html!

JSON và XML

Vì HTTP là một giao thức để truyền văn bản nên dữ liệu yêu cầu thông qua API web cần được tuần tự thành định dạng chuỗi. Thông thường, việc tuần tự hóa này sử dụng Ký hiệu đối tượng JavaScript (JSON). Các đối tượng JavaScript trông khá giống với kiểu từ điển trong Python:

```
{ "title": "Data Science Book",  
  "author": "Joel Grus",  
  "publicationYear": "2019",  
  "topic": ["data", "science", "data science"]}
```

Cú pháp JSON có thể phân tích bằng sử dụng mô-đun *json* của Python. Cụ thể, sử dụng hàm *loads* để giải tuần tự hóa một chuỗi biểu thị một đối tượng JSON thành một đối tượng Python:

```
import json  
serialized = """{ "title": "Data Science Book",  
  "author": "Joel Grus",  
  "publicationYear": "2019",  
  "topic": ["data", "science", "data science"]} """  
#Phân tích JSON để tạo một dict Python  
deserialized = json.loads(serialized)  
assert deserialized["publicationYear"] == 2019  
assert "data science" in deserialized["topics"]
```

Đôi khi nhà cung cấp API chỉ cung cấp kết quả dạng XML như sau:

```
<Book>  
<Title>Data Science Book </Title>  
<Author>Joe Grus</Author>  
<PublicationYear>2019</PublicationYear>  
<Topics>  
<Topic>data</Topic>  
<Topic>science</Topic>  
<Topic>data science</Topic>  
</Topics>  
</Book>
```

Khi đó có thể sử dụng Beautiful Soup để lấy dữ liệu từ XML tương tự như cách lấy dữ liệu từ HTML.

Sử dụng API chưa được xác thực

Hầu hết các API hiện nay yêu cầu phải tự xác thực trước khi có thể sử dụng. Theo đó để đơn giản, xem xét API của GitHub, ta có thể thực hiện một số thao tác mà không cần xác thực:

```
import requests, json

github_user = "joelgrus"
endpoint = f"https://api.github.com/users/{github_user}/repos"

repos = json.loads(requests.get(endpoint).text)
```

Giá trị của *repos* trả về là một danh sách các *dicts* Python, mỗi phần tử biểu diễn cho một kho lưu trữ tệp - *repository* trong tài khoản GitHub của người dùng (cần thay thế tên người dùng và lấy dữ liệu từ *repository* từ GitHub tương ứng). Dựa vào đó, có thể tìm được tháng nào và ngày nào trong tuần mà nhiều *repository* được tạo ra nhất. Vấn đề ở đây là ngày tháng trong kết quả là dạng chuỗi như sau:

```
"created_at": "2013-07-05T02:02:28Z"
```

Hàm mặc định của Python không có trình phân tích cú pháp thực sự tốt, vì vậy cần cài đặt một trình phân tích cú pháp *python-dateutil*:

```
python -m pip install python-dateutil
```

Sau đó, ta chỉ cần sử dụng hàm *dateutil.parser.parse* như sau:

```
from collections import Counter
from dateutil.parser import parse

dates = [parse(repo["created_at"]) for repo in repos]
month_counts = Counter(date.month for date in dates)
weekday_counts = Counter(date.weekday() for date in dates)
```

Tương tự, dễ dàng lấy được các ngôn ngữ lập trình được sử dụng của năm *repository* gần nhất:

```
last_5_repositories = sorted(repos,
key=lambda r: r["pushed_at"],
reverse=True)[:5]
last_5_languages = [repo["language"]
for repo in last_5_repositories]
```

Tìm APIs Nếu cần dữ liệu từ một trang web cụ thể, hãy tìm phần “*developers*” hoặc “*API*” của trang web để biết chi tiết và thử tìm kiếm “*python <sitename> api*” trên

web để tìm thư viện. Trên thực tế, đều có một số thư viện cho một số nền tảng như Yelp API, Instagram API, Spotify API, v.v. Danh sách các API được Python đóng gói, thì có một danh sách hay từ *Real Python* trên GitHub ².

Ví dụ: Sử dụng Twitter APIs

Twitter là một nguồn dữ liệu tốt để bắt đầu thử nghiệm vì nó cho phép nhận dữ liệu tin tức theo thời gian thực và có thể sử dụng để đo phản ứng của người dùng Twitter với các sự kiện hiện tại hoặc tìm các liên kết liên quan đến một chủ đề cụ thể. Và đặc biệt có thể truy cập vào dữ liệu thông qua các API của Twitter. Để tương tác với Twitter API cần sử dụng thư viện *Twython* (`python -m pip install twython`). Có khá nhiều thư viện Python Twitter khác, nhưng đây là thư viện dễ sử dụng và khuyến khích cho người mới bắt đầu.

Lấy thông tin xác thực Để sử dụng API của Twitter, cần có thông tin đăng nhập như tài khoản Twitter. Dưới đây là các bước thực hiện:

1. Đi đến đường dẫn <https://developer.twitter.com/>.
2. Đăng nhập tài khoản Twitter
3. Trong trang <https://developer.twitter.com/>, nhấp vào *Apply* để đăng ký tài khoản nhà phát triển.
4. Yêu cầu quyền truy cập cho mục đích sử dụng cá nhân.
5. Điền vào đơn đăng ký, yêu cầu 300 từ về lý do tại sao bạn cần quyền truy cập.
6. Đợi một khoảng thời gian để cấp quyền sử dụng
7. Sau khi được phê duyệt, hãy quay lại developer.twitter.com, tìm phần *Apps - Ứng dụng* và nhấp vào “Create an app” tạo ứng dụng.
8. Điền hết các trường bắt buộc và nhấn *CREATE*.

Sau khi được chấp nhận, một Key và token với phần “*Consumer API keys*” - Khóa API dành cho người dùng được cung cấp bao gồm một “*API key*” và “*API secret key*”.

Chú ý 2.2 Không chia sẻ khóa, một giải pháp đơn giản là lưu trữ các khóa này trong tệp *credentials.json* và sử dụng *json.loads* để truy xuất chúng. Một giải pháp khác là lưu trữ chúng trong các biến môi trường và sử dụng *os.environ* để truy xuất.

Sử dụng Twython

Phần khó nhất khi sử dụng API Twitter là xác thực. Nhà cung cấp API muốn đảm bảo được phép truy cập dữ liệu của Twitter và không vượt quá giới hạn sử dụng.

²<https://github.com/realpython/list-of-python-api-wrappers>

Với một *API key* và *API secret key* (đôi khi được gọi là *consumer key* và *consumer secret*) có thể bắt đầu khai báo sử dụng Twitter API như sau:

```
import os
#Đien key va secret tuong ung
CONSUMER_KEY = os.environ.get("TWITTER_CONSUMER_KEY")
CONSUMER_SECRET = os.environ.get("TWITTER_CONSUMER_SECRET")
```

Sau đó, khởi tạo ứng dụng từ Twitter API:

```
import webbrowser
from twython import Twython

#Nhan mot client tam thoi de nhan xac thuc URL
temp_client = Twython(CONSUMER_KEY, CONSUMER_SECRET)
temp_creds = temp_client.get_authentication_tokens()
url = temp_creds['auth_url']

# Truy cap mot UR de cap quyen cho ung dung va nhan ma PIN
print(f"Di toi {url} nhan ma PIN va nhap ma duoi day")
webbrowser.open(url)
PIN_CODE = input("Vui long nhap ma PIN: ")

#Su dung PIN_CODE de nhan token thuc te
auth_client = Twython(CONSUMER_KEY,
CONSUMER_SECRET,
temp_creds['oauth_token'],
temp_creds['oauth_token_secret'])
final_step = auth_client.get_authorized_tokens(PIN_CODE)
ACCESS_TOKEN = final_step['oauth_token']
ACCESS_TOKEN_SECRET = final_step['oauth_token_secret']

twitter = Twython(CONSUMER_KEY,
CONSUMER_SECRET,
ACCESS_TOKEN,
ACCESS_TOKEN_SECRET)
```

Lưu ý, có thể cần nhắc lưu *ACCESS_TOKEN* và *ACCESS_TOKEN_SECRET* ở một nơi an toàn nào đó để lần sau không phải trải qua quy trình này.

```
# Tim kiem cac tweet co cum tu "data science"
for status in twitter.search(q='data science')['statuses']:
user = status["user"]["screen_name"]
text = status["text"]
print(f"{user}: {text}\n")
```

Sau khi chạy, một số tweet sẽ được trả về như:

haithemnyc: Data scientists with the technical savvy & analytical chops to derive meaning from big data are in demand. <http://t.co/HsF9Q0dShP>

RPubsRecent: Data Science <http://t.co/6hcHUz2PHM>

spleonard1: Using #dplyr in #R to work through a procrastinated assignment for @rdpeng in @coursera data science specialization. So easy and Awesome.

Phần lớn Twitter Search API chỉ hiển thị cho bất kỳ kết quả nào gần đây, thường ta muốn có nhiều tweet hơn. Khi đó sử dụng *Streaming API* sẽ hữu ích hơn. Để truy cập *AStreaming API* bằng *Twython*, cần định nghĩa một lớp kế thừa từ *TwythonStreamer* và ghi đè phương thức *on_success* của nó và có thể là phương thức *on_error*:

```
from twython import TwythonStreamer

tweets = []

class MyStreamer(TwythonStreamer):
    def on_success(self, data):
        """
        What do we do when Twitter sends us data?
        Here data will be a Python dict representing a tweet.
        """
        # Khi chi muon thu thap du lieu cac tweet tieng Anh
        if data.get('lang') == 'en':
            tweets.append(data)
            print(f"received tweet #{len(tweets)}")
            # Dung thu thap den khi thu thap du
            if len(tweets) >= 100:
                self.disconnect()
        def on_error(self, status_code, data):
            print(status_code, data)
            self.disconnect()
```

MyStreamer sẽ kết nối với luồng Twitter và đợi Twitter cung cấp dữ liệu cho luồng đó. Mỗi khi nó nhận được một số dữ liệu (một tweet được biểu thị dưới dạng một đối tượng trong Python), nó sẽ chuyển dữ liệu đó tới phương thức *on_success*, phương thức này sẽ thêm vào danh sách tweet nếu ngôn ngữ của tweet là tiếng Anh, sau đó ngắt kết nối với streamer sau khi nó đã thu thập được 1.000 tweet. Tất cả những gì còn lại là khởi tạo và bắt đầu chạy:

```
stream = MyStreamer(CONSUMER_KEY, CONSUMER_SECRET,
                    ACCESS_TOKEN, ACCESS_TOKEN_SECRET)
# Bat dau su dung cac tweet cong khai co chua tu khoa "data"
stream.statuses.filter(track='data')
```

Khi thu thập được 100 tweet (hoặc cho đến khi nó gặp lỗi) chương trình sẽ dừng lại, lúc đó có thể bắt đầu phân tích các tweet đó. Chẳng hạn, tìm thấy các thẻ - hashtag bắt đầu bằng #:

```
top_hashtags = Counter(hashtag['text'].lower()
for tweet in tweets
for hashtag in tweet["entities"]["hashtags"])
print(top_hashtags.most_common(5))
```

Mỗi tweet chứa rất nhiều dữ liệu để phát triển các ứng dụng khác có thể tìm hiểu tài liệu về API Twitter ³.

Chú ý 2.3 Việc thu thập dữ liệu cần lưu trữ lâu dài thay vì chỉ lưu trong khi chạy tại biến *list*, do đó, cần lưu trữ các dữ liệu này thành file hoặc một cơ sở dữ liệu nào đó để có thể sử dụng dữ liệu vĩnh viễn.

Có rất nhiều các kỹ thuật khác nhau để sử dụng cho việc thu thập dữ liệu và với mỗi một trang web, dựa vào chính sách của nó cần áp dụng linh hoạt các phương pháp khác nhau cho phù hợp. Để phát triển thêm kỹ năng về thu thập dữ liệu ta nên tìm hiểu một số kỹ thuật trong Web Scraping với Python như sử dụng Scrapy, Splash , Selenium... Từ đó, vận dụng vào các dự án thực tiễn.

2.2 Làm sạch dữ liệu

Việc làm sạch dữ liệu rất quan trọng bởi vì quá trình thu thập dữ liệu thường có sai sót. Có một số nguyên nhân gây ra thiếu giá trị hoặc sai số trong quá trình thu thập dữ liệu. Một số ví dụ như sau:

- Một số công nghệ thu thập dữ liệu, chẳng hạn như bằng cảm biến, vốn đã không chính xác do những hạn chế về phần cứng liên quan đến việc thu thập và truyền tải. Đôi khi các cảm biến có thể bị giảm số lần quét do lỗi phần cứng hoặc hết pin.
- Dữ liệu được thu thập bằng công nghệ quét có thể có sai sót do kỹ thuật nhận dạng ký tự quang học chưa bao giờ hoàn hảo. Hơn nữa, kỹ thuật chuyển lời nói thành văn bản cũng dễ bị lỗi.
- Người dùng thường không muốn cung cấp chi tiết thông tin cá nhân của họ vì lý do riêng tư hoặc họ cố ý tình đưa ra thông tin không chính xác. Ví dụ: đôi khi người dùng cung cấp ngày sinh giả trên các trang đăng ký tự động, chẳng hạn như các trang mạng xã hội. Trong một số trường hợp, người dùng có thể

³<https://developer.twitter.com/en/docs/twitter-api/v1/data-dictionary/object-model/tweet>

chọn để trồng một số trường dữ liệu.

- Một lượng lớn dữ liệu được tạo thủ công. Lỗi thủ công rất phổ biến trong quá trình nhập dữ liệu.
- Đơn vị phụ trách thu thập dữ liệu có thể bỏ qua một số thông tin trong hồ sơ nếu việc đó quá tốn kém. Do đó, hồ sơ có thể không đầy đủ.

Các vấn đề đã nói ở trên đều có thể là một nguồn cơn khiến các ứng dụng khai thác dữ liệu thiếu tính chính xác. Do đó, cần thiết phải có các phương pháp để loại bỏ hoặc sửa các giá trị nhập bị thiếu và sai sót khỏi dữ liệu. Có một số khía cạnh quan trọng của việc làm sạch dữ liệu:

1. *Xử lý các giá trị bị thiếu*: Nhiều giá trong dữ liệu có thể không được xác định do yếu tố trong quá trình thu thập dữ liệu hoặc bản chất vốn có của dữ liệu. Các giá trị bị thiếu như vậy có thể cần phải được ước tính. Quá trình ước tính các giá trị bị thiếu cũng được gọi là *quy ước* (imputation).
2. *Xử lý giá trị sai*: Trong trường hợp cùng một thông tin đến từ nhiều nguồn thì có thể phát hiện sự không thống nhất. Việc loại bỏ những mâu thuẫn như vậy là một phần của quá trình phân tích. Một phương pháp khác để phát hiện các giá trị sai là sử dụng kiến thức chuyên ngành đã biết về dữ liệu. Ví dụ: nếu chiều cao của một người được liệt kê là 6 m, rất có thể điều đó là không chính xác. Tổng quát hơn, các điểm dữ liệu không tương thích với phân phối của dữ liệu còn lại thường bị nhiễu. Các điểm dữ liệu như vậy được coi là *ngoại lệ*. Tuy nhiên, thật nguy hiểm khi cho rằng các điểm dữ liệu đó luôn do lỗi gây ra. Ví dụ: một bản ghi về gian lận thẻ tín dụng có khả năng không thống nhất đối với các mẫu trong hầu hết dữ liệu (thông thường) nhưng ta không nên xóa nó với lý do dữ liệu “sai”.

2.2.1 Xử lý giá trị bị thiếu

Thiếu giá trị là hiện tượng phổ biến trong cơ sở dữ liệu khi các phương pháp thu thập dữ liệu thường không hoàn hảo. Ví dụ: các cuộc khảo sát người dùng thường không thể thu thập được đầy đủ câu trả lời cho tất cả các câu hỏi. Trong các cuộc khảo sát tự nguyện, các câu hỏi đưa ra hầu như luôn bị trả lời thiếu. Ba kỹ thuật để xử lý các giá trị bị thiếu là:

1. Loại bỏ hoàn toàn các bản ghi dữ liệu nào bị thiếu. Tuy nhiên, cách tiếp cận này có thể không thực tế khi hầu hết các bản ghi chứa các mục bị thiếu.
2. Ước tính hoặc quy ước các giá trị còn thiếu. Tuy nhiên, sai số của việc quy ước có thể ảnh hưởng đến kết quả của thuật toán khai thác dữ liệu.

3. Thiết kế các nhóm phân tích sao cho nó có thể hoạt động với các giá trị còn thiếu. Nhiều phương pháp khai thác dữ liệu vốn được thiết kế để hoạt động hiệu quả với các giá trị bị thiếu. Đây là cách tiếp cận ưa thích nhất vì nó tránh được những sai lệch phát sinh vốn có trong quá trình quy ước.

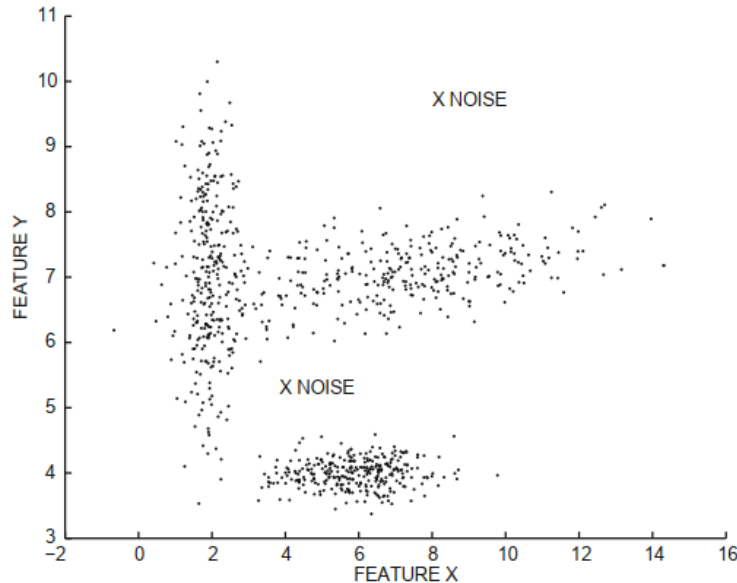
Trong trường hợp dữ liệu phụ thuộc có định hướng, chẳng hạn như dữ liệu theo chuỗi thời gian hoặc không gian, việc ước tính giá trị bị thiếu sẽ đơn giản hơn nhiều. Trong trường hợp này, ta sử dụng các giá trị hành vi của các bản ghi lân cận theo ngữ cảnh cho việc quy ước. Ví dụ: trong tập dữ liệu chuỗi thời gian, ta có thể dùng giá trị trung bình của các giá trị tại đầu thời gian ngay trước hoặc sau giá trị bị thiếu để ước tính. Nếu cần, ta có thể sử dụng các giá trị hành vi của n điểm dữ liệu đầu thời gian cuối cùng để nội suy tuyến tính ra giá trị còn thiếu. Đối với trường hợp dữ liệu theo không gian, quy trình ước tính hoàn toàn tương tự, khi đó ta sử dụng giá trị trung bình của các giá trị tại các vị trí không gian lân cận.

2.2.2 Xử lý giá trị sai hoặc không nhất quán

Một số phương pháp chính được sử dụng để xử lý các giá trị sai hoặc không nhất quán như sau:

1. *Phát hiện sự không nhất quán*: Điều này thường được thực hiện khi dữ liệu có sẵn từ các nguồn khác nhau ở các định dạng khác nhau. Ví dụ: tên của một người có thể được viết đầy đủ trong một nguồn, trong khi nguồn kia chỉ có thể chứa tên viết tắt và họ. Trong những trường hợp như vậy, phát hiện trùng lặp và phát hiện sự không nhất quán là điều quan trọng. Kỹ thuật này được nghiên cứu trong chủ đề tích hợp dữ liệu trong lĩnh vực cơ sở dữ liệu.
2. *Kiến thức miền*: Một lượng kiến thức miền rất lớn luôn có sẵn dưới dạng phạm vi của các thuộc tính hoặc quy tắc xác định mối quan hệ giữa các thuộc tính khác nhau. Ví dụ: nếu trường quốc gia là “Hoa Kỳ” thì trường thành phố không được là “Shanghai”. Nhiều công cụ *làm sạch dữ liệu* và *kiểm tra dữ liệu* đã được phát triển sử dụng kiến thức miền và các ràng buộc đó để phát hiện các giá trị nhập không chính xác.
3. *Phương pháp tập trung vào dữ liệu*: Sử dụng *hành vi thống kê* của dữ liệu để phát hiện các giá trị ngoại lai. Ví dụ: hai điểm dữ liệu bị cô lập trong Hình 2.1 được đánh dấu là “nhiều” là các giá trị ngoại lệ. Những điểm cô lập này có thể đã phát sinh do sai sót trong quá trình thu thập dữ liệu. Tuy nhiên, điều này có thể không phải lúc nào cũng đúng vì sự bất thường có thể là kết quả của hành vi thú vị của hệ đang khảo sát. Do đó, khi phát hiện bất kỳ ngoại lệ nào,

cần phải được kiểm tra thủ công trước khi bị loại bỏ. Việc sử dụng các phương pháp tập trung vào dữ liệu để làm sạch đôi khi có thể nguy hiểm vì chúng có thể dẫn đến việc loại bỏ kiến thức hữu ích khỏi hệ đang khảo sát.



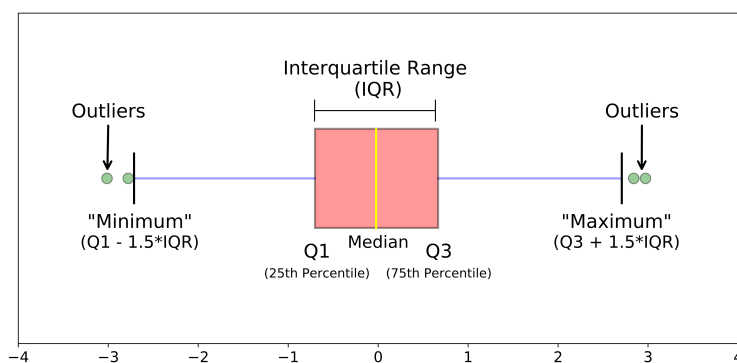
Hình 2.1: Tìm nhiễu bằng phương pháp tập trung vào dữ liệu

Dưới đây giới thiệu hai cách xác định giá trị ngoại lai bằng việc sử dụng biểu đồ hộp; sử dụng giá trị trung bình và độ lệch tiêu chuẩn.

Sử dụng biểu đồ hộp phát hiện giá trị ngoại lai

- Biểu đồ hộp mô tả một số đặc trưng quan trọng của tập dữ liệu như tâm, độ phân tán, mức độ đối xứng và cũng là một cách để phát hiện các quan sát bất thường. Biểu đồ hộp cho biết 3 điểm tứ phân vị Q_1 , Q_2 , Q_3 , min, max trên một hộp chữ nhật.
- Một cạnh của hình chữ nhật nằm tại vị trí tứ phân vị thứ nhất Q_1 , cạnh đối diện ở vị trí của điểm tứ phân vị thứ 3, Q_3 , khoảng tứ phân vị $IQR = Q_3 - Q_1$.
- Từ điểm Q_1 ta vẽ đoạn thẳng theo hướng đi ra hướng giá trị nhỏ nhất của dữ liệu với độ dài là $1,5 * IQR$ và từ điểm Q_3 vẽ đoạn thẳng đi ra hướng giá trị lớn nhất của dữ liệu với độ dài là $1,5 * IQR$ (các đoạn thẳng này được gọi là "đuôi dưới" và "đuôi trên"). Các quan sát nằm ngoài hình chữ nhật và hai đuôi này được biểu diễn bằng các ký hiệu "o" đơn lẻ. Các điểm nằm ngoài hai đuôi được xem là các giá trị ngoại lai (Hình 2.2).

Như vậy, để xác định một dãy dữ liệu có giá trị ngoại lai hay không chúng ta chỉ cần xác định có quan sát nào nằm ngoài khoảng $(Q_1 - 1,5 * IQR; Q_3 + 1,5 * IQR)$?



Hình 2.2: Biểu đồ hộp.

Sử dụng $\bar{x}$ và s phát hiện giá trị ngoại lai

Nếu dữ liệu tuân theo phân bố chuẩn thì khoảng 95% số quan sát nằm trong khoảng $\bar{x} - 2s, \bar{x} + 2s$ và 99,74% số quan sát sẽ nằm trong khoảng $(\bar{x} - 3s, \bar{x} + 3s)$, trong đó $\bar{x}$ là trung bình mẫu và s là độ lệch chuẩn mẫu. Các quan sát có giá trị không nằm trong khoảng $(\bar{x} - 2s, \bar{x} + 2s)$ là quan sát bất thường. Như vậy để tìm quan sát bất thường theo phương pháp này ta thực hiện hai bước sau:

- Tính trung bình mẫu $\bar{x}$ và độ lệch chuẩn mẫu s .
- Tìm khoảng $(\bar{x} - 2s, \bar{x} + 2s)$ và xác định bất thường.

Một số tài liệu coi quan sát bất thường là quan sát không thuộc khoảng $(\bar{x} - 3s, \bar{x} + 3s)$. Việc sử dụng khoảng $(\bar{x} - 2s, \bar{x} + 2s)$ (khoảng có độ rộng $4s$) hay khoảng $(\bar{x} - 3s, \bar{x} + 3s)$ (khoảng có độ rộng $6s$) tùy thuộc vào người dùng và tập dữ liệu.

2.3 Co giãn và chuẩn hóa dữ liệu

Trong nhiều tình huống, các thuộc tính khác nhau của dữ liệu sử dụng các thang co giãn khác nhau và do đó không thể so sánh các thuộc tính với nhau được. Ví dụ: một thuộc tính như tuổi tác được vẽ theo tỷ lệ rất khác so với một thuộc tính như tiền lương. Thuộc tính tiền lương thường có độ lớn lớn hơn thuộc tính tuổi tác. Kết quả là, bất kỳ hàm tổng hợp nào được tính toán trên các thuộc tính khác nhau (ví dụ: khoảng cách Euclide) sẽ bị chi phối bởi thuộc tính có độ lớn lớn hơn.

Để giải quyết vấn đề này, người ta thường sử dụng phép *chuẩn hóa dữ liệu*. Giả sử thuộc tính thứ j có kỳ vọng μ_j và độ lệch chuẩn σ_j . Khi đó, giá trị x_i^j của thuộc tính thứ j trong bản ghi $\bar{X}_i$ có thể được chuẩn hóa như sau:

$$z_i^j = \frac{x_i^j - \mu_j}{\sigma_j}. \quad (2.1)$$

Phần lớn các giá trị được chuẩn hóa sẽ nằm trong đoạn $[-3, 3]$ khi nó tuân theo phân phối chuẩn.

Cách tiếp cận thứ hai sử dụng *phép co giãn cực đại–cực tiểu* để ánh xạ tất cả các thuộc tính vào đoạn $[0, 1]$. Đặt $\min_j$ và $\max_j$ tương ứng là giá trị nhỏ nhất, giá trị lớn nhất của thuộc tính j . Khi đó, giá trị x_i^j của thuộc tính thứ j trong bản ghi $\bar{X}_i$ được co giãn như sau:

$$y_i^j = \frac{x_i^j - \min_j}{\max_j - \min_j}. \quad (2.2)$$

Cách tiếp cận này không hiệu quả khi các giá trị cực đại và cực tiểu là các giá trị ngoại lệ do một số lỗi trong quá trình thu thập dữ liệu. Ví dụ: giả sử thuộc tính tuổi bị lỗi trong quá trình thu thập dữ liệu làm thêm một số 0 vào một tuổi, dẫn đến giá trị tuổi là 800 năm thay vì 80. Trong trường hợp này, hầu hết dữ liệu được co giãn theo thuộc tính tuổi sẽ nằm trong đoạn $[0, 0.1]$, do đó thuộc tính này có thể không được nhấn mạnh. Với các tình huống như này nên dùng phép chuẩn hoá dữ liệu.

2.4 Giảm chiều và biến đổi dữ liệu

Mục tiêu của việc giảm chiều dữ liệu là để biểu diễn nó gọn hơn. Khi kích thước dữ liệu nhỏ hơn, việc áp dụng các thuật toán phức tạp và tốn kém về mặt tính toán sẽ dễ dàng hơn nhiều. Việc giảm chiều dữ liệu có thể là giảm về số lượng hàng (số bản ghi) hoặc về số lượng cột (số chiều). Giảm chiều dữ liệu dẫn đến mất mát thông tin. Đôi khi có thể sử dụng một thuật toán phức tạp hơn để bù đắp cho việc mất thông tin do giảm chiều dữ liệu. Nhiều kiểu giảm chiều dữ liệu khác nhau được sử dụng trong các ứng dụng khác nhau:

1. *Lấy mẫu dữ liệu*: Từ các bản ghi dữ liệu ban đầu, ta lấy mẫu để tạo cơ sở dữ liệu mới nhỏ hơn. Trong trường hợp phát trực tuyến thì việc lấy mẫu sẽ khó hơn bởi vì mẫu cần đảm bảo tính duy trì động.
2. *Lựa chọn đặc trưng*: Ta sử dụng một tập hợp con các đặc trưng từ dữ liệu ban đầu trong quá trình phân tích. Thông thường, các tập hợp con này được chọn theo cách riêng cho từng ứng dụng. Ví dụ: một phương pháp lựa chọn đặc trưng hoạt động tốt khi phân cụm nhưng có thể không hoạt động tốt để phân loại và ngược lại. Do đó, mục này chỉ thảo luận vấn đề lựa chọn tập hợp đặc trưng con theo một cách hạn chế và sẽ thảo luận chi tiết hơn ở các chương sau.
3. *Giảm chiều dữ liệu*: Các mối tương quan trong dữ liệu được tận dụng để thể hiện dữ liệu đó trong số chiều nhỏ hơn.
4. *Biến đổi dữ liệu*: Hình thức giảm dữ liệu này liên quan chặt chẽ đến tính di

động của kiểu dữ liệu. Ví dụ: chuỗi thời gian được chuyển đổi thành dữ liệu đa chiều có kích thước nhỏ hơn và độ phức tạp nhỏ hơn bằng các phép biến đổi sóng nhỏ rời rạc. Tương tự như vậy, ta có thể chuyển đổi các biểu đồ thành các biểu diễn đa chiều bằng cách sử dụng các kỹ thuật nhúng.

2.4.1 Lấy mẫu

Ưu điểm chính của lấy mẫu là nó đơn giản, trực quan và tương đối dễ thực hiện. Có thể thay đổi kiểu lấy mẫu tùy theo ứng dụng hiện tại.

Lấy mẫu cho dữ liệu tĩnh

Việc lấy mẫu dữ liệu sẽ đơn giản hơn nhiều khi toàn bộ dữ liệu đã có sẵn và khi đó ta đã biết trước số lượng điểm dữ liệu cơ sở. Trong phương pháp lấy mẫu khách quan, một phần f định trước của các điểm dữ liệu được chọn và giữ lại để phân tích. Cách thực hiện này cực kỳ đơn giản và có thể đạt được theo hai cách khác nhau: lấy mẫu có hoàn lại và không hoàn lại.

Khi lấy mẫu không hoàn lại từ tập dữ liệu $\mathcal{D}$ có n bản ghi, ta chọn ngẫu nhiên $\lceil n \cdot f \rceil$ bản ghi từ dữ liệu. Khi đó, trong mẫu không có hai bản ghi trùng nhau, trừ khi tập dữ liệu gốc $\mathcal{D}$ cũng chứa các bản ghi trùng nhau. Khi lấy mẫu có hoàn lại từ tập dữ liệu $\mathcal{D}$ có n bản ghi, các bản ghi được lấy mẫu tuần tự và độc lập với nhau toàn bộ tập dữ liệu $\mathcal{D}$ tổng cộng $\lceil n \cdot f \rceil$ lần. Do đó có thể xảy ra trường hợp hai bản ghi trùng nhau vì cùng một bản ghi có thể được đưa vào mẫu qua các lựa chọn tuần tự. Nói chung, hầu hết các ứng dụng không sử dụng cách chọn mẫu có hoàn lại vì các bản ghi trùng nhau có thể gây phiền toái cho một số ứng dụng khai thác dữ liệu, chẳng hạn như phát hiện ngoại lệ. Một số hình thức lấy mẫu chuyên dụng khác như sau:

1. *Lấy mẫu chệch*: Ta thực hiện lấy mẫu chệch khi một số phần của dữ liệu được nhấn mạnh có chủ ý vì chúng có tầm quan trọng lớn hơn đối với phân tích. Một ví dụ thông thường là lấy mẫu chệch phân rã theo thời gian, khi đó các bản ghi gần đây hơn có cơ hội được đưa vào mẫu lớn hơn và các bản ghi cũ có ít cơ hội được đưa vào mẫu hơn. Trong lấy mẫu chệch phân rã theo cấp số nhân, giả sử bản ghi $\bar{X}$ được ghi từ δt đơn vị thời gian trước, xác suất $p(\bar{X})$ lấy mẫu được $\bar{X}$ tỷ lệ với hàm phân rã theo cấp số nhân được điều chỉnh bởi tham số phân rã λ :

$$p(\bar{X}) \propto e^{-\lambda \cdot \delta t} \quad (2.3)$$

Ở đây e là cơ số của logarit tự nhiên. Bằng cách sử dụng các giá trị khác nhau

của λ , ta có thể điều chỉnh tác động của sự phân rã theo thời gian một cách thích hợp.

2. *Lấy mẫu phân tầng*: Trong một số bộ dữ liệu, do độ hiếm của chúng, các phần quan trọng của dữ liệu có thể không được thể hiện đầy đủ bằng cách lấy mẫu. Khi đó, ta thực hiện lấy mẫu phân tầng bằng cách trước tiên phân vùng dữ liệu thành một tập hợp các tầng mong muốn, sau đó lấy mẫu độc lập từ mỗi tầng này dựa trên các tỷ lệ định trước theo cách dành riêng cho ứng dụng.

Ví dụ, xét một cuộc khảo sát sự đa dạng về kinh tế trong lối sống của các cá nhân khác nhau trong một cộng đồng dân cư. Ngay cả một mẫu gồm 1 triệu người tham gia cũng có thể để lọt một người tỷ phú vì độ hiếm tương đối của những người là tỷ phú. Tuy nhiên, một mẫu phân tầng (theo thu nhập) sẽ lấy mẫu độc lập phần trăm người tham gia định trước từ mỗi nhóm thu nhập để đảm bảo phân tích vững chắc hơn.

Có thể có nhiều hình thức lấy mẫu chệch khác. Ví dụ: trong lấy mẫu thiên về mật độ, các điểm trong vùng có mật độ cao hơn được tính trọng số thấp hơn để đảm bảo tính đại diện cao hơn cho các vùng hiếm trong mẫu.

Hồ chứa mẫu cho luồng dữ liệu

Một hình thức lấy mẫu đặc biệt thú vị là *hồ chứa mẫu* (reservoir sampling) cho luồng dữ liệu. Trong hồ chứa mẫu, ta luôn duy trì *động* một mẫu gồm k điểm từ luồng dữ liệu. Nhắc lại rằng một luồng dữ liệu có dung lượng cực lớn và do đó người ta không thể lưu trữ nó trên ổ cứng để lấy mẫu. Do đó, đối với mỗi điểm dữ liệu đến trong luồng dữ liệu, người ta phải sử dụng một tập các hoạt động có thể triển khai hiệu quả để *duy trì* mẫu.

Đối với dữ liệu tĩnh có lực lượng là n , xác suất đưa một điểm dữ liệu vào trong mẫu cỡ k là k/n . Trong trường hợp luồng dữ liệu, “bộ dữ liệu” không tĩnh và không thể lưu trữ trên ổ cứng. Hơn nữa, giá trị của n không ngừng tăng lên khi có nhiều điểm mới và các điểm dữ liệu trước đó (bên ngoài mẫu) đã bị loại bỏ. Do đó, phương pháp lấy mẫu phải làm việc với kiến thức không đầy đủ về lịch sử trước đây của luồng tại bất kỳ thời điểm nào. Nói cách khác, với mỗi điểm dữ liệu mới xuất hiện trong luồng cần *liên tục* đưa ra hai quyết định để *kiểm soát đầu vào*:

1. Nên sử dụng quy tắc lấy mẫu nào để quyết định có đưa điểm dữ liệu mới vào mẫu hay không?
2. Nên sử dụng quy tắc nào để quyết định cách đẩy một điểm dữ liệu ra khỏi mẫu để “nhường chỗ” cho điểm dữ liệu mới được chèn vào?

May mắn thay, việc thiết kế một thuật toán hồ chứa mẫu cho luồng dữ liệu là tương đối đơn giản. Đối với một hồ chứa mẫu có kích thước k , ta khởi tạo hồ chứa bằng k điểm dữ liệu đầu tiên của luồng dữ liệu. Sau đó, với điểm dữ liệu thứ n của luồng, ta áp dụng hai quyết định kiểm soát đầu vào như sau:

1. Chèn điểm thứ n vào hồ chứa mẫu với xác suất k/n .
2. Nếu điểm thứ n được chèn vào thì ta loại bỏ một điểm cũ theo cách ngẫu nhiên để nhường chỗ cho điểm mới.

Có thể chỉ ra rằng quy tắc đã nói ở trên duy trì một hồ chứa mẫu không chệch từ luồng dữ liệu.

Bổ đề 2.1 Sau khi có n điểm đến từ luồng dữ liệu, xác suất được đưa vào hồ chứa mẫu của bất kỳ điểm luồng nào là như nhau và bằng k/n .

Chứng minh. Ta chứng minh bằng phương pháp quy nạp. Khi khởi tạo k điểm dữ liệu đầu tiên, định lý hiển nhiên đúng vì xác suất đưa vào hồ chứa mẫu của các điểm đều bằng 1. Giả sử bổ đề đúng sau khi nhận $(n-1)$ điểm dữ liệu từ luồng và do đó xác suất của mỗi điểm được đưa vào hồ chứa mẫu là $k/(n-1)$. Xác suất của điểm mới từ luồng được đưa vào hồ chứa mẫu là k/n , và do đó bổ đề đúng với điểm dữ liệu mới. Ta còn phải chứng minh kết quả cho các điểm còn lại trong luồng dữ liệu. Có hai sự kiện *xung khắc* có thể xảy ra đối với một điểm dữ liệu mới đến và xác suất cuối cùng của một điểm được đưa vào hồ chứa mẫu là tổng xác suất của hai sự kiện này:

- I:** Điểm dữ liệu mới không được đưa vào hồ chứa. Xác suất của sự kiện này là $(n-k)/n$. Theo giả thiết quy nạp, xác suất của một điểm bất kỳ được đưa vào hồ chứa mẫu là $k/(n-1)$, nên xác suất của một điểm được đưa vào hồ chứa mẫu và xảy ra sự kiện I là $p_1 = \frac{k(n-k)}{n(n-1)}$.
- II:** Điểm dữ liệu mới được đưa vào hồ chứa mẫu. Xác suất xảy ra sự kiện II bằng xác suất chèn một điểm mới là k/n . Do đó, các điểm trong hồ chứa hiện tại được giữ lại với xác suất $(k-1)/k$ vì có đúng một điểm trong số chúng bị loại. Theo giả thiết quy nạp, bất kỳ điểm nào trước đó của luồng dữ liệu đều có xác suất xuất hiện trong hồ chứa bằng $k/(n-1)$, điều này kéo theo xác suất của một điểm được thêm vào hồ chứa và sự kiện II xảy bằng tích của ba xác suất nói trên:

$$p_2 = \frac{k}{n} \cdot \frac{k-1}{k} \cdot \frac{k}{n-1} = \frac{k(k-1)}{n(n-1)}. \quad (2.4)$$

Do đó, xác suất của một điểm luồng sau điểm dữ liệu thứ n được đưa vào hồ chứa

được tính bằng tổng của p_1 và p_2 và bằng k/n . ■

Ta có thể mở rộng phương pháp hồ chứa mẫu cho các trường hợp có chệch thời gian trong luồng dữ liệu. Đặc biệt, trường hợp chệch theo cấp số nhân đã được giải quyết trong [2].

2.4.2 Lựa chọn đặc trưng

Kỹ thuật thứ hai để tiền xử lý dữ liệu là lựa chọn đặc trưng. Một số đặc trưng có thể bị loại bỏ khi chúng không liên quan. Những đặc trưng có liên quan là những đặc trưng nào? Rõ ràng, quyết định này phụ thuộc vào ứng dụng hiện có. Có hai cách lựa chọn đặc trưng chính:

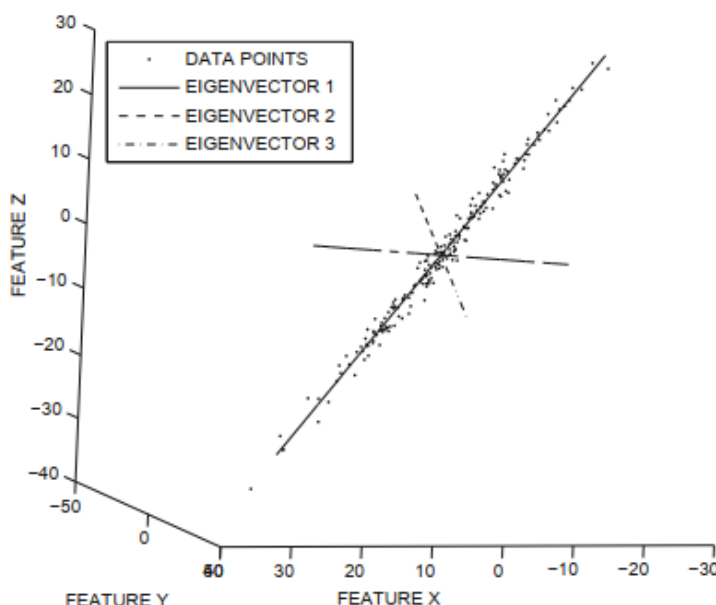
1. *Lựa chọn đặc trưng không giám sát:* Điều này tương ứng với việc loại bỏ các thuộc tính nhiễu và dư thừa khỏi dữ liệu. Phương pháp này nên được áp dụng cho các ứng dụng phân cụm, mặc dù khả năng ứng dụng rộng hơn nhiều. Thật khó để mô tả một cách toàn diện các phương pháp lựa chọn đặc trưng không giám sát mà không sử dụng bài toán phân cụm như một ngữ cảnh thích hợp.
2. *Lựa chọn đặc trưng có giám sát:* Cách lựa chọn đặc trưng này có liên quan đến bài toán phân loại dữ liệu. Trong trường hợp này, những đặc trưng phù hợp nhất là những đặc trưng có thể dự đoán được thuộc tính của lớp một cách hiệu quả. Các phương pháp lựa chọn đặc trưng thường được tích hợp chặt chẽ với các phương pháp phân tích để phân loại.

Lựa chọn đặc trưng là một phần quan trọng của quá trình khai thác dữ liệu vì nó xác định chất lượng của dữ liệu đầu vào.

2.4.3 Giảm chiều dữ liệu

Trong các tập dữ liệu thực, giữa các thuộc tính khác nhau của dữ liệu có mối tương quan đáng kể. Trong một số trường hợp, một số thuộc tính được xác định duy nhất thông qua các ràng buộc hoặc quy tắc cứng với các thuộc tính khác. Ví dụ, ngày sinh của một cá nhân hoàn toàn tương quan với tuổi của người đó. Trong hầu hết các trường hợp, các mối tương quan có thể không hoàn hảo, nhưng vẫn có thể tồn tại những phụ thuộc đáng kể giữa các đặc trưng khác nhau. Các tập dữ liệu thực chứa nhiều phần dư thừa như vậy khiến nhà phân tích không chú ý trong giai đoạn tạo dữ liệu ban đầu. Các mối tương quan và ràng buộc này tương ứng với sự dư thừa ẩn vì thông qua chúng, từ kiến thức đã biết của một số chiều đặc trưng ta có thể dự đoán các giá trị của các chiều đặc trưng khác. Ví dụ, hãy xem xét tập dữ liệu 3 chiều được minh họa trong Hình 2.3. Trong trường hợp này, nếu trục được xoay

theo hướng được minh họa trong hình, các mối tương quan và dư thừa trong các giá trị đặc trưng mới được chuyển đổi sẽ bị loại bỏ. Nhờ phép loại bỏ dư thừa này, toàn bộ dữ liệu có thể được biểu diễn (xấp xỉ) dọc theo một đường thẳng 1 chiều. Do đó, *chiều nội tại* của tập dữ liệu 3 chiều này là 1. Hai trục còn lại tương ứng với các chiều có phương sai thấp. Nếu dữ liệu được biểu diễn dưới dạng tọa độ trong hệ trục mới được minh họa trong Hình 2.3, thì các giá trị tọa độ dọc theo các chiều có phương sai thấp này sẽ không thay đổi nhiều. Do đó, sau khi xoay hệ trục, ta có thể loại bỏ các chiều này mà không làm mất nhiều thông tin.



Hình 2.3: Dữ liệu có độ tương quan cao được biểu thị bằng ba chiều trong một hệ trục xoay thích hợp

Một câu hỏi tự nhiên đặt ra là làm thế nào để xác định hệ trục giúp loại bỏ tương quan như trong Hình 2.3 theo cách tự động. Hai phương pháp tự nhiên để đạt được mục tiêu này là phương pháp *phân tích thành phần chính* (PCA) và *phân tích giá trị kỳ dị* (SVD). Hai phương pháp này, mặc dù không hoàn toàn giống nhau ở cấp độ định nghĩa, nhưng có liên quan chặt chẽ với nhau. Mặc dù phương pháp phân tích thành phần chính trực quan dễ hiểu hơn, nhưng phương pháp SVD là một khung tổng quát hơn và có thể thực thi PCA như một trường hợp riêng.

Phương pháp phân tích thành phần chính

Phương pháp PCA thường được áp dụng sau khi trừ đi giá trị trung bình của tập dữ liệu từ mỗi điểm dữ liệu. Tuy nhiên, ta cũng có thể áp dụng trực tiếp phương pháp này mà không cần định tâm giá trị trung bình trước, miễn là giá trị trung bình được

lưu trữ riêng. Thao tác này được gọi là *căn giữa giá trung bình* và nó giúp tập dữ liệu được căn giữa tại điểm gốc. Mục tiêu của phương pháp PCA là xoay dữ liệu vào một hệ trục trong đó lượng phương sai lớn nhất được ghi lại trong một số lượng nhỏ các chiều. Rõ ràng bằng trực giác từ ví dụ trong Hình 2.3 ta thấy một hệ trục như vậy bị ảnh hưởng bởi mối tương quan giữa các thuộc tính. Một quan sát quan trọng sẽ trình bày dưới đây là biểu diễn phương sai của một tập dữ liệu theo một hướng cụ thể dưới dạng ma trận hiệp phương sai của nó.

Gọi C là ma trận hiệp phương sai đối xứng cỡ $d \times d$ của ma trận dữ liệu D cỡ $n \times d$. Nghĩa là phần tử c_{ij} ở vị trí (i, j) của C là hiệp phương sai giữa cột thứ i và j (các chiều) của ma trận dữ liệu D . Đặt μ_i giá trị trung bình dọc theo chiều thứ i . Cụ thể, nếu x_k^m là chiều thứ m của bản ghi thứ k thì giá trị của phần tử hiệp phương sai c_{ij} như sau:

$$c_{ij} = \frac{\sum_{k=1}^n x_k^i x_k^j}{n} - \mu_i \mu_j \quad \forall i, j \in \{1, 2, \dots, d\}. \quad (2.5)$$

Ký hiệu $\bar{\mu} = (\mu_1, \dots, \mu_d)$ là vectơ d chiều với các thành phần là các giá trị trung bình dọc theo các chiều khác nhau. Khi đó, ta có dạng ma trận của (2.5) như sau

$$C = \frac{D^T D}{n} - \bar{\mu}^T \bar{\mu}. \quad (2.6)$$

Chú ý rằng d phần tử trên đường chéo của C là d phương sai. Ma trận hiệp phương sai C là ma trận nửa xác định dương bởi vì ta có thể chứng minh được với vectơ $\bar{v}$ bất kỳ, giá trị của $\bar{v}^T C \bar{v}$ bằng với phương sai của phép chiếu $D\bar{v}$ của tập dữ liệu D trên $\bar{v}$.

$$\bar{v}^T C \bar{v} = \frac{(D\bar{v})^T D\bar{v}}{n} - (\bar{\mu} \bar{v})^2 = \text{Phương sai của các điểm trong } D\bar{v} \geq 0. \quad (2.7)$$

Trên thực tế, mục tiêu của kĩ thuật PCA là xác định liên tiếp các vectơ trực giao $\bar{v}$ cực đại hóa $\bar{v}^T C \bar{v}$. Làm thế nào để ta có thể xác các vectơ như vậy? Vì ma trận hiệp phương sai có tính đối xứng và nửa xác định dương nên có thể được chéo hóa như sau:

$$C = P \Lambda P^T. \quad (2.8)$$

Các cột của ma trận P chứa các vectơ riêng trực chuẩn của C và Λ là một ma trận đường chéo chứa các giá trị riêng không âm của C . Phần tử Λ_{ii} là giá trị riêng ứng với vectơ riêng thứ i của ma trận P . Các vectơ riêng này là các nghiệm trực giao liên tiếp của bài toán tối ưu hóa đã nói ở trên nhằm cực đại hóa phương sai $\bar{v}^T C \bar{v}$ dọc theo hướng đơn vị $\bar{v}$.

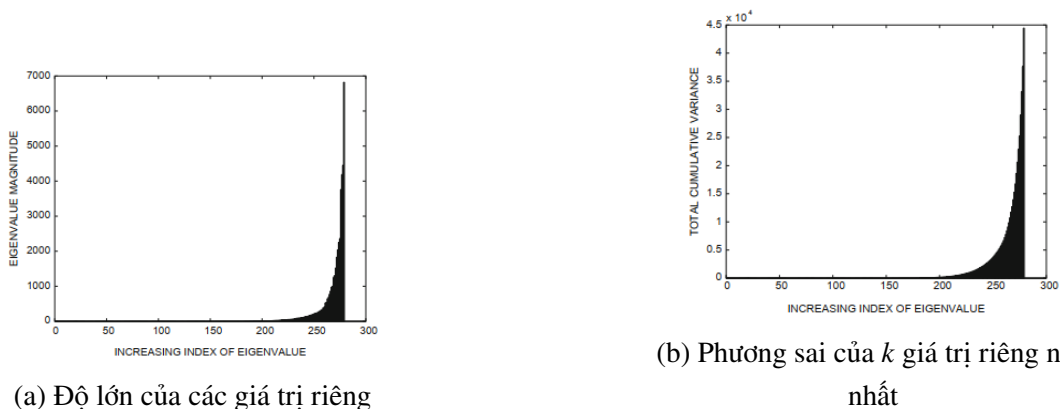
Một tính chất thú vị của phép chéo hóa này là cả vectơ riêng và giá trị riêng đều có cách diễn giải hình học theo phân phối của dữ liệu. Cụ thể, nếu hệ trục biểu diễn dữ liệu được xoay sang tập các vectơ riêng trực giao trong các cột của P , thì có thể chỉ ra rằng toàn bộ $\binom{d}{2}$ hiệp phương sai của các giá trị đặc trưng mới sau biến đổi bằng không. Nói cách khác, *hướng bảo toàn phương sai lớn nhất cũng là hướng loại bỏ tương quan*. Hơn nữa, các giá trị riêng biểu diễn các phương sai của dữ liệu dọc theo các vectơ riêng tương ứng. Thực chất, ma trận đường chéo Λ là ma trận hiệp phương sai mới sau phép quay hệ trục. Do đó, các vectơ riêng tương ứng với giá trị riêng lớn bảo toàn phương sai lớn hơn và còn được gọi là *thành phần chính*. Do bản chất của công thức tối ưu hóa được sử dụng để tạo ra phép biến đổi này, một hệ trục mới chỉ chứa các vectơ riêng có giá trị riêng lớn nhất được tối ưu hóa để duy trì phương sai cực đại trong một số chiều cố định. Ví dụ, biểu đồ phân tán trong Hình 2.3 minh họa các vectơ riêng khác nhau, và rõ ràng là vectơ riêng với phương sai lớn nhất là tất cả những gì cần thiết để có một biểu diễn bảo toàn phương sai. Thông thường ta chỉ cần giữ lại một số lượng nhỏ các vectơ riêng ứng với giá trị riêng lớn.

Không mất tính tổng quát, có thể giả sử rằng các cột của P (và tương ứng ma trận đường chéo Λ) được sắp xếp từ trái sang phải sao cho chúng tương ứng với các giá trị riêng giảm dần. Khi đó, ma trận dữ liệu sau biến đổi D' trong hệ tọa độ mới sau khi xoay trục thành các cột trực chuẩn của P được tính bằng phép biến đổi tuyến tính sau:

$$D' = DP. \quad (2.9)$$

Mặc dù ma trận dữ liệu sau chuyển đổi D' cũng có kích thước $n \times d$, nhưng chỉ $k \ll d$ cột đầu tiên (ngoài cùng bên trái) có sự thay đổi giá trị đáng kể. Mỗi cột trong $d - k$ cột còn lại của D' sẽ xấp xỉ bằng giá trị trung bình của dữ liệu trong hệ trục mới. Đối với dữ liệu trung bình, giá trị của các $(d - k)$ cột này sẽ gần bằng 0. Do đó, chiều của dữ liệu có thể được giảm xuống và chỉ k cột đầu tiên của ma trận dữ liệu sau chuyển đổi D' có thể cần được giữ lại để biểu diễn. Hơn nữa, có thể kiểm tra rằng ma trận hiệp phương sai của dữ liệu sau chuyển đổi $D' = DP$ là ma trận đường chéo Λ bằng cách áp dụng định nghĩa hiệp phương sai trong (2.6) cho DP (dữ liệu sau chuyển đổi) và $\bar{\mu}P$ (giá trị trung bình sau chuyển đổi) thay vì D và $\bar{\mu}$, tương ứng. Ma trận hiệp phương sai thu được có thể được biểu diễn thông qua dạng ma trận hiệp phương sai C ban đầu bằng $P^T C P$. Thay $C = P \Lambda P^T$ ta thu được ma trận hiệp sai của D' vì $P^T P = P P^T = I$. Nói cách khác, trong dữ liệu sau chuyển đổi, các mối tương quan đã bị loại bỏ khỏi dữ liệu vì Λ là ma trận đường chéo.

Phương sai của tập dữ liệu xác định bởi các phép chiếu dọc theo các k vectơ riêng đầu tiên bằng tổng của k giá trị riêng tương ứng. Nhiều ứng dụng cho thấy các giá trị riêng sụt giảm nhanh chóng sau một vài giá trị đầu tiên. Ví dụ: đáng điều của các giá trị riêng của với tập dữ liệu *Arrhythmia* với 279 chiều lấy từ *Kho lưu trữ máy học UCI* được minh họa trong Hình 2.4. Hình 2.4a minh họa giá trị tuyệt đối của các giá trị riêng theo thứ tự tăng dần, trong khi Hình 2.4b minh họa tổng lượng phương sai được giữ lại trong k giá trị riêng đầu. Hình 2.4b có thể được suy ra bằng cách sử dụng tổng tích lũy của các giá trị riêng nhỏ nhất trong Hình 2.4a. Lưu ý rằng 215 giá trị riêng nhỏ nhất chứa ít hơn 1% tổng phương sai trong dữ liệu và do đó có thể được loại bỏ với ít thay đổi đối với kết quả của các ứng dụng dựa trên độ tương tự. Lưu ý rằng bộ dữ liệu *Arrhythmia* không phải là bộ dữ liệu có mối tương quan chặt chẽ với nhiều cặp chiều. Tuy nhiên, việc giảm kích thước là rất lớn do tác động *tích lũy* của các mối tương quan trên nhiều chiều khác nhau.



Hình 2.4: Phương sai ứng với số lượng giá trị riêng tăng dần của tập dữ liệu *Arrhythmia*

Các vectơ riêng của ma trận C có thể được xác định bằng phương pháp số bất kỳ hoặc bằng một công cụ giải vectơ riêng có sẵn. Ta có thể mở rộng kỹ thuật PCA cho các phép nhúng phi tuyến tính bằng cách sử dụng một phương pháp được gọi là *thuật hạt nhân*.

Phương pháp phân tích giá trị kỳ dị

Phương pháp phân tích giá trị kỳ dị (SVD) có liên quan chặt chẽ với phương pháp phân tích thành phần chính (PCA). Tuy nhiên, hai phương pháp riêng biệt này đôi khi bị nhầm lẫn với nhau do có mối quan hệ chặt chẽ. Trước khi bắt đầu thảo luận về SVD, ta so sánh nó với PCA. SVD tổng quát hơn PCA vì nó cho ra *hai bộ vectơ cơ sở* thay vì một. SVD cung cấp các vectơ cơ sở của cả hàng và cột của ma trận dữ liệu, trong khi PCA chỉ cung cấp các vectơ cơ sở của các hàng của ma trận dữ liệu.

Hơn nữa, SVD tạo ra cùng cơ sở giống như PCA cho các hàng của ma trận dữ liệu trong một số trường hợp đặc biệt:

SVD tạo ra cùng vectơ cơ sở và phép biến đổi dữ liệu như PCA đối với các tập dữ liệu có giá trị trung bình của mỗi thuộc tính bằng 0.

Các vectơ cơ sở của PCA là bất biến đối với phép tịnh tiến giá trị trung bình, trong khi các vectơ của SVD thì không. Khi dữ liệu không định tâm giá trị trung bình, các vectơ cơ sở của SVD và PCA khác nhau và có thể thu được các kết quả định lượng khác nhau. SVD thường được áp dụng cho tập dữ liệu không âm thừa mà không cần định tâm giá trị trung bình, chẳng hạn như ma trận thuật ngữ tài liệu. SVD là kỹ thuật phân tích ma trận thành tích của ba ma trận:

$$D = Q\Sigma P^T. \quad (2.10)$$

Ở đây, Q là ma trận vuông cấp n với các cột trực chuẩn, mà được gọi là các *vectơ kì dị trái*. Σ là một ma trận đường chéo cỡ $n \times d$ chứa các *giá trị kì dị* luôn không âm, và để cho tiện ta sắp xếp theo thứ tự không tăng. Ngoài ra, P là một ma trận vuông cấp d với các cột trực chuẩn, mà được gọi là các *vectơ kì dị phải*. Chú ý rằng ma trận đường chéo Σ là một ma trận hình chữ nhật, không phải là ma trận vuông, nhưng ta gọi nó là ma trận đường chéo bởi vì chỉ các phần tử Σ_{ii} khác không. Kết quả cơ bản trong đại số tuyến tính khẳng định rằng luôn tồn tại một phép phân tích như vậy. Số phần tử khác không trên đường chéo của Σ bằng hạng của ma trận D , do đó hạng của D lớn nhất bằng $\min\{n, d\}$. Ngoài ra, do tính trực chuẩn của các vectơ kì dị, cả $P^T P$ và $Q^T Q$ đều là ma trận đơn vị. Ta có một số nhận xét sau:

1. Các cột của ma trận Q , tức các vectơ kì dị trái, là các vectơ riêng trực chuẩn của DD^T . Thật vậy, $DD^T = Q\Sigma(P^T P)\Sigma^T Q^T = Q\Sigma\Sigma^T Q^T$. Do đó, bình phương của các giá trị kì dị khác không, tức các phần tử đường chéo của ma trận đường chéo $\Sigma\Sigma^T$, chính là các giá trị riêng khác không của DD^T .
2. Các cột của P , tức các vectơ kì dị phải, là các vectơ riêng trực chuẩn của DD^T . Bình phương của các giá trị kì dị khác không, tức các phần tử đường chéo của ma trận đường chéo $\Sigma\Sigma^T$, chính là các giá trị riêng khác không của $D^T D$. Chú ý rằng các giá trị riêng khác không của DD^T và $D^T D$ là một. Ma trận P rất quan trọng bởi vì nó cung cấp cho ta các vectơ cơ sở, tương tự với các vectơ riêng của ma trận hiệp phương sai trong PCA.
3. Bởi vì ma trận hiệp phương sai của dữ liệu định tâm giá trị trung bình là $\frac{D^T D}{n}$ và các vectơ kì dị phải của SVD là các vectơ riêng của $D^T D$, ta suy ra các

vectơ riêng của PCA là một với các vectơ kì dị phải của SVD khi dữ liệu được định tâm giá trị trung bình. Ngoài ra, bình phương của giá trị kì dị trong SVD bằng bội n của các giá trị riêng của PCA. Sự tương đương này cho thấy tại sao SVD và PCA có thể cung cấp cùng một phép biến đổi cho dữ liệu định tâm giá trung bình.

4. Không mất tính tổng quát, ta có thể giả sử các phần tử đường chéo của Σ được sắp xếp theo thứ tự giảm dần, các cột của ma trận P và Q cũng được sắp thứ tự tương ứng. Ký hiệu P_k, Q_k tương ứng là các ma trận chặt cột cỡ $d \times k$ và $n \times k$ thu được từ P và Q bằng cách chọn k cột đầu tiên. Ký hiệu Σ_k là ma trận vuông cấp k chứa k giá trị kì dị đầu tiên. Khi đó, phép biến đổi SVD kéo theo một cách biểu diễn dữ liệu xấp xỉ d chiều của tập dữ liệu D ban đầu:

$$D \approx Q_k \Sigma_k P_k^T. \quad (2.11)$$

Các cột của P_k là hệ cơ sở k chiều của tập dữ liệu sau rút gọn. Tập dữ liệu rút gọn chiều trong hệ cơ sở k chiều này được xác định bởi tập dữ liệu $D'_k = DP_k = Q_k \Sigma_k$, mà có cỡ $n \times k$, như trong (2.9) của PCA. Mỗi hàng trong n hàng của D'_k chứa k tọa độ của điểm dữ liệu sau biến đổi trong hệ trục mới. Thông thường, giá trị của k của nhỏ hơn n và d nhiều. Ngoài ra, khác với kĩ thuật PCA, $(d - k)$ cột ngoài cùng bên phải của ma trận dữ liệu sau biến đổi đủ d chiều $D' = DP$ sẽ xấp xỉ bằng 0 (thay vì giá trị trung bình của dữ liệu), cho dù dữ liệu có được định tâm giá trị trung bình hay không. Nói chung, PCA chiếu dữ liệu lên một siêu phẳng số chiều thấp đi qua gốc tọa độ. PCA nắm bắt phương sai (hoặc, bình phương khoảng cách Euclide quanh giá trị trung bình) của dữ liệu càng nhiều càng tốt, trong khi SVD nắm bắt bình phương khoảng cách Euclide tổng hợp quanh gốc tọa độ càng nhiều càng tốt. Phương pháp xấp xỉ ma trận dữ liệu này được gọi là SVD *chặt cột*.

Phần tiếp theo sẽ chỉ ra rằng kĩ thuật SVD chặt cột cực đại hóa bình phương khoảng cách Euclidean tổng hợp quanh gốc tọa độ của các điểm dữ liệu sau chuyển đổi. Gọi $\bar{v}$ là một vectơ cột d chiều và $D\bar{v}$ là hình chiếu của tập dữ liệu D trên $\bar{v}$. Xét bài toán xác định vectơ *đơn vị* $\bar{v}$ cực đại hóa tổng bình phương khoảng cách Euclide $(D\bar{v})^T (D\bar{v})$ quanh gốc tọa độ của các điểm dữ liệu chiếu. Việc đặt gradient của phép giảm dư Lagrangian $\bar{v}^T D^T D \bar{v} - \lambda (\|\bar{v}\|^2 - 1)$ bằng 0 tương đương với điều kiện $D^T D \bar{v} - \lambda \bar{v} = 0$. Bởi vì các vectơ kỳ dị phải là các vectơ riêng của $D^T D$, nên các vectơ riêng (vectơ kì dị phải) với k giá trị riêng lớn nhất (bình phương giá trị kì dị) cung cấp một cơ sở giúp cực đại hóa năng lượng được bảo toàn trong ma trận

dữ liệu sau biến đổi và rút gọn $D'_k = DP_k = Q_k \Sigma_k$. Vì tổng bình phương khoảng cách Euclide từ gốc tọa độ bất biến đối với phép xoay trục nên phép SVD hạng k là phép phân tích cực đại hóa năng lượng bảo toàn. Kết quả này được gọi là định lý Eckart–Young.

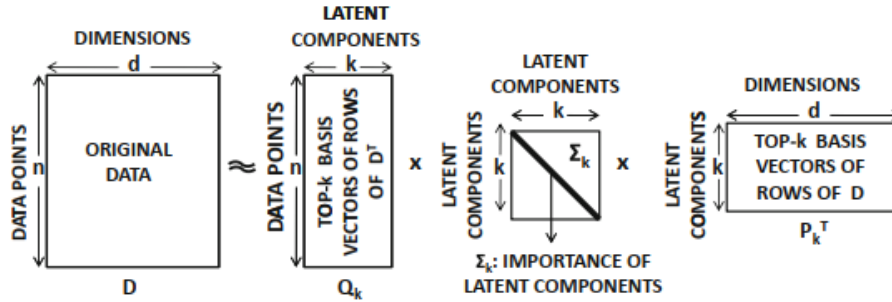
Khi được xét hoàn toàn dưới dạng phân tích vectơ riêng, SVD cho ta hai quan điểm khác nhau để hiểu về biến đổi và giảm chiều dữ liệu. Ta có thể coi ma trận dữ liệu sau biến đổi là hình chiếu DP_k của ma trận dữ liệu D trên k vectơ riêng cơ sở đầu tiên P_k của ma trận phân tán $D^T D$ có cấp $d \times d$, hoặc ta có thể trực tiếp coi nó là các vectơ riêng được co giãn tỷ lệ $Q_k \Sigma_k = DP_k$ của ma trận đồng dạng DD^T cỡ $n \times n$. Mặc dù việc tìm vectơ riêng của một ma trận nhân đồng dạng cỡ $n \times n$ rất tốn kém về mặt tính toán, nhưng cách tiếp cận này cũng tổng quát hóa cho phương pháp giảm chiều dữ liệu phi tuyến trong đó các khái niệm về vectơ cơ sở tuyến tính không tồn tại trong không gian ban đầu. Trong những trường hợp như vậy, ma trận nhân đồng dạng được thay thế bằng ma trận đồng dạng tự phức tạp hơn để trích xuất phép nhúng phi tuyến.

SVD tổng quát hơn PCA và có thể được sử dụng để xác định đồng thời một tập con gồm k vectơ cơ sở của ma trận dữ liệu và chuyển vị của nó với năng lượng cực đại. Việc xác định chuyển vị của k vectơ cơ sở với năng lượng cực đại sẽ hữu ích trong việc hiểu các tính chất biến đổi bổ sung của D^T . Các cột trực chuẩn của Q_k là một hệ cơ sở k chiều để biến đổi (xấp xỉ) các “điểm dữ liệu” tương ứng với các hàng của D^T và ma trận $D^T Q_k = P_k \Sigma_k$ chứa các tọa độ tương ứng. Ví dụ: trong ma trận đánh giá người dùng-hàng hóa, người ta có thể muốn xác định biểu diễn rút gọn của người dùng hoặc biểu diễn rút gọn của hàng hóa. SVD cung cấp các vectơ cơ sở cho cả hai phép rút gọn. SVD bị chắt chít biểu diễn dữ liệu theo k thành phần ẩn trội. Thành phần ẩn thứ i được phát biểu thông qua vectơ cơ sở thứ i của cả D và D^T và tầm quan trọng tương đối của nó trong dữ liệu được xác định bởi giá trị kì dị thứ i . Bằng cách phân tích tích ma trận $Q_k \Sigma_k P_k^T$ thành các vectơ cột của Q_k và P_k (nghĩa là các vectơ cơ sở trội của D^T và D), ta thu được tổng của k thành phần ẩn:

$$Q_k \Sigma_k P_k^T = \sum_{i=1}^k \bar{q}_i \sigma_i \bar{p}_i^T = \sum_{i=1}^k \sigma_i (\bar{q}_i \bar{p}_i^T). \quad (2.12)$$

Ở đây $\bar{q}_i$ là cột thứ i của Q , $\bar{p}_i$ là cột thứ i của P và σ_i là phần tử đường chéo thứ i của Σ . Mỗi thành phần ẩn $\sigma_i (\bar{q}_i \bar{p}_i^T)$ là một ma trận cỡ $n \times d$ có hạng bằng 1 và có năng lượng bằng σ_i^2 . Phép phân tích trên được gọi là phép *phân tích phổ*. Mỗi quan

hệ của các vectơ cơ sở rút gọn với phép phân tích ma trận SVD được minh họa trong Hình 2.5.



Hình 2.5: Bổ sung cơ sở cho ma trận phân tích trong SVD

Ta minh họa phép phân tích SVD chặt cắt hạng 2 của ma trận 6×6 như sau:

$$\begin{aligned}
 D &= \begin{bmatrix} 2 & 2 & 1 & 2 & 0 & 0 \\ 2 & 3 & 3 & 3 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 2 & 2 & 2 & 3 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 2 & 1 & 2 \end{bmatrix} \approx Q_2 \Sigma_2 P_2^T \\
 &\approx \begin{bmatrix} -0.41 & 0.17 \\ -0.65 & 0.31 \\ -0.23 & 0.13 \\ -0.56 & -0.20 \\ -0.10 & -0.46 \\ -0.19 & -0.78 \end{bmatrix} \begin{bmatrix} 8.4 & 0 \\ 0 & 3.1 \end{bmatrix} \begin{bmatrix} -0.41 & -0.49 & -0.44 & -0.61 & -0.10 & -0.12 \\ 0.21 & 0.31 & 0.26 & -0.37 & -0.44 & -0.68 \end{bmatrix} \\
 &= \begin{bmatrix} 1.55 & 1.87 & \underline{1.67} & 1.91 & 0.10 & 0.04 \\ 2.46 & 2.98 & 2.66 & 2.95 & 0.10 & -0.03 \\ 0.89 & 1.08 & 0.96 & 1.04 & 0.01 & -0.04 \\ 1.81 & 2.11 & 1.91 & 3.14 & 0.77 & 1.03 \\ 0.02 & -0.05 & -0.02 & 1.06 & 0.74 & 1.11 \\ 0.10 & -0.02 & 0.04 & 1.89 & 1.28 & 1.92 \end{bmatrix}
 \end{aligned}$$

Chú ý rằng ma trận hạng 2 là một phép xấp xỉ tốt của ma trận ban đầu. Phần tử có sai số lớn nhất là phần tử được gạch chân trong ma trận xấp xỉ cuối cùng. Phần tử này cũng không tương thích với cấu trúc của phần còn lại của ma trận trong dữ liệu ban đầu. Kỹ thuật SVD chặt cắt thường cố gắng sửa các phần tử không tương thích, và đôi khi tính chất này được tận dụng để giảm nhiễu trong các tập dữ liệu dễ bị lỗi.

Ứng dụng của PCA và SVD

Mặc dù PCA và SVD chủ yếu được sử dụng để giảm và nén dữ liệu, nhưng chúng có nhiều ứng dụng khác trong khai thác dữ liệu. Một số ví dụ như sau:

1. *Giảm nhiễu:* Mặc dù việc bỏ các vectơ riêng/vectơ kì dị nhỏ trong kĩ thuật PCA và SVD có thể dẫn đến mất thông tin, nhưng điều này cũng có thể cải thiện chất lượng biểu diễn dữ liệu trong nhiều trường hợp một cách đáng ngạc nhiên. Lý do chính là các biến thể dọc theo các vectơ riêng nhỏ thường là hậu quả do nhiễu gây ra và việc loại bỏ chúng nói chung là có lợi. Một ví dụ là ứng dụng LSA trong xử lý văn bản khi việc loại bỏ các thành phần nhỏ hơn dẫn đến việc tăng cường các đặc điểm ngữ nghĩa của văn bản. SVD cũng được sử dụng để làm mờ hình ảnh nhiễu. Các kết quả về xử lý văn bản và hình ảnh này cũng đúng trong các miền dữ liệu tùy ý [1]. Do đó, việc giảm dữ liệu không chỉ hiệu quả về mặt không gian mà còn thực sự mang lại lợi ích định tính trong nhiều trường hợp.
2. *Quy ước dữ liệu:* SVD và PCA có thể được sử dụng cho các ứng dụng quy ước dữ liệu [3], chẳng hạn như lọc thông thường, vì ta có thể ước tính các ma trận rút gọn Q_k, Σ_k và P_k với các giá trị k nhỏ ngay cả từ các ma trận dữ liệu không đầy đủ. Do đó, toàn bộ ma trận có thể được xây dựng lại gần đúng thành $Q_k \Sigma_k P_k^T$.
3. *Phương trình tuyến tính:* Nhiều ứng dụng khai thác dữ liệu là các bài toán tối ưu trong đó lời giải được chuyển thành một hệ phương trình tuyến tính. Với bất kỳ hệ phương trình tuyến tính $A\bar{y} = 0$, mọi vectơ kỳ dị phải của A có giá trị kỳ dị bằng 0 là nghiệm của hệ phương trình. Do đó, mọi tổ hợp tuyến tính của các vectơ kì dị ứng với giá trị kì dị 0 đều là một nghiệm.
4. *Phép nghịch đảo ma trận:* Ta có thể sử dụng SVD để tìm ma trận nghịch đảo của ma trận vuông D cỡ $d \times d$. Giả sử D có phân tích là $Q\Sigma P^T$. Khi đó, nghịch đảo của D là $D^{-1} = P\Sigma^{-1}Q^T$. Lưu ý rằng ta có thể tính Σ^{-1} một cách đơn giản bằng cách đảo ngược các phần tử trên đường chéo của Σ . Ta cũng có thể tổng quát phương pháp này để tính ma trận nghịch đảo giả Moore–Penrose D^+ của ma trận D hạng k bằng cách chỉ đảo ngược các phần tử đường chéo khác không của Σ . Ta cũng có thể mở rộng phương pháp này cho ma trận không vuông bằng cách thực hiện thao tác bổ sung là lấy chuyển vị Σ . Việc tính ma trận nghịch đảo xuất hiện trong nhiều ứng dụng khai thác dữ liệu như hồi quy bình phương nhỏ nhất.
5. *Đại số ma trận:* Nhiều ứng dụng khai thác mạng cần áp dụng các phép toán đại số như tính toán lũy thừa của ma trận. Điều này phổ biến trong các phương pháp bước đi ngẫu nhiên, trong đó có thể cần phải tính lũy thừa bậc k của ma trận kề đối xứng của một mạng vô hướng. Các ma trận kề đối xứng như vậy có

thể được phân tách thành dạng $Q\Delta Q^T$. Lũy thừa bậc k của phép phân tích trên được tính một cách hiệu quả là $D^k = Q\Delta^k Q^T$. Trên thực tế, bất kỳ hàm đa thức ma trận nào đều có thể được tính toán một cách hiệu quả.

Phép phân tích SVD và PCA cực kỳ hữu ích vì trong khai thác dữ liệu, các phép toán đại số tuyến tính và ma trận rất phổ biến. Phép phân tích SVD và PCA giúp cho việc tính toán ma trận như vậy đơn giản hơn bằng cách cung cấp các phép phân tích thuận tiện và các biểu diễn cơ sở.

2.4.4 Biến đổi dữ liệu

Trong các phương pháp này, ta kết hợp kỹ thuật giảm chiều dữ liệu với kỹ thuật chuyển đổi kiểu dữ liệu. Trong hầu hết các trường hợp, dữ liệu được chuyển đổi từ loại phức tạp hơn sang loại ít phức tạp hơn, chẳng hạn như dữ liệu đa chiều. Do đó, các phương pháp này phục vụ mục đích kép giảm chiều dữ liệu và tính di động (portability) của dữ liệu. Mục này trình bày hai phương pháp biến đổi như vậy:

1. *Từ dữ liệu theo thời gian sang dữ liệu đa chiều:* Một số phương pháp được sử dụng ví dụ như phép biến đổi Fourier rời rạc và biến đổi sóng nhỏ rời rạc. Mặc dù các phương pháp này cũng có thể được xem như một phép quay của một hệ trục được xác định bởi các dấu thời gian khác nhau của thuộc tính ngữ cảnh, nhưng dữ liệu không còn được định hướng phụ thuộc sau khi xoay. Do đó, tập dữ liệu kết quả có thể được xử lý theo cách tương tự như dữ liệu đa chiều. Chúng ta sẽ nghiên cứu biến đổi sóng nhỏ Haar vì tính đơn giản trực quan của nó.
2. *Từ dữ liệu biểu đồ có trọng số sang dữ liệu đa chiều:* Ta sử dụng các phương pháp co giãn và phổ đa chiều để nhúng các biểu đồ có trọng vào không gian đa chiều, sao cho giá trị tương tự hoặc khoảng cách trên các cạnh được ghi lại bằng một phép nhúng đa chiều.

Bài tập Chương 2

Bài tập 2.1 Xét chuỗi thời gian $(-3, -1, 1, 3, 5, 7, *)$. Ở đây dấu $*$ là một phần tử bị thiếu. Đánh giá giá trị của phần tử bị thiếu bằng phép nội suy tuyến tính trên một ô kích thước 3.

Bài tập 2.2 Giả sử bạn có một loạt tài liệu văn bản và bạn muốn xác định tất cả các cá nhân được đề cập trong các tài liệu này. Bạn sẽ sử dụng loại công nghệ nào để đạt được mục tiêu này?

Bài tập 2.3 Tải dữ liệu *Arrhythmia* từ *UCI Machine Learning Repository* [213]. Chuẩn hóa các bản ghi để có giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1. Rồi raster hóa mỗi thuộc tính số thành (a) 10 đoạn độ rộng bằng nhau và (b) 10 đoạn độ sâu bằng nhau.

Bài tập 2.4 Cho dãy số liệu sau

199, 201, 236, 269, 271, 278, 283, 291, 301, 303, 371.

Xác định bất thường của dãy số liệu trên sử dụng khoảng $(\bar{x} - 2s, \bar{x} + 2s)$ và sử dụng khoảng $(\bar{x} - 3s, \bar{x} + 3s)$

Bài tập 2.5 Cho dãy số liệu sau

199, 201, 236, 269, 271, 278, 283, 291, 301, 303, 371

Tính Q_1, Q_2, Q_3, IQR và cho biết có giá trị ngoại lai không? Minh họa bằng biểu đồ hộp trên python.

Bài tập 2.6 Viết chương trình python xác định giá trị ngoại lai cho các biến có trong dữ liệu *Boston.csv* theo hai cách:

- Sử dụng biểu đồ hộp,
- Sử dụng $\bar{x}$ và s .

Bài tập 2.7 Viết chương trình python chuẩn hóa dữ liệu cho các biến định lượng trong dữ liệu *Boston.csv*.

Bài tập 2.8 Giả sử mẫu dữ liệu sau:

13, 16, 19, 22, 23, 38, 47, 56, 58, 63, 65, 70, 71,

được trích ra từ một biến ngẫu nhiên có phân phối chuẩn với kỳ vọng là 43,15 và độ lệch tiêu chuẩn là 22,13. Chuẩn hóa mẫu dữ liệu đã cho.

Bài tập 2.9 Giả sử rằng bạn có một tập hợp các đối tượng tùy ý có kiểu bất kì đại diện cho các đặc điểm khác nhau của các vật dụng. Một chuyên gia cung cấp cho bạn giá trị tương tự giữa mọi cặp đối tượng. Bạn sẽ chuyển đổi các đối tượng này thành một tập dữ liệu đa chiều để phân cụm như thế nào?

Bài tập 2.10 Giả sử bạn có một bộ dữ liệu sao cho mỗi điểm dữ liệu tương ứng với các nhiệt độ bề mặt biển trên một dặm vuông có độ phân giải 10×10 . Nói cách khác, mỗi bản ghi dữ liệu chứa một lưới 10×10 giá trị nhiệt độ theo vị trí không

gian. Bạn cũng có một số văn bản được liên kết với mỗi lưới 10×10 . Bạn sẽ chuyển đổi dữ liệu này thành tập dữ liệu đa chiều như thế nào?

Bài tập 2.11 Giả sử rằng bạn có một tập hợp các dãy protein sinh học rời rạc được chú thích bằng đoạn văn mô tả các đặc tính của protein. Bạn sẽ tạo một biểu diễn đa chiều từ tập dữ liệu không đồng nhất này như thế nào?

Bài tập 2.12 Tải bộ dữ liệu *Musk* từ *UCI Machine Learning Repository* [213]. Áp dụng PCA cho tập dữ liệu, tìm các vectơ riêng và giá trị riêng.

Bài tập 2.13 Lặp lại bài trên với SVD.

Bài tập 2.14 Cho bộ dữ liệu được định tâm giá trị trung bình với các điểm $\bar{X}_1, \dots, \bar{X}_n$, chứng minh rằng

$$\|\bar{X}_i\|^2 + \|\bar{X}_j\|^2 \frac{\sum_{p=1}^n \|\bar{X}_i - \bar{X}_p\|^2}{n} + \frac{\sum_{q=1}^n \|\bar{X}_j - \bar{X}_q\|^2}{n} - \frac{\sum_{p=1}^n \sum_{q=1}^n \|\bar{X}_p - \bar{X}_q\|^2}{n^2}.$$

Bài tập 2.15 Xét chuỗi thời gian 1, 1, 3, 3, 3, 3, 1, 1. Thực hiện phép phân tích sóng nhỏ trên chuỗi thời gian. Có bao nhiêu hệ số của chuỗi khác không?

Bài tập 2.16 Tải tập dữ liệu *Intel Research Berkeley*. Áp dụng phép phân tích sóng nhỏ cho các giá trị nhiệt độ trong cảm biến đầu tiên.

Bài tập 2.17 Coi các biến định lượng trong tập dữ liệu *KDD CUP 1999 Network Intrusion* lấy trong *UCI Machine Learning Repository* [213] là một chuỗi thời gian. Thực hiện phép phân tích sóng nhỏ cho chuỗi thời gian này.

Bài tập 2.18 Tạo một mẫu kích thước $n = 1, 10, 100, 1000, 10000$ bản ghi từ tập dữ liệu trong bài tập trước, xác định giá trị trung bình e_i của mỗi cột số lượng i bằng cách lấy mẫu. Gọi μ_i và σ_i là trung bình tổng thể và độ lệch chuẩn tổng thể trên toàn bộ tập dữ liệu. Tính mức độ lệch z_i của e_i khỏi μ_i

$$z_i = \frac{e_i - \mu_i}{\sigma_i}.$$

z_i thay đổi như thế nào với các giá trị của n ?

Bài tập 2.19 Triển khai trình thu thập dữ liệu bằng cách sử dụng thuật toán theo chiều rộng.

Bài tập 2.20 Xét chuỗi *ababcdef*. Liệt kê tất cả 2-shingles và 3-shingles, sử dụng mỗi bảng chữ cái làm một token.

Bài tập 2.21 Thảo luận về lý do tại sao nên thêm văn bản neo (*anchor text*) vào trang Web mà nó trỏ đến cho mục đích khai phá dữ liệu, nhưng nó thường gây nhầm lẫn cho trang mà nó xuất hiện.

Bài tập 2.22 Thực hành tìm hiểu một số website dữ liệu có cung cấp API như website về chỉ số thời tiết và dân số. Sau đó, thu thập và hiển thị dữ liệu thu thập được.

Bài tập 2.23 Tìm hiểu và thu thập dữ liệu ba dự án thực tế sau:

1. Champions League Table [ESPN]
2. Product Tracker [Amazon]
3. Scraper Application [GUI].



3. Trực quan hóa dữ liệu

Chương 3 có nội dung chính là tìm hiểu sâu công cụ Matplotlib trực quan hoá dữ liệu cho các loại biểu đồ khác nhau trong phân tích dữ liệu.

Matplotlib là thư viện trực quan hóa dữ liệu được xây dựng trên mảng NumPy và được thiết kế để hoạt động với ngăn xếp SciPy. Nó được John Hunter đưa ra vào năm 2002, cho phép vẽ sơ đồ kiểu MATLAB tương tác thông qua gnuplot từ dòng lệnh IPython.

Một trong những tính năng quan trọng nhất của Matplotlib là khả năng hoạt động tốt với đa nền tảng nhiều hệ điều hành, điều này dẫn đến một số lượng lớn người dùng với sự phổ biến trong thế giới khoa học dữ liệu sử dụng Python.

3.1 Cơ bản về Matplotlib

Trước khi đi sâu vào chi tiết tạo trực quan hóa bằng Matplotlib, có một số mẹo hữu ích nên biết về việc sử dụng Matplotlib như sau.

Nạp thư viện matplotlib

Sử dụng viết tắt tiêu chuẩn khi nạp thư viện Matplotlib:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

plt bao gồm nhiều hàm sẽ được sử dụng thường xuyên trong suốt chương này.

Thiết lập Style sử dụng Matplotlib

Thiết lập *plt.style* để chọn phong cách, các kiểu phù hợp cho các biểu đồ trực quan. Trong giáo trình này sẽ chọn kiểu *classic*, để đảm bảo rằng các biểu đồ được tạo ra sử dụng kiểu Matplotlib cổ điển ta thực hiện câu lệnh sau:

```
plt.style.use('classic')
```

Cách hiển thị các biểu đồ

Để phác hoạ và hiển thị một biểu đồ sử dụng Matplotlib có thể sử dụng một *script* - ngôn ngữ kịch bản, *IPython shell* hoặc *IPython notebook* như sau:

Trực quan dữ liệu bằng một Script

Sử dụng một tệp có tên *myplot.py* chứa các đoạn lệnh sau:

```
# ----- file: myplot.py -----
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)

plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x))

plt.show()
```

Có thể chạy file trên từ giao diện dòng lệnh command-line prompt và kết quả sẽ hiển thị một biểu đồ trong cửa sổ mới bằng câu lệnh:

```
$ python myplot.py
```

Lệnh `plt.show()` thực hiện việc kết hợp nhiều thành phần của một biểu đồ và thường chỉ được sử dụng một lần ở cuối tập lệnh để hiển thị biểu đồ. Nhiều lệnh `show()` có thể dẫn đến sự phụ thuộc vào các thành phần của biểu đồ không thể đoán trước vì vậy nên hạn chế điều này.

Trực quan dữ liệu bằng IPython shell

IPython được xây dựng để hoạt động tốt với Matplotlib, để bật chế độ này, có thể sử dụng lệnh `%matplotlib` sau khi khởi động *ipython*:

```
In [1]: %matplotlib
Using matplotlib backend: TkAgg

In [2]: import matplotlib.pyplot as plt
```

Bất kỳ lệnh *plt plot* nào cũng sẽ mở cửa sổ biểu đồ ra và các lệnh khác có thể được chạy để cập nhật biểu đồ. Một số thay đổi (chẳng hạn như sửa đổi thuộc tính của các đường đã được vẽ) sẽ không được cập nhật; để bắt buộc cập nhật, sử dụng `plt.draw()`.

Trực quan dữ liệu bằng IPython notebook

IPython Notebook là một công cụ phân tích dữ liệu tương tác dựa trên trình duyệt có thể kết hợp mã nguồn, đồ họa, các phần tử HTML, v.v. vào một tài liệu thực thi duy nhất. Vẽ biểu đồ tương tác trong IPython Notebook có thể được thực hiện bằng lệnh `%matplotlib` và hoạt động theo cách tương tự với IPython shell. Trong *IPython Notebook*, cũng có tùy chọn nhúng đồ họa trực tiếp vào Notebook, với hai tùy chọn:

- `%matplotlib notebook` các biểu đồ tương tác được nhúng trong Notebook.
- `%matplotlib inline` các biểu đồ tĩnh được nhúng trong Notebook.

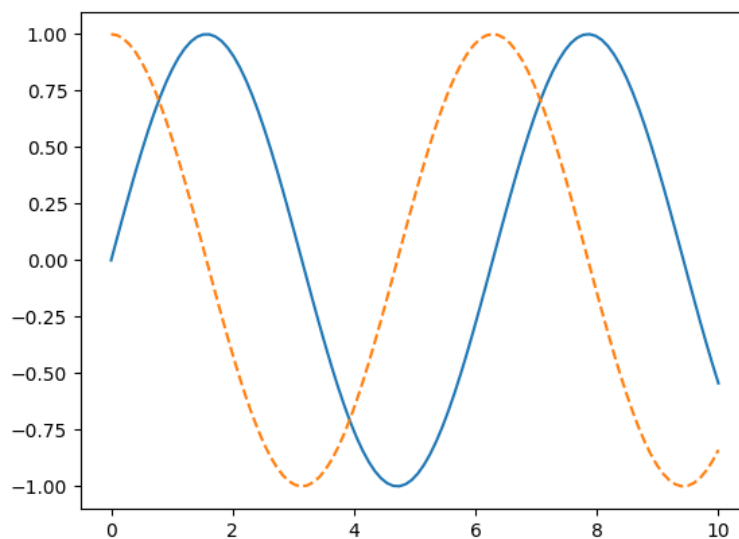
Để thống nhất trong toàn chương trình, các hướng dẫn trong chương trình lựa chọn `%matplotlib inline`:

```
In[3]: %matplotlib inline
```

Sau khi chạy lệnh này, bất kỳ ô - cell nào trong Notebook để tạo biểu đồ sẽ nhúng hình ảnh PNG trong kết quả Hình 3.1:

```
import numpy as np
x = np.linspace(0, 10, 100)

fig = plt.figure()
plt.plot(x, np.sin(x), '-b')
plt.plot(x, np.cos(x), '--r');
```



Hình 3.1: Ví dụ cơ bản về trực quan hoá.

Lưu ảnh vào tệp

Một tính năng của Matplotlib là khả năng lưu hình ảnh ở nhiều định dạng khác nhau, có thể lưu một hình bằng lệnh `savefig()`. Ví dụ: để lưu hình trước đó dưới dạng tệp PNG, có thể chạy lệnh sau:

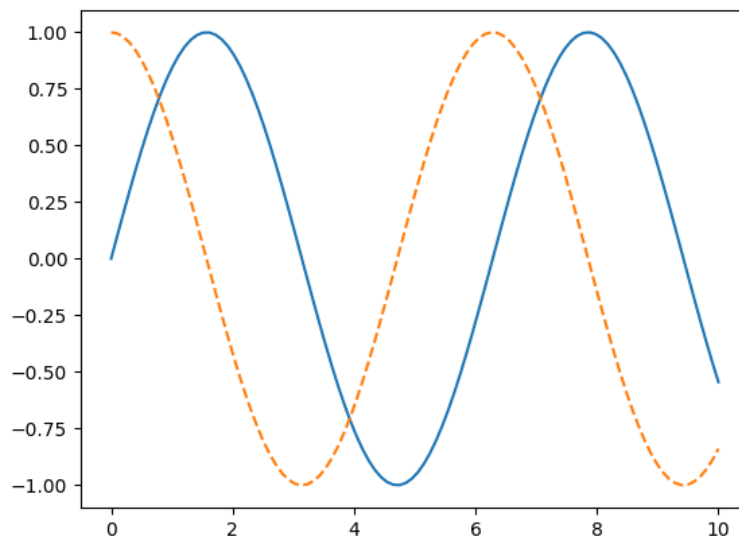
```
fig.savefig('my_figure.png')
```

Khi đó một tệp có tên `my_figure.png` được lưu trong thư mục làm việc hiện tại:

```
!ls -lh my_figure.png
-rw-r--r-- 1 mycomputer staff 37K Jul 24 11:46 my_figure.png
```

Để xác nhận rằng file ảnh vừa lưu chứa những gì, hãy sử dụng đối tượng `IPython.Image` để hiển thị nội dung của tệp này (3.2):

```
from IPython.display import Image
Image('my_figure.png')
```



Hình 3.2: Kết xuất PNG của hình ảnh cơ bản.

Trong `savefig()`, định dạng tệp được suy ra từ phần mở rộng của tên tệp đã cho và có thể tìm thấy danh sách các loại tệp được hỗ trợ bằng cách sử dụng phương thức `canvas` sau của đối tượng hình:

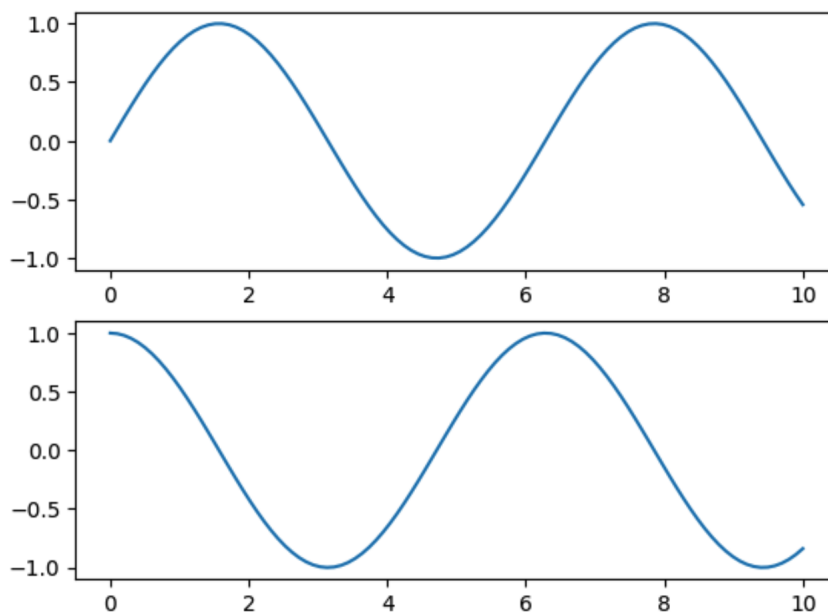
```
In[8]: fig.canvas.get_supported_filetypes()
Out[8]: {'eps': 'Encapsulated Postscript',
         'jpg': 'Joint Photographic Experts Group',
         'jpeg': 'Joint Photographic Experts Group',
```

```
'pdf': 'Portable Document Format',  
'pgf': 'PGF code for LaTeX',  
'png': 'Portable Network Graphics',  
'ps': 'Postscript',  
'raw': 'Raw RGBA bitmap',  
'rgba': 'Raw RGBA bitmap',  
'svg': 'Scalable Vector Graphics',  
'svgz': 'Scalable Vector Graphics',  
'tif': 'Tagged Image File Format',  
'tiff': 'Tagged Image File Format',  
'webp': 'WebP Image Format'}
```

Trực quan hoá nhiều biểu đồ trong một *Figure*

Một trong những điểm nổi bật của Matplotlib là có thể biểu diễn nhiều biểu đồ trên cùng một giao diện ảnh. Ví dụ dưới đây để vẽ hai biểu đồ trên cùng một ảnh (Hình 3.3):

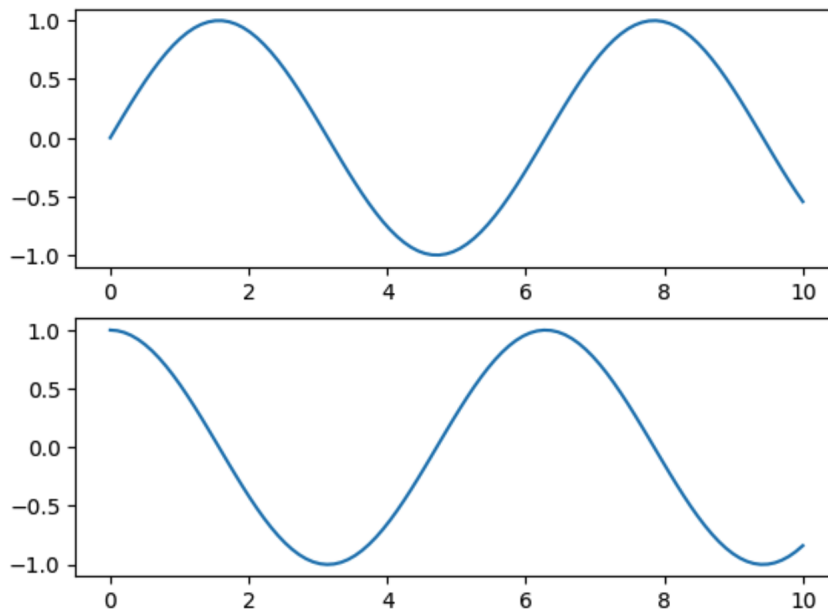
```
plt.figure() # Tao mot cua so figure de ve cac do thi  
# Tao bieu do dau tien trong hai bieu do  
plt.subplot(2, 1, 1) # (rows, columns, panel number)  
plt.plot(x, np.sin(x))  
# Tao bieu do thu hai  
plt.subplot(2, 1, 2)  
plt.plot(x, np.cos(x));
```



Hình 3.3: Sử dụng *subplot* để vẽ nhiều biểu đồ trong một *Figure*.

Trong trường hợp khi muốn hiển thị nhiều biểu đồ trong một hình cần sử dụng phương thức `subplots()`. Phương thức này lấy hai đối số `nrows` và `ncols` là số lượng hàng và số cột. Nó tạo ra hai đối tượng: *figure* và *axes* mà lưu trữ trong các biến *fig* và *ax* dưới đây có thể dùng để thay đổi các thuộc tính mức *figure* và *axes* tương ứng. Xem xét ví dụ kết hợp 2 biểu đồ (Hình 3.4) và 4 biểu đồ (Hình 3.5):

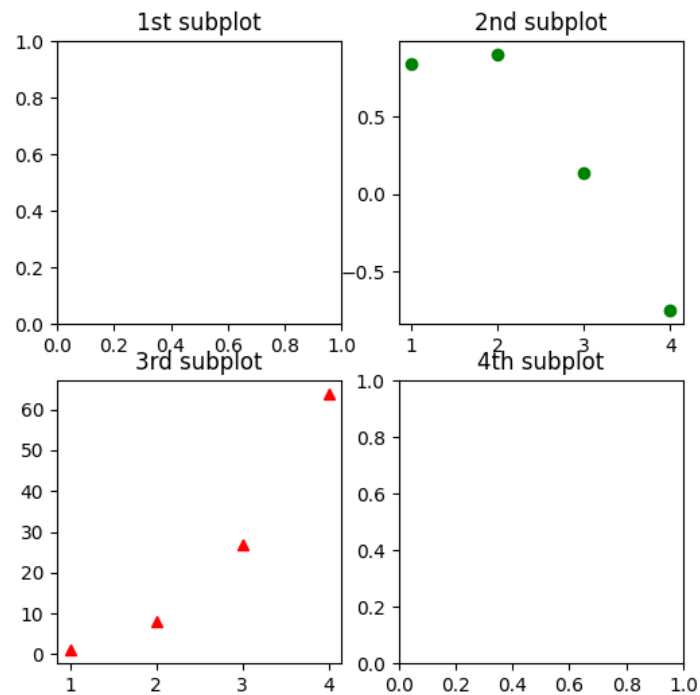
```
# Tao mot luoi cac o cho bieu do
# ax la mot mang bao gom hai phan tu Axes
fig, ax = plt.subplots(2)
# Goi phuong thuc plot() tren cac do thi tuong ung
ax[0].plot(x, np.sin(x))
ax[1].plot(x, np.cos(x));
```



Hình 3.4: Sử dụng *subplot* để vẽ 2 biểu đồ trong một *Figure*.

```
x = np.arange(1,5)
y = x**3
# Tao mot luoi cac o cho bieu do
# ax la mot mang bao gom 4 phan tu Axes (2 hang, 2 cot)
fig, ax = plt.subplots(nrows = 2, ncols = 2, figsize = (6,6))

# Goi phuong thuc plot() tren cac do thi tuong ung
ax[0,1].plot(x, np.sin(x), 'go')
ax[1,0].plot(x, y, 'r~')
ax[0,0].set_title('1st subplot')
ax[0,1].set_title('2nd subplot')
ax[1,0].set_title('3rd subplot')
ax[1,1].set_title('4th subplot')
```



Hình 3.5: Sử dụng *subplot* để vẽ 4 biểu đồ trong một Figure.

Các phương pháp trực quan hoá dữ liệu để trình bày, mô tả và tóm tắt dữ liệu rất hữu ích trong việc quan sát dữ liệu. Phần tiếp theo của chương này sẽ tìm hiểu cách trình bày dữ liệu một cách trực quan bằng các biểu đồ sau:

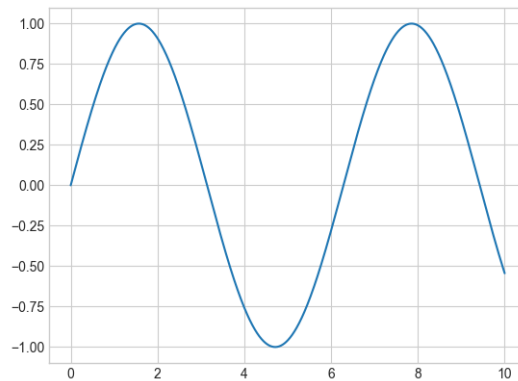
- Đồ thị dạng đường thẳng
- Đồ thị điểm rời rạc
- Trực quan hóa các biểu đồ lỗi
- Đồ thị đường viền
- Histograms và mật độ
- Văn bản và chú thích
- Đồ thị ba chiều
- Đồ thị dữ liệu địa lý

3.2 Đồ thị dạng đường

Đồ thị dạng đường thẳng có lẽ là đồ thị đơn giản nhất trong tất cả các đồ thị, nó hình dung của một hàm duy nhất $y = f(x)$. Đầu tiên là việc thiết Notebook để vẽ đồ thị và nhập các hàm sẽ thực hiện bằng các câu lệnh dưới đây, sau đó tạo một hình - *figure* và một trục - *ax*:

```
In[1]: %matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
import numpy as np
In[2]: fig = plt.figure()
ax = plt.axes()
```

Khi đã tạo một trục *ax*, có thể sử dụng hàm *ax.plot* để vẽ một số dữ liệu. Hãy bắt đầu với một hình *sin* đơn giản (Hình 3.6):



Hình 3.6: Đồ thị đường hình *sin*.

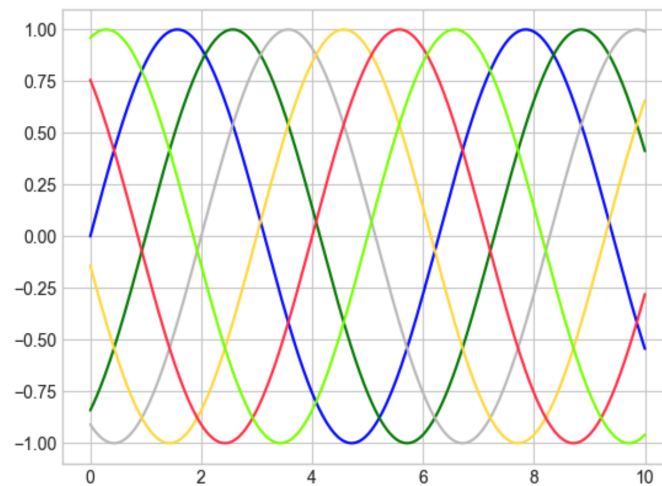
Điều chỉnh màu và kiểu dạng của đồ thị

Điều chỉnh có thể thực hiện đối với một biểu đồ là màu sắc (*colors*) và kiểu dáng của đồ thị (*styles*). Hàm *plt.plot()* chứa các đối số sử dụng để chỉ định các điều chỉnh này. Với màu sử dụng từ khóa *color* (Hình 3.7). Trong trường hợp không khai báo một màu cụ thể thì Matplotlib sẽ tự động lựa chọn một tập các màu mặc định.

```
plt.plot(x, np.sin(x - 0), color='blue') #Màu cụ thể theo tên
plt.plot(x, np.sin(x - 1), color='g') #Mã màu bằng viết tắt (rgbcmk)
plt.plot(x, np.sin(x - 2), color='0.75') #Thang xám - Grayscale từ 0 đến 1
plt.plot(x, np.sin(x - 3), color='#FFDD44') #Mã Hex (RRGGBB từ 00 - FF)
plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3)) #RGB, giá trị giữa 0 và 1
plt.plot(x, np.sin(x - 5), color='chartreuse'); #Tên màu HTML được hỗ trợ
```

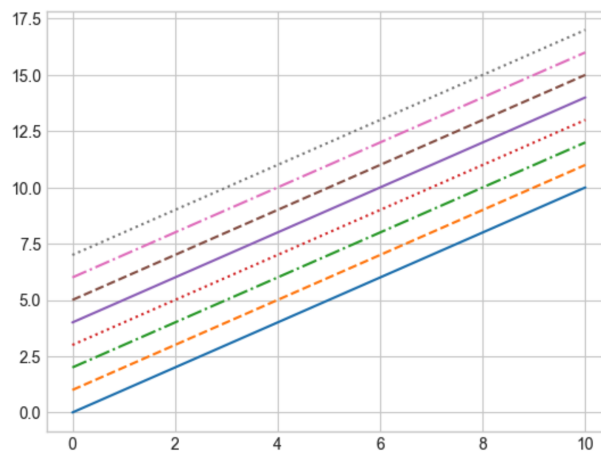
Tương tự đối với *colors*, để thay đổi dạng của đường trong đồ thị cần thông qua từ khóa *linestyle* (Hình 3.8):

```
plt.plot(x, x + 0, linestyle='solid')
plt.plot(x, x + 1, linestyle='dashed')
plt.plot(x, x + 2, linestyle='dashdot')
plt.plot(x, x + 3, linestyle='dotted');
```

Hình 3.7: Thay đổi màu sắc sử dụng hàm *plot*.

Đang viết tắt, có thể sử dụng đoạn chương trình sau:

```
plt.plot(x, x + 4, linestyle='--') # solid
plt.plot(x, x + 5, linestyle='--') # dashed
plt.plot(x, x + 6, linestyle='-.') # dashdot
plt.plot(x, x + 7, linestyle=':'); # dotted
```

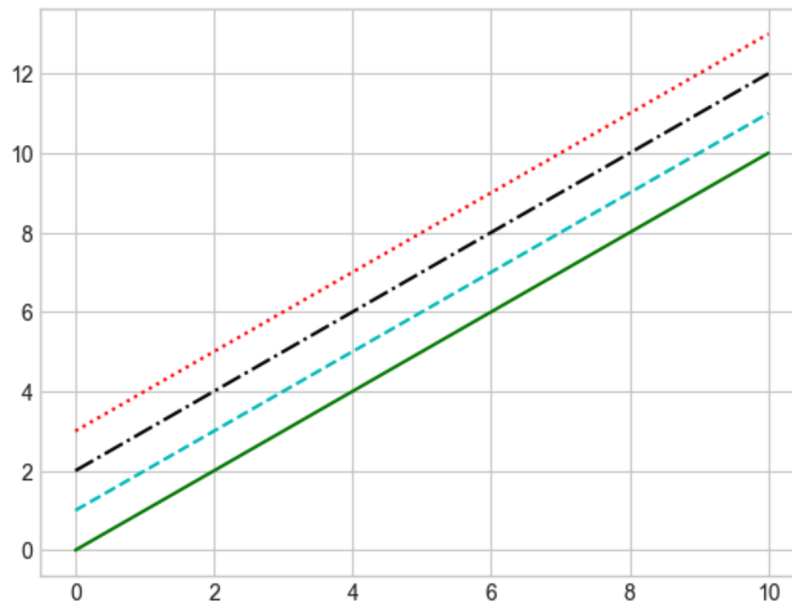


Hình 3.8: Các kiểu dạng của đồ thị đường.

Nếu muốn cực kỳ ngắn gọn, các mã màu và kiểu đường này có thể được kết hợp thành một đối số mà không phải từ khóa duy nhất cho hàm *plt.plot()* (Hình 3.9): *linestyle* (Hình 3.8):

```
plt.plot(x, x + 0, '-g') # solid green
plt.plot(x, x + 1, '--c') # dashed cyan
plt.plot(x, x + 2, '-.k') # dashdot black
```

```
plt.plot(x, x + 3, 'r:'); # dotted red
```



Hình 3.9: Điều chỉnh màu sắc và kiểu đồ thị bằng cách rút gọn.

Các mã màu gồm một ký tự tương ứng với các chữ viết tắt tiêu chuẩn trong hệ màu RGB (Red/Green/Blue) và CMYK (Cyan/Magenta/Yellow/blacK). Có nhiều từ khóa khác có thể được sử dụng để tinh chỉnh hình thức của đồ thị; để biết thêm chi tiết, có thể xem trong tài liệu của hàm `plt.plot()` bằng các công cụ trợ giúp của IPython (*help tool*).

Điều chỉnh giới hạn của trục đồ thị - Axes

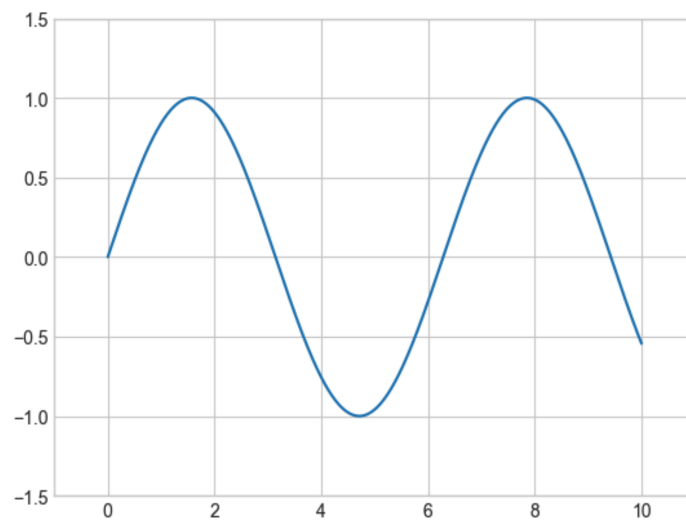
Matplotlib thực hiện tốt việc chọn giới hạn biên trục tọa độ mặc định đồ thị, nhưng việc kiểm soát được các thông số này cũng rất hữu ích. Cách cơ bản nhất để điều chỉnh giới hạn trục là sử dụng các phương thức `plt.xlim()` và `plt.ylim()` (Hình 3.10):

```
plt.plot(x, np.sin(x))
plt.xlim(-1, 11)
plt.ylim(-1.5, 1.5);
```

Phương thức `plt.axis()` cho phép đặt các giới hạn của trục x và trục y với một lần gọi, bằng cách truyền một danh sách `[xmin, xmax, ymin, ymax]` (Hình 3.11):

```
plt.plot(x, np.sin(x))
plt.axis([-1, 11, -1.5, 1.5]);
```

Phương thức `plt.axis()` còn cho phép thực hiện những việc như tự động loại bỏ



Hình 3.10: Thay đổi biên của các trục tọa độ.

phần giới hạn xung quanh đồ thị hiện tại mà không chứa đồ thị bằng sử dụng từ khoá "tight" (Hình 3.12) hay cho tỷ lệ các trục bằng nhau bằng "equal" (Hình 3.13):

```
plt.plot(x, np.sin(x))
plt.axis('tight');
```

```
plt.plot(x, np.sin(x))
plt.axis('equal');
```

Để biết thêm thông tin về giới hạn trục và các tham số của phương thức `plt.axis()`, hãy tham khảo *docstring* của `plt.axis()`.

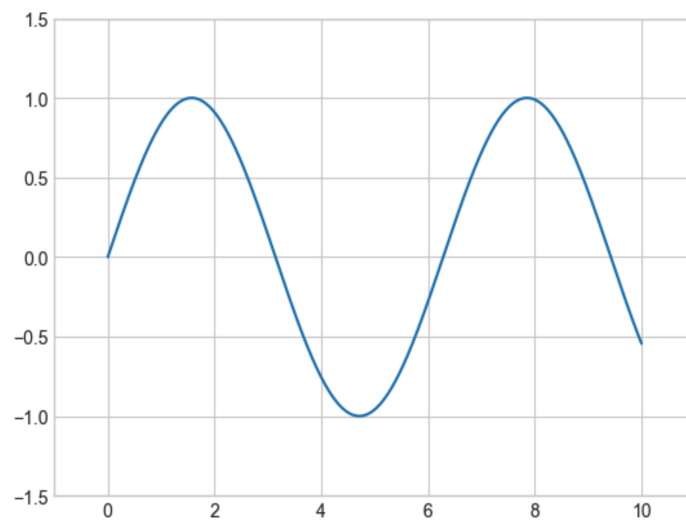
Tên đồ thị và nhãn các trục đồ thị

Nhãn các biểu đồ bao gồm tiêu đề tên đồ thị, nhãn trục và các lời chú giải. Sử dụng các phương thức dưới đây để có thể để giá trị cho tên đồ thị và nhãn trục (Hình 3.14):

```
plt.plot(x, np.sin(x))
plt.title("A Sine Curve")
plt.xlabel("x")
plt.ylabel("sin(x)");
```

Khi hiển thị nhiều đường đồ thị cần thiết việc tạo chú giải nhãn cho từng loại đường, điều này được thực hiện thông qua phương thức `plt.legend()` dưới đây (Hình 3.15):

```
plt.plot(x, np.sin(x), '-g', label='sin(x)')
```



Hình 3.11: Thay đổi biên của các trục tọa độ bằng *plt.axis()*

```
plt.plot(x, np.cos(x), 'b', label='cos(x)')
plt.legend();
```

3.3 Đồ thị điểm rời rạc

Đồ thị điểm rời rạc các điểm thay vì nối với nhau bằng đoạn thì được biểu diễn riêng lẻ bằng dấu chấm, hình tròn hoặc hình dạng khác bằng phương thức *plt.plot/ax.plot*:

```
plt.plot(x, np.sin(x), 'o', color='black');
```

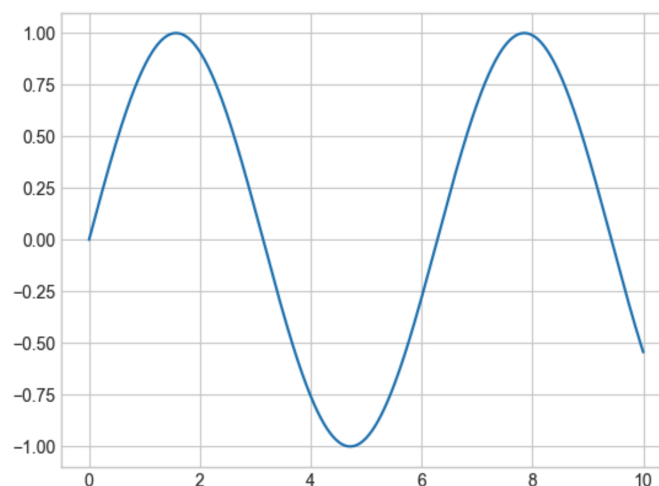
Đối số thứ ba trong *plt.plot* là đại diện cho ký hiệu sử dụng để vẽ đồ thị. Chỉ định tùy chọn như '-' và '-' cho đồ thị kiểu đường, các ký hiệu có sẵn trong tài liệu hướng dẫn sử dụng của *plt.plot*. Dưới đây là một số loại ký tự hay được sử dụng (Hình 3.17):

```
rng = np.random.RandomState(0)
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.rand(5), rng.rand(5), marker,
             label="marker='{0}'".format(marker))
plt.legend(numpoints=1)
plt.xlim(0, 1.8);
```

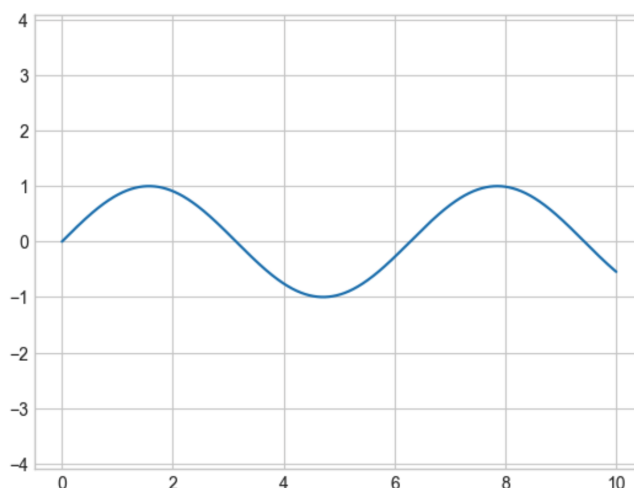
Để có nhiều thuộc tính hơn, các mã ký tự có thể được sử dụng cùng với mã đường và mã màu để vẽ các điểm cùng với nhau trên một đường (Hình 3.18):

```
plt.plot(x, y, '-sk'); # line (-), square marker (s), black (k)
```

Vẽ đồ thị điểm rời rạc bằng *plt.scatter*



Hình 3.12: Thay đổi biên của các trục tọa độ bằng *tight*.



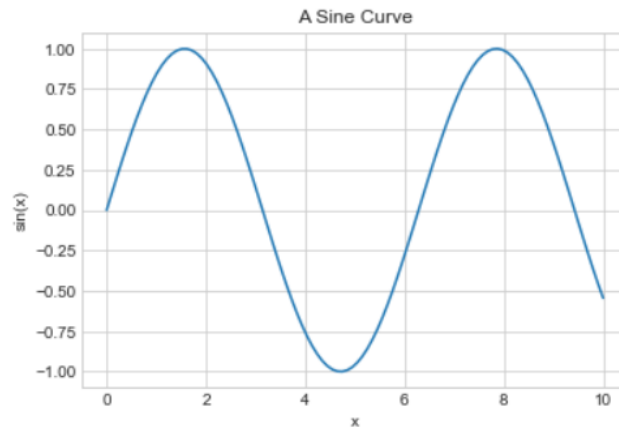
Hình 3.13: Thay đổi biên của các trục tọa độ bằng *equal*.

Phương thức thứ hai, mạnh mẽ hơn để tạo đồ thị phân tán là hàm *plt.scatter*, được sử dụng rất giống với hàm *plt.plot*:

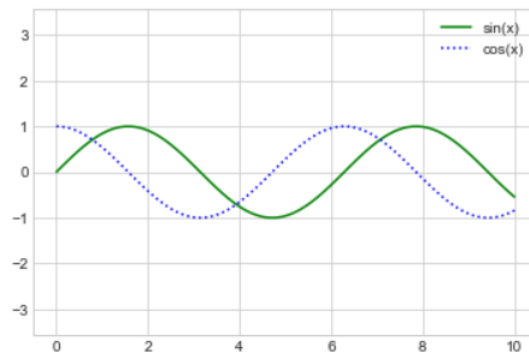
```
plt.scatter(x, y, marker='o');
```

Sự khác biệt chính của *plt.scatter* so với *plt.plot* là sử dụng để tạo đồ thị với các điểm mà các thuộc tính của từng điểm là riêng lẻ và khác nhau (kích thước, màu của từng điểm, màu cạnh bao, vv..). Đồ thị dưới đây tạo ngẫu nhiên với các điểm có nhiều màu sắc và kích cỡ khác nhau. Để thấy rõ hơn các điểm trùng nhau sử dụng từ khóa *alpha* để điều chỉnh mức độ trong suốt của các điểm (Hình 3.20):

```
rng = np.random.RandomState(0)
```



Hình 3.14: Ví dụ về đặt tên đồ thị và nhãn trục.



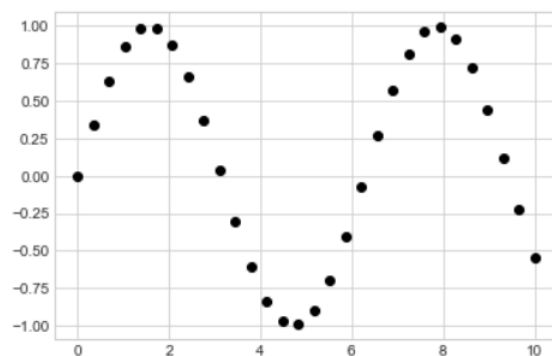
Hình 3.15: Ví dụ chú giải nhiều đường.

```
x = rng.randn(100)
y = rng.randn(100)
colors = rng.rand(100)
sizes = 1000 * rng.rand(100)

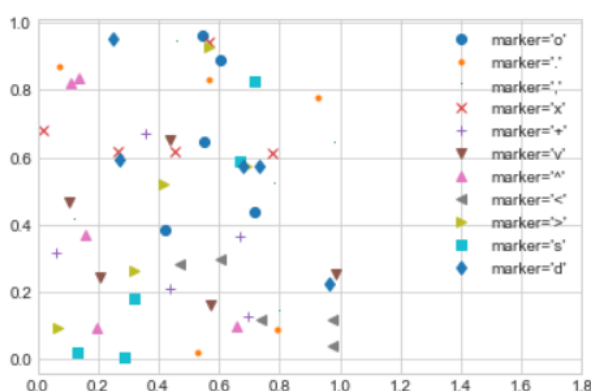
plt.scatter(x, y, c=colors, s=sizes, alpha=0.3, cmap='viridis')
plt.colorbar(); # Hien thi thang mau
```

So sánh *plot* với *scatter* về hiệu năng

Mặc dù nó không có sự khác biệt đối với lượng dữ liệu nhỏ, nhưng khi bộ dữ liệu lớn hơn vài nghìn điểm, *plt.plot* có thể hiệu quả hơn đáng kể so với *plt.scatter*. Lý do là *plt.scatter* hiển thị kích thước và/hoặc màu sắc khác nhau cho từng điểm, vì vậy phải thực hiện việc xây dựng từng điểm riêng lẻ. Trong khi *plt.plot*, các điểm về cơ bản luôn là bản sao của nhau, việc xác định hình thức các điểm chỉ được thực hiện một lần cho toàn bộ tập hợp dữ liệu. Với bộ dữ liệu lớn, *plt.plot* nên được ưu tiên hơn *plt.scatter*.



Hình 3.16: Đồ thị điểm rời rạc.



Hình 3.17: Đồ thị điểm rời rạc.

3.4 Trực quan hóa lỗi

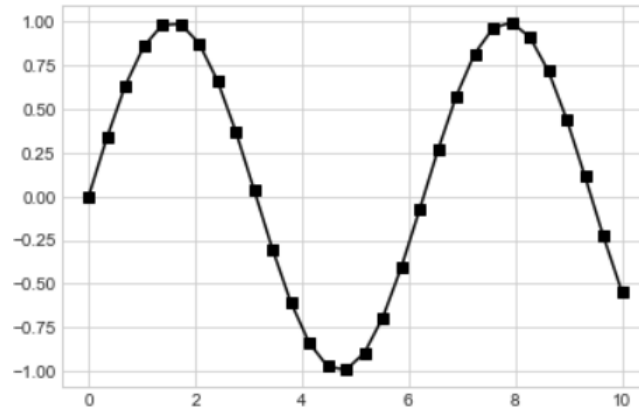
Đối với bất kỳ phép đo khoa học nào, việc tính toán chính xác các lỗi cũng quan trọng không kém việc báo cáo chính xác con số. Khi trực quan hóa dữ liệu và các kết quả, việc hiển thị các lỗi một cách hiệu quả có thể giúp đồ thị truyền tải thông tin đầy đủ hơn nhiều.

Trực quan bằng thanh lỗi

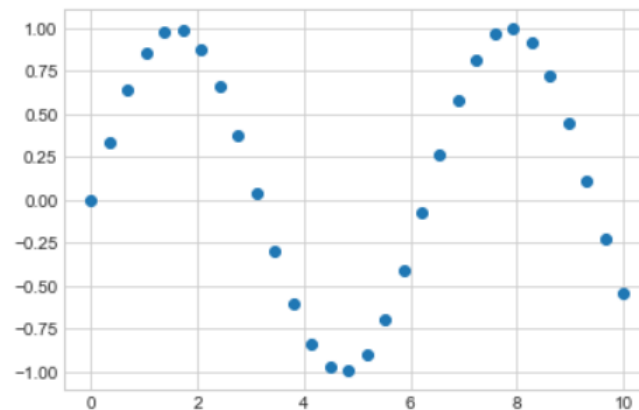
Trực quan lỗi bằng thanh được tạo bằng một lệnh gọi hàm *Matplotlib* duy nhất (Hình 3.21):

```
x = np.linspace(0, 10, 50)
dy = 0.8
y = np.sin(x) + dy * np.random.randn(50)
plt.errorbar(x, y, yerr=dy, fmt='k');
```

Ở đây, *fmt* là một mã định dạng của các đường và điểm. Ngoài các tùy chọn cơ



Hình 3.18: Đồ thị kết hợp điểm và đường.

Hình 3.19: Đồ thị điểm rời rạc sử dụng `plt.scatter`.

bản, đồ thị thanh lỗi còn có nhiều tùy chọn để tinh chỉnh đầu ra, làm cho các thanh lỗi có độ nhật hơn các điểm (Hình 3.22):

```
plt.errorbar(x, y, yerr=dy, fmt='o', color='black',
             ecolor='lightgray', elinewidth=3, capsize=0);
```

Trực quan bằng miền lỗi liên tục

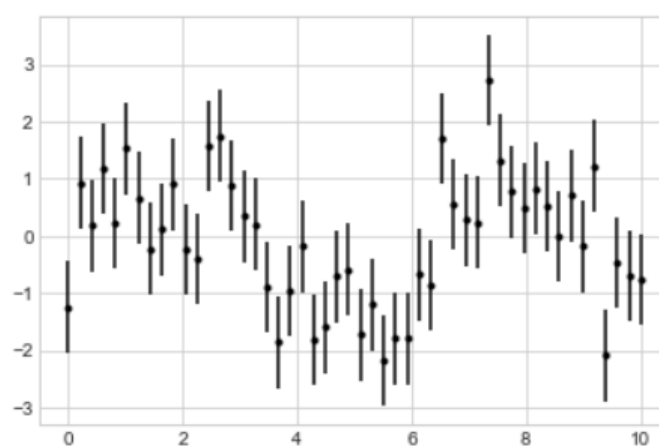
Trong một số trường hợp, mong muốn trực quan đồ thị lỗi trên miền liên tục, có thể kết hợp hàm `plt.plot` và `plt.fill_between`. Ví dụ minh họa dưới đây thực hiện hồi quy Gaussian đơn giản - *Gaussian process regression (GPR)*, sử dụng API Scikit-Learn như một phép đo lỗi trên miền liên tục như sau:

```
from sklearn.gaussian_process import GaussianProcessRegressor as GPR
from sklearn.gaussian_process.kernels import RBF

# Khởi tạo giá trị các điểm dữ liệu
```



Hình 3.20: Thay đổi kích thước, màu sắc và độ trong suốt ở các điểm phân tán.



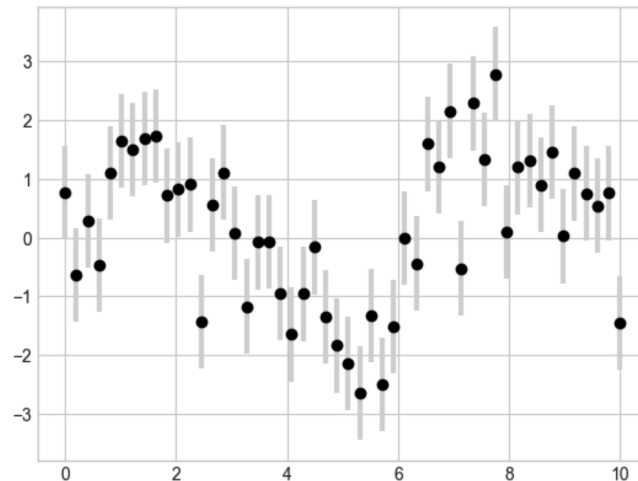
Hình 3.21: Trục quan bằng thanh lỗi.

```
model = lambda x: x * np.sin(x)
xdata = np.linspace(0, 10, 1_000).reshape(-1, 1)
ydata = model(xdata)

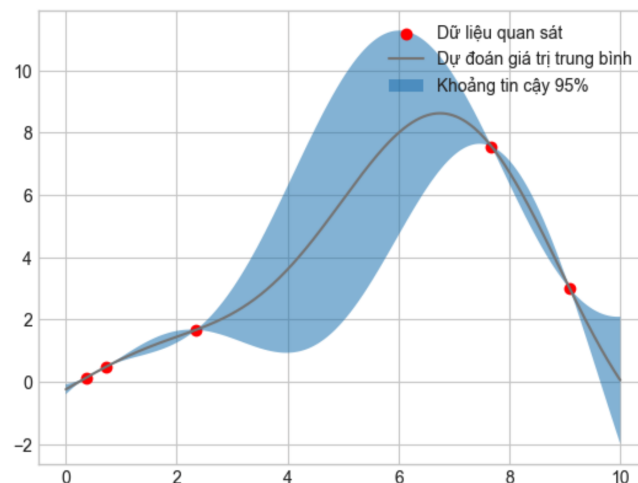
# Tính toán hồi quy Gaussian
rng = np.random.RandomState(1)
dtrain = rng.choice(np.arange(ydata.size), size=5)
X_train, y_train = xdata[dtrain], ydata[dtrain]

kernel = 1 * RBF(length_scale=1.0, length_scale_bounds=(1e-2, 1e2))
gp = GPR(kernel=kernel, n_restarts_optimizer=9)
gp.fit(X_train, y_train)
mean, std = gp.predict(X, return_std=True)
```

Ở đây X_{train} , y_{train} , $xdata$ và $mean$ để lấy mẫu sự phù hợp liên tục với dữ liệu, có thể dùng hàm `plt.errorbar` như trên, nhưng nếu không muốn vẽ 1.000 điểm với



Hình 3.22: Trực quan bằng thanh lỗi.



Hình 3.23: Trực quan miền lỗi liên tục.

1.000 với thanh lỗi. Thay vào đó có thể sử dụng hàm `plt.fill_between` với màu để trực quan lỗi liên tục này (Hình 3.23):

```
plt.scatter(X_train, y_train, label="Dữ liệu quan sát", color = 'red')
plt.plot(xdata, mean, label="Dự đoán giá trị trung bình", color = 'gray')
plt.fill_between( xdata.ravel(), mean - 1.96 * std, mean + 1.96 * std,
                  alpha=0.5, label=r"Khoảng tin cậy 95%" )
plt.legend()
```

Lưu ý với hàm `fill_between` truyền một giá trị x được vẽ, sau đó là giá trị giới hạn dưới của y (*lower y-bound*), sau đó là giá trị giới hạn trên của y (*upper y-bound*) và kết quả là miền giữa các vùng này được lấp đầy. Hình 3.23 cung cấp một cái nhìn rất trực quan về thuật toán hồi quy Gaussian đang thực hiện: ở vùng gần điểm dữ

liệu được quan sát, mô hình bị hạn chế mạnh và điều này phản ánh các lỗi mô hình nhỏ; ở những vùng cách xa điểm dữ liệu được đo, quan sát thì mô hình không bị hạn chế mạnh và sai số mô hình tăng lên.

3.5 Đồ thị đường viền

Đôi khi sẽ hữu ích khi hiển thị dữ liệu ba chiều ở không gian hai chiều bằng cách sử dụng các đường viền hoặc vùng mã màu chuyển tiếp khác nhau. Có ba hàm Matplotlib có thể hữu ích cho nhiệm vụ này: *plt.contour* cho các ô đường viền, *plt.contourf* cho các ô đường viền được tô màu và *plt.imshow* để hiển thị hình ảnh. Đầu tiên cần cách thiết lập *Notebook* để vẽ đồ thị và nhập các hàm sẽ sử dụng:

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-white')
import numpy as np
```

Trực quan hóa một hàm ba chiều

Ví dụ về một biểu đồ đường đồng mức trực quan hoá sử dụng hàm $z = f(x, y)$:

```
def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

Tạo biểu đồ đường viền bằng hàm *plt.contour* với ba đối số: các giá trị x , giá trị y và giá trị z . Các giá trị x và y đại diện cho các vị trí trên biểu đồ và giá trị z sẽ được biểu thị bằng các mức đường viền. Sử dụng hàm *np.meshgrid* xây dựng các điểm dữ liệu hai chiều từ mảng một chiều:

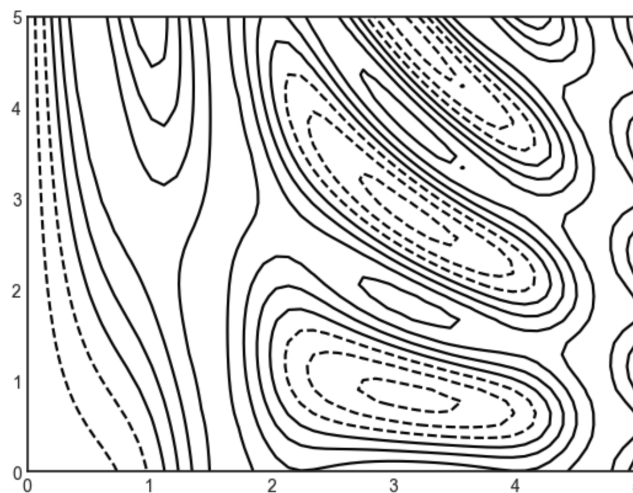
```
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)

X, Y = np.meshgrid(x, y)
Z=f(X,Y)
```

Một biểu đồ đường viền tiêu chuẩn chỉ có các đường viền đồng mức như Hình 3.24 dưới đây:

```
plt.contour(X, Y, Z, colors='black');
```

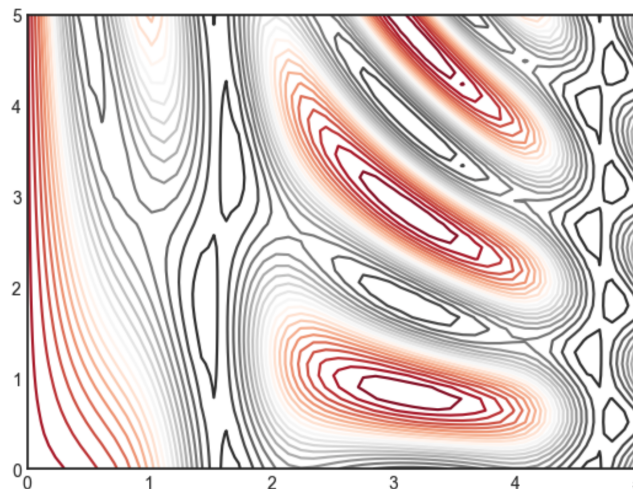
Theo mặc định khi một màu được sử dụng, các giá trị âm được biểu thị bằng các đường nét đứt và các giá trị dương được biểu thị bằng các đường liền nét. Ngoài ra, có thể đánh mã màu cho các dòng bằng cách chỉ định một bản đồ màu với đối số *cmap*. Ví dụ khi muốn vẽ nhiều đường hơn như khoảng 20 cách đều nhau trong



Hình 3.24: Trực quan hóa dữ liệu ba chiều với các đường viền.

phạm vi dữ liệu sử dụng câu lệnh sau (Hình 3.25):

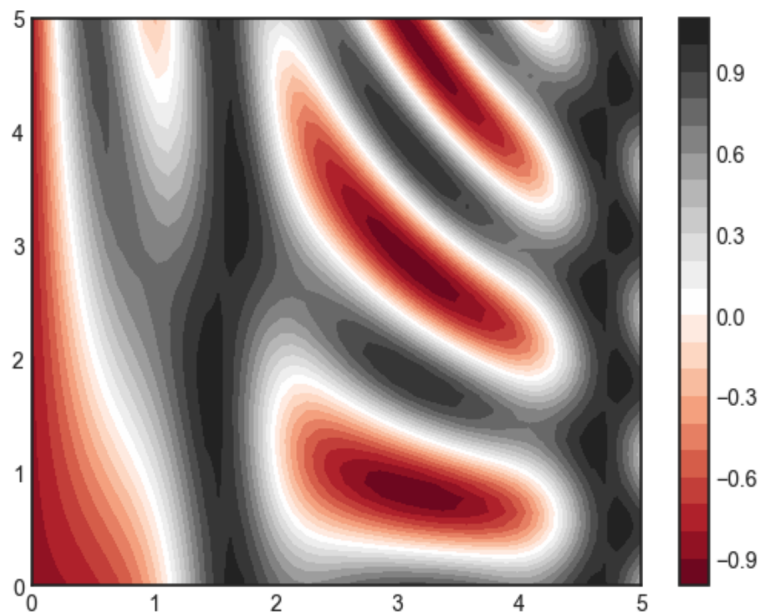
```
plt.contour(X, Y, Z, 20, cmap='RdGy');
```



Hình 3.25: Trực quan hóa dữ liệu ba chiều với các đường viền nhiều màu và nhiều mức.

Biểu đồ trông đẹp hơn so với đường viền một màu xám, nhưng khoảng cách giữa các dòng có thể làm mất tập trung. Điều này có thể thay đổi bằng cách chuyển sang biểu đồ đường viền được chuyển màu liên tục bằng cách sử dụng hàm *plt.contourf()* (chú ý *f* ở cuối), hàm này sử dụng phần lớn cú pháp giống như *plt.con tour()*. Ngoài ra, thêm một lệnh *plt.colorbar()*, lệnh này sẽ tự động tạo một trục bổ sung với thông tin màu được gắn nhãn cho biểu đồ (Hình 3.26):

```
plt.contourf(X, Y, Z, 20, cmap='RdGy')
plt.colorbar();
```



Hình 3.26: Trực quan hóa dữ liệu các đường viền chuyển màu liên tục.

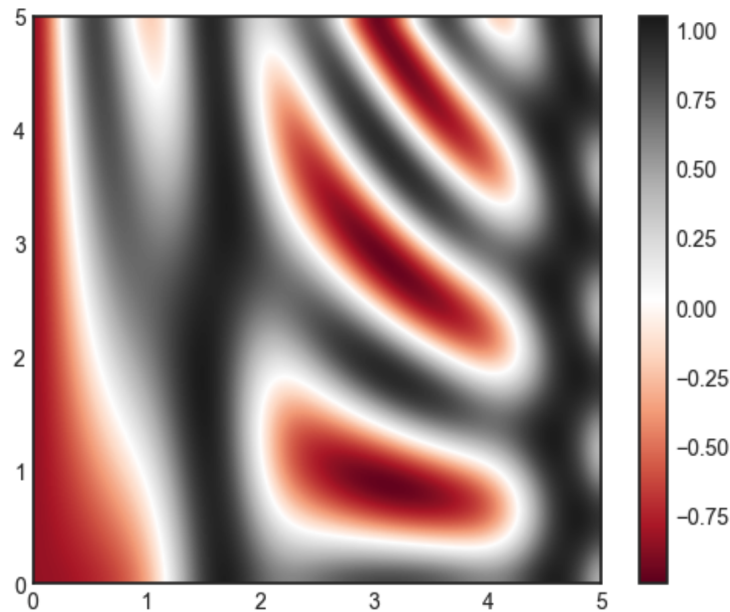
Một vấn đề tiềm ẩn với cách trực quan hoá này là các bước màu rời rạc mà không mượt mà. Điều này có thể khắc phục bằng cách đặt số lượng đường viền với số lượng lớn, nhưng nó dẫn đến kém hiệu quả do Matplotlib phải hiển thị nhiều bước. Cách tốt hơn để xử lý việc này là sử dụng hàm *plt.imshow()*, hàm này biểu diễn lưới dữ liệu hai chiều dưới dạng một hình ảnh. Hình 3.27 là kết quả trực quan hoá bằng mã lệnh dưới đây:

```
plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',
           cmap='RdGy', interpolation="bicubic")
plt.colorbar()
```

Các tham số trong *imshow()* bao gồm:

- *plt.imshow()* nhận phạm vi các giá trị [*xmin*, *xmax*, *ymin*, *ymax*] của hình ảnh trên biểu đồ.
- *plt.imshow()* theo mặc định tuân theo định nghĩa mảng hình ảnh tiêu chuẩn trong đó điểm gốc nằm ở phía trên bên trái, không phải ở phía dưới bên trái như trong hầu hết các biểu đồ đường viền. Điều này phải được thay đổi khi hiển thị dữ liệu dạng lưới.
- *plt.imshow()* sẽ tự động điều chỉnh tỷ lệ khung hình trục để khớp với dữ liệu đầu vào. Ngoài ra, có thể sử dụng Phép nội suy hai khối - *Bicubic interpolation*

được sử dụng khi phóng to ảnh vì thực tế các biểu đồ có xu hướng pixel mờ hơn sẽ được sử dụng yêu thích sử dụng nhiều hơn.



Hình 3.27: Trực quan hóa dữ liệu các đường viền liên tục.

Đôi khi đối với biểu đồ đường viền cần kết hợp với các đường đồng mức như trực quan của Hình 3.28. Biểu đồ này sử dụng một ảnh nền trong suốt một phần (với độ trong suốt được thiết lập thông qua tham số *alpha*) và vẽ chồng lên các đường viền đồng mức bằng nhãn trên chính các đường viền đó (sử dụng hàm `plt.clabel()`):

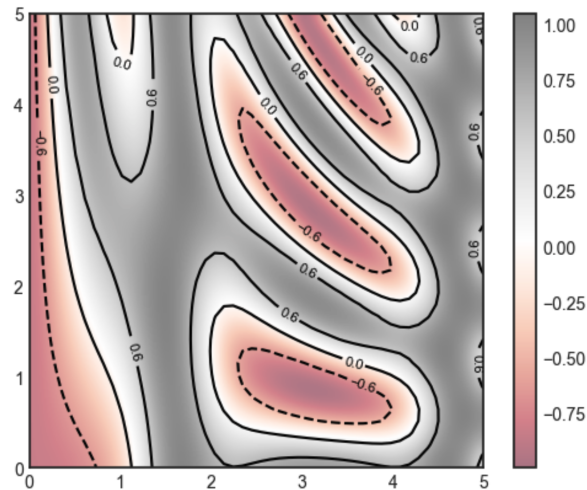
```
contours = plt.contour(X, Y, Z, 3, colors='black')
plt.clabel(contours, inline=True, fontsize=8)

plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower', cmap='RdGy',
           alpha=0.5, interpolation="bicubic")
plt.colorbar();
```

Sự kết hợp của ba hàm `plt.contour`, `plt.contourf` và `plt.imshow` tạo ra khả năng gần như vô hạn để hiển thị loại dữ liệu ba chiều trong một đồ thị đường viền hai chiều. Để biết thêm các tùy chọn có sẵn trong các hàm này, hãy tham khảo tài liệu hướng dẫn của các hàm trên.

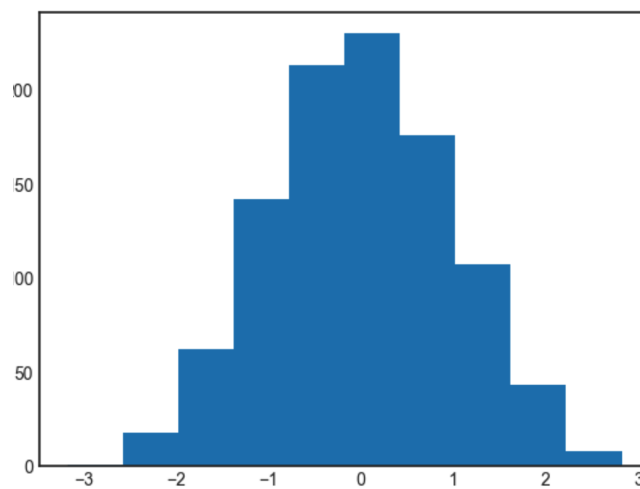
3.6 Histograms và mật độ

Histograms là một biểu đồ đơn giản để hiểu tập dữ liệu, đồ thị cơ bản được trực quan hoá bằng một dòng mã lệnh, sau khi nhập một tập dữ liệu như sau:



Hình 3.28: Trực quan hóa dữ liệu các đường viền liên tục và đường đồng mức.

```
data = np.random.randn(1000)
plt.hist(data);
```



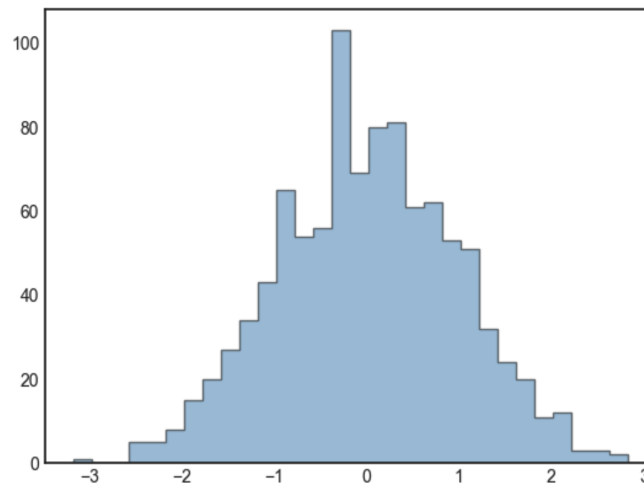
Hình 3.29: Một đồ thị *Histogram* đơn giản.

Hàm `hist()` có nhiều tùy chọn để điều chỉnh dưới đây là một ví dụ về đồ thị Histogram tùy chỉnh hơn (Hình 3.30):

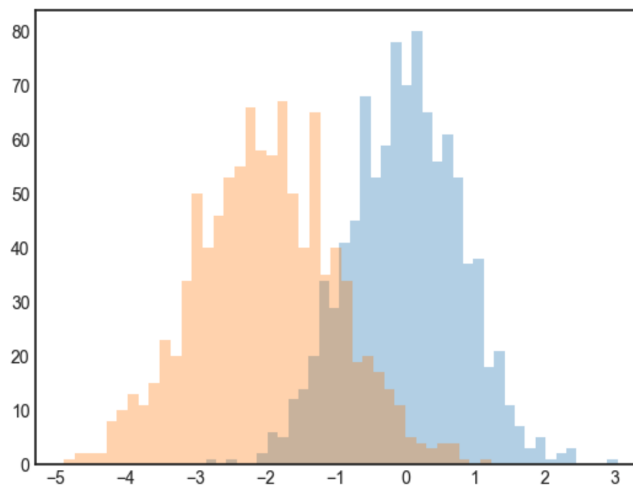
```
plt.hist(data, bins=30, alpha=0.5, histtype='stepfilled',
         color='steelblue', edgecolor='none');
```

Có thể kết hợp `histtype='stepfills'` cùng với hệ số trong suốt `alpha` khi so sánh biểu đồ của nhiều dữ liệu phân phối khác nhau:

```
x1 = np.random.normal(0, 0.8, 1000)
```

Hình 3.30: Một đồ thị *Histogram* tùy chỉnh.

```
x2 = np.random.normal(-2, 1, 1000)
kwargs = dict(histtype='stepfilled', alpha=0.3, bins=40)
plt.hist(x1, **kwargs)
plt.hist(x2, **kwargs)
```

Hình 3.31: Một đồ thị *Histogram* kết hợp.

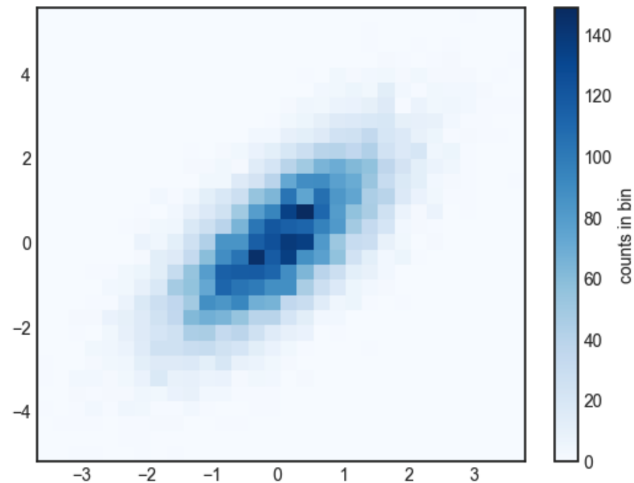
Đồ thị histograms hai chiều và biểu đồ mật độ

Giống như tạo biểu đồ trong một chiều bằng cách chia số dòng thành các ngăn. Sử dụng dữ liệu gồm x và y được rút ra từ phân phối Gaussian đa biến như sau:

```
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
```

Một cách đơn giản để trực quan hoá histograms hai chiều là sử dụng hàm `plt.hist2d`:

```
plt.hist2d(x, y, bins=30, cmap='Blues')
cb = plt.colorbar()
cb.set_label('counts in bin')
```



Hình 3.32: Đồ thị *histogram* hai chiều.

Biểu đồ histograms hai chiều tạo ra một dãy các ô vuông trên các trục, ngoài ra có thể khác sử dụng các ô vuông này thành hình lục giác đều cho tự nhiên hơn. Với mục đích này, Matplotlib cung cấp một gói `plt.hexbin`, đại diện cho một tập dữ liệu hai chiều được đánh dấu trong một lưới các hình lục giác (Hình 3.33):

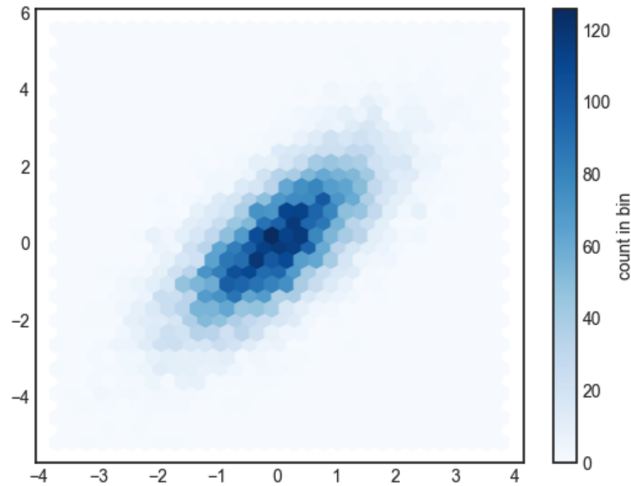
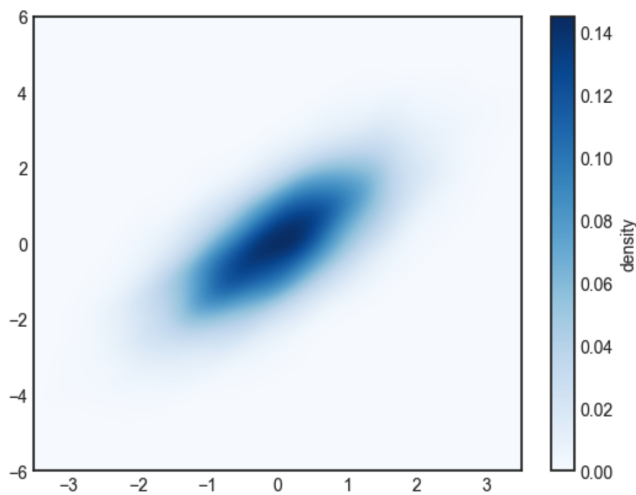
```
plt.hexbin(x, y, gridsize=30, cmap='Blues')
cb = plt.colorbar(label='count in bin')
```

`plt.hexbin` có một số tùy chọn khác bao gồm khả năng chỉ định trọng số cho từng điểm và thay đổi đầu ra trong mỗi ngăn thành bất kỳ tổng hợp *NumPy* nào (giá trị trung bình của trọng số, độ lệch chuẩn của trọng số, v.v.).

Ước tính mật độ nhân

Một phương pháp phổ biến khác để đánh giá mật độ theo nhiều chiều là ước tính mật độ nhân (*KDE*). KDE có thể được coi là một cách để “làm mờ” các điểm trong không gian và thêm vào kết quả để có được một hàm làm mịn dữ liệu. Có cách triển khai KDE cực kỳ nhanh chóng và đơn giản trong gói `scipy.stats`. Dưới đây là một ví dụ nhanh về việc sử dụng KDE trên dữ liệu này (Hình 3.34):

```
from scipy.stats import gaussian_kde
# Tạo dữ liệu bằng một mảng có kích thước [Ndim, Nsamples]
data = np.vstack([x, y])
```

Hình 3.33: Đồ thị *histogram* hai chiều.Hình 3.34: Đồ thị *histogram* hai chiều.

```

kde = gaussian_kde(data)
# Danh gia tren mot dai gia tri chinh quy
xgrid = np.linspace(-3.5, 3.5, 40)
ygrid = np.linspace(-6, 6, 40)
Xgrid, Ygrid = np.meshgrid(xgrid, ygrid)
Z = kde.evaluate(np.vstack([Xgrid.ravel(), Ygrid.ravel()]))
# Truc quan bang imshow
plt.imshow(Z.reshape(Xgrid.shape), origin='lower', aspect='auto',
            extent=[-3.5, 3.5, -6, 6], cmap='Blues',
            interpolation="bicubic")
cb = plt.colorbar()
cb.set_label("density")

```


3.7 Đồ thị ba chiều

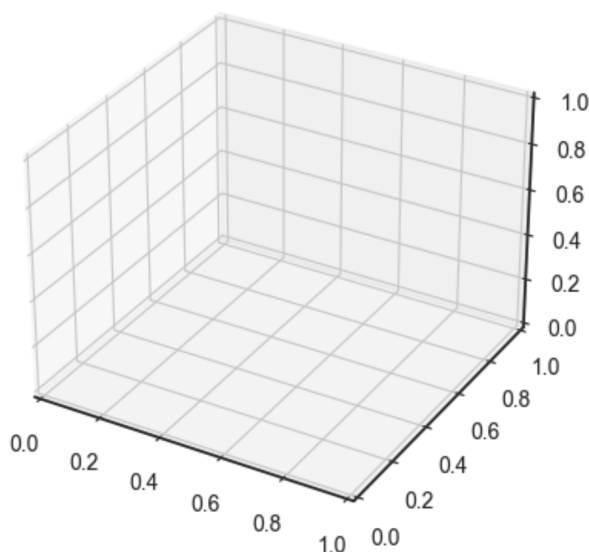
Matplotlib ban đầu được thiết kế chỉ với mục đích vẽ đồ thị hai chiều. Từ phiên bản 1.0, một số gói vẽ sơ đồ ba chiều đã được xây dựng trên màn hình hai chiều của *Matplotlib* để trực quan hóa dữ liệu ba chiều. Để sử dụng biểu đồ ba chiều bằng cách nhập bộ công cụ *mplot3d* (Hình 3.35):

```
from mpl_toolkits import mplot3d
```

Sau khi mô-đun này được nhập, có thể tạo các trục ba chiều bằng cách chuyển từ khóa *projector='3d'* cho bất kỳ quá trình trực quan hoá tạo trục nào:

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()
ax = plt.axes(projection='3d')
```



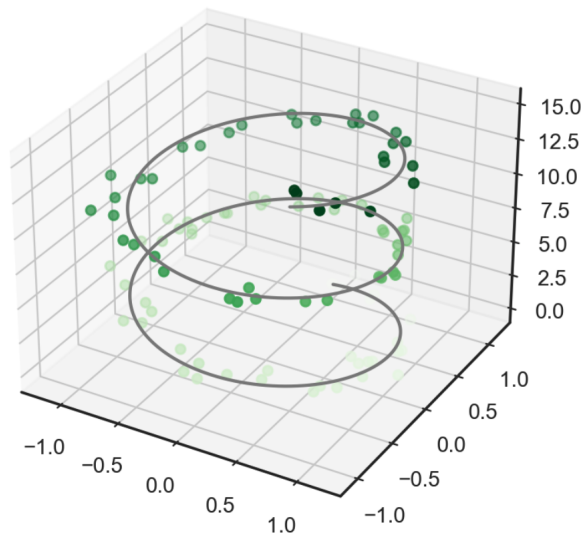
Hình 3.35: Khởi tạo trục đồ thị 3D.

Chú ý 3.1 Với các trục 3D này có thể vẽ nhiều loại đồ thị ba chiều khác nhau. Đồ thị ba chiều có một chức năng mang lại lợi ích to lớn từ việc xem các số liệu đó là tương tác với đồ thị thay vì dữ liệu tĩnh trong *Notebook*; để sử dụng các số liệu tương tác, có thể sử dụng *%matplotlib notebook* thay vì *%matplotlib inline* khi chạy mã nguồn trực quan hoá các đồ thị 3D này.

Điểm và đường ba chiều

Đồ thị ba chiều đơn giản nhất là một biểu đồ đường hoặc phân tán các điểm dữ liệu được tạo từ các bộ ba (x, y, z) . Tương tự với các biểu đồ hai chiều, trực quan hoá đồ thị 3 chiều bằng cách sử dụng các hàm *ax.plot3D* và *ax.scatter3D*. Ví dụ minh hoạ dưới đây sẽ vẽ một đường xoắn ốc lượng giác, cùng với một số điểm được vẽ ngẫu nhiên gần đường đó (Hình 3.36):

```
%matplotlib notebook
ax = plt.axes(projection='3d')
# Dữ liệu cho đường 3 chiều
zline = np.linspace(0, 15, 1000)
xline = np.sin(zline)
yline = np.cos(zline)
ax.plot3D(xline, yline, zline, 'gray')
# Dữ liệu cho các điểm rơi rác 3 chiều
zdata = 15 * np.random.random(100)
xdata = np.sin(zdata) + 0.1 * np.random.randn(100)
ydata = np.cos(zdata) + 0.1 * np.random.randn(100)
ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='Greens');
```



Hình 3.36: Đồ thị điểm và đường ba chiều.

Trực quan hoá đường viền ba chiều

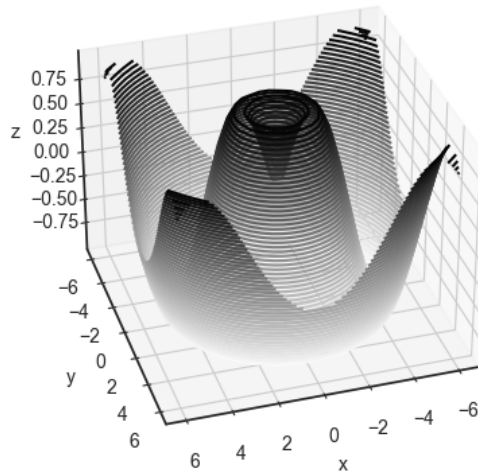
Tương tự như các biểu đồ đường viền, *mplot3d* chứa các công cụ để tạo các biểu đồ viền ba chiều bằng cách sử dụng cùng các đầu vào. Giống như các đồ thị *ax.contour* hai chiều, *ax.contour3D* yêu cầu tất cả dữ liệu đầu vào phải ở dạng lưới thông

thường hai chiều, với dữ liệu Z được đánh giá tại mỗi điểm. Ví dụ dưới đây là một đồ thị đường bao ba chiều của một hàm hình sin ba chiều (Hình 3.37):

```
def f(x, y):
    return np.sin(np.sqrt(x ** 2 + y ** 2))

x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)
Z=f(X,Y)

ax = plt.axes(projection='3d')
ax.contour3D(X, Y, Z, 50, cmap='binary')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z');
```



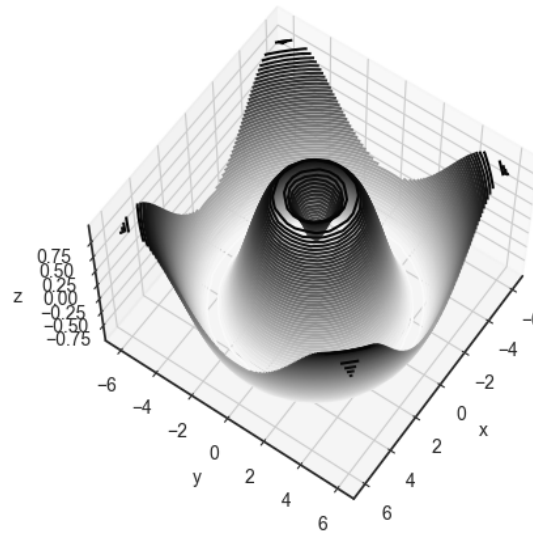
Hình 3.37: Đồ thị đường viền ba chiều.

Khi sử dụng *%matplotlib notebook* có thể xoay các góc đồ thị 3 chiều theo nhiều các góc khác nhau, trong trường hợp muốn quay một góc hoặc độ cao đồ thị mặc định sẵn có thể sử dụng phương thức *view_init* để đặt độ cao và góc phương vị của đồ thị. Trong ví dụ trên để hiển thị độ cao 60 độ so với mặt phẳng x - y và góc phương vị 35 độ nghĩa là xoay 35 độ ngược chiều kim đồng hồ về trục z (Hình 3.38):

```
ax.view_init(60, 35)
fig
```

Khung lưới - wireframe và bề mặt

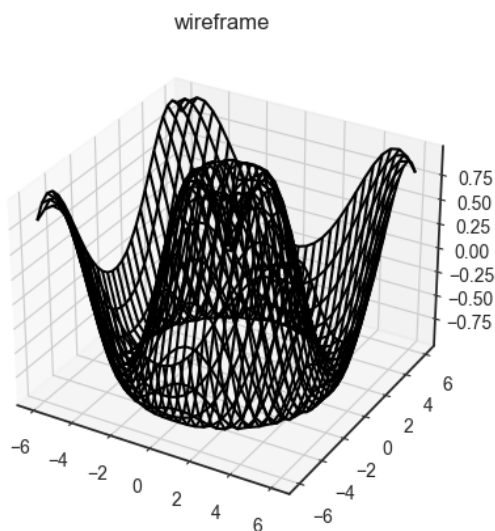
Hai loại biểu đồ ba chiều khác hoạt động trên dữ liệu dạng lưới là khung lưới - *wireframe* và biểu đồ bề mặt - *surface*. Dưới đây là một ví dụ sử dụng *wireframe*



Hình 3.38: Đồ thị đường viền ba chiều tùy chỉnh góc mặc định.

(Hình 3.39):

```
ax = plt.axes(projection='3d')
ax.plot_wireframe(X, Y, Z, color='black')
ax.set_title('wireframe');
```

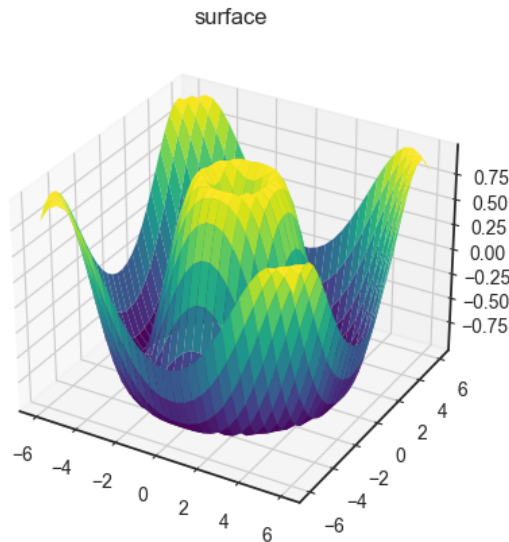


Hình 3.39: Đồ thị khung lưới *wireframe*.

Biểu đồ bề mặt - *surface* giống như biểu đồ khung lưới - *wireframe*, nhưng mỗi mặt của *wireframe* là một đa giác được lấp đầy. Việc thêm một bản đồ màu (*colormap*) vào các đa giác được hỗ trợ bằng các tham số về cấu trúc của bề mặt (Hình 3.40):

```
ax = plt.axes(projection='3d')
```

```
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,
               cmap='viridis', edgecolor='none')
ax.set_title('surface');
```



Hình 3.40: Đồ thị bề mặt *surface*.

Lưu ý mặc dù lưới các giá trị cho biểu đồ bề mặt cần phải là hai chiều, nhưng nó không nhất thiết phải là các giá trị tuyến tính. Dưới đây là một ví dụ về việc tạo một lưới phân cực một phần, mà khi được sử dụng với biểu đồ *surface3D* có thể tạo một lát cắt khi trực quan hoá (Hình 3.41):

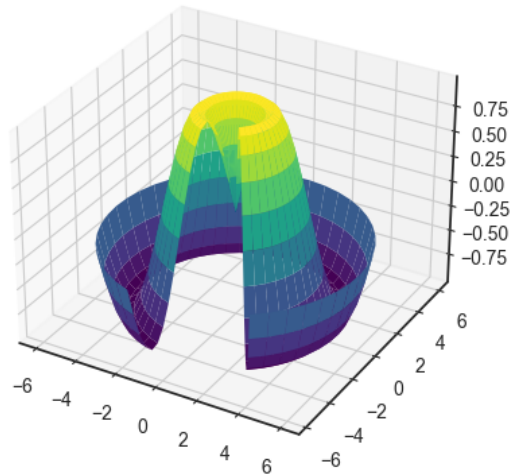
```
r = np.linspace(0, 6, 20)
theta = np.linspace(-0.9 * np.pi, 0.8 * np.pi, 40)
r, theta = np.meshgrid(r, theta)

X = r * np.sin(theta)
Y = r * np.cos(theta)
Z=f(X,Y)

ax = plt.axes(projection='3d')
ax.plot_surface(X, Y, Z, rstride=1, cstride=1,
               cmap='viridis', edgecolor='none');
```

Ví dụ: Trực quan hóa dải Möbius

Một dải Möbius tương tự như một mảnh giấy hình chữ nhật dài được dán vào một vòng xoắn nửa (*half-twist*). Về mặt tô pô, nó khá thú vị bởi vì bề ngoài nó chỉ có một mặt! Ở đây nó sẽ trực quan hóa ba chiều bằng *Matplotlib*. Để tạo dải Möbius là tạo tham số hóa cho nó: đó là dải dữ liệu hai chiều, gọi các điểm này là θ , nằm



Hình 3.41: Đồ thị bề mặt cực - *polar surface*.

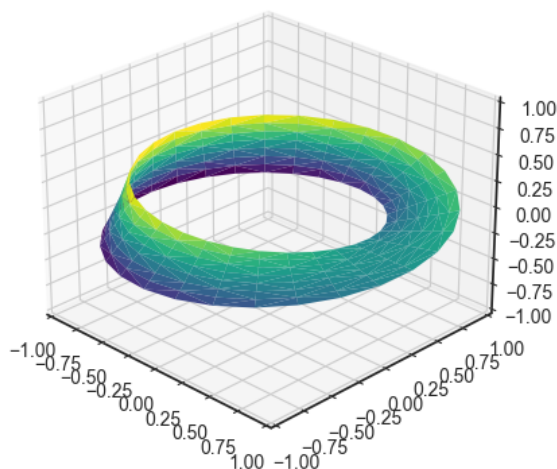
trong khoảng từ 0 đến 2π xung quanh vòng lặp và w nằm trong khoảng từ -1 đến 1 trên chiều rộng của dải:

```
from matplotlib.tri import Triangulation
tri = Triangulation(np.ravel(w), np.ravel(theta))

ax = plt.axes(projection='3d')
ax.plot_trisurf(x, y, z, triangles=tri.triangles,
                cmap='viridis', linewidths=0.2);

ax.set_xlim(-1, 1);
ax.set_ylim(-1, 1);
ax.set_zlim(-1, 1);
```

Một dải Mobius có hai chuyển động quay đang xảy ra: một là vị trí của vòng dây quanh tâm của nó (gọi là θ), hai là sự xoắn của dải quanh trục của nó (gọi là ϕ). Đối với dải Mobius, phải để dải xoắn một nửa trong toàn bộ vòng lặp hay nói cách khác $\Delta\phi = \Delta\theta/2$. Giá trị r xác định khoảng cách của mỗi điểm đến trung tâm và sử dụng giá trị này để tìm tọa độ x, y, z . Sử dụng *Triangulation* để xác định phép đo tam giác và chiếu phép đo tam giác vào không gian ba chiều của dải Mobius (Hình 3.42):



Hình 3.42: Trục quan hóa dải Mobius.

3.8 Dữ liệu địa lý

Một loại trục quan hóa phổ biến trong khoa học dữ liệu là dữ liệu địa lý. Công cụ chính của *Matplotlib* cho kiểu trục quan hóa này là bộ công cụ Bản đồ cơ sở - *Basemap*, đây là một trong một số bộ công cụ của *Matplotlib* nằm trong miền *mpl_toolkits*. Bên cạnh đó hiện nay API Google Maps được sử dụng phổ biến và là lựa chọn tốt hơn để trục quan hóa bản đồ chuyên sâu. Tuy nhiên, *Basemap* vẫn là một công cụ hữu ích để người dùng Python có trong bộ công cụ ảo. Phần này, sẽ trình bày một số ví dụ về kiểu trục quan hóa bản đồ có thể thực hiện được với bộ công cụ *Basemap*, do đó, cần cài đặt thư viện *basemap* trước khi tiến hành thực thi việc trục quan hóa dữ liệu địa lý.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap

fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='ortho', resolution=None, lat_0=50, lon_0=-100)
m.blumapble(scale=0.5);
```

Basemap không chỉ hiển thị được một hình ảnh bản đồ trái đất đơn thuần mà nó còn là một hàm của *Matplotlib* có đầy đủ chức năng từ các tọa độ cho phép dễ dàng



Hình 3.43: Phép chiếu "*bluemarble*" của Trái đất.

vẽ các dữ liệu các địa điểm trên bản đồ. Ví dụ: có thể sử dụng một phép chiếu bản đồ địa điểm khác như phóng to Bắc Mỹ và vẽ vị trí của Seattle (Hình 3.44):



Hình 3.44: Vẽ dữ liệu và nhãn trên bản đồ.

```
fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='lcc', resolution=None,
            width=8E6, height=8E6,
            lat_0=45, lon_0=-100,)
m.etopo(scale=0.5, alpha=0.5)

# Anh xa (long, lat) sang (x, y) de truc quan hoa
x, y = m(-122.3, 47.6)
```



```
plt.plot(x, y, 'ok', markersize=5)
plt.text(x, y, 'Seattle', fontsize=12);
```

Phép chiếu bản đồ

Đối với dữ liệu bản đồ việc lựa chọn phép chiếu là điều đầu tiên cần quyết định để sử dụng. Tùy thuộc vào mục đích sử dụng của phép chiếu có một số thuộc tính nhất định của bản đồ như hướng, diện tích, khoảng cách, hình dạng...

Gói *Basemap* thực hiện các phép chiếu như vậy, tất cả được tham chiếu bằng một mã định danh. Có thể bắt đầu bằng cách xác định một phương thức đơn giản để vẽ bản đồ với các điểm chỉ bằng các đường kinh độ và vĩ độ với hàm *draw_map* như sau:

```
from itertools import chain
def draw_map(m, scale=0.2):
    # Vẽ một ảnh bóng địa hình
    m.shadedrelief(scale=scale)

    # lats và longs được trả về như một tu dien - dict
    lats = m.drawparallels(np.linspace(-90, 90, 13))
    lons = m.drawmeridians(np.linspace(-180, 180, 13))

    # Các khoa chưa các màu plt.Line2D
    lat_lines = chain(*(tup[1][0] for tup in lats.items()))
    lon_lines = chain(*(tup[1][0] for tup in lons.items()))
    all_lines = chain(lat_lines, lon_lines)

    # Thiết lập các kiểu tương ứng cho các dòng
    for line in all_lines:
        line.set(linestyle='-', alpha=0.3, color='w')
```

Phép chiếu hình trụ - *Cylindrical projections*

Phép chiếu bản đồ đơn giản nhất là các phép chiếu hình trụ *Cylindrical projections*, trong đó các đường có vĩ độ và kinh độ không đổi lần lượt được ánh xạ tới các đường ngang và đường dọc. Kiểu lập bản đồ này thể hiện ở các vùng xích đạo khá tốt, nhưng dẫn đến sự biến dạng nhiều hơn ở gần các cực. Khoảng cách của các đường vĩ tuyến khác nhau giữa các hình chiếu hình trụ khác nhau, dẫn đến sự bảo toàn các đặc tính khác nhau và sự biến dạng không đồng đều ở các cực. Trong Hình 3.45, chỉ ra một ví dụ về phép chiếu hình trụ đều, chọn một vĩ độ chia tỷ lệ bảo toàn khoảng cách dọc theo các kinh tuyến. Các phép chiếu hình trụ khác là phép chiếu *Mercator* (projection = 'merc') và phép chiếu có diện tích hình trụ bằng nhau (projection = 'cea').

```
fig = plt.figure(figsize=(8, 6), edgecolor='w')
m = Basemap(projection='cyl', resolution=None,
            llcrnrlat=-90, urcrnrlat=90,
            llcrnrlon=-180, urcrnrlon=180, )
draw_map(m)
```



Hình 3.45: Phép chiếu hình trụ.

Các đối số bổ sung của *Basemap* cho chế độ này như chỉ định vĩ độ (*lat*) và kinh độ (*lon*) của góc dưới bên trái (*llcrnr*) và góc trên bên phải (*urcrnr*) cho bản đồ mong muốn, theo đơn vị độ.

Phép chiếu trụ giả - *Pseudo-cylindrical projections*

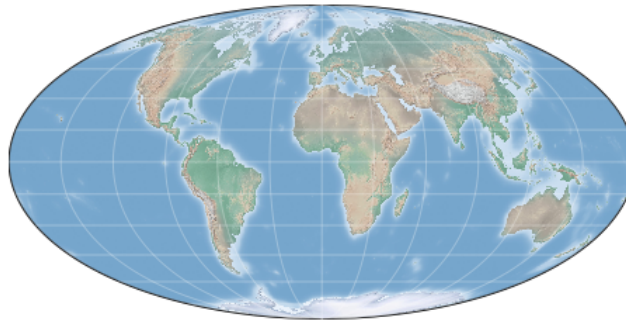
Các phép chiếu trụ giả nói lỏng yêu cầu các kinh tuyến (đường kinh độ không đổi) vẫn thẳng đứng; điều này có thể mang lại những đặc tính tốt hơn ở gần các cực của phép chiếu. Phép chiếu *Mollweide* (phép chiếu='moll') là một ví dụ phổ biến về phép chiếu trụ giả, trong đó tất cả các kinh tuyến đều là các cung hình elip (Hình 3.46). Nó được xây dựng để bảo toàn diện tích trên bản đồ: mặc dù vẫn có sự biến dạng gần các cực, diện tích của các mảng nhỏ phản ánh diện tích thực. Các phép chiếu trụ giả khác có thể kể đến như phép chiếu hình sin (projection='sinu') và phép chiếu Robinson (projection='robin').

```
fig = plt.figure(figsize=(8, 6), edgecolor='w')
m = Basemap(projection='moll', resolution=None,
            lat_0=0, lon_0=0)
draw_map(m)
```

Các đối số bổ sung cho *Basemap* là vĩ độ trung tâm (*lat_0*) và kinh độ (*lon_0*) cho bản đồ.

Phép chiếu phối cảnh - *Perspective projections*

Các phép chiếu phối cảnh được xây dựng bằng cách sử dụng một lựa chọn điểm phối cảnh cụ thể, tương tự như khi chụp ảnh Trái đất từ một điểm cụ thể trong không



Hình 3.46: Phép chiếu Mollweide.

gian. Một ví dụ phổ biến là phép chiếu trực giao (projection = *'ortho'*), cho thấy một mặt của Trái đất khi nhìn từ một điểm ở khoảng cách rất xa. Do đó, nó chỉ có thể hiển thị một nửa địa cầu tại một thời điểm. Các phép chiếu dựa trên phối cảnh khác bao gồm phép chiếu hải đồ Gnomonic (projection = *'gnom'*) và phép chiếu lập thể (projection = *'stere'*). Đây thường là đơn giản nhất để hiển thị các phần nhỏ của bản đồ.

```
fig = plt.figure(figsize=(8, 8))  
m = Basemap(projection='ortho', resolution=None,  
             lat_0=50, lon_0=0)  
draw_map(m)
```



Hình 3.47: Phép chiếu trực giao.

Phép chiếu hình nón - *Conic projections*

Phép chiếu hình nón chiếu bản đồ lên một hình nón duy nhất, sau đó hình nón này sẽ được mở ra. Điều này có thể dẫn đến các đặc tính cục bộ rất tốt, nhưng các vùng ở xa tiêu điểm của hình nón có thể trở nên rất méo mó. Một ví dụ về điều này là phép chiếu hình nón Lambert (projection = 'lcc'), mà đã biểu diễn trước đó trên bản đồ Bắc Mỹ. Nó chiếu bản đồ lên một hình nón được sắp xếp theo cách sao cho hai điểm tương đồng (được chỉ định trong Bản đồ cơ sở bằng *lat_1* và *lat_2*) có khoảng cách được thể hiện rõ ràng, với tỷ lệ giảm dần giữa hai điểm và tăng dần khi xa hai điểm. Các phép chiếu hình nón hữu ích khác là phép chiếu hình nón đều (projection = 'eqdc') và phép chiếu các diện tích bằng nhau *Albers* (projection = 'aea') (Hình 3.48). Các phép chiếu hình nón, giống như các phép chiếu phối cảnh, có xu hướng là những lựa chọn tốt để biểu diễn các mảng nhỏ đến trung bình của địa cầu.

```
fig = plt.figure(figsize=(8, 8))
m = Basemap(projection='lcc', resolution=None,
             lon_0=0, lat_0=50, lat_1=45, lat_2=55,
             width=1.6E7, height=1.2E7)
draw_map(m)
```



Hình 3.48: Phép chiếu Alberse.

Các phép chiếu khác:

Trong trường hợp cần các yêu cầu khác với trực quan hóa bản đồ, có thể đọc thêm các gói trong *Basemap*.

Vẽ nền bản đồ

Trong các phần trên đã đưa ra các phương thức *bluemarble()* và *shadedrelief()* để chiếu các hình ảnh toàn cầu trên bản đồ, cũng như các phương thức *drawparallels()* và *drawmeridians()* để vẽ các đường kinh độ và vĩ độ không đổi. Gói *Basemap* chứa một loạt các chức năng hữu ích để vẽ đường viền của các đối tượng địa lý như lục địa, đại dương, hồ và sông, cũng như các ranh giới chính trị như quốc gia, tiểu bang và quận của Hoa Kỳ. Dưới đây là một số chức năng vẽ có sẵn bằng cách sử dụng các thuộc tính từ IPython:

- Ranh giới vật lý và vùng nước:

drawcoastlines(): Vẽ đường bờ biển lục địa

drawlsmask(): Vẽ mặt nạ giữa đất liền và biển, để sử dụng với việc chiếu hình ảnh lên mặt này hay mặt khác.

drawmapboundary(): Vẽ ranh giới bản đồ, bao gồm tô màu cho các đại dương.

drawrivers(): Vẽ sông trên bản đồ.

fillcontinents(): Tô màu các lục địa với một màu nhất định; tùy chọn với hồ với màu khác.

- Ranh giới chính trị:

drawcountries(): Vẽ ranh giới quốc gia.

drawstates(): Vẽ ranh giới tiểu bang Hoa Kỳ.

drawcounties(): Vẽ ranh giới quận của Hoa Kỳ.

- Thuộc tính của bản đồ:

drawgreatcircle(): Vẽ một vòng tròn lớn giữa hai điểm.

drawparallels(): Vẽ các đường vĩ tuyến không đổi.

drawmeridians(): Vẽ các đường kinh tuyến không đổi.

drawmapscale(): Vẽ tỷ lệ tuyến tính trên bản đồ.

- Hình ảnh bản đồ toàn cầu:

bluemarble(): Chiếu hình ảnh Viên Bi Xanh của NASA lên bản đồ.

shadedrelief(): Chiếu hình ảnh phù điêu bóng mờ lên bản đồ.

etopo(): Vẽ hình phù điêu etopo lên bản đồ.

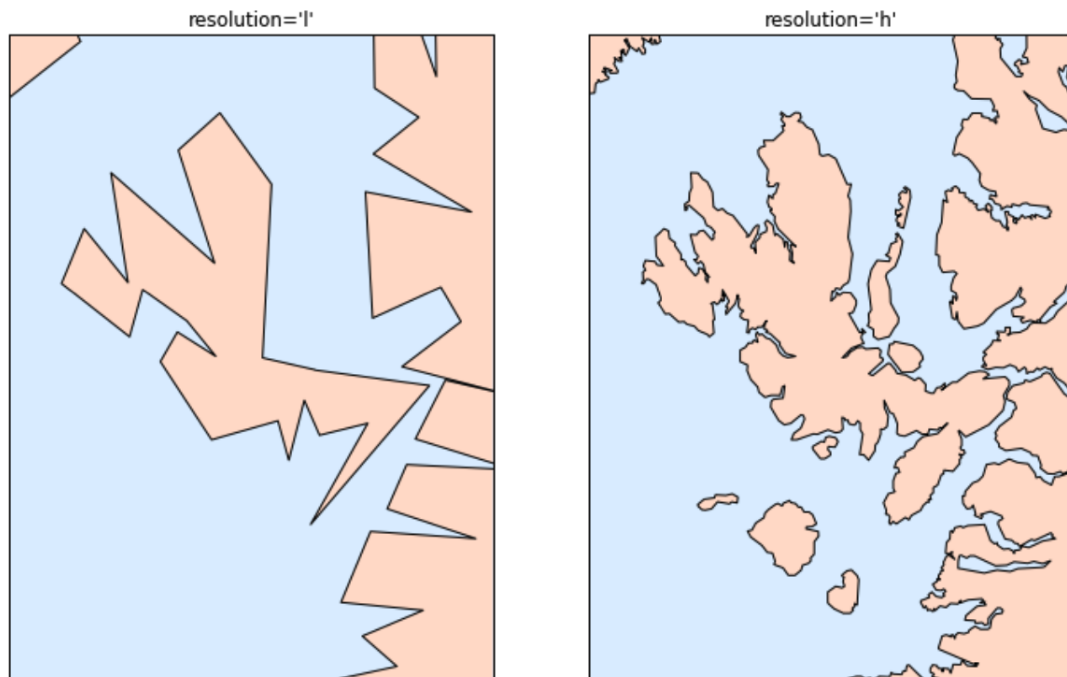
warpimage(): Chiếu hình ảnh do người dùng cung cấp lên bản đồ.

Đối với các thuộc tính dựa trên ranh giới cần phải đặt độ phân giải mong muốn khi tạo ảnh trong *Basemap*. Đối số độ phân giải của lớp *Basemap* đặt mức độ chi tiết trong các ranh giới như 'c' (crude), 'l' (low), 'i' (intermediate), 'h' (high), 'f' (full)

, hoặc None nếu không có ranh giới nào được sử dụng.

Dưới đây là một ví dụ về vẽ ranh giới đất liền/biển và ảnh hưởng của tham số độ phân giải sẽ tạo cả bản đồ có độ phân giải thấp và độ phân giải cao của *Isle of Skye* của Scotland. Nó nằm ở tọa độ $57,3^{\circ}B$ và $6,2^{\circ}T$, và bản đồ 90.000×120.000 km cho thấy rõ điều đó (Hình 3.49):

```
fig, ax = plt.subplots(1, 2, figsize=(12, 8))
for i, res in enumerate(['l', 'h']):
    m = Basemap(projection='gnom', lat_0=57.3, lon_0=-6.2,
                width=90000, height=120000, resolution=res,
                ax=ax[i])
    m.fillcontinents(color="#FFDDCC", lake_color='#DDEEFF')
    m.drawmapboundary(fill_color="#DDEEFF")
    m.drawcoastlines()
    ax[i].set_title("resolution='{0}'".format(res));
```



Hình 3.49: Bản đồ ranh giới ở độ phân giải thấp và cao.

Lưu ý: các đường bờ biển có độ phân giải thấp không phù hợp với mức thu phóng này, trong khi độ phân giải cao hoạt động tốt. Tuy nhiên, độ phân giải thấp sẽ chỉ hoạt động tốt đối với chế độ xem toàn cầu và sẽ nhanh hơn nhiều so với việc tải dữ liệu đường viền có độ phân giải cao cho toàn bộ địa cầu.

3.8.1 Trực quan dữ liệu trên bản đồ

Có lẽ phần hữu ích nhất của bộ công cụ Basemap là khả năng vẽ nhiều loại dữ liệu lên nền bản đồ. Đối với biểu đồ và văn bản đơn giản hay bất kỳ chức năng *plt* nào hoạt động trên bản đồ đều có thể sử dụng phiên bản Basemap để chiếu tọa độ kinh độ và vĩ độ thành tọa độ (x, y) trên đồ thị với *plt*, như phần trước đó trong ví dụ về Seattle.

Ngoài ra, có nhiều chức năng dành riêng cho bản đồ có sẵn dưới dạng các phương thức của Basemap. Một số phương pháp dành riêng cho bản đồ này là:

contour()/contourf(): Vẽ đường đồng mức hoặc đường viền.

imshow(): Vẽ một hình ảnh.

pcolor()/pcolormesh(): Vẽ một biểu đồ giả màu cho các lưới.

plot(): Vẽ các đường và/hoặc đánh dấu.

scatter(): Vẽ các điểm rời rạc.

quiver(): Vẽ vectơ.

barbs(): Vẽ các cột.

drawgreatcircle(): Vẽ một vòng tròn lớn.

Xem xét một số ví dụ dưới đây:

Ví dụ: Dân số các thành phố Việt Nam

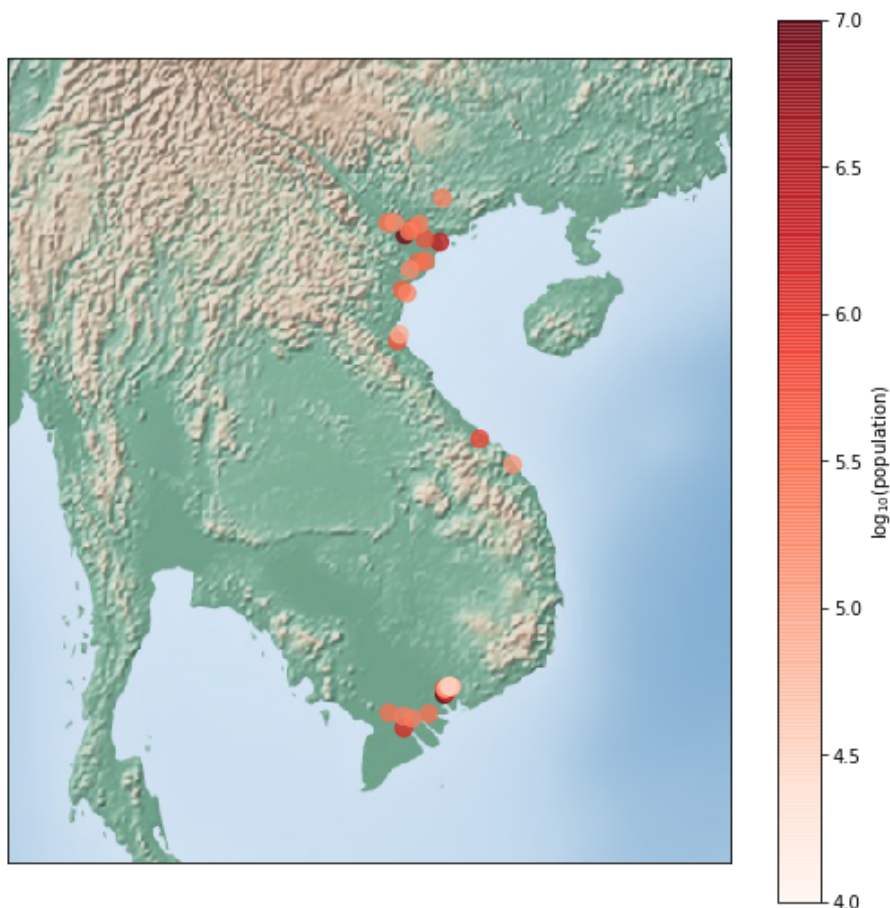
Trong ví dụ này sẽ tạo ra một biểu đồ về dân số các thành phố ở Việt Nam bằng cách sử dụng Basemap để đưa dữ liệu vào trong bản đồ bao gồm việc sử dụng màu sắc trong một biểu đồ phân tán để truyền đạt thông tin về vị trí và dân số (Hình 3.50).

Đầu tiên là tải dữ liệu *Vietnam Cities Database*¹ dưới dạng file *.csv* và sử dụng các câu lệnh dưới đây:

```
import pandas as pd
cities = pd.read_csv('vn.csv')

# Trích xuất dữ liệu cần sử dụng
lat = cities['lat'].values
lon = cities['lng'].values
population = cities['population'].values
fig = plt.figure(figsize=(9, 9))
m = Basemap(projection='lcc', resolution=None,
             lat_0=16, lon_0=105, width=1.8E6, height=2.0E6)
m.shadedrelief()
```

¹<https://simplemaps.com/data/vn-cities>



Hình 3.50: Biểu đồ phân tán trên nền bản đồ.

```
# Phan tan du lieu thanh pho voi mau sac phan anh dan so
m.scatter(lon, lat, latlon=True, c=np.log10(population),
          s=90, cmap='Reds', alpha=0.75)

# Tao thanh mau va chu thích
plt.colorbar(label=r'$\log_{10}(\text{population})$')
plt.clim(4, 7)
```

Điều này cho thấy thành phố tập trung nhiều người sinh sống ở Việt Nam: họ tập trung gần các đồng bằng lớn như các vùng ở Hà Nội và Hồ Chí Minh các trung tâm bằng phẳng.

3.9 Văn bản và chú giải

Tạo ra một hình ảnh trực quan liên quan mật thiết đến việc hướng dẫn người đọc hình dung được ý nghĩa của tập hợp các dữ liệu. Trong một số trường hợp, các hình

ảnh này có thể được kể theo cách hoàn toàn trực quan mà không cần thêm các chú thích bằng văn bản, nhưng trong những trường hợp khác, các đánh dấu và nhãn văn bản là điều cần thiết. Các loại chú thích cơ bản nhất sẽ sử dụng là nhãn của các trục tọa độ và tiêu đề của hình ảnh. Hãy xem xét một số dữ liệu và cách có thể trực quan hóa cũng như chú thích dữ liệu đó để giúp truyền tải thông tin. Bắt đầu bằng cách thiết lập Notebook để vẽ và nhập các hàm sẽ sử dụng:

```
%matplotlib inline
import matplotlib.pyplot as plt
import matplotlib as mpl
plt.style.use('seaborn-whitegrid')
import numpy as np
import pandas as pd
```

Các chú thích văn bản có thể được ghi chú vào các vị trí dữ liệu đặc biệt như đỉnh hoặc các đáy dữ liệu. Đôi khi, văn bản có thể được chú thích vào một vị trí trên trục hoặc cần một số dịch chuyển giữa các hệ tọa độ. Ví dụ: một điểm dữ liệu tại $(x, y) = (1, 1)$ cần được biểu diễn bằng cách nào đó tại một vị trí nhất định trên hình, do đó cần phải được biểu thị bằng pixel trên màn hình. Về mặt toán học, các phép biến đổi tọa độ như vậy tương đối đơn giản và *Matplotlib* có một bộ công cụ được phát triển để thực hiện chúng trong mô-đun *matplotlib.transforms*. Có ba phép biến đổi được xác định trước có thể hữu ích trong trường hợp này:

ax.transData: Biến đổi liên quan đến tọa độ dữ liệu.

ax.transAxes: Biến đổi liên quan đến các trục (tính theo đơn vị kích thước trục).

fig.transFigure: Biến đổi liên quan đến hình (tính theo đơn vị kích thước hình).

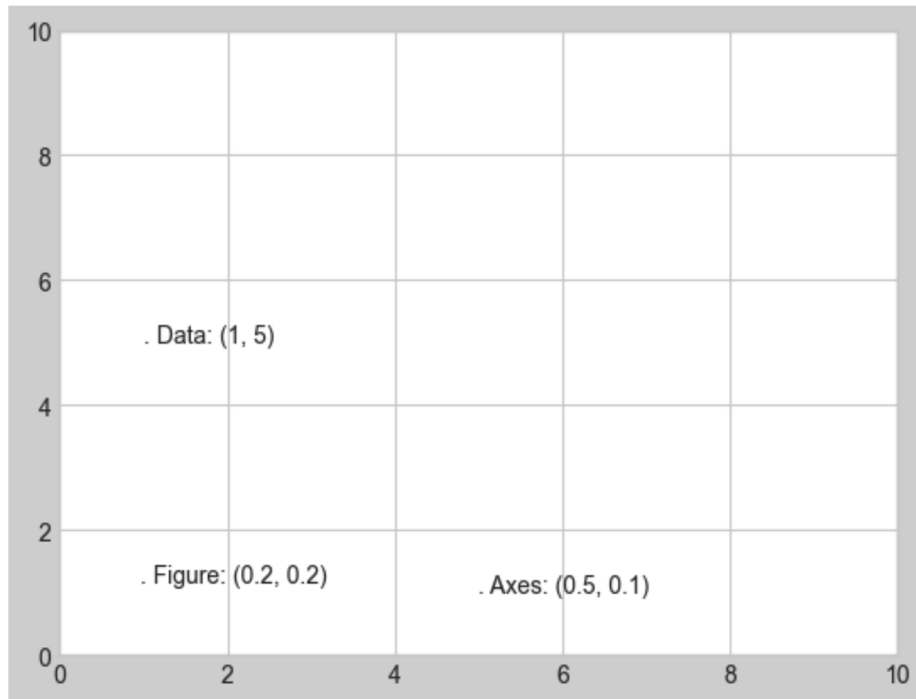
Dưới đây là một ví dụ về ghi chú ở nhiều vị trí khác nhau bằng cách sử dụng các phép biến đổi trên (Hình 3.51):

```
fig, ax = plt.subplots(facecolor='lightgray')
ax.axis([0, 10, 0, 10])

# transform = ax.transData là mặc định
ax.text(1, 5, ". Data: (1, 5)", transform=ax.transData)
ax.text(0.5, 0.1, ". Axes: (0.5, 0.1)", transform=ax.transAxes)
ax.text(0.2, 0.2, ". Figure: (0.2, 0.2)", transform=fig.transFigure);
```

Lưu ý theo mặc định, văn bản được căn chỉnh phía trên và bên trái của tọa độ đã chỉ định bởi dấu “.” đánh dấu vị trí tọa độ đã cho.

Tọa độ *transData* cung cấp tọa độ dữ liệu được liên kết với nhãn trục x và y. Tọa



Hình 3.51: So sánh các hệ tọa độ của Matplotlib.

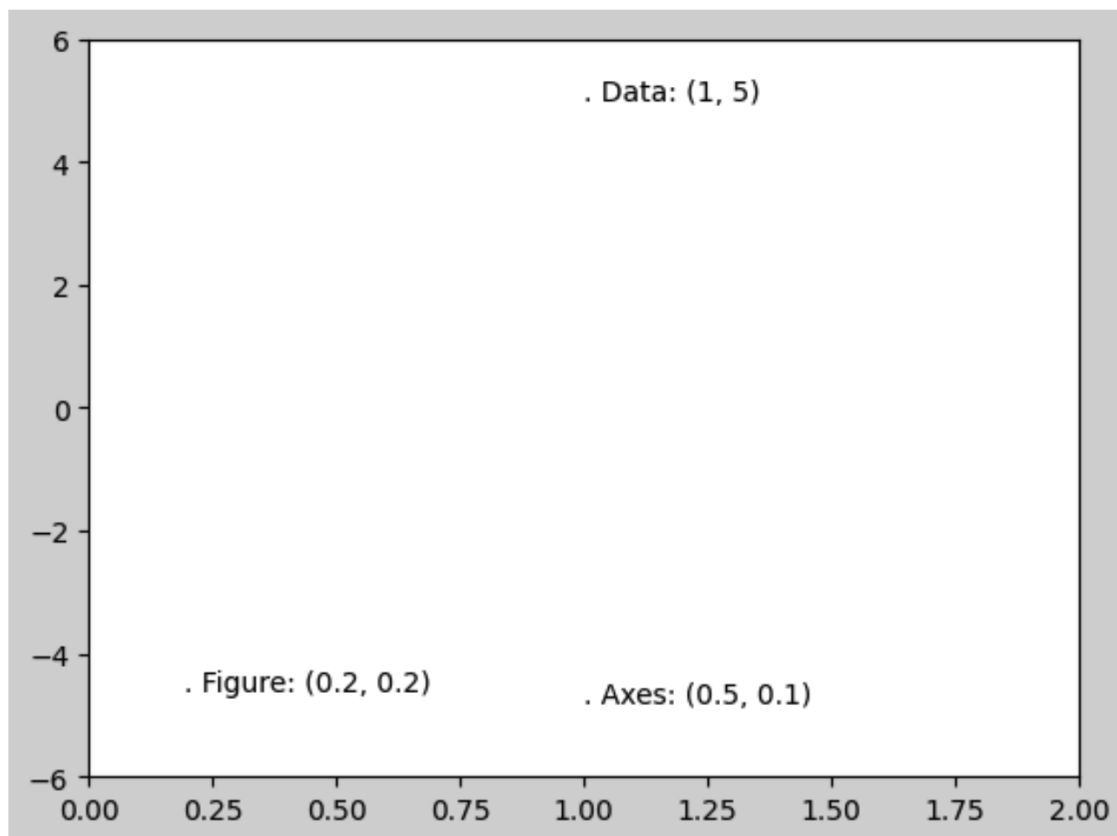
độ *transAxes* cung cấp vị trí từ góc dưới bên trái của các trục dưới dạng một phần của kích thước trục. Tọa độ *transfigure* chỉ định vị trí từ dưới cùng bên trái của hình dưới dạng một phần của kích thước hình. Nên nếu thay đổi giới hạn trục thì chỉ có tọa độ *transData* sẽ bị ảnh hưởng, trong khi các tọa độ khác vẫn không đổi (Hình 3.52):

```
ax.set_xlim(0, 2)
ax.set_ylim(-6, 6)
fig
```

3.9.1 Tuỳ chỉnh mũi tên chú giải

Cùng với kiểu chú thích sử dụng dấu chấm các điểm dữ liệu trên tọa độ với văn bản chú thích, một dấu chú thích bằng mũi tên đơn giản cũng được sử dụng một cách hữu ích.

Vẽ mũi tên trong *Matplotlib* thường sử dụng hàm *plt.arrow()*, tuy nhiên, các mũi tên mà nó tạo ra là các đối tượng SVG sẽ tuân theo tỷ lệ khung hình khác nhau của các ô và kết quả ít khi như mong muốn của người dùng. Thay vào đó, nên sử dụng hàm *plt.annotate()*. Hàm này tạo ra một số văn bản chú thích và một mũi tên được chỉ định rất linh hoạt (Hình 3.53):



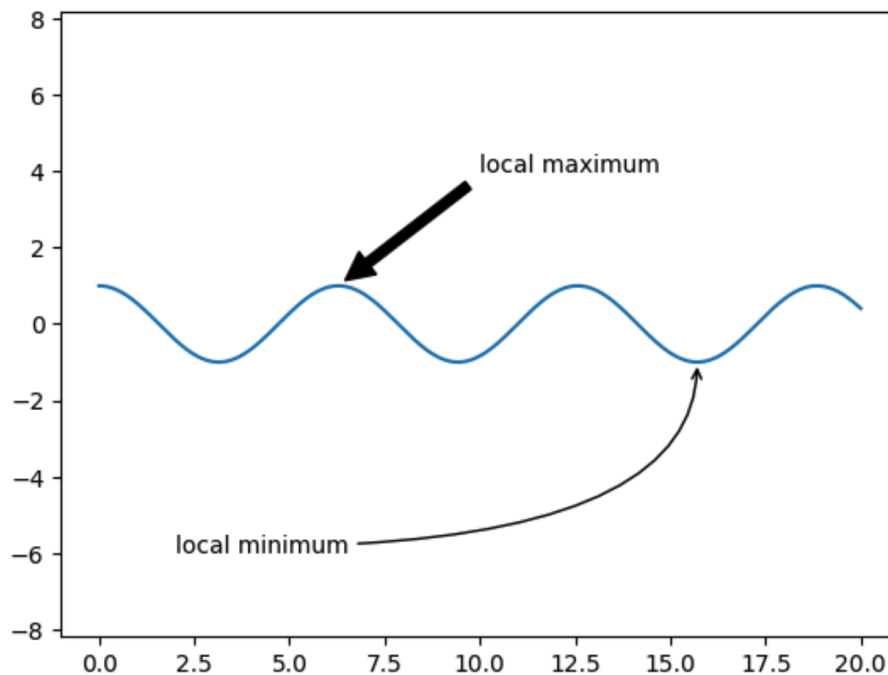
Hình 3.52: So sánh các hệ tọa độ của Matplotlib.

```
fig, ax = plt.subplots()
x = np.linspace(0, 20, 1000)
ax.plot(x, np.cos(x))
ax.axis('equal')
ax.annotate('local maximum', xy=(6.28, 1), xytext=(10, 4),
            arrowprops=dict(facecolor='black', shrink=0.05))
ax.annotate('local minimum', xy=(5 * np.pi, -1),
            xytext=(2, -6), arrowprops=dict(arrowstyle="->",
            connectionstyle="angle3,angleA=0,
            angleB=-90"));
```

Kiểu mũi tên được tùy chỉnh thông qua kiểu dữ liệu từ điển trong *arrowprops*. Dưới đây là một số lựa chọn tùy chỉnh đó (Hình 3.54):

```
fig, ax = plt.subplots()
x = np.linspace(0, 20, 1000)
ax.plot(x, np.cos(x))
ax.axis('equal')

ax.annotate('first point', xy=(0, 1), xytext=(0, 4),
            arrowprops=dict(arrowstyle="->",
```



Hình 3.53: Ví dụ về tùy chỉnh mũi tên chú giải.

```

connectionstyle="arc3,rad=-0.2"))

ax.annotate('local maximum', xy=(6.28, 1), xytext=(10, 20),
            bbox=dict(boxstyle="round", fc="none", ec="gray"),
            textcoords='offset points', ha='center',
            arrowprops=dict(arrowstyle="->"))

ax.annotate('local minimum', xy=(3 * np.pi, -1), xycoords='data',
            xytext=(4, -4), bbox=dict(boxstyle="round4,pad=.5", fc="0.9"),
            arrowprops=dict(arrowstyle="->",
                            connectionstyle="angle,angleA=0,angleB=80,rad=20"))

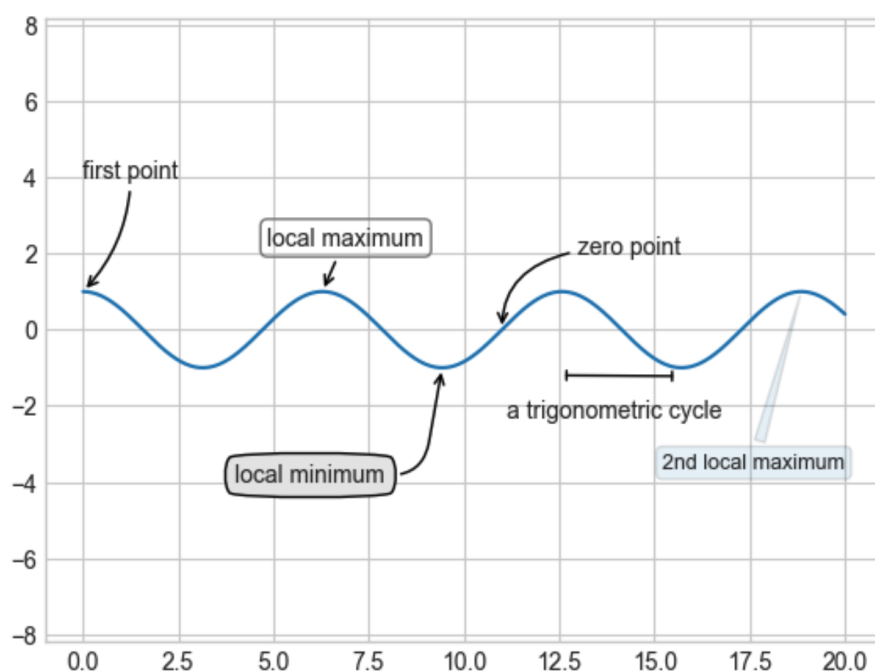
ax.annotate('zero point', xy=(3.5 * np.pi, 0), xytext=(13,2),
            arrowprops=dict(arrowstyle="->",
                            connectionstyle="angle3,angleA=0,angleB=-90"))

ax.annotate('a trigonometric cycle', xy=(11, -1.95),
            xytext=(2, -6),textcoords='offset points')
ax.annotate(' ', xy=(4 * np.pi, -1.2), xytext=(5 * np.pi, -1.4),
            textcoords='data', arrowprops={'arrowstyle': '|-|',
                                           widthA=0.2,widthB=0.2'})

ax.annotate('2nd local maximum', xy=(6 * np.pi, 1), xycoords='data',
            xytext=(20, -3.5),size=9, ha='right', va="center",
            bbox=dict(boxstyle="round", alpha=0.1),

```

```
arrowprops=dict(arrowstyle="wedge,tail_width=0.5",
alpha=0.1));
```



Hình 3.54: Tùy chỉnh các loại mũi tên chú giải.

3.9.2 Tùy chỉnh cách hiển thị của trục biểu đồ

Các định vị và định dạng hiển thị trục tọa độ mặc định của Matplotlib được thiết kế nhìn chung là đủ trong nhiều tình huống phổ biến, nhưng không có cách nào tối ưu cho mọi biểu đồ. Phần này sẽ đưa ra một số ví dụ về điều chỉnh vị trí đánh dấu và định dạng hiển thị của các trục tọa độ cho các loại biểu đồ cụ thể.

Mỗi trục có các thuộc tính *xaxis* và *yaxis*, lần lượt có các thuộc tính chứa tất cả các thuộc tính của đường, dấu tích đánh dấu đơn vị và nhãn tạo nên các trục của biểu đồ.

Các điểm đánh dấu chính và phụ trên các trục tọa độ

Trong mỗi trục đều có thể có điểm đánh dấu chính (dấu tích chính - *major tick*) và điểm đánh dấu phụ (dấu tích phụ - *minor tick*). Các điểm *major tick* thường lớn hơn hoặc rõ ràng hơn, trong khi *minor tick* thường nhỏ hơn. Theo mặc định, Matplotlib hiếm khi sử dụng các điểm *minor tick*, một ví dụ có thể thấy chúng là trong các ô logarit (Hình 3.55):

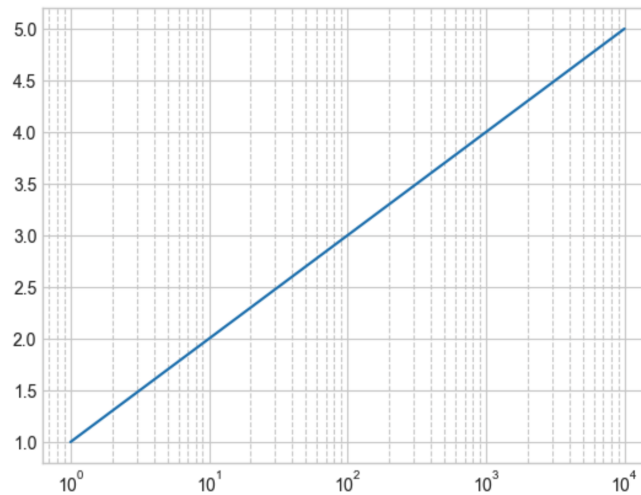
```
import matplotlib.pyplot as plt
```

```
import matplotlib.ticker as ticker

x = [1, 10, 100, 1000, 10000]
y = [1, 2, 3, 4, 5]

ax.xaxis.set_major_locator(ticker.LogLocator(base=10.0, numticks=5))
ax.xaxis.set_minor_locator(ticker.LogLocator(base=10.0,
                                             subs=(0.2, 0.4, 0.6, 0.8), numticks=5))
ax.xaxis.set_minor_formatter(ticker.FormatStrFormatter("%.2f"))

plt.semilogx(x, y)
plt.grid(True, which="minor", ls="--")
plt.show()
```



Hình 3.55: Ví dụ về thang đo logarit và nhân.

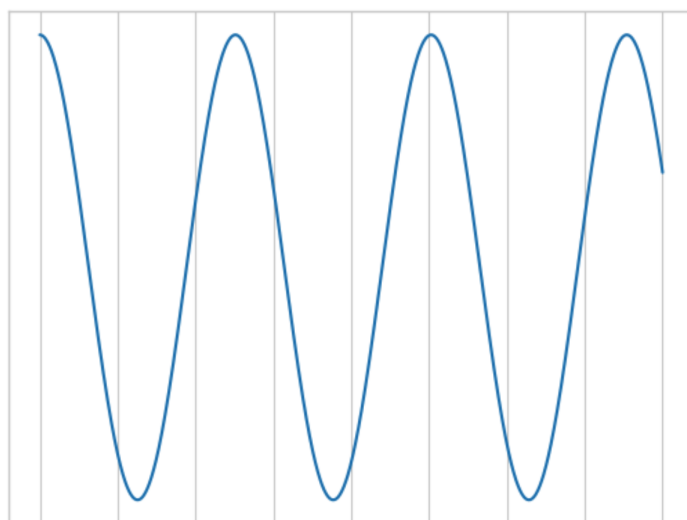
Ở đây, mỗi dấu *major tick* hiển thị một dấu tích lớn và một nhãn, trong khi mỗi dấu *minor tick* hiển thị một dấu tích nhỏ hơn không có nhãn. Có thể tùy chỉnh các thuộc tính đánh dấu này và vị trí và nhãn bằng cách đặt các đối tượng định dạng và định vị của mỗi trục bằng `set_major_locator` và `set_minor_locator`.

Ẩn các điểm đánh dấu trên trục tọa độ và nhãn

Thao tác ẩn các điểm đánh dấu trên trục tọa độ và các nhãn trên trục có thể thực hiện việc này bằng các thao tác `plt.NullLocator()` và `plt.NullFormatter()`, như minh họa ở đây (Hình 3.57):

```
ax = plt.axes()
x = np.linspace(0, 20, 1000)
ax.plot(x, np.cos(x))
ax.yaxis.set_major_locator(plt.NullLocator())
```

```
ax.xaxis.set_major_formatter(plt.NullFormatter())
```



Hình 3.56: Vẽ biểu đồ ẩn các nhãn đánh dấu trục x và trục y.

Việc không có đánh dấu nào trên các trục tọa độ có thể hữu ích trong nhiều trường hợp, chẳng hạn như khi muốn hiển thị một lưới hình ảnh. Ví dụ Hình 3.55 bao gồm hình ảnh của các khuôn mặt khác nhau, cách này thường được sử dụng trong các bài toán học máy có giám sát.

```
fig, ax = plt.subplots(5, 5, figsize=(5, 5))
fig.subplots_adjust(hspace=0, wspace=0)
# Lay du lieu anh tu thu vien scikit-learn
from sklearn.datasets import fetch_olivetti_faces
faces = fetch_olivetti_faces().images
for i in range(5):
    for j in range(5):
        ax[i, j].xaxis.set_major_locator(plt.NullLocator())
        ax[i, j].yaxis.set_major_locator(plt.NullLocator())
        ax[i, j].imshow(faces[10 * i + j], cmap="bone")
```

Giảm hoặc tăng số lượng các nhãn đánh dấu trên các trục tọa độ

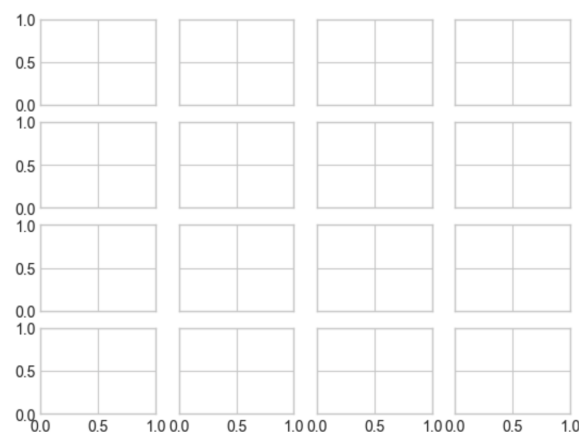
Một vấn đề phổ biến mặc định của các đồ thị là các đồ thị con (subplot) nhỏ hơn có thể có nhiều nhãn đánh dấu trên trục. Có thể thấy điều này trong lưới đồ thị được hiển thị trong Hình 3.58:

```
fig, ax = plt.subplots(4, 4, sharex=True, sharey=True)
```

Để biết thêm thông tin về bất kỳ điều nào trong số này, hãy tham khảo các tài



Hình 3.57: Ấn đánh dấu các trục tọa độ trong biểu đồ hiển thị các hình ảnh.



Hình 3.58: Một ô mặc định có nhiều dấu tích.

liệu trực tuyến Matplotlib.

Bài tập Chương 3

Bài tập 3.1 Tùy chỉnh Matplotlib: Cấu hình và Bảng định kiểu (Stylesheets)

1. Thay đổi mặc định: rcParams
2. Stylesheets
 - a. Phong cách *FiveThirtyEight*
 - b. ggplot
 - c. Phương pháp Bayesian dành cho phong cách hacker (*bhm*)
 - d. Nền tối

- e. Thang độ xám
- f. Phong cách *seaborn*

Bài tập 3.2 Trực quan hoá các đồ thị bằng Seaborn:

1. Histograms
2. Biểu đồ ước lượng mật độ (KDE)
3. Đồ thị kết hợp
 - a. Pair plots
 - b. Faceted histograms
 - c. Factor plots
 - d. Joint distributions
 - e. Bar plots



4. Học máy

Học máy hay Machine Learning (ML) là một tập con của trí tuệ nhân tạo (AI) tập trung vào phát triển các thuật toán máy tính tự động cải thiện thông qua trải nghiệm và sử dụng dữ liệu. Học máy cho phép máy tính học từ dữ liệu và đưa ra quyết định hoặc dự đoán mà không cần được lập trình rõ ràng để làm việc đó.

Chương 4 trình bày những phương pháp trong học máy bao gồm trích chọn đặc trưng và các phương pháp học có giám sát, học không giám sát, học bán giám sát và học kết hợp.

4.1 Các khái niệm cơ bản

Học máy liên quan đến việc xây dựng các mô hình toán học để giúp hiểu dữ liệu, việc tạo và sử dụng các mô hình được học từ dữ liệu như vậy còn được gọi là mô hình dự đoán hoặc khai thác dữ liệu. Mục tiêu sẽ là sử dụng dữ liệu hiện có để phát triển các mô hình có thể sử dụng để dự đoán các kết quả khác nhau cho dữ liệu mới. “*Learning*” - *Học*: là quá trình khi cung cấp cho các mô hình này các thông số có thể điều chỉnh được *tunable parameters* để có thể điều chỉnh phù hợp với dữ liệu được quan sát, theo cách này, chương trình có thể được coi là “*học*” từ dữ liệu. Khi các mô hình phù hợp với dữ liệu quan sát trước đó, chúng có thể được sử dụng để dự đoán và hiểu các khía cạnh của dữ liệu mới được quan sát, ví dụ như:

- Liệu một email có phải là thư rác hay không
- Giao dịch thẻ tín dụng có gian lận hay không
- Quảng cáo nào người mua hàng có nhiều khả năng nhấp vào
- Đội bóng nào sẽ vô địch.

Phân loại học máy

Ở cấp độ cơ bản nhất, học máy có thể được phân thành hai loại chính: học có giám sát (*Supervised learning*) và học không giám sát (*Unsupervised learning*).

- *Học có giám sát* liên quan đến việc mô hình hóa bằng cách nào đó mối quan hệ giữa các thuộc tính của dữ liệu và các nhãn dữ liệu; một khi mô hình này được xác định, nó có thể được sử dụng để áp dụng nhãn cho dữ liệu mới. Học có giám sát được chia nhỏ thành các nhiệm vụ phân loại và nhiệm vụ hồi quy: trong phân loại, nhãn là các danh mục riêng biệt, trong khi ở hồi quy, nhãn là số lượng liên tục.
- *Học không giám sát* liên quan đến việc lập mô hình các thuộc tính của tập dữ liệu mà không tham chiếu đến bất kỳ nhãn nào và thường được mô tả là “để tập dữ liệu tự nói”. Những mô hình này bao gồm các nhiệm vụ như phân cụm và giảm kích thước. Các thuật toán phân cụm xác định các nhóm dữ liệu riêng biệt, trong khi các thuật toán giảm kích thước tìm kiếm các cách trình bày dữ liệu ngắn gọn hơn.

Ngoài ra, còn có một số loại học máy như học máy bán giám sát (*semisupervised learning*)(trong đó chỉ một số ít dữ liệu được gán nhãn), học trực tuyến (*online learning*)(mô hình cần liên tục điều chỉnh theo dữ liệu mới đến) và học tăng cường (*reinforcement learning*) (sau khi đưa ra một loạt dự đoán, mô hình nhận được thông tin cho biết nó đã hoạt động tốt như thế nào để cải thiện dự đoán).

4.1.1 Học và suy diễn

Trong học máy, có hai giai đoạn chính quan trọng là training (học - huấn luyện) và inference (suy diễn - dự đoán).

- **Training** (Học - Huấn luyện)
 - *Định nghĩa*: Giai đoạn huấn luyện trong học máy là quá trình mà mô hình máy học học từ dữ liệu. Mục tiêu là để mô hình hóa mối quan hệ hoặc cấu trúc trong dữ liệu để có thể dự đoán hoặc phân loại các điểm dữ liệu mới.
 - *Cách thức hoạt động*: Trong giai đoạn huấn luyện, mô hình được đào tạo bằng cách sử dụng dữ liệu huấn luyện có thông tin đầu vào và đầu ra tương ứng. Mô hình sẽ điều chỉnh các tham số của mình thông qua việc tối ưu hóa hàm mất mát (loss function) để dự đoán chính xác nhất có thể các kết quả cho dữ liệu mới.
- **Inference** (Suy diễn - Dự đoán)
 - *Định nghĩa*: Giai đoạn inference xảy ra sau khi mô hình đã được huấn luyện. Đây là quá trình sử dụng mô hình đã học để dự đoán hoặc phân loại dữ liệu mới mà mô hình chưa từng thấy trước đó.
 - *Cách thức hoạt động*: Khi được triển khai, mô hình đã huấn luyện sẽ được

sử dụng để thực hiện dự đoán trên dữ liệu mới. Dữ liệu mới được đưa vào mô hình thông qua quá trình inference để sinh ra dự đoán hoặc phân loại dựa trên thông tin đã học trong quá trình huấn luyện.

Sự khác nhau giữa Training và Inference:

1. *Dữ liệu sử dụng*: Trong training, mô hình sử dụng dữ liệu huấn luyện để học. Trong inference, mô hình sử dụng dữ liệu mới để tạo ra dự đoán hoặc phân loại.
2. *Mục tiêu*: Trong training, mục tiêu là tối ưu hóa mô hình để dự đoán chính xác trên dữ liệu huấn luyện. Trong inference, mục tiêu là sử dụng mô hình đã huấn luyện để dự đoán kết quả cho dữ liệu mới.
3. *Quá trình*: Trong training, mô hình thay đổi và cải thiện thông qua việc điều chỉnh các tham số để cải thiện dự đoán. Trong inference, mô hình không thay đổi và chỉ được sử dụng để dự đoán dữ liệu mới.

Sự phân biệt giữa training và inference quan trọng để hiểu rõ quá trình hoạt động của mô hình machine learning và cách triển khai mô hình học máy trong các ứng dụng thực tế.

4.1.2 Đánh giá mô hình

Đánh giá mô hình trong học máy là quá trình đo lường hiệu suất của mô hình dựa trên các chỉ số, số liệu hoặc phương pháp đánh giá khác nhau. Mục tiêu chính của việc đánh giá mô hình là xác định xem mô hình của bạn hoạt động như thế nào trên dữ liệu mà nó chưa từng thấy trước đó trong bộ dữ liệu kiểm tra.

Các phương pháp phổ biến để đánh giá mô hình trong học máy chia tập dữ liệu thành tập huấn luyện (Training set) và tập kiểm tra (Test set):

- Tập huấn luyện (Training set): Dữ liệu được sử dụng để huấn luyện mô hình.
- Tập kiểm tra (Test set): Dữ liệu không được mô hình thấy trong quá trình huấn luyện, dùng để đánh giá hiệu suất dự đoán của mô hình trên dữ liệu mới.

Các phép đo đánh giá hiệu suất trong machine learning thường dựa trên bốn kết quả chính từ việc dự đoán:

- True Positive (TP): Số lượng điểm dữ liệu thực sự là positive và được dự đoán đúng là positive.
- False Positive (FP): Số lượng điểm dữ liệu thực sự là negative nhưng được dự đoán là positive.
- True Negative (TN): Số lượng điểm dữ liệu thực sự là negative và được dự đoán đúng là negative.

- False Negative (FN): Số lượng điểm dữ liệu thực sự là positive nhưng được dự đoán là negative.

Dựa trên các chỉ số này, các phép đo đánh giá hiệu suất phổ biến được định nghĩa như sau:

- Accuracy (Độ chính xác):
 - Định nghĩa: Tỷ lệ dự đoán đúng trên tổng số dự đoán.
 - Sử dụng khi: Dữ liệu của các lớp (classes) trong bài toán phân loại cân bằng.
 - Công thức: sử dụng TP và TN: Tỷ lệ dự đoán đúng trên tổng số dự đoán:

$$Accuracy = \frac{\text{correct predictions}}{\text{all predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision (Độ chính xác của dự đoán tích cực):
 - Định nghĩa: Tỷ lệ các dự đoán tích cực (positive) đúng so với tất cả các dự đoán tích cực.
 - Sử dụng khi: Quan trọng khi sự chính xác của dự đoán tích cực là ưu tiên hàng đầu, và việc giảm số lượng dự đoán sai là quan trọng.
 - Công thức: sử dụng TP và FP: Tỷ lệ các dự đoán tích cực (positive) đúng so với tất cả các dự đoán tích cực:

$$Precision = \frac{TP}{TP + FP}$$

- Recall (Tỷ lệ dự đoán đúng tích cực):
 - Định nghĩa: Tỷ lệ các dự đoán tích cực đúng so với tất cả các điểm dữ liệu thực sự là tích cực.
 - Sử dụng khi: Quan trọng khi việc bỏ sót các trường hợp tích cực là rủi ro cao, và việc giảm false negatives là quan trọng.
 - Công thức: sử dụng TP và FN: Tỷ lệ các dự đoán tích cực đúng so với tất cả các điểm dữ liệu thực sự là tích cực:

$$Recall = \frac{TP}{TP + FN}$$

- F1-score:
 - Định nghĩa: Kết hợp giữa Precision và Recall để đo lường hiệu suất tổng thể của mô hình.
 - Sử dụng khi: Cần cân nhắc cả Precision và Recall một cách cân bằng.

– kết hợp giữa Precision và Recall:

$$F1 = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

Bên cạnh đó còn một số các phương pháp đánh giá hiệu suất như:

- *Cross-validation* (CV) là một kỹ thuật quan trọng trong machine learning, bộ dữ liệu được chia thành các phần khác nhau để huấn luyện và kiểm tra, đánh giá hiệu suất trung bình của mô hình.
- *Confusion Matrix* (Ma trận nhầm lẫn): Hiển thị số lượng dự đoán chính xác và sai lệch theo từng loại dự đoán. Ma trận này thể hiện sự phân loại của mô hình dựa trên dữ liệu thực tế và dự đoán được bao gồm 4 ô TP, TN, FP và FN.

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Hình 4.1: Confusion Matrix (Ma trận nhầm lẫn).

- *ROC Curve* và *AUC* (*Receiver Operating Characteristic Curve* và *Area Under the Curve*): Đánh giá hiệu suất của mô hình phân loại dựa trên trade-off giữa tỷ lệ True Positive Rate (TPR) và False Positive Rate (FPR), được tính dựa trên TP, TN, FP và FN.
- *Bias* và *Variance*: Kiểm tra sự đồng đều (bias) và sự dao động (variance) của mô hình để hiểu cách mô hình phản ứng với dữ liệu mới.
- Đánh giá thông qua một số thử nghiệm và mô hình hóa: Sử dụng các kỹ thuật như Grid Search hoặc Random Search để tìm kiếm siêu tham số tốt nhất cho mô hình.

Bên cạnh đó có một số phép đo phổ biến được sử dụng để đánh giá hiệu suất của các mô hình Regression như:

- *Mean Absolute Error (MAE)*: Trung bình cộng của giá trị tuyệt đối của sự chênh lệch giữa giá trị dự đoán và giá trị thực tế.
 - Ưu điểm: Dễ hiểu, không nhạy cảm với các giá trị ngoại lệ.
 - Nhược điểm: Không phản ánh sự chênh lệch giữa các sai số.
- *Mean Squared Error (MSE)*: Trung bình của bình phương của sự chênh lệch giữa giá trị dự đoán và giá trị thực tế.
 - Ưu điểm: Phản ánh rõ ràng sự chênh lệch giữa các sai số.
 - Nhược điểm: Đơn vị của MSE là bình phương của đơn vị gốc, khó hiểu trong một số ngữ cảnh.
- *Root Mean Squared Error (RMSE)*: Căn bậc hai của MSE, cho giá trị trung bình của sự chênh lệch giữa giá trị dự đoán và giá trị thực tế.
 - Ưu điểm: Có cùng đơn vị với giá trị gốc, dễ hiểu hơn MSE.
 - Nhược điểm: Nhược điểm tương tự như MSE.
- *R-squared (Coefficient of Determination)*: Phần trăm phương sai của biến phụ thuộc mà mô hình có thể giải thích được. Giá trị càng gần 1 càng tốt.
 - Ưu điểm: Cho biết tỷ lệ phương sai của biến phụ thuộc mô hình có thể giải thích được.
 - Nhược điểm: Có thể không phản ánh được độ quan trọng của các biến đầu vào.
- Ngoài ra, một số các phương pháp có thể kể đến như Adjusted R-squared, Mean Absolute Percentage Error (MAPE), Akaike Information Criterion (AIC) và Bayesian Information Criterion (BIC).

Việc chọn phương pháp đánh giá phụ thuộc vào loại mô hình, bài toán cụ thể và tập dữ liệu. Kết quả đánh giá mô hình giúp tối ưu hóa và cải thiện hiệu suất của mô hình trên dữ liệu thực tế.

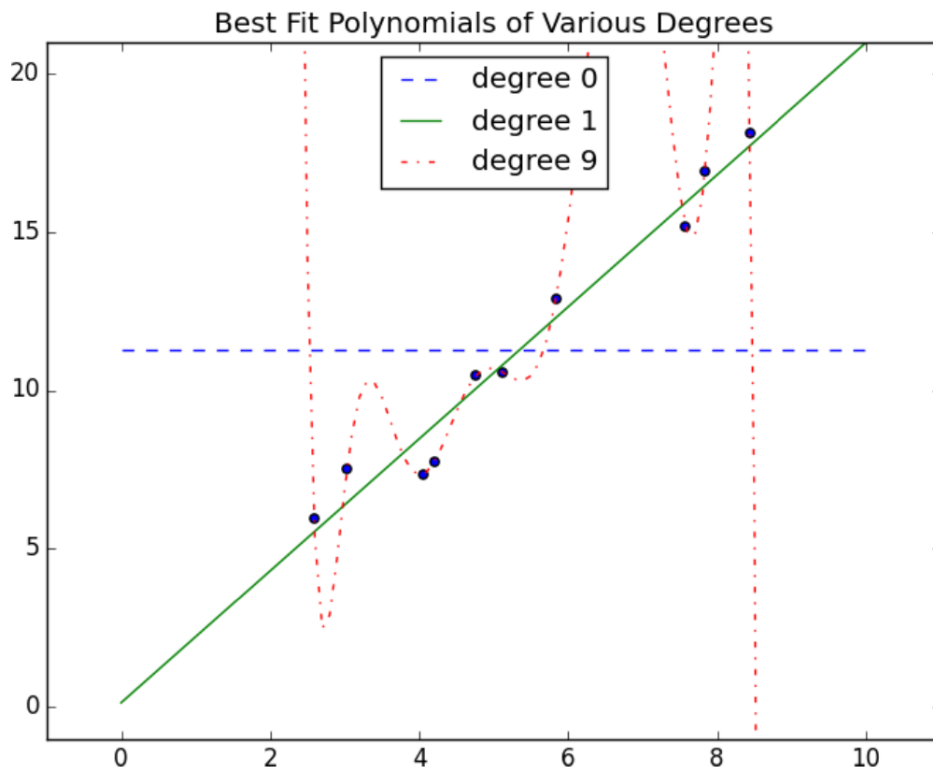
4.1.3 Overfitting và Underfitting

Overfitting và Underfitting là hai vấn đề quan trọng trong machine learning, liên quan đến việc mô hình hóa dữ liệu. Chúng xảy ra khi mô hình không thể tổng quát hóa tốt trên dữ liệu mới mà nó chưa từng nhìn thấy.

- Overfitting:
 - Overfitting xảy ra khi mô hình học "quá nhiều" từ dữ liệu huấn luyện, bao gồm cả các điểm dữ liệu nhiễu hoặc các đặc trưng không cần thiết.
 - Dấu hiệu: Hiệu suất cao trên tập huấn luyện nhưng hiệu suất kém trên tập kiểm tra. Mô hình quá phức tạp và nhớ các chi tiết không cần thiết trong

dữ liệu huấn luyện.

- Giải pháp: Sử dụng các kỹ thuật như Regularization, giảm độ phức tạp của mô hình, sử dụng thêm dữ liệu, hoặc loại bỏ các đặc trưng không cần thiết.
- Underfitting:
 - Underfitting xảy ra khi mô hình không học đủ từ dữ liệu huấn luyện và không thể tổng quát hóa được.
 - Dấu hiệu: Hiệu suất kém cả trên tập huấn luyện và tập kiểm tra. Mô hình quá đơn giản để diễn tả mối quan hệ trong dữ liệu.
 - Giải pháp: Sử dụng mô hình phức tạp hơn, thêm các đặc trưng, tăng độ phức tạp của mô hình, hoặc tăng kích thước tập dữ liệu huấn luyện.



Hình 4.2: Overfitting và underfitting.

Trong hình 4.2 đã khớp ba đa thức với một mẫu dữ liệu. Đường ngang hiển thị đa thức mức 0 (hằng số) - *degree 0* phù hợp nhất với dữ liệu, tuy nhiên, trường hợp này được gọi là *Underfitting*. Đa thức mức độ 9 (10 tham số) - *degree 9* đi qua mọi điểm dữ liệu huấn luyện một cách chính xác, nếu thêm một vài điểm dữ liệu, rất có thể nó sẽ bỏ lỡ rất nhiều điểm dữ liệu khác nên đường này được gọi là *Overfitting*. Và đường cấp 1 - *degree 1* tạo ra sự cân bằng tốt nhất, nó khá gần với mọi điểm và

nếu những dữ liệu này mang tính đại diện cho đường này cũng có thể sẽ gần với các điểm dữ liệu mới.

Các biện pháp chống Overfitting và Underfitting:

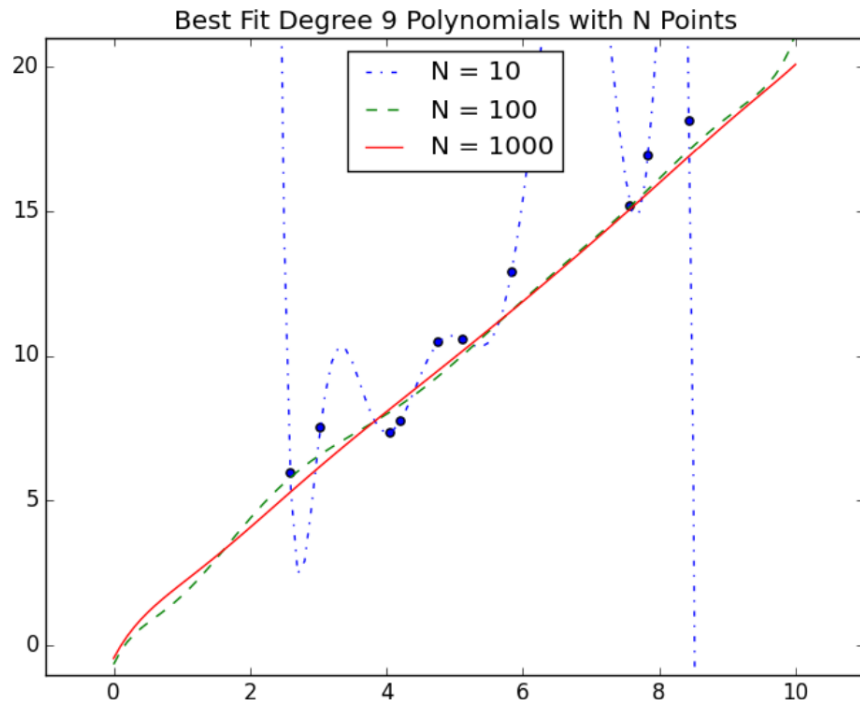
- Regularization: Giảm độ phức tạp của mô hình bằng cách áp dụng penalty cho các trọng số lớn.
- Cross-validation: Đánh giá hiệu suất của mô hình trên nhiều tập dữ liệu để kiểm tra sự tổng quát hóa.
- Thêm dữ liệu: Mở rộng tập dữ liệu huấn luyện để mô hình có thể học được nhiều hơn về dữ liệu.
- Feature selection: Loại bỏ các đặc trưng không quan trọng hoặc dùng ít.
- Simpler model: Sử dụng mô hình đơn giản hơn nếu dữ liệu không đủ để học mô hình phức tạp.

Điều quan trọng là tìm một sự cân bằng giữa độ phức tạp của mô hình và khả năng tổng quát hóa trên dữ liệu mới để tránh hiện tượng Overfitting và Underfitting trong machine learning.

4.1.4 Bias - Sai số định kiến và Variance - Phương sai

Bias - Sai số định kiến và Variance - Phương sai là hai khía cạnh quan trọng trong việc đánh giá hiệu suất của một mô hình machine learning. Đây là hai yếu tố đặc biệt quan trọng để hiểu về tính tổng quát hóa và độ chính xác của mô hình. Một cách nghĩ khác về vấn đề overfitting là sự cân bằng giữa bias và phương sai.

Ví dụ: mô hình *degree 0* Hình 4.2 sẽ mắc rất nhiều lỗi đối với hầu hết mọi tập huấn luyện, điều đó có nghĩa là nó có sai số định kiến bias cao. Tuy nhiên, bất kỳ hai tập huấn luyện được chọn ngẫu nhiên nào cũng sẽ cho các mô hình khá giống nhau (vì bất kỳ hai tập huấn luyện được chọn ngẫu nhiên nào cũng phải có giá trị trung bình khá giống nhau). Điều này được nói rằng mô hình có phương sai thấp. *High bias* - Sai số định kiến cao và *low variance* - phương sai thấp thường tương ứng với việc Overfitting. Mặt khác, mô hình *degree 9* hoàn toàn phù hợp với tập huấn luyện. Nó bias rất thấp nhưng phương sai rất cao tương ứng với underfitting. Trong Hình 4.3, với mô hình đa thức bậc 9 - *degree 9* với các mẫu có kích thước khác nhau dựa trên 10 điểm dữ liệu. Thay vào đó, nếu huấn luyện trên 100 điểm dữ liệu thì overfitting sẽ ít hơn. Và mô hình được đào tạo từ 1.000 điểm dữ liệu trông rất giống với mô hình *degree 1*. Khi độ phức tạp của mô hình không đổi, càng có nhiều dữ liệu thì càng khó bị overfitting. Mặt khác, nhiều dữ liệu hơn sẽ không giúp ích gì cho bias. Nếu mô hình không sử dụng đủ các thuộc tính dữ liệu để nắm bắt các quy



Hình 4.3: Overfitting và underfitting.

luật trong dữ liệu thì việc đưa thêm dữ liệu vào mô hình sẽ không giúp ích gì.

Cả hai Bias - Sai số định kiến và Variance - Phương sai đều là thước đo những gì sẽ xảy ra nếu mô hình huấn luyện lại nhiều lần trên các tập dữ liệu huấn luyện khác nhau.

- Bias (Sai số định kiến):
 - Bias đo lường mức độ độc lập của mô hình dự đoán so với giá trị thực tế. Nó biểu thị cho sai số do giả định định kiến trong quá trình học mô hình.
 - Dấu hiệu nhận biết: Một mô hình với bias cao có thể dẫn đến kết quả dự đoán không chính xác và không phản ánh đúng mối quan hệ giữa các biến đầu vào và biến mục tiêu.
 - Nguyên nhân: Bias thường xảy ra khi mô hình quá đơn giản để diễn tả mối quan hệ giữa các biến đầu vào và biến mục tiêu.
 - Giải pháp: Sử dụng mô hình phức tạp hơn, thêm các đặc trưng hoặc tăng độ phức tạp của mô hình để giảm bias.
- Variance (Phương sai):
 - Định nghĩa: Variance đo lường mức độ biến đổi của dự đoán của mô hình trên các tập dữ liệu khác nhau. Nó chỉ ra mức độ mà mô hình thay đổi khi được huấn luyện trên các tập dữ liệu khác nhau.

- Dấu hiệu nhận biết: Một mô hình có variance cao thường cho kết quả khác nhau đáng kể trên các tập dữ liệu khác nhau, thậm chí có thể dẫn đến hiện tượng overfitting.
- Nguyên nhân: Variance thường xảy ra khi mô hình quá phức tạp và "nhớ" các chi tiết không cần thiết hoặc nhiễu trong dữ liệu huấn luyện.
- Giải pháp: Sử dụng các kỹ thuật như Regularization, giảm độ phức tạp của mô hình, hoặc sử dụng thêm dữ liệu để giảm variance.
- Trade-off giữa Bias và Variance:
 - Một mô hình machine learning lý tưởng sẽ đạt được sự cân bằng tốt giữa bias và variance, tức là có độ chính xác cao và ổn định trên cả dữ liệu huấn luyện và dữ liệu kiểm tra.

Việc hiểu rõ về bias và variance giúp chọn mô hình phù hợp và thực hiện các điều chỉnh để đảm bảo mô hình có khả năng tổng quát hóa tốt nhất trên dữ liệu mới.

4.2 Lựa chọn và trích chọn đặc trưng

Trong các ứng dụng thực tế, dữ liệu hiếm khi ở dạng $[n_samples, n_features]$ (bao gồm $n_samples$ mẫu dữ liệu, mỗi dữ liệu có $n_features$ thuộc tính). Do đó, một trong những bước quan trọng trong việc sử dụng máy học trong thực tế là lựa chọn và trích chọn đặc trưng - nghĩa là lấy bất kỳ thông tin nào của dữ liệu và biến nó thành những con số có thể sử dụng để xây dựng ma trận đặc tính của dữ liệu. Phần này sẽ đưa ra một số ví dụ phổ biến về nhiệm vụ kỹ thuật lựa chọn đặc trưng: các thuộc tính để biểu diễn dữ liệu phân loại, các thuộc tính để biểu diễn văn bản và các thuộc tính để biểu diễn hình ảnh. Thông thường quá trình này được gọi là vectơ hóa *vectorization*, vì nó liên quan đến việc chuyển đổi dữ liệu tùy ý thành các vectơ.

4.2.1 Đặc trưng phân loại

Đặc trưng phân loại (Categorical Features) là loại đặc trưng trong dữ liệu mà mỗi giá trị đại diện cho một loại hay một nhóm cụ thể, không có mức độ liên tục giữa các giá trị.

Có hai loại chính của đặc trưng phân loại:

- Đặc trưng Nominal (Nominal Features): Đây là loại đặc trưng mà các giá trị không có thứ tự hay mức độ xác định. Để sử dụng trong các mô hình machine learning, cần thực hiện mã hóa (encoding) để chuyển đổi các giá trị này thành dạng số để mô hình có thể hiểu được. Có nhiều phương pháp mã hóa như One-Hot Encoding, Label Encoding, và Ordinal Encoding.

- Ví dụ: Màu sắc (đỏ, xanh, vàng), giới tính (nam, nữ), loại hình chủ sở hữu (cá nhân, doanh nghiệp, tổ chức).
- Đặc trưng Ordinal (Ordinal Features): Đây là loại đặc trưng mà các giá trị có thứ tự, mức độ xác định. Cần sắp xếp các giá trị theo thứ tự và mã hóa chúng thành các số thể hiện thứ tự tương ứng, có thể sử dụng Label Encoding hoặc ánh xạ các giá trị sang các số tương ứng.
 - Ví dụ: Trình độ học vấn (Tiểu học, Trung học, Đại học), Đánh giá sản phẩm (Tệ, Trung bình, Tốt).

Một ví dụ điển hình thường thấy trong bài toán trích chọn đặc trưng đó là bài toán định giá giá nhà, trong đó, các thuộc tính có giá trị số bao gồm giá nhà, số phòng bên cạnh đó cũng có các thuộc tính đó là khu vực. Khi đó, dữ liệu sẽ có dạng như sau:

```
# Dữ liệu mẫu
du_lieu = [
    {'gia': 850000, 'so_phong': 4, 'khu_vuc': 'Ha Dong'},
    {'gia': 700000, 'so_phong': 3, 'khu_vuc': 'Hoan Kiem'},
    {'gia': 650000, 'so_phong': 3, 'khu_vuc': 'Hai Ba Trung'},
    {'gia': 600000, 'so_phong': 2, 'khu_vuc': 'Hoan Kiem'}
]
```

Các tiếp cận đơn giản là đưa dữ liệu *khu_vuc* về dạng số bằng cách sử dụng một ánh xạ như:

```
{ 'Ha Dong': 1, 'Hoan Kiem': 2, 'Hai Ba Trung': 3};
```

Tuy nhiên, không phải lúc nào cách tiếp cận này cũng hữu ích vì một số mô hình này mặc định rằng các đặc trưng số phản ánh các đặc trưng cho các đại lượng đại số. Do đó, một ánh xạ như vậy sẽ ngầm định, Hà Đông < Hoàn Kiếm < Hai Bà Trưng, hoặc thậm chí Hai Bà Trưng - Hoàn Kiếm = Hà Đông, điều này không có nhiều ý nghĩa về dữ liệu.

Xử lý Đặc trưng phân loại thường sử dụng các phương pháp:

- One-Hot Encoding: Chuyển đổi mỗi giá trị trong đặc trưng phân loại thành một vectơ nhị phân, tạo ra các cột mới thể hiện có hoặc không có giá trị đó trong mỗi điểm dữ liệu.
- Label Encoding: Ánh xạ các giá trị phân loại sang các số nguyên duy nhất, thích hợp cho đặc trưng ordinal.
- Ordinal Encoding: Mã hóa giá trị phân loại thành các số nguyên tương ứng với thứ tự hoặc mức độ.

Trong ví dụ trên, một kỹ thuật mã hoá one-hot, tạo ra các cột dữ liệu cho thấy sự hiện diện hoặc không của một phân loại với giá trị tương ứng là 1 hoặc 0. Khi dữ liệu biểu diễn dưới dạng danh sách các từ điển, *DictVectorizer* của Scikit-Learn hoặc áp dụng mã hoá one-hot cho đặc trưng 'khu_vuc' bằng hàm của pandas để thực hiện việc này:

```
from sklearn.feature_extraction import DictVectorizer
vec = DictVectorizer(sparse=False, dtype=int)
vec.fit_transform(du_lieu)
```

Kết quả sử dụng vector one-hot như sau:

```
array([[850000, 1, 0, 0, 4],
       [700000, 0, 0, 1, 3],
       [650000, 0, 1, 0, 3],
       [600000, 0, 0, 1, 2]])
```

vec

▼ DictVectorizer
DictVectorizer(dtype=<class 'int'>, sparse=False)

Hình 4.4: Sử dụng one-hot vector.

Để xem ý nghĩa của từng cột, kiểm tra tên các thuộc tính bằng câu lệnh sau:

```
vec.get_feature_names_out()

array(['gia', 'khu_vuc=Ha Dong', 'khu_vuc=Hai Ba Trung',
       'khu_vuc=Hoan Kiem', 'so_phong'], dtype=object)
```

Hình 4.5: Ý nghĩa tên của từng cột.

4.2.2 Đặc trưng văn bản

Đặc trưng văn bản (Text Features) là loại đặc trưng quan trọng trong machine learning được sử dụng để biểu diễn và xử lý dữ liệu văn bản. Khi làm việc với dữ liệu văn bản, việc biểu diễn và rút trích thông tin từ văn bản đó là rất quan trọng để áp dụng các mô hình machine learning.

Một số phương pháp phổ biến để biểu diễn đặc trưng văn bản trong machine learning bao gồm:

- Bag-of-Words (BoW) - Túi từ: Mô hình không quan tâm đến cấu trúc hay thứ

tự của từ trong câu, chỉ quan trọng đếm số lần xuất hiện của từ. BoW chuyển đổi mỗi văn bản thành vectơ dựa trên số lần xuất hiện của các từ trong từ điển. Ưu điểm của BoW là đơn giản, dễ hiểu và dễ triển khai, tuy nhiên, mô hình BoW dễ mất thông tin về thứ tự từ và không xử lý được các ý nghĩa từ ghép.

- TF-IDF (Term Frequency-Inverse Document Frequency): Đánh giá tầm quan trọng của một từ trong một văn bản so với tất cả các văn bản khác trong tập dữ liệu. TF-IDF tính toán tần số của từ trong một văn bản (TF) và lưu ý đến mức độ quan trọng của từ đó trong toàn bộ tập dữ liệu (IDF) với ưu điểm giảm thiểu tần suất xuất hiện của các từ không quan trọng, tăng độ quan trọng của các từ hiếm, tuy nhiên, TF-IDF vẫn mất thông tin về thứ tự và ngữ cảnh của từ.
- Word Embeddings (Word Vectors) - nhúng từ: Biểu diễn từ dưới dạng vector trong không gian nhiều chiều, giữ nguyên thông tin về ngữ cảnh và ý nghĩa của từ. Các phương pháp nhúng từ được kể đến như Word2Vec, GloVe, FastText. Các phương pháp Word Embeddings đảm bảo toàn thông tin về ngữ cảnh và ý nghĩa của từ, hỗ trợ khả năng ánh xạ từ sang không gian vector. Bên cạnh đó, các phương pháp đòi hỏi dữ liệu lớn để huấn luyện, có thể cần thời gian và tài nguyên tính toán lớn.

Một vấn đề đối với đặc trưng văn bản là khi xử lý văn bản trước khi biểu diễn cần thực hiện:

- Loại bỏ stop words: Từ không mang nhiều ý nghĩa như "là", "và", "cái" có thể bị loại bỏ để giảm kích thước của vector.
- Chuẩn hóa văn bản: Chuyển văn bản về dạng chữ thường, loại bỏ dấu câu, số hoặc kí tự đặc biệt không cần thiết.

Ví dụ: hãy xem xét bộ ba cụm từ sau:

```
sample = ['Paris is the capital of France ',  
          'France is a country',  
          'A country is a distinct part of the world']
```

Để vector hóa dữ liệu dựa trên số từ, có thể xây dựng một cột biểu thị từ “*Paris*”, từ “*is*”, từ “*capital*”, v.v. Sử dụng *CountVectorizer* của Scikit-Learn như sau:

```
from sklearn.feature_extraction.text import CountVectorizer  
  
vec = CountVectorizer()  
X = vec.fit_transform(sample)  
X
```

Kết quả trả về một ma trận vector bao gồm có 3 hàng (3 đoạn văn bản trong sample)

và 10 cột tương ứng 10 từ riêng biệt trong cả 3 đoạn văn bản trong sample. Kết quả

```
<3x10 sparse matrix of type '<class 'numpy.int64'>'
      with 16 stored elements in Compressed Sparse Row format>
```

là một ma trận thưa ghi lại số lần mỗi từ xuất hiện; việc kiểm tra sẽ dễ dàng hơn khi chuyển đổi dữ liệu này thành DataFrame với các cột được gắn nhãn:

```
import pandas as pd
pd.DataFrame(X.toarray(), columns=vec.get_feature_names_out())
```

	capital	country	distinct	france	is	of	paris	part	the	world
0	1	0	0	1	1	1	1	0	1	0
1	0	1	0	1	1	0	0	0	0	0
2	0	1	1	0	1	1	0	1	1	1

Cách tiếp cận (TF-IDF), tính trọng số của số từ bằng thước đo tần suất xuất hiện trong các bản ghi/tài liệu. Cú pháp tính toán tương tự như ví dụ trước:

```
from sklearn.feature_extraction.text import TfidfVectorizer

vec = TfidfVectorizer()
X = vec.fit_transform(sample)
pd.DataFrame(X.toarray(), columns=vec.get_feature_names_out())
```

	capital	country	distinct	france	is	of	paris	part	the	world
0	0.49483	0.000000	0.000000	0.376331	0.292254	0.376331	0.49483	0.000000	0.376331	0.000000
1	0.00000	0.619805	0.000000	0.619805	0.481334	0.000000	0.00000	0.000000	0.000000	0.000000
2	0.00000	0.337295	0.443503	0.000000	0.261940	0.337295	0.00000	0.443503	0.337295	0.443503

4.2.3 Đặc trưng ảnh

Đặc trưng ảnh (Image Features) trong machine learning đóng vai trò quan trọng trong việc biểu diễn và xử lý dữ liệu hình ảnh. Việc trích chọn đặc trưng từ hình ảnh giúp mô hình hiểu và phân tích hình ảnh để thực hiện các nhiệm vụ như phân loại, nhận dạng đối tượng, phát hiện đối tượng, và nhiều ứng dụng khác.

Dưới đây là một số phương pháp phổ biến để trích chọn đặc trưng từ ảnh trong machine learning:

- Mô hình trích chọn đặc trưng được huấn luyện trước - *Pre-trained Feature Extraction Model*: VGG, ResNet, Inception, và các mô hình CNN khác: Các mô hình CNN được huấn luyện trước trên tập dữ liệu lớn như ImageNet đã học cách trích chọn các đặc trưng cấp cao từ ảnh.
- Transfer Learning: Sử dụng các mô hình này và loại bỏ phần phân loại (fully connected layers) để lấy các đặc trưng trước của ảnh. Sau đó, các đặc trưng này có thể được sử dụng trong các mô hình machine learning khác.

Phương pháp trích chọn đặc trưng cơ bản bao gồm Histogram của các đặc trưng màu sắc (Tính toán các histogram của các kênh màu (RGB hoặc HSV) để biểu diễn thông tin màu sắc trong ảnh) và HOG (Histogram of Oriented Gradients) (đặc trưng hình dạng của vật thể trong ảnh bằng cách tính toán hướng gradient của các vùng pixel.)

Dưới đây là các ví dụ và đoạn mã sử dụng thư viện OpenCV trong Python để trích xuất các đặc trưng bằng histogram của các đặc trưng màu sắc:

```
import cv2
import matplotlib.pyplot as plt

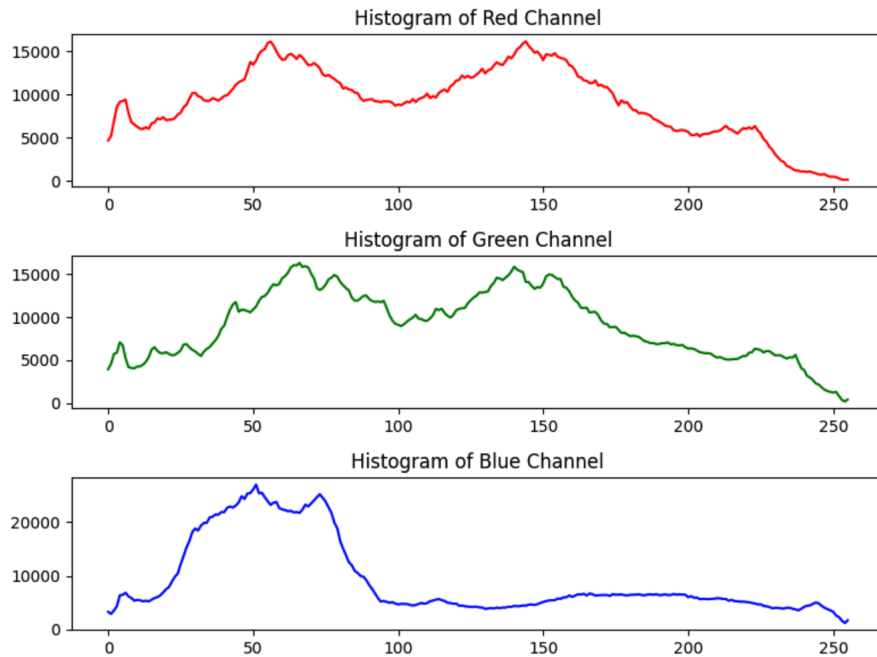
# Load Image from a file
img_path = 'path_to_your_image.jpg'
img = cv2.imread(img_path)

# Convert color space from BGR to RGB (OpenCV reads image as BGR)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Calculate the histograms of the color channels (R, G, B)
hist_r = cv2.calcHist([img_rgb], [0], None, [256], [0, 256])
hist_g = cv2.calcHist([img_rgb], [1], None, [256], [0, 256])
hist_b = cv2.calcHist([img_rgb], [2], None, [256], [0, 256])

# Draw a histogram of the color channels
plt.figure(figsize=(8, 6))
plt.subplot(3, 1, 1)
plt.plot(hist_r, color='red')
plt.title('Histogram of Red Channel')
plt.subplot(3, 1, 2)
plt.plot(hist_g, color='green')
plt.title('Histogram of Green Channel')
plt.subplot(3, 1, 3)
plt.plot(hist_b, color='blue')
plt.title('Histogram of Blue Channel')
plt.tight_layout()
plt.show()
```

Kết quả histogram các đặc trưng màu sắc của ảnh hiển thị như sau:



Tương tự với đặc trưng HOG như sau:

```
import cv2
from skimage.feature import hog
from skimage import exposure
import matplotlib.pyplot as plt

#Read images from file
img_path = 'path_to_your_image.jpg'
img = cv2.imread(img_path)

# Convert image to grayscale image
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

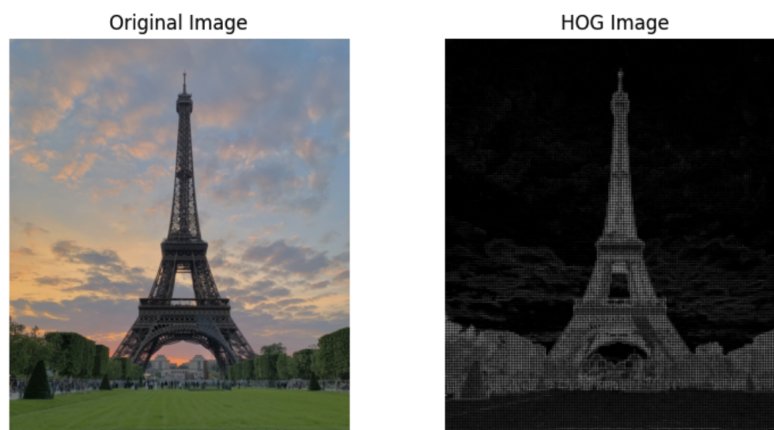
# Calculate HOG features
fd, hog_image = hog(gray_img, orientations=9, pixels_per_cell=(8, 8),
                    cells_per_block=(2, 2), visualize=True)

# Adjust the brightness of HOG photos
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0,
10))

# Displays original images and HOG images
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Original Image')
```

```
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap='gray')
plt.title('HOG Image')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Kết quả histogram các đặc trưng màu sắc của ảnh hiển thị như sau:



Bên cạnh đó là phương pháp trích chọn đặc trưng bằng Convolutional Neural Networks (CNN): CNN được thiết kế để tự động học các đặc trưng từ dữ liệu hình ảnh bằng cách sử dụng các lớp convolutional và pooling layers. CNN có khả năng tự động học các đặc trưng cấp cao từ ảnh, bao gồm cả hình dạng, cấu trúc, và các đặc điểm quan trọng của đối tượng. Mô hình CNN nổi tiếng như VGG, ResNet, Inception, và EfficientNet.

Trích chọn đặc trưng bằng Convolutional Autoencoders cũng được sử dụng bằng kiến trúc autoencoder với các lớp convolutional để học biểu diễn ẩn (latent) của ảnh, sau đó sử dụng các đặc trưng ẩn này cho các mô hình machine learning. Với ưu điểm có thể học biểu diễn tốt của ảnh mà không cần dữ liệu được gán nhãn.

Dưới đây là một ví dụ đơn giản sử dụng thư viện Python TensorFlow để trích xuất đặc trưng từ hình ảnh bằng CNN:

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import preprocess_input
import numpy as np

# Load pre-trained VGG16 model (weights on ImageNet dataset)
model = VGG16(weights='imagenet', include_top=False)
```

```

# Load and preprocess an example image
img_path = 'path_to_your_image.jpg' # Replace 'path_to_your_image.jpg'
    with the actual path to your image
img = image.load_img(img_path, target_size=(224, 224)) # VGG16 model
    requires input shape (224, 224)
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

# Extract features using VGG16 model
features = model.predict(x)

# Display extracted features shape
print("Kích thước của ảnh được trích xuất:", features.shape)

```

Kết quả kích thước của ảnh sẽ là một ảnh (7, 7, 512):

```

Downloading data from https://
pplications/vgg16/vgg16_weight
58889256/58889256 [=====
1/1 [=====
Shape of extracted features: (

```

4.3 Học có giám sát

Học có giám sát (Supervised Learning) trong machine learning là một phương pháp mà mô hình được huấn luyện từ dữ liệu đã được gán nhãn trước đó. Điều này có nghĩa là mỗi mẫu dữ liệu trong tập huấn luyện đã được gán nhãn với đầu ra mong muốn, giúp mô hình học từ các cặp dữ liệu đầu vào (input) và đầu ra mong muốn (outcome hoặc label) cặp dữ liệu này còn được gọi (*data, label*).

Các bước chính của học có giám sát:

- Thu thập dữ liệu: Tập hợp dữ liệu được gán nhãn đầy đủ với các đặc trưng (features) và nhãn (labels).
- Chuẩn bị dữ liệu: Tiền xử lý dữ liệu, bao gồm việc xử lý dữ liệu thiếu, chuẩn

hóa dữ liệu, mã hóa các đặc trưng phân loại, chia tập dữ liệu thành tập huấn luyện và tập kiểm tra.

- **Chọn mô hình:** Lựa chọn một loại mô hình phù hợp với bài toán cụ thể, ví dụ như Linear Regression, Decision Trees, Support Vector Machines (SVM), Neural Networks, và các mô hình khác.
- **Huấn luyện mô hình:** Dùng tập dữ liệu huấn luyện để huấn luyện mô hình, mô hình sẽ cố gắng học từ dữ liệu và điều chỉnh các tham số để dự đoán nhãn tốt nhất có thể.
- **Đánh giá và tinh chỉnh:** Sử dụng tập dữ liệu kiểm tra để đánh giá hiệu suất của mô hình. Nếu cần, thực hiện tinh chỉnh mô hình bằng cách thay đổi các siêu tham số hoặc kiến trúc của mô hình.
- **Dự đoán và triển khai:** Sử dụng mô hình đã được huấn luyện để dự đoán nhãn cho dữ liệu mới hoặc triển khai mô hình trong môi trường sản xuất.

Học có giám sát thường được sử dụng trong nhiều bài toán như dự đoán, phân loại, và hồi quy. Phương pháp này đòi hỏi dữ liệu đã được gán nhãn đúng và thường yêu cầu nỗ lực lớn trong việc thu thập và chuẩn bị dữ liệu. Tuy nhiên, khi có đủ dữ liệu huấn luyện, học có giám sát thường mang lại kết quả mạnh mẽ và chính xác trong việc dự đoán hoặc phân loại dữ liệu mới.

Dưới đây là các dạng chính và công thức của học có giám sát:

- **Phân loại (Classification):** Phân loại là bài toán dự đoán một nhãn (label) thuộc một tập hợp hữu hạn các lớp hoặc danh mục cho mỗi mẫu dữ liệu mới. Bài toán phân loại được biểu diễn dưới dạng:

$$f : X \rightarrow Y$$

Trong đó, X là không gian các đặc trưng (input variables), Y tập các nhãn hoặc lớp (output space), và f là hàm ánh xạ từ X đến Y .

- **Hồi quy (Regression):** dự đoán giá trị liên tục hoặc số thực dựa trên các đặc trưng đầu vào biểu diễn dưới dạng:

$$f : X \rightarrow \mathbb{R}$$

Trong đó, X là không gian các đặc trưng (input variables), $\mathbb{R}$ là không gian số thực (output space), và f là hàm ánh xạ từ X đến $\mathbb{R}$.

Với $X = \{x_1, x_2, \dots, x_N\}$ là không gian các thuộc tính (input variables), $Y = \{y_1, y_2, \dots, y_N\}$ là tập các nhãn, lớp hoặc giá trị (output labels), với x_i, y_i là vectơ. Các cặp dữ liệu đã biết $(x_i, y_i) \in X \times Y$ được gọi là tập dữ liệu huấn luyện - *training set*. Từ tập dữ

liệu huấn luyện này, cần tạo một hàm ánh xạ f từng phần từ tập X tới phần tử (gần đúng) tương ứng của tập Y :

$$y_i \approx f(x_i), \forall i = 1, 2, \dots, N$$

Mục tiêu là xấp xỉ hàm f đủ tốt để khi có dữ liệu x mới, có thể tính nhản tương ứng của nó $y = f(x)$. Điều này giúp mô hình có khả năng dự đoán đầu ra chính xác cho dữ liệu mới mà nó chưa từng thấy trước đó.

Trong học có giám sát có nhiều phương pháp và mô hình khác nhau được sử dụng để dự đoán hoặc phân loại dữ liệu dựa trên các cặp (*data*, *label*) đã được gán nhản trong tập dữ liệu huấn luyện. Một số phương pháp phổ biến trong học có giám sát bao gồm:

- *Linear Regression (Hồi quy tuyến tính)* dùng để dự đoán giá trị liên tục dựa trên một hoặc nhiều đặc trưng đầu vào như dự đoán giá nhà, doanh số bán hàng dự kiến, dự báo thời tiết và các vấn đề liên quan đến dự đoán giá trị số.. Tuy nhiên, hồi quy tuyến tính có độ chính xác thấp khi dữ liệu có mối quan hệ phức tạp.
- *Logistic Regression (Hồi quy Logistic)* sử dụng để phân loại các mẫu dựa trên các đặc trưng vào các nhóm hoặc lớp khác nhau. Hồi quy logistic rất hiệu quả trong phân loại nhị phân và các bài toán phân loại đơn giản như ự đoán khả năng xảy ra sự kiện (có hoặc không), phân loại email vào thư rác hoặc không phải thư rác, và các bài toán phân loại khác nhưng không thể xử lý các mối quan hệ phi tuyến tính.
- *Decision Trees (Cây quyết định)* tạo ra các quyết định phân loại bằng cách đặt câu hỏi về các đặc trưng của dữ liệu; Random Forests (Rừng ngẫu nhiên) kết hợp nhiều cây quyết định để tăng tính chính xác. Hai phương pháp này có khả năng xử lý cả dữ liệu số học và phân loại như hệ thống giám sát giao thông, hệ thống khuyến nghị, và phân loại tín dụng nhưng dễ bị overfitting.
- *Support Vector Machines (SVM)* tìm ra ranh giới phân chia tốt nhất giữa các lớp hoặc nhóm dữ liệu bằng cách tìm ra siêu phẳng (hyperplane) phân chia tốt nhất giữa chúng. SVM hiệu suất tốt trong không gian chiều cao, đa dạng kernel function để xử lý các dữ liệu phi tuyến tính như Phân loại hình ảnh, nhận dạng ký tự.
- *Naive Bayes* là một mô hình dự đoán dựa trên nguyên tắc của Định lý Bayes, giả định các đặc trưng độc lập với nhau. Naive Bayes có hiệu suất nhanh chóng tốt trong một số trường hợp với dữ liệu lớn như phân loại văn bản, phân loại

tin tức, tuy nhiên, giả định đặc trưng độc lập có thể không phù hợp với thực tế và khả năng dự đoán yếu khi các đặc trưng không độc lập.

- *Neural Networks (Deep Learning)*: Mạng Neural Network với nhiều lớp (Deep Learning) có khả năng học biểu diễn phức tạp từ dữ liệu đầu vào, với khả năng học biểu diễn phức tạp từ dữ liệu và hiệu suất cao đối với nhiều bài toán phức tạp được áp dụng cho nhận diện hình ảnh, xử lý ngôn ngữ tự nhiên, dự đoán chuỗi thời gian, và nhiều ứng dụng khác nhau, đòi hỏi lượng dữ liệu lớn để huấn luyện hiệu quả và tài nguyên tính toán và thời gian huấn luyện lớn.

4.4 Học không giám sát

Học không có giám sát (Unsupervised Learning) trong machine learning là một phương pháp mà mô hình học từ dữ liệu mà không có nhãn hoặc kết quả đầu ra được gán trước cho từng điểm dữ liệu. Trong học không có giám sát, mục tiêu chính là tìm ra cấu trúc ẩn hoặc những đặc điểm chung trong dữ liệu mà không cần thông tin nhãn.

Các dạng chính và công thức của học không có giám sát bao gồm:

- **Phân cụm (Clustering)**: Phân cụm là bài toán nhóm các điểm dữ liệu các thuộc tính tương tự vào các nhóm hoặc cụm mà không cần biết nhãn trước. Mục tiêu là tìm cách chia tập dữ liệu thành các nhóm sao cho các điểm trong mỗi nhóm có tính tương đồng cao nhất có thể và các nhóm khác biệt lẫn nhau.

Hai thuật toán phổ biến trong bài toán phân cụm là:

- **K-Means Clustering**: Phân chia dữ liệu thành các cụm dựa trên sự tương tự giữa các điểm dữ liệu.
- **Hierarchical Clustering**: Xây dựng cấu trúc phân cấp của các cụm dựa trên sự tương đồng giữa các điểm.
- **Phân tích thành phần chính (Principal Component Analysis - PCA)**: là một phương pháp giảm chiều dữ liệu để tìm ra các thành phần chính (principal components) chứa lượng thông tin lớn nhất trong dữ liệu ban đầu. PCA tìm ra các vector riêng (eigenvectors) của ma trận hiệp phương sai (covariance matrix) tương ứng với các giá trị riêng (eigenvalues) lớn nhất để biểu diễn dữ liệu ở các chiều mới có kích thước nhỏ hơn.
- **Phát hiện bất thường (Anomaly Detection)**: Phát hiện các điểm dữ liệu bất thường, không phù hợp, hay các mẫu hiếm trong tập dữ liệu. Mục tiêu là tìm ra các điểm dữ liệu khác thường so với phần lớn dữ liệu còn lại, không cần biết nhãn trước.

- LOF (Local Outlier Factor) là một phương pháp sử dụng trong phát hiện bất thường để phát hiện các điểm dữ liệu bất thường dựa trên độ "khác biệt" so với các hàng xóm xung quanh.
- Khai phá luật kết hợp (Association Rule Learning) là phân tích mối quan hệ giữa các mục tiêu trong dữ liệu, nhưng không có mục tiêu dự đoán cụ thể và tìm ra các quy luật kết hợp giữa các mục tiêu hoặc biểu thị mối quan hệ giữa chúng.
 - Apriori Algorithm là một phương pháp điển hình phân tích mối quan hệ giữa các mục tiêu trong dữ liệu để tìm các quy tắc liên kết hoặc mẫu thường xuyên.
- Mạng neural không giám sát (Unsupervised Neural Networks): là một loại mạng neural được sử dụng trong học không giám sát, nơi mô hình học từ dữ liệu không có nhãn và tự động tìm ra cấu trúc hoặc thông tin ẩn trong dữ liệu đầu vào. Có một số loại mạng neural không giám sát, trong đó một trong những phổ biến nhất là "Autoencoder" mô hình hóa tự động dữ liệu đầu vào, học cách biểu diễn dữ liệu mà không cần nhãn.

Trong học không có giám sát, không có thông tin nhãn được cung cấp cho mô hình. Mục tiêu là khám phá cấu trúc, tính chất hoặc mối quan hệ trong dữ liệu mà không cần sự can thiệp của con người. Phương pháp này thường được sử dụng để hiểu sâu hơn về dữ liệu, phát hiện cấu trúc ẩn và tạo ra cái nhìn tổng quan về tập dữ liệu mà không cần thông tin nhãn cụ thể.

4.5 Học bán giám sát

Học bán giám sát (Semi-Supervised Learning) trong machine learning là một phương pháp kết hợp giữa học có giám sát và học không giám sát, trong đó mô hình được huấn luyện từ cả dữ liệu có nhãn và dữ liệu không có nhãn.

Trong học bán giám sát, thông thường chỉ một phần nhỏ dữ liệu được gán nhãn, trong khi phần lớn dữ liệu không có nhãn. Mô hình sẽ sử dụng thông tin từ cả hai loại dữ liệu này để học và tối ưu hóa hiệu suất dự đoán hoặc phân loại.

Học bán giám sát hoạt động theo phương thức sau:

- Dữ liệu có nhãn: Sử dụng dữ liệu có nhãn để huấn luyện mô hình theo cách thông thường trong học có giám sát. Dữ liệu này cung cấp thông tin rõ ràng về đầu ra mong muốn.
- Dữ liệu không có nhãn: Sử dụng dữ liệu không có nhãn để học các đặc trưng, cấu trúc, hoặc thông tin ẩn trong dữ liệu mà không cần thông tin nhãn cụ thể.

Các phương pháp học không giám sát như gom cụm (clustering), PCA, hoặc autoencoders có thể được sử dụng để khám phá thông tin từ dữ liệu không có nhãn.

- Kết hợp thông tin: Mô hình học bán giám sát sử dụng thông tin từ cả dữ liệu có nhãn và không có nhãn để cải thiện hiệu suất dự đoán hoặc phân loại. Thông tin từ dữ liệu không có nhãn có thể giúp mô hình học các đặc trưng tổng quát hơn và giảm thiểu hiện tượng quá khớp (overfitting) đối với dữ liệu huấn luyện nhỏ.

Có nhiều ứng dụng của học bán giám sát trong thực tế như:

- Y học: Dữ liệu y khoa thường có rất ít dữ liệu có nhãn (ví dụ: dữ liệu về bệnh nhân được gán nhãn), nhưng có rất nhiều dữ liệu không có nhãn (ví dụ: dữ liệu y học từ hồ sơ bệnh viện). Học bán giám sát có thể giúp cải thiện việc dự đoán hoặc phân loại trong y học.
- Phân loại văn bản: Trong phân loại văn bản, chỉ một phần nhỏ văn bản có nhãn, trong khi hầu hết văn bản không có nhãn. Học bán giám sát có thể được sử dụng để cải thiện hiệu suất phân loại.

Có một số phương pháp và thuật toán được sử dụng trong học bán giám sát để kết hợp thông tin từ cả dữ liệu có nhãn và không có nhãn:

- Mô hình Hỗn hợp (Mixture Models): Sử dụng mô hình phân phối hỗn hợp để mô hình hóa cả dữ liệu có nhãn và không có nhãn.
- Mô hình Đồng huấn luyện (Co-Training): Một phương pháp trong đó mô hình được huấn luyện trên hai tập dữ liệu không trùng lặp, và sau đó mô hình sẽ trao đổi thông tin để cải thiện hiệu suất.
- Học theo đệ quy (Semi-Supervised Recursive Learning): Sử dụng cấu trúc dữ liệu đệ quy để kết hợp thông tin từ cả dữ liệu có nhãn và không có nhãn.
- Mô hình Transductive Learning: Transductive SVM sử dụng SVM (Support Vector Machine) để dự đoán nhãn cho các điểm dữ liệu không có nhãn bằng cách tận dụng thông tin cấu trúc của dữ liệu.
- Học tập kết hợp (Ensemble Learning): Semi-Supervised Ensemble Learning kết hợp các mô hình học máy được huấn luyện trên dữ liệu có nhãn và không có nhãn để cải thiện hiệu suất.
- Học gom cụm và dự đoán (Cluster-then-Predict): Cluster-then-Predict: phân cụm dữ liệu không có nhãn, sau đó áp dụng mô hình học có giám sát trong từng nhóm để dự đoán nhãn cho các điểm dữ liệu.
- Transfer Learning sử dụng thông tin từ một nhiệm vụ có nhãn để cải thiện hiệu

suất cho một nhiệm vụ có ít hoặc không có nhãn.

Học bán giám sát là một lựa chọn hữu ích khi không có đủ dữ liệu có nhãn hoặc khi việc gán nhãn dữ liệu tốn kém hoặc khó khăn. Đây là một trong những cách tiếp cận mạnh mẽ trong việc xử lý dữ liệu khi thông tin nhãn không có sẵn đầy đủ.

4.6 Học tăng cường

Học tăng cường (Reinforcement Learning) trong machine learning là một nhánh quan trọng, trong đó một tác nhân (agent) học từ trải nghiệm tương tác với một môi trường để đạt được mục tiêu cụ thể và tối ưu hóa một phần thưởng (reward) hoặc hàm mục tiêu. Cụ thể, tác nhân thực hiện các hành động trong một môi trường và nhận phản hồi từ môi trường qua các phần thưởng hoặc hình phạt. Mục tiêu của tác nhân là tìm ra chiến lược tối ưu để tăng cường phần thưởng hoặc giảm hình phạt.

Học tăng cường là cách tiếp cận mà tác nhân học từ các quyết định mà nó thực hiện, thông qua quá trình thử nghiệm và lặp lại trong môi trường để tìm ra hành động tốt nhất dựa trên các phần thưởng nhận được. Quá trình học này thường bao gồm:

- Tác nhân (Agent): Đại diện cho thuật toán hoặc hệ thống cần học để tối ưu hóa các quyết định.
- Môi trường (Environment): Biểu diễn cho không gian mà tác nhân tương tác và nhận phản hồi từ đó.
- Hành động (Action): Các hành động tác nhân thực hiện trong môi trường.
- Phần thưởng (Reward): Phản hồi từ môi trường về việc tác nhân thực hiện hành động đó.

Học tăng cường có nhiều ứng dụng trong thực tế, bao gồm những lĩnh vực sau:

- Robot và Tự động hóa: Học tăng cường được sử dụng để điều khiển robot trong môi trường động để đạt được mục tiêu như di chuyển, vận chuyển vật phẩm, hoặc thực hiện công việc cụ thể. Các hệ thống tự động hoá, như hệ thống quản lý kho hoặc hệ thống điều khiển tự động cũng có thể áp dụng học tăng cường để tối ưu hóa hiệu suất và quy trình.
- Trò chơi: Học tăng cường được sử dụng rộng rãi trong việc huấn luyện AI chơi các trò chơi như cờ vua, game video, hoặc các trò chơi chiến lược phức tạp như StarCraft hoặc Dota 2. Bên cạnh đó là quản lý Nhân vật trong trò chơi video, AI có thể học từ trải nghiệm để quản lý nhân vật, chuyển động, và ra quyết định trong môi trường ảo.
- Tự lái và Xe tự hành: Học tăng cường được sử dụng trong phát triển các hệ

thống tự lái ô tô để học từ trải nghiệm và tối ưu hóa hành vi lái xe.

- **Quản lý Tài chính và Tài nguyên:** Các thuật toán học tăng cường được sử dụng để tối ưu hóa quản lý rủi ro và đầu tư tài chính. Trong nông nghiệp hoặc quản lý môi trường, học tăng cường có thể được áp dụng để tối ưu hóa việc quản lý nguồn tài nguyên như nước, năng lượng, và đất đai.
- **Quản lý Dịch vụ và Tối ưu hóa Hệ thống:** Học tăng cường có thể giúp tối ưu hóa quy trình quản lý dịch vụ khách hàng và việc ra quyết định trong thực tế. Trong lĩnh vực công nghiệp và sản xuất, học tăng cường có thể được sử dụng để tối ưu hóa hoạt động của hệ thống sản xuất và quản lý chuỗi cung ứng.

Các thuật toán học tăng cường như Q-learning, SARSA, Deep Q Networks (DQN), hoặc các thuật toán như Policy Gradient, Actor-Critic là những phương pháp phổ biến trong lĩnh vực này:

- **Q-learning:** Một trong những thuật toán học tăng cường cơ bản, dựa trên giá trị Q, đo lường giá trị của một cặp (trạng thái, hành động). Nó cập nhật giá trị Q dựa trên phần thưởng nhận được và tìm ra chiến lược tối ưu để chọn hành động.
- **SARSA (State-Action-Reward-State-Action):** Thuật toán học tăng cường khác, giống như Q-learning nhưng cập nhật giá trị Q dựa trên cặp (trạng thái, hành động) thực tế thay vì tối ưu.
- **Deep Q Networks (DQN):** Sử dụng mạng neural sâu để xấp xỉ hàm giá trị Q, cho phép học từ các môi trường tương tác phức tạp hơn. DQN đặc biệt hiệu quả trong việc giải quyết các bài toán học tăng cường trong trò chơi video.
- **Policy Gradient:** Tối ưu hóa chính sách hành động trực tiếp thay vì ước lượng hàm giá trị Q. Thường được sử dụng cho các bài toán có không gian hành động lớn hoặc liên tục.
- **Actor-Critic:** Kết hợp giữa các mô hình học chính sách (Actor) và học giá trị (Critic) để cùng nhau học và cải thiện chính sách hành động. Hiệu quả trong việc giải quyết các bài toán học tăng cường phức tạp.
- **Proximal Policy Optimization (PPO):** Thuật toán cải tiến của Policy Gradient, giúp ổn định quá trình học và tối ưu hóa chính sách hành động. PPO được sử dụng rộng rãi trong robotica và các bài toán tối ưu hóa chính sách.
- **Trust Region Policy Optimization (TRPO):** Tối ưu hóa chính sách hành động với việc giữ cho các bước cập nhật nhỏ, đảm bảo tính ổn định của học tập. TRPO sử dụng trong các nhiệm vụ có thể dễ bị phá vỡ nếu các bước cập nhật lớn.

Reinforcement Learning đã được áp dụng rộng rãi trong các lĩnh vực như robotica, tự động hóa, trò chơi, quản lý tài chính, và nhiều lĩnh vực khác nữa. Điều này cho thấy sự mạnh mẽ và tiềm năng của phương pháp này trong việc tìm hiểu và tối ưu hóa từ trải nghiệm tương tác với môi trường.

Bài tập Chương 4

Bài tập 4.1 Mục tiêu của việc đánh giá mô hình là gì? Nêu các độ đo đánh giá cho mô hình phân loại: Accuracy, Precision, Recall, F1-score?

Bài tập 4.2 Giả sử có 2000 ảnh cần phân loại bệnh nhân có bị bệnh ung thư da hay không, trong dữ liệu có 1800 ảnh là bệnh nhân bị mắc bệnh, 200 ảnh không phải là ảnh bệnh nhân bị mắc bệnh. Một mô hình X cho kết quả dự đoán như sau:

- Trong 1800 ảnh là bệnh nhân bị mắc bệnh, 1600 ảnh được dự đoán đúng, 200 ảnh được dự đoán sai.
- Trong 200 ảnh không phải là ảnh bệnh nhân bị mắc bệnh thì có 170 ảnh dự đoán đúng và 30 ảnh dự đoán sai.

Hãy tính các độ đo Accuracy, Precision, Recall, F1-score và nhận xét về độ chính xác của mô hình?

Bài tập 4.3 1. Nêu các độ đo đánh giá mô hình hồi quy: MSE và MAE?

2. Cho dữ liệu sau:

STT	1	2	3	4	5	6	7	8	9	10	11
Giới tính	1	2	1	2	2	1	2	2	1	1	1
Giờ ôn tập	5	10	5	3	8	5	8	5	6	11	2
Điểm Thi cuối kỳ	70	93	69	74	88	69	79	80	78	97	70

Điểm thi cuối kỳ được dự đoán theo giới tính (1 = nữ, 2 = nam) và giờ ôn tập theo phương trình sau: $Diem_thi = 42.5 + 7.942 * (Gioi_tinh) + 3.235 * (Gio_on_tap)$. Hãy tính các độ đo MSE và MAE cho mô hình và nhận xét?

Bài tập 4.4 Xác định từ điển và các vectơ đặc trưng của các câu văn trong đoạn văn sau bằng phương pháp “bag-of-word”:

“Trong một khu vườn rộng, có nhiều người thích những hoạt động khác nhau. Lan yêu thích việc vẽ tranh và đọc sách. An cũng thích vẽ tranh, nhưng bạn ấy không thích đọc sách. Hoa không thích vẽ tranh, nhưng bạn ấy rất thích đọc sách và chơi piano. Hoàng không hề thích cả hai hoạt động này, thay vào đó bạn ấy thích chơi thể thao và đi du lịch.”

Bài tập 4.5 Các bài tập thảo luận: Lựa chọn các đề tài thảo luận dưới đây về các

thuật toán machine learning:

1. *Phân loại Nhãn (Classification)*: Sử dụng tập dữ liệu như Iris, MNIST hoặc CIFAR-10 để xây dựng mô hình phân loại học máy để phân loại các đối tượng hoặc hình ảnh vào các nhóm cụ thể.
2. *Dự đoán Giá (Regression)*: Sử dụng dữ liệu về giá nhà, doanh số bán hàng, hoặc thông tin tài chính để xây dựng mô hình học máy để dự đoán giá trị số liệu liên tục.
3. *Phân loại Văn bản (Text Classification)*: Xây dựng mô hình phân loại để phân loại văn bản thành các danh mục cụ thể như phân loại email vào thư rác hoặc phân loại tin tức vào các chủ đề khác nhau.
4. *Dự đoán Chuyển bay Trễ (Flight Delay Prediction)*: Xây dựng mô hình dự đoán việc chuyển bay sẽ trễ dựa trên các dữ liệu lịch sử.
5. *Dự đoán Sản phẩm mua tiếp theo (Next Purchase Prediction)*: Xây dựng mô hình để dự đoán sản phẩm mà người dùng có thể mua tiếp theo dựa trên lịch sử mua hàng của họ.
6. *Dự đoán Bệnh (Disease Prediction)*: Sử dụng dữ liệu y tế để dự đoán nguy cơ mắc các bệnh như tiểu đường, ung thư dựa trên các chỉ số y tế.
7. *Xử lý Ảnh (Image Processing)*: Sử dụng các mô hình học sâu để nhận diện đối tượng trong ảnh, như việc nhận diện khuôn mặt, đếm vật thể, hoặc xác định biển báo giao thông.
8. *Tìm kiếm thông tin (Information Retrieval)*: Xây dựng hệ thống tìm kiếm thông tin đa dạng dựa trên các thuật toán xử lý ngôn ngữ tự nhiên (NLP) để tìm kiếm, phân loại và trích xuất thông tin từ văn bản.
9. *Tự lái (Self-Driving)*: Xây dựng mô hình tự lái ô tô sử dụng dữ liệu từ cảm biến và ảnh để điều khiển ô tô tự động và an toàn trên đường.



5. Cơ sở dữ liệu và SQL

Cơ sở dữ liệu đóng vai trò quan trọng trong việc truy xuất và quản lý dữ liệu, hỗ trợ quyết định kinh doanh, phát triển ứng dụng và website, cũng như cung cấp nền tảng quan trọng cho việc quản lý dự án... Chương 5 giới thiệu những khái niệm cơ bản về cơ sở dữ liệu, sự quan trọng của việc tổ chức và lưu trữ thông tin. Đồng thời giới thiệu về ngôn ngữ truy vấn SQL để tương tác với cơ sở dữ liệu, từ một số nội dung cơ bản như cách tạo bảng, thêm, sửa đổi, xóa dữ liệu,... đến cơ sở dữ liệu nâng cao như truy vấn con, tối ưu truy vấn và NoSQL.

5.1 Cơ sở dữ liệu cơ bản

Trong lĩnh vực khoa học dữ liệu, dữ liệu thường được lưu trữ trong các cơ sở dữ liệu (database) - một hệ thống được thiết kế để lưu trữ, truy vấn hiệu quả dữ liệu. Phần lớn trong số các hệ cơ sở dữ liệu là cơ sở dữ liệu quan hệ như là PostgreSQL, MySQL và SQL Server, thường chứa dữ liệu trong các bảng và thường được truy vấn bằng ngôn ngữ truy vấn có cấu trúc (SQL) - một ngôn ngữ khai báo để thao tác với dữ liệu. SQL là một phần khá quan trọng trong bộ công cụ của nhà khoa học dữ liệu. Chương 5 này sẽ giới thiệu về các câu lệnh SQL cơ bản cũng như một số kiến thức nâng cao về quản trị hệ cơ sở dữ liệu. Trong thực tế, để xây dựng một quy trình xử lý dữ liệu hoàn chỉnh, quản trị hệ cơ sở dữ liệu là một trong những bước đầu tiên và quan trọng. Dữ liệu khởi nguồn từ hệ quản trị cơ sở dữ liệu và khi xử lý xong cũng sẽ được lưu trữ trở lại cơ sở dữ liệu. Khi hệ thống hoạt động, số lượng và tần suất truy vấn cơ sở dữ liệu là rất lớn. Do đó, việc tối ưu quản trị hệ cơ sở dữ liệu là yếu tố quyết định đến hiệu năng của hệ thống.

5.1.1 CREATE TABLE và INSERT

Một cơ sở dữ liệu quan hệ là một tập hợp các bảng và các mối quan hệ giữa chúng. Một bảng đơn giản chỉ là một tập hợp các hàng, tương tự như một số ma trận đã làm

việc. Tuy nhiên, một bảng cũng đi kèm với một lược đồ cố định bao gồm tên cột và loại cột.

Ví dụ, một bộ dữ liệu người dùng chứa thông tin cho từng người dùng với mã người dùng (`user_id`), tên (`user_name`) người dùng, và số lượng bạn của người dùng đó (`num_friends`)

```
users = [[0, "Hero", 0],
[1, "Dunn", 2],
[2, "Sue", 3],
[3, "Chi", 3]]
```

Trong ngôn ngữ SQL, có thể tạo bảng với câu lệnh sau:

```
CREATE TABLE users (
    user_id int NOT NULL,
    name varchar (200),
    num_friends int);
```

Lưu ý rằng đã chỉ định rằng `user_id` và `num_friends` phải là số nguyên (và `user_id` không được phép là NULL, điều này chỉ ra giá trị thiếu và giống như None) và `user_name` nên là một chuỗi có độ dài 200 Ký tự hoặc ít hơn. Sử dụng các kiểu dữ liệu SQL một cách tương tự.

Một hàng có thể được thêm vào bảng user này bằng cách sử dụng câu lệnh INSERT như sau:

```
INSERT INTO users (user_id, name, num_friends) VALUES (0, 'Hero', 0);
```

Trong một cơ sở dữ liệu SQL thực tế, phải chỉ định liệu bất kỳ cột cụ thể nào có thể chứa giá trị null (None); để đơn giản hóa sẽ nói rằng bất kỳ cột nào cũng có thể.

Ngoài ra, một số phương pháp cho phép xem xét một bảng như một List[Row], chủ yếu sẽ sử dụng nó để kiểm thử mã nguồn:

```
def __getitem__(self, idx: int) -> Row:
    return self.rows[idx]
def __iter__(self) -> Iterator[Row]:
    return iter(self.rows)
def __len__(self) -> int:
    return len(self.rows)
```

Cũng có thể in bảng dữ liệu ra như sau:

```
def __repr__(self):
    """Pretty representation of the table: columns then rows"""
    rows = "\n".join(str(row) for row in self.rows)
    return f"{self.columns}\n{rows}"
```


Có thể thử tạo một bảng Users theo câu lệnh bên dưới đây:

```
# Constructor requires column names and types
users = Table(['user_id', 'name', 'num_friends'], [int, str, int])
users.insert([0, "Hero", 0])
users.insert([1, "Dunn", 2])
users.insert([2, "Sue", 3])
users.insert([3, "Chi", 3])
users.insert([4, "Thor", 3])
users.insert([5, "Clive", 2])
users.insert([6, "Hicks", 3])
users.insert([7, "Devin", 2])
users.insert([8, "Kate", 2])
users.insert([9, "Klein", 3])
users.insert([10, "Jen", 1])
```

5.1.2 UPDATE

Trong quá trình thao tác với cơ sở dữ liệu, cần phải cập nhật lại dữ liệu đã được lưu trong cơ sở dữ liệu. Ví dụ, người dùng có tên Dunn có thêm một người bạn mới, cần cập nhật lại dữ liệu cho Dunn.

```
UPDATE users
SET num_friends = 3 WHERE user_id = 1;
```

Điểm cần lưu ý khi cập nhật dữ liệu là:

- Bảng nào cần cập nhật dữ liệu
- Hàng nào cần cập nhật dữ liệu
- Trường nào cần cập nhật dữ liệu
- Giá trị cập nhật mới là giá trị nào

Để định nghĩa một phương thức cho phép cập nhật dữ liệu trong bảng

```
def update(self, updates: Dict[str, Any], predicate: WhereClause = lambda row:
    True):
    # First make sure the updates have valid names and types
    for column, new_value in updates.items():
        if column not in self.columns:
            raise ValueError(f"invalid column: {column}")
        typ3 = self.col2type(column)
        if not isinstance(new_value, typ3) and new_value is not None:
            raise TypeError(f"expected type {typ3}, but got {new_value}")
    # Now update
    for row in self.rows:
        if predicate(row):
            for column, new_value in updates.items():
```

```
row[column] = new_value
```

Sau đó, để cập nhật một dữ liệu mới, có thể gọi phương thức bên trên

```
assert users[1]['num_friends'] == 2 # Original value
users.update({'num_friends' : 3}, lambda row: row['user_id'] == 1)
assert users[1]['num_friends'] == 3
```

5.1.3 DELETE

Có hai cách để xóa dòng dữ liệu của một bảng trong SQL. Có thể xóa toàn bộ hàng trong một bảng bằng câu lệnh sau

```
DELETE FROM users;
```

Tuy nhiên, xóa tất cả các dòng trong bảng khá nguy hiểm, thường thì một mệnh đề điều kiện sẽ được sử dụng để lọc ra những hàng nào cần phải xóa

```
def delete(self, predicate: WhereClause = lambda row: True) -> None:
    DELETE FROM users WHERE user_id = 1;
```

Một phương thức được sử dụng để xóa dữ liệu này được định nghĩa như sau để thực thi được câu lệnh SQL bên trên

```
self.rows = [row for row in self.rows if not predicate(row)]
```

Sau khi định nghĩa, có thể gọi phương thức này để xóa một hàng với điều kiện tương ứng.

```
# We're not actually going to run these
users.delete(lambda row: row["user_id"] == 1) # Deletes rows with user_id ==
1
users.delete() # Deletes every row
```

5.1.4 SELECT

Thường thì không kiểm tra trực tiếp các bảng SQL. Thay vào đó, chúng truy vấn chúng bằng một câu lệnh SELECT:

```
SELECT * FROM users; -- get the entire contents
SELECT * FROM users LIMIT 2; -- get the first two rows
SELECT user_id FROM users; -- only get specific columns
SELECT user_id FROM users WHERE name = 'Dunn'; -- only get specific rows
```

Cũng có thể sử dụng câu lệnh SELECT để tính toán các trường

```
SELECT LENGTH(name) AS name_length FROM users;
```

Thêm phương thức truy vấn SELECT để có thể sử dụng trong mã nguồn tương tác với cơ sở dữ liệu như sau:

```
def select(self, keep_columns: List[str] = None, additional_columns: Dict[str,
    Callable] = None) -> 'Table':
    if keep_columns is None: # If no columns specified,
        keep_columns = self.columns # return all columns
    if additional_columns is None:
        additional_columns = {}
    # New column names and types
    new_columns = keep_columns + list(additional_columns.keys())
    keep_types = [self.col2type(col) for col in keep_columns]
    # This is how to get the return type from a type annotation. # It will crash
    if 'calculation' doesn't have a return type.
    add_types = [calculation.__annotations__['return']
        for calculation in additional_columns.values()]
    # Create a new table for results
    new_table = Table(new_columns, keep_types + add_types)
    for row in self.rows:
        new_row = [row[column] for column in keep_columns]
        for column_name, calculation in additional_columns.items():
            new_row.append(calculation(row))
    new_table.insert(new_row)
    return new_table
```

5.1.5 GROUP BY

Một thao tác SQL phổ biến khác là GROUP BY, nó nhóm các hàng có giá trị giống nhau trong các cột được chỉ định và tạo ra các giá trị tổng hợp như MIN và MAX và COUNT và SUM.

Ví dụ, có thể muốn tìm số lượng người dùng và user_id nhỏ nhất cho mỗi độ dài tên có thể:

```
SELECT LENGTH(name) as name_length, MIN(user_id) AS min_user_id, COUNT(*) AS
    num_users
FROM users
GROUP BY LENGTH(name);
```

Mọi trường SELECT cần phải ở trong mệnh đề GROUP BY hoặc là một phép tính tổng hợp (mà min_user_id và num_users là). SQL cũng hỗ trợ một mệnh đề HAVING hoạt động tương tự như một mệnh đề WHERE, ngoại trừ việc bộ lọc của nó được áp dụng cho các phép tổng hợp (trong khi WHERE sẽ lọc ra các hàng trước khi thực hiện tổng hợp).

Khi muốn biết số lượng bạn bè trung bình cho người dùng có tên bắt đầu bằng

các chữ cái cụ thể nhưng chỉ xem kết quả cho các chữ cái có số trung bình tương ứng lớn hơn 1.

```
SELECT SUBSTR(name, 1, 1) AS first_letter, AVG(num_friends) AS
    avg_num_friends
FROM users
GROUP BY SUBSTR(name, 1, 1) HAVING AVG(num_friends) > 1;
```

Nếu tính tổng hợp toàn bộ, trong trường hợp đó, có thể bỏ qua mệnh đề GROUP BY:

```
SELECT SUM(user_id) as user_id_sum FROM users
WHERE user_id > 1;
```

5.1.6 ORDER BY

Đôi khi muốn sắp xếp các kết quả trả về từ các câu truy vấn. Ví dụ, có thể muốn sắp xếp tên người dùng trong bảng dữ liệu theo thứ tự alphabet. Có thể sử dụng câu truy vấn sau:

```
SELECT * FROM users
ORDER BY name
LIMIT 2;
```

Có thể xây dựng phương thức dựa trên câu lệnh truy vấn này như sau:

```
def order_by(self, order: Callable[[Row], Any]) -> 'Table':
    new_table = self.select().new_table.rows.sort(key=order)
    return new_table
```

Sau đó, có thể sử dụng phương thức này để sắp xếp dữ liệu trả về

```
friendliest_letters
    (avg_friends_by_letter.order_by(lambda row:
        row["avg_num_friends"]).limit(4))
```

5.1.7 JOIN

Cơ sở dữ liệu quan hệ thường được chuẩn hóa, nghĩa là chúng được tổ chức để loại bỏ những dư thừa. Ví dụ muốn lưu trữ một vài sở thích của người dùng thông qua danh sách các sở thích này. Tuy nhiên SQL database không cho phép lưu trữ một list trong một phần tử của bảng. Để lưu trữ thông tin này cần tạo ra một bảng khác lưu trữ thông tin các sở thích và liên kết với bảng user thông qua quan hệ one-to-many để cho phép người dùng có thể có nhiều sở thích cùng lúc. Câu lệnh sử dụng để tạo ra bảng phụ này là:

```
CREATE TABLE user_interests (  
    user_id INT NOT NULL,  
    interest VARCHAR(100) NOT NULL)  
;
```

Khi dữ liệu nằm ở nhiều bảng khác nhau, làm sao có thể phân tích chúng. Bằng việc kết hợp các bảng với nhau. Câu lệnh JOIN trong SQL cho phép nối các hàng trong bảng bên trái với hàng tương ứng trong bảng bên phải. Định nghĩa "tương ứng" ở đây được hiểu dựa trên cách chỉ ra cụ thể là nối gì. Ví dụ, để tìm những người dùng thích SQL trong dữ liệu, thực hiện truy vấn như sau:

```
SELECT users.name  
FROM users  
JOIN user_interests  
ON users.user_id = user_interests.user_id WHERE user_interests.interest =  
    'SQL'
```

Câu truy vấn JOIN sẽ tiến hành như sau, với mỗi hàng trong bảng users, nên nhìn vào user_id và những liên hệ của hàng đó với tất cả các hàng trong bảng user_interests có chứa cùng user_id. Chú ý, phải chỉ rõ bảng nào để JOIN và cột nào để JOIN. Đây được gọi là INNER JOIN, trả về sự kết hợp của các hàng khớp với các điều kiện nối cụ thể.

Có một cách JOIN khác gọi là LEFT JOIN - ngoài các kết hợp của các hàng khớp nhau - LEFT JOIN trả về một hàng cho mỗi hàng trong bảng trái mà không có hàng khớp (trong trường hợp này, các trường sẽ đến từ bảng bên phải đều là NULL).

Sử dụng LEFT JOIN, việc đếm số lượng sở thích của mỗi người dùng có thể được thực hiện như sau:

```
SELECT users.id, COUNT(user_interests.interest) AS num_interests FROM users  
LEFT JOIN user_interests  
ON users.user_id = user_interests.user_id
```

LEFT JOIN đảm bảo rằng users không có sở thích nào vẫn có thể có hàng trong dataset được nối (với giá trị NULL cho những trường tới từ bảng user_interests), và COUNT chỉ đếm những giá trị không phải NULL.

Trong SQL, RIGHT JOIN cũng được sử dụng để giữ những hàng trong bảng bên phải mà không tìm được hàng tương ứng. Trong khi đó FULL OUTER JOIN được sử dụng để giữ lại những hàng từ cả hai bảng mà không tìm thấy hàng tương ứng khớp nhau.

5.2 Cơ sở dữ liệu nâng cao

5.2.1 Truy vấn con

Trong SQL, có thể SELECT từ (và JOIN) kết quả của các truy vấn nếu chúng là các bảng. Vì vậy, nếu muốn tìm user_id nhỏ nhất của bất kỳ ai có sở thích SQL, có thể sử dụng một truy vấn con (subquery). (Tất nhiên, cũng có thể thực hiện cùng một phép truy vấn tương đương bằng cách sử dụng JOIN, nhưng điều đó sẽ không minh họa truy vấn con.)

```
SELECT MIN(user_id) AS min_user_id FROM
(SELECT user_id FROM user_interests WHERE interest = 'SQL') sql_interests;
```

Để tìm các dòng chứa một giá trị cụ thể (ví dụ, khi name là "Hero"), hệ quản trị cơ sở dữ liệu phải kiểm tra mỗi dòng trong bảng. Nếu bảng có nhiều dòng, điều này có thể mất rất nhiều thời gian.

Tương tự, thuật toán JOIN của cũng có thời gian tính toán lâu. Đối với mỗi dòng trong bảng trái, nó kiểm tra từng dòng trong bảng phải để xem liệu chúng có khớp không. Nếu cả hai bảng đều lớn, điều này có thể mất rất nhiều thời gian.

Ngoài ra, khi muốn áp dụng ràng buộc cho một số cột. Ví dụ, trong bảng người dùng, khi không muốn cho phép hai người dùng khác nhau có cùng user_id.

Indexes giải quyết tất cả những vấn đề này. Nếu bảng user_interests có một index trên user_id, một thuật toán JOIN thông minh có thể tìm các kết quả trực tiếp thay vì quét toàn bộ bảng. Nếu bảng người dùng có một index "duy nhất" trên user_id, sẽ nhận được một lỗi nếu cố gắng chèn một bản sao.

Mỗi bảng trong cơ sở dữ liệu có thể có một hoặc nhiều indexes, cho phép nhanh chóng tìm kiếm các dòng theo các cột khóa, JOIN bảng một cách hiệu quả và áp dụng ràng buộc duy nhất cho các cột hoặc sự kết hợp của các cột.

5.2.2 Tối ưu truy vấn

Tối ưu trong ngữ cảnh của truy vấn cơ sở dữ liệu thường đề cập đến việc đạt được hiệu suất hoặc hiệu quả tốt nhất về thời gian thực hiện và sử dụng tài nguyên. Điều này liên quan đến việc tìm ra cách hiệu quả nhất để truy xuất và xử lý dữ liệu từ cơ sở dữ liệu.

Việc xác định khi nào một truy vấn là "tối ưu" phụ thuộc vào nhiều yếu tố, bao gồm:

- Thời Gian Thực Hiện: Thời gian mà truy vấn mất để hoàn thành. Truy vấn tối ưu được kỳ vọng sẽ thực hiện nhanh chóng.

- **Sử Dụng Tài Nguyên:** Bao gồm lượng CPU, bộ nhớ và I/O đĩa mà truy vấn tiêu thụ. Một truy vấn tối ưu nên sử dụng tài nguyên một cách hiệu quả.
- **Khả Năng Mở Rộng:** Truy vấn nên thực hiện tốt khi kích thước của bộ dữ liệu tăng lên. Một truy vấn tối ưu nên mở rộng một cách linh hoạt.
- **Dễ Bảo Trì:** Truy vấn nên được viết một cách dễ hiểu, dễ bảo trì và sửa đổi nếu cần.
- **Tính Đồng Nhất:** Kết quả của truy vấn nên chính xác và đồng nhất với dữ liệu trong cơ sở dữ liệu.

Đạt được tính tối ưu thường đòi hỏi một sự cân đối giữa các yếu tố này. Ví dụ, việc giảm bớt bảng cơ sở dữ liệu để tối ưu hóa thời gian thực hiện có thể cải thiện thời gian thực hiện nhưng có thể ảnh hưởng đến khả năng bảo trì. Tương tự, việc tạo các chỉ mục có thể làm tăng tốc độ các thao tác đọc nhưng có thể làm chậm các thao tác ghi.

Quyết định khi nào một truy vấn là "đủ" và tối ưu phụ thuộc vào yêu cầu và ràng buộc cụ thể của ứng dụng hoặc hệ thống. Thường cần thử nghiệm và phân tích các phương pháp khác nhau để tìm ra sự cân bằng tốt nhất cho một trường hợp sử dụng cụ thể.

Một số ví dụ trong việc tối ưu hóa truy vấn SQL.

1. Chỉ lấy những dòng cần thiết

```
-- Not optimized
SELECT * FROM users;
-- Optimized
SELECT user_id, name FROM users;
```

2. Sử dụng INDEX cho những cột quan trọng

```
-- Not optimized
SELECT * FROM users WHERE name = 'John';
-- Optimized
CREATE INDEX idx_name ON users(name);
SELECT * FROM users WHERE name = 'John';
```

3. Tránh sử dụng SELECT* mà chỉ lấy những cột cần thiết

```
-- Not optimized
SELECT * FROM orders WHERE order_date > '2023-01-01';
-- Optimized
SELECT order_id, customer_id, order_date FROM orders WHERE
    order_date > '2023-01-01';
```

4. Sử dụng JOIN thay vì Subquery

```
-- Not optimized
SELECT * FROM customers WHERE customer_id IN (SELECT customer_id
      FROM orders);
-- Optimized
SELECT customers.* FROM customers JOIN orders ON
      customers.customer_id = orders.customer_id;
```

5. Sử dụng GROUP BY và HAVING đúng cách

Ngoài ra, việc tối ưu hóa truy vấn SQL đòi hỏi sự hiểu biết về cấu trúc cơ sở dữ liệu, sự sẵn có của chỉ mục (index), và đặc điểm của dữ liệu cụ thể trong từng hệ thống khác nhau

5.2.3 NoSQL

NoSQL là một thuật ngữ dùng để mô tả các hệ thống quản lý cơ sở dữ liệu (DBMS) không sử dụng ngôn ngữ truy vấn SQL (Structured Query Language) hoặc không tuân theo mô hình quan hệ của cơ sở dữ liệu quan hệ truyền thống (RDBMS). Thay vào đó, NoSQL tập trung vào việc lưu trữ và truy xuất dữ liệu có cấu trúc linh hoạt, thường được gọi là dữ liệu phi cấu trúc.

Các hệ thống NoSQL thường được thiết kế để xử lý khối lượng lớn dữ liệu, tích hợp linh hoạt và có khả năng mở rộng tốt. Các loại cơ sở dữ liệu NoSQL phổ biến bao gồm: Redis, DynamoDB, HBase, MongoDB, Neo4j.

Các hệ thống NoSQL thường được sử dụng trong các ứng dụng yêu cầu mức độ mở rộng, linh hoạt cao, và có thể xử lý dữ liệu không đồng nhất hoặc thay đổi nhanh. Tuy nhiên, quyết định giữa việc sử dụng NoSQL hay SQL phụ thuộc vào yêu cầu cụ thể của dự án và tính chất của dữ liệu.

Dưới đây là một ví dụ đơn giản về cách sử dụng MongoDB để lưu trữ dữ liệu về người dùng trong một ứng dụng

1. **Kết nối đến MongoDB:** Trước hết, cần thiết lập kết nối với cơ sở dữ liệu MongoDB từ ứng dụng. Sử dụng thư viện MongoDB cho ngôn ngữ lập trình đang sử dụng. Dưới đây là một ví dụ bằng SQL sử dụng thư viện pymongo:

```
from pymongo import MongoClient

# Initiating connection
client = MongoClient('mongodb://localhost:27017/')
# create a database
db = client['mydatabase']
# create collection
user_collection = db['users']
```


2. **Thêm người dùng vào cơ sở dữ liệu:** Giả sử có một class hoặc dictionary chứa thông tin về người dùng, có thể thêm người dùng vào collection như sau

```
user_data = {
    'username': 'john_doe',
    'email': 'john@example.com',
    'password': 'hashed_password',
    'age': 25
}

# add users
user_id = user_collection.insert_one(user_data).inserted_id
print(f"User added with ID: {user_id}")
```

3. **Truy vấn dữ liệu người dùng:** có thể truy vấn dữ liệu người dùng dựa trên các điều kiện nhất định:

```
# find user name 'john_doe'
result = user_collection.find_one({'username': 'john_doe'})
print(result)
```

4. **Cập nhật thông tin người dùng:** Nếu cập nhật thông tin người dùng, có thể sử dụng phương thức update_one hoặc update_many:

```
# update email for user 'john_doe'
user_collection.update_one({'username': 'john_doe'}, {'$set': {'email':
    'new_email@example.com'}})
```

Bài tập Chương 5

Bài tập 5.1 Cho bảng dữ liệu sau:

employee_id	employee_name	department	salary
1	Alice	HR	50000
2	Bob	Marketing	60000
3	Charlie	Sales	55000
4	David	HR	47000
5	Eve	Sales	51000
6	Frank	Marketing	62000

Dựa trên bảng dữ liệu trên, thực hiện một số câu truy vấn SQL sau:

- Hiển thị tất cả thông tin của tất cả nhân viên trong bảng.
- Lọc thông tin của những nhân viên trong phòng ban HR.
- Sắp xếp danh sách nhân viên theo mức lương từ thấp đến cao.
- Tính tổng lương của tất cả nhân viên.

- (e) Hiển thị tên và mức lương của những nhân viên có mức lương trên 55000.
- (f) Tính trung bình mức lương của các nhân viên trong mỗi phòng ban.
- (g) Đếm số lượng nhân viên trong mỗi phòng ban.
- (h) Cập nhật mức lương của nhân viên có employee_id là 4 lên thành 50000.
- (i) Xóa nhân viên có employee_id là 6 khỏi bảng.

Bài tập 5.2 Cho các bảng dữ liệu sau:

Student (Sinh viên):

student_id	student_name	date_of_birth	department_id
1	Alice	1998 – 05 – 20	1
2	Bob	1997 – 09 – 15	2
3	Charlie	1999 – 02 – 10	1
4	David	2000 – 07 – 08	3
5	Eve	1996 – 11 – 30	2

Department (Khoa)

department_id	department_name
1	Computer Science
2	Mathematics
3	Physics

Grade (điểm)

student_id	course	grade
1	Math	85
1	Physics	78
2	Math	92
2	Physics	88
3	Math	70
3	Physics	65
4	Math	75
4	Physics	80
5	Math	90
5	Physics	95

Thực hiện các yêu cầu sau:

- (a) Liệt kê tất cả thông tin về sinh viên.
- (b) Hiển thị tên và ngày sinh của sinh viên có student_id là 3.
- (c) Liệt kê tên khoa và số lượng sinh viên trong mỗi khoa.
- (d) Tính điểm trung bình của mỗi sinh viên.

- (e) Hiển thị tên sinh viên, khoa và điểm của những sinh viên có điểm toán (Math) trên 80.
- (f) Tính điểm cao nhất và thấp nhất của từng khoa.

Bài tập 5.3 Xây dựng một cơ sở dữ liệu quản lý Thực tập nghề nghiệp của sinh viên có tên là “ThucTap” gồm các sơ đồ quan hệ sau:

Khoa(makhoa char(10), tenkhoa char(30), dienthoai char(10))

GiangVien(magv int, hotengv char(30), luong decimal(5,2), makhoa char(10))

SinhVien(masv int, hotensv char(30), makhoa char(10), namsinh int, quequan char(30))

DeTai(madt char(10), tendt char(30), kinhphi int, NoiThucTap char(30))

HuongDan(masv int, madt char(10), magv int, ketqua decimal(5,2))

Bài tập 5.4 Sử dụng cơ sở dữ liệu ở Bài 5.1, chỉ sử dụng 01 lệnh SQL trả lời các yêu cầu sau:

- (a)
 - Đưa ra thông tin gồm mã số, họ tên và tên khoa của tất cả các giảng viên.
 - Đưa ra thông tin gồm mã số, họ tên và tên khoa của các giảng viên của khoa ‘DIA LY và QLTN’.
 - Cho biết số sinh viên của khoa ‘CONG NGHE SINH HOC’.
 - Đưa ra danh sách gồm mã số, họ tên và tuổi của các sinh viên khoa ‘TOAN’.
 - Cho biết số giảng viên của khoa ‘CONG NGHE SINH HOC’.
 - Cho biết thông tin về sinh viên không tham gia thực tập.
 - Đưa ra mã khoa, tên khoa và số giảng viên của mỗi khoa.
 - Cho biết số điện thoại của khoa mà sinh viên có tên ‘Le van son’ đang theo học.
- (b)
 - Cho biết mã số và tên của các đề tài do giảng viên ‘Tran son’ hướng dẫn.
 - Cho biết tên đề tài không có sinh viên nào thực tập.
 - Cho biết mã số, họ tên, tên khoa của các giảng viên hướng dẫn từ 3 sinh viên trở lên.
 - Cho biết mã số, tên đề tài của đề tài có kinh phí cao nhất.
 - Cho biết mã số và tên các đề tài có nhiều hơn 2 sinh viên tham gia thực tập.
 - Đưa ra mã số, họ tên và điểm của các sinh viên khoa ‘DIALY và QLTN’.
 - Đưa ra tên khoa, số lượng sinh viên của mỗi khoa.

- Cho biết thông tin về các sinh viên thực tập tại quê nhà.
- Hãy cho biết thông tin về những sinh viên chưa có điểm thực tập.
- Đưa ra danh sách gồm mã số, họ tên các sinh viên có điểm thực tập bằng 0.

Bài tập 5.5 Dữ liệu dưới đây sẽ sử dụng một bảng “Orders” (Đơn hàng) bao gồm thông tin về các đơn hàng. Thực hiện các truy vấn con (subqueries) cho các yêu cầu dưới đây:

orders:

order_id	customer_id	order_date	total_amount
1	101	2023 – 01 – 10	150.00
2	102	2023 – 01 – 12	200.00
3	101	2023 – 01 – 15	100.00
4	103	2023 – 01 – 18	250.00
5	102	2023 – 01 – 20	180.00

- Liệt kê tất cả đơn hàng có số tiền lớn hơn trung bình của tất cả các đơn hàng.
- Hiển thị thông tin về các đơn hàng của khách hàng có ID là 101.
- Tính tổng số lượng đơn hàng của mỗi khách hàng.
- Hiển thị thông tin về các đơn hàng có ngày đặt hàng trước ngày 2023-01-15.
- Liệt kê thông tin về các đơn hàng có số tiền lớn hơn đơn hàng có order_id là 2.
- Hiển thị thông tin về các đơn hàng có số tiền lớn hơn trung bình của từng khách hàng.

Bài tập 5.6 Dưới đây là một bảng dữ liệu “Employees” (Nhân viên). Sử dụng truy vấn con thực hiện các yêu cầu sau:

employees:

employee_Id	employee_name	department	salary
1	Alice	HR	50000
2	Bob	Marketing	60000
3	Charlie	Sales	55000
4	David	HR	47000
5	Eve	Sales	51000
6	Frank	Marketing	62000

- Hiển thị tên và bộ phận của những nhân viên có mức lương lớn hơn mức lương trung bình của toàn bộ công ty.
- Liệt kê tên nhân viên và bộ phận của những người làm việc trong bộ phận có ít nhất hai nhân viên.

- (c) Tính tổng mức lương của các bộ phận và hiển thị các bộ phận có tổng mức lương lớn hơn mức lương trung bình của công ty.
- (d) Hiển thị thông tin về những nhân viên có mức lương cao hơn nhân viên có employee_id là 3.
- (e) Liệt kê tên và mức lương của những nhân viên có mức lương cao nhất trong từng bộ phận.



6. Hệ khuyến nghị

Chương 6 trình bày về hệ khuyến nghị, một trong những ứng dụng nổi bật của khoa học dữ liệu trong cách mạng công nghiệp lần thứ 4.

6.1 Giới thiệu chung

6.1.1 Hệ khuyến nghị là gì

Hệ khuyến nghị là một hệ thống dựa trên học máy giúp cho người dùng khám phá những sản phẩm và dịch vụ mới. Khi người dùng sử dụng các nền tảng thương mại điện tử trực tuyến, hệ khuyến nghị đang được sử dụng để hướng người dùng đến những sản phẩm mà họ muốn mua nhất dựa trên những dữ liệu về hành vi người dùng đã thu thập từ trước. Khi số lượng sản phẩm dịch vụ trong thế giới số càng ngày càng tăng, hệ khuyến nghị là một thành phần không thể thiếu vì người dùng thường bị choáng ngợp bởi số lượng lớn các sản phẩm, dịch vụ trên các nền tảng số. Người dùng cần sự giúp đỡ của hệ khuyến nghị để tìm được đúng sản phẩm mình cần tìm một cách tự nhiên, dẫn đến sự hài lòng cho họ, người bán sẽ bán được nhiều hàng hóa hơn, từ đó hoa hồng cho các nền tảng thương mại điện tử trực tuyến cũng tăng lên. Hệ khuyến nghị giống như một người bán hàng hiểu người mua hàng muốn gì dựa trên lịch sử mua hàng của khách hàng và thói quen mua sắm của họ.

Hiện nay, hệ khuyến nghị phổ biến đến nỗi người dùng sử dụng chúng hằng ngày mà thậm chí còn không nhận ra. Bởi vì người dùng không thể nào duyệt qua được tất cả các sản phẩm hoặc các nội dung trên website, hệ khuyến nghị đóng một vai trò quan trọng trong việc giúp cho người dùng có trải nghiệm tốt hơn, trong khi cũng giúp cho người dùng khám phá ra những sản phẩm dịch vụ mới mà có thể người dùng quan tâm.

Một số ví dụ của hệ khuyến nghị trong các ứng dụng thực tế có thể kể đến hệ khuyến nghị của Amazon, Netflix gợi ý các bộ phim, chương trình TV; hệ khuyến nghị của Youtube gợi ý các videos mà người dùng quan tâm; hệ khuyến nghị của Spotify gợi ý các bài hát, bản nhạc mà người dùng có thể thích nghe; Facebook gợi ý tin cho người dùng và Google gợi ý các quảng cáo sản phẩm mà người dùng quan tâm.

Một trong những thành phần quan trọng nhất của bất kỳ một hệ khuyến nghị nào nêu trên là hàm khuyến nghị. Hàm này nhận đầu vào là thông tin từ người dùng và dự đoán xếp hạng (rating) mà người dùng này có thể gán cho các sản phẩm. Việc dự đoán được xếp hạng này trước khi người dùng đưa ra đánh giá xếp hạng của họ cho sản phẩm làm cho các hệ khuyến nghị trở thành một công cụ mạnh mẽ trong thực tế.

6.1.2 Hệ khuyến nghị hoạt động thế nào

Các hệ khuyến nghị hoạt động dựa trên nguyên tắc hiểu rõ các quan hệ của các thực thể trong hệ thống. Các quan hệ cung cấp cho hệ khuyến nghị những góc nhìn sâu sắc về bản chất của các hành vi mà người dùng thực hiện trên các nền tảng. Thông qua các quan hệ này, hệ khuyến nghị hiểu rõ về khách hàng khi họ có nhu cầu mua các sản phẩm. Có 3 loại quan hệ hay được sử dụng trong hệ khuyến nghị: (1) quan hệ giữa người dùng và sản phẩm; (2) quan hệ giữa sản phẩm và sản phẩm; và (3) quan hệ giữa người dùng và người dùng.

Mỗi quan hệ giữa người dùng và sản phẩm xuất hiện khi một số người dùng có cùng sở thích với một loại sản phẩm cụ thể nào đó mà họ cần. Ví dụ, một người chơi bóng đá có thể có sở thích mua các mặt hàng có liên quan đến bóng đá, do đó các trang thương mại điện tử sẽ xây dựng một mối quan hệ giữa người dùng và sản phẩm là người chơi và bóng đá. Trong khi đó, mối quan hệ giữa sản phẩm và sản phẩm xuất hiện khi các mặt hàng có tính chất tương tự nhau, về hình thức hoặc mô tả. Một số ví dụ bao gồm sách hoặc nhạc cùng thể loại, các món ăn từ cùng một nền ẩm thực hoặc các bài báo từ một sự kiện cụ thể. Mối quan hệ giữa người dùng và người dùng xuất hiện khi một số người dùng có sở thích tương tự đối với một sản phẩm hoặc dịch vụ cụ thể. Ví dụ bao gồm bạn bè chung, xuất thân tương tự, tuổi tác tương tự, v.v.

Các hệ khuyến nghị hoạt động đằng sau rất nhiều các nền tảng website lớn. Các website thương mại điện tử thường sử dụng hệ khuyến nghị để tăng sự gắn bó, yêu thích của khách hàng với nền tảng và tăng lượng giao dịch. Tuy nhiên, những gợi ý được tạo ra trên các nền tảng này phụ thuộc lớn vào chất lượng và số lượng dữ liệu

họ có được. Do đó, việc xử lý một lượng lớn dữ liệu (thông tin) là tối quan trọng để xây dựng một hệ khuyến nghị tốt phục vụ cho nhu cầu của các nền tảng này.

Khi mua hàng hoặc tìm kiếm thứ gì đó trên Amazon, chúng ta có thể sẽ thấy những sản phẩm được gợi khuyến nghị dựa trên sở thích của chúng ta hoặc những kết quả hiển thị lên trang tìm kiếm. Hoặc khi đánh giá một chương trình TV hoặc một bộ phim trên nền tảng đánh giá phim trực tuyến IMDb, nền tảng này sẽ gợi ý những chương trình hoặc bộ phim khác dựa trên những chi tiết quan trọng như diễn viên, thể loại, cốt truyện, tóm tắt. Ví dụ như nếu bạn xem một bộ phim với thể loại phim tài liệu, trang web sẽ gợi ý cho bạn những bộ phim cùng thể loại tài liệu đó với xác suất bạn sẽ thích những bộ phim này và xem nó. Trong khi đó Facebook và Instagram sử dụng các hệ khuyến nghị với một quy mô lớn hơn để gợi ý bạn bè và các bài đăng của người khác trên newsfeed. Thông thường, người dùng sẽ thấy những mẫu tin tức tương tự nhau dồn dập trên newsfeed dựa trên những lượt yêu thích và những trang mà họ theo dõi, điều này cũng có thể làm cho việc gợi ý trở nên nhàm chán. Các nền tảng thường sẽ phải có thêm những thuật toán lọc thông minh dựa trên các tương tác thực tế của người dùng với các bài tin tức để đưa ra những gợi ý phù hợp hơn. Với cỗ máy tìm kiếm Google, mỗi lần chúng ta đăng nhập và tìm kiếm thứ gì đó, thông tin sẽ được ghi lại vào lịch sử tìm kiếm và sẽ gợi ý những sản phẩm dựa trên những tìm kiếm trước đó. Kết quả là, hơn 90% doanh thu của Google tới từ các hoạt động quảng cáo sản phẩm dựa trên lịch sử tìm kiếm của người dùng. Những kết quả tìm kiếm trả về cho người dùng cũng tính toán đến cả vị trí của người dùng tại thời điểm tìm kiếm.

Tóm lại, hệ khuyến nghị là một mảng khá rộng của học máy và có tuổi đời ít hơn so với các hệ phân loại vì Internet mới chỉ thực sự bùng nổ khoảng hai thập kỷ trở lại đây. Có hai thực thể chính trong Hệ gợi ý là users và items. Users là người dùng. Items là sản phẩm, ví dụ như các bộ phim, bài hát, cuốn sách, clip, hoặc cũng có thể là các users khác trong bài toán gợi ý kết bạn. Mục đích chính của các hệ khuyến nghị là dự đoán mức độ quan tâm của một người dùng tới một sản phẩm nào đó, qua đó có chiến lược khuyến nghị phù hợp, tăng tính tương tác của người dùng với sản phẩm, làm tăng khả năng chuyển đổi từ gợi ý sang mua hàng.

6.2 Lọc dựa trên nội dung

Lọc dựa trên nội dung là một trong hai nhóm lớn của các hệ khuyến nghị. Lọc dựa trên nội dung đánh giá đặc tính của các sản phẩm được khuyến nghị. Ví dụ: Một người dùng xem rất nhiều bộ phim hành động, vậy thì gợi ý một bộ phim trong cơ

Bảng 6.1: Ví dụ Utility matrix

	P1	P2	P3	P4
Nhiệm vụ bất khả thi	4	5	1	1
Avatar: Dòng chảy của nước	5	4		
La la land			4	5
Me before you	1	2	5	
Vệ binh dải ngân hà	4		2	3

sở dữ liệu có chung đặc tính *hành động* tới người dùng này, ví dụ phim *Fast and Furious*. Cách tiếp cận này yêu cầu việc sắp xếp các sản phẩm vào từng nhóm hoặc đi tìm đặc trưng của từng sản phẩm. Tuy nhiên, có những sản phẩm không có nhóm cụ thể và việc xác định nhóm hoặc đặc trưng của từng sản phẩm đôi khi là không thể thực hiện được.

6.2.1 Utility matrix

Như đã đề cập đến ở phần trước, có hai thực thể chính tồn tại ở trong các hệ khuyến nghị là người dùng và sản phẩm. Mỗi người dùng có mức độ quan tâm đến từng sản phẩm khác nhau. Mức độ quan tâm này, nếu đã được biết trước bằng một cách nào đó, được gán cho một giá trị tương ứng với mỗi cặp user-item. Giả sử mức độ quan tâm được đo đạc bằng giá trị mà người dùng đánh giá cho sản phẩm (rating). Tập hợp tất cả các ratings, bao gồm cả giá trị chưa biết cần phải dự đoán, tạo nên một ma trận gọi là utility matrix. Xét ví dụ sau:

Trong ví dụ này, có 4 người dùng P1, P2, P3, P4 và 5 bộ phim. Các ô có giá trị trong bảng thể hiện một người dùng đã đánh giá một bộ phim với điểm tương ứng từ 0 (không thích) tới 5 (rất thích). Các ô trống còn lại tương ứng với những bộ phim mà user chưa có đánh giá. Hệ khuyến nghị muốn đưa ra được gợi ý cho người dùng trên hệ thống cần phải dự đoán được giá trị trong các ô còn trống giá trị.

Trong ví dụ này có thể thấy có 2 thể loại phim khác nhau là phim *hành động* và phim *tình cảm*. Từ dữ liệu này, ta có thể có những gợi ý nhất định cho các user còn lại về những bộ phim tương ứng mà họ thích. Giả sử như hệ thống chỉ có những bộ phim thuộc 2 thể loại này, khi có một bộ phim mới, ta chỉ cần phân lớp nó vào thể loại nào, sau đó ứng với các user thích thể loại tương ứng, ta sẽ gợi ý bộ phim mới này cho các users.

Trong thực tế, số lượng users và items là rất lớn trong hệ thống, mỗi user thường đưa ra rất ít đánh giá về các items, thậm chí có những user không đánh giá bất kỳ một item nào (với những user này thì tốt nhất là gợi ý những bộ phim phổ biến nhất

có nhiều người xem nhất). Vì vậy, số lượng ô trống trong ma trận là rất lớn, lượng ô có giá trị được điền chiếm tỉ lệ nhỏ. Một điều có thể thấy rõ là nếu càng nhiều ô có giá trị được điền thì độ chính xác của hệ khuyến nghị sẽ càng được cải thiện. Vì vậy, các hệ thống luôn hỏi người dùng về sự quan tâm của họ tới sản phẩm, dịch vụ, và muốn người dùng đánh giá càng nhiều sản phẩm, dịch vụ càng tốt. Việc đánh giá này là rất quan trọng với các hệ khuyến nghị.

Không có Utility matrix, chỉ có cách khuyến nghị các sản phẩm phổ biến nhất. Vì vậy, trong các hệ khuyến nghị, việc xây dựng Utility Matrix là rất cần thiết. Tuy nhiên, việc xây dựng ma trận này thường có gặp nhiều khó khăn. Có hai hướng tiếp cận phổ biến để xác định giá trị rating cho mỗi vị trí trong Utility Matrix

- Nhờ người dùng đánh giá càng nhiều càng tốt
- Dựa vào hành vi của người dùng để điền thêm vào các vị trí chưa có ratings. Dựa vào những dữ liệu trong quá khứ của người dùng để dự đoán ra các giá trị này. Việc người dùng ở lại với nền tảng càng lâu sẽ tạo ra càng nhiều những thông tin hữu ích để dự đoán ra ratings của người dùng cho các sản phẩm, dịch vụ.

6.2.2 Hồ sơ của sản phẩm (item profiles)

Trong các hệ thống khuyến nghị dựa trên nội dung (nội dung của các sản phẩm), cần xây dựng một bộ hồ sơ của từng sản phẩm. Hồ sơ này được biểu diễn dưới dạng một vector đặc trưng. Trong các trường hợp đơn giản, đặc trưng này có thể được trích xuất ra từ bản thân của sản phẩm. Ví dụ các đặc trưng của một bộ phim có thể được sử dụng như diễn viên, đạo diễn, năm phát hành, thể loại, quốc gia. Có rất nhiều yếu tố khác dùng để mô tả chính xác bộ phim cũng có thể được sử dụng. Việc chọn lựa được một bộ đặc trưng tốt cho các items là điều quan trọng dẫn đến sự hiệu quả của mô hình gợi ý. Một bộ đặc trưng tốt là bộ đặc trưng có thể mô tả chính xác được các items. Trong ví dụ đơn giản trong bảng 1 phía trên, chúng ta đơn giản hóa bằng cách xây dựng một vector đặc trưng ba chiều cho mỗi bộ phim: chiều thứ nhất là mức độ *hành động*, chiều thứ hai là mức độ *tình cảm*, chiều thứ ba là mức độ *hài hước*. Đặt các vector đặc trưng cho các bộ phim là x_1, x_2, x_3, x_4, x_5 . Giả sử có được giá trị của các vector đặc trưng cho các bài hát ở hình bên dưới. Nhiệm vụ là đi tìm từng mô hình người dùng θ_i để tối ưu hóa trên những ratings đã có trong bảng.

Bài toán đi tìm mô hình θ_i cho mỗi user có thể được coi là bài toán *hồi quy* trong trường hợp ratings là một dải giá trị, hoặc bài toán *phân loại* trong trường hợp ratings là một vài trường hợp cụ thể. Dữ liệu huấn luyện để xây dựng các mô hình

Bảng 6.2: Xây dựng item profile

	P1	P2	P3	P4	film's feature vector
Nhiệm vụ bất khả thi	4	5	1	1	$x_1 = [0.99, 0.02, 0.5]$
Avatar: Dòng chảy của nước	5	4			$x_2 = [0.99, 0.1, 0.4]$
La la land			4	5	$x_3 = [0.02, 0.98, 0.4]$
Me before you	1	2	5		$x_4 = [0.01, 0.99, 0.6]$
Vệ binh dải ngân hà	4		2	3	$x_5 = [0.99, 0.2, 0.9]$

θ_i là các cặp (feature vector, ratings) tương ứng với các items mà user đó đã có đánh giá. Việc điền các giá trị còn thiếu trong ma trận là các đầu ra cho các item chưa có đánh giá khi áp dụng mô hình θ_i .

6.2.3 Xây dựng hàm mất mát

Giả sử số users là N , số items là M , ma trận utility là $\mathbf{Y}$. Phần tử ở hàng m , cột n trong ma trận $\mathbf{Y}$ thể hiện mức độ quan tâm (rating) mà user thứ n đánh giá cho sản phẩm thứ m . Ma trận $\mathbf{R}$ thể hiện rằng một sản phẩm có được đánh giá bởi user tương ứng hay không. $r_{ij} = 1$ nếu item thứ i được đánh giá bởi user thứ j , ngược lại $r_{ij} = 0$.

Giả sử mỗi user có thể tìm được một mô hình θ_i được minh họa bởi vector cột hệ số w_i , và độ lệch b_n . Từ đó, mức độ quan tâm của user tới một item có thể được tính bằng một hàm tuyến tính:

$$y_{mn} = x_m w_n + b_n \quad (6.1)$$

Với x_m là vector đặc trưng - vector hàng, w_n là vector cột.

Với một user thứ n bất kỳ, ta có tập hợp các thành phần đã được đánh giá tương ứng của y_n , hàm mất mát có thể được xây dựng như sau:

$$\mathcal{L}_n = \frac{1}{2} \sum_{m:r_{mn}=1} (x_m w_n + b_n - y_{mn})^2 + \frac{\lambda}{2} \|w_n\|_2^2 \quad (6.2)$$

Trong đó, thành phần thứ hai là regularization để ngăn cho mô hình không bị overfit. Thông thường hàm mất mát sẽ được tính trên trung bình cộng của tất cả các vị trí có đánh giá của *user* nên hàm mất mát $\mathcal{L}_n$ được viết lại như sau:

$$\mathcal{L}_n = \frac{1}{2s_n} \sum_{m:r_{mn}=1} (x_m w_n + b_n - y_{mn})^2 + \frac{\lambda}{2s_n} \|w_n\|_2^2 \quad (6.3)$$

Trong đó s_n là số lượng các *items* mà *user* thứ n đã đánh giá.

Vì hàm mất mát chỉ được tính toán dựa trên các *items* đã được đánh giá nên có thể rút gọn lại hàm mất mát này bằng cách trích xuất vector cột $\hat{y}_n$ từ vector y . vector cột $\hat{y}_n$ chỉ chứa các phần tử có giá trị được đánh giá trong ma trận utility. Đồng thời, $\hat{X}_n$ là ma trận được trích xuất tương tự như vector $\hat{y}_n$. Khi đó, hàm mất mát được viết lại như sau:

$$\mathcal{L}_n = \frac{1}{2s_n} \|\hat{X}_n w_n + b_n e_n - \hat{y}_n\|_2^2 + \frac{\lambda}{2s_n} \|w_n\|_2^2 \quad (6.4)$$

Trong đó, e_n là vector cột chứa s_n phần tử 1. Cặp nghiệm w_n, b_n tương ứng với mô hình được tìm ra bằng việc áp dụng các phương pháp lan truyền ngược như Gradient Descent (GD), Stochastic Gradient Descent (SGD).

6.2.4 Ví dụ hàm mất mát cho user P2

Quay trở lại với ví dụ về đánh giá các bộ phim của một tập đối tượng người dùng trong Bảng 6.2, ma trận đặc trưng cho các items là:

$$X = \begin{bmatrix} 0.99 & 0.02 & 0.5 \\ 0.99 & 0.1 & 0.4 \\ 0.02 & 0.98 & 0.4 \\ 0.01 & 0.99 & 0.6 \\ 0.99 & 0.2 & 0.9 \end{bmatrix} \quad (6.5)$$

Với user P2 ở cột thứ $n = 2$, $y_2 = [5, 4, *, 2, *]^T$ với các vị trí trong y_2 có dấu "*" là những bộ phim mà người dùng P2 chưa có đánh giá. Vì người dùng P2 chỉ đánh giá cho bộ phim thứ 1,2,4 nên ta có thể viết rút gọn lại ma trận đặc trưng cho người dùng P2 này như sau:

$$\hat{X}_2 = \begin{bmatrix} 0.99 & 0.02 & 0.5 \\ 0.99 & 0.1 & 0.4 \\ 0.01 & 0.99 & 0.6 \end{bmatrix}, \hat{y}_2 = \begin{bmatrix} 5 \\ 4 \\ 2 \end{bmatrix}, e_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad (6.6)$$

Khi đó, hàm mất mát cho hệ số tương ứng với người dùng P2 sẽ được viết lại như sau:

$$\mathcal{L}_2 = \frac{1}{6} \left\| \begin{bmatrix} 0.99 & 0.02 & 0.5 \\ 0.99 & 0.1 & 0.4 \\ 0.01 & 0.99 & 0.6 \end{bmatrix} w_2 + b_2 \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 5 \\ 4 \\ 2 \end{bmatrix} \right\|_2^2 + \frac{\lambda}{6} \|w_2\|_2^2 \quad (6.7)$$

Trong thực tế, khi xây dựng các mô hình gợi ý sử dụng các ngôn ngữ như *python*, *C++*, chúng ta có thể sử dụng các hàm có sẵn trong các thư viện để tiến hành huấn luyện cho các mô hình với từng người dùng như bên trên. Việc hiểu cách thức xây dựng hàm mất mát cũng như quá trình huấn luyện sẽ giúp cho người đọc có thể điều chỉnh các tham số một cách phù hợp.

Lọc dựa trên nội dung là phương pháp đơn giản để xây dựng các hệ gợi ý. Đặc điểm của phương pháp này là quá xây dựng các mô hình cho từng người dùng không phụ thuộc vào các người dùng khác. Việc xây dựng mô hình cho từng người dùng có thể được coi là bài toán hồi quy hoặc phân loại với dữ liệu huấn luyện là các cặp (hồ sơ item, đánh giá) mà người dùng đã đánh giá. Hồ sơ item không phụ thuộc vào người dùng, mà phụ thuộc vào các đặc điểm dùng để mô tả item hoặc cũng có thể được xác định bằng việc yêu cầu người dùng đưa thêm thông tin về item.

6.3 Lọc cộng tác

Trong phần trước, lọc dựa trên nội dung được xây dựng dựa trên các bản mô tả (đặc trưng) của items giúp hệ thống tiết kiệm bộ nhớ và tính toán. Tuy nhiên, phương pháp này có hai nhược điểm cơ bản. Thứ nhất, mô hình được xây dựng cho từng user và không tận dụng được thông tin của các user khác mặc dù những thông tin này rất hữu ích khi gợi ý item. Thứ hai, không phải lúc nào việc trích xuất đặc trưng cho các item cũng thực hiện được. Hơn nữa, việc yêu cầu user đưa ra các đánh giá cụ thể để sử dụng làm đặc trưng cho các item rất khó thực hiện hoặc là dù có thực hiện được nhưng độ tin cậy sẽ không cao. Những nhược điểm này có thể được giải quyết bằng lọc cộng tác (Collaborative Filtering). Lọc cộng tác được xây dựng với ý tưởng là xác định mức độ quan tâm của một user tới một item dựa trên các users khác gần giống đặc điểm với user này. Việc xác định sự giống nhau giữa các users sẽ dựa trên mức độ quan tâm của các users và các items mà hệ thống đã ghi nhận.

Để xây dựng một hệ thống dựa trên lọc cộng tác, có hai câu hỏi cần phải trả lời:

- Làm thế nào xác định được sự giống nhau của các users.
- Khi đã xác định được các users gần giống nhau, làm sao dự đoán được mức độ quan tâm của một user lên một item.

Cách tiếp cận như bên trên được gọi là lọc cộng tác dựa trên người dùng (user-user collaborative filtering). Có một cách tiếp cận khác là đi tìm sự liên quan giữa các items sau đó gợi ý cho user những items tương tự với các items mà người dùng có mức độ quan tâm cao. Cách tiếp cận này được gọi là lọc cộng tác dựa trên sản phẩm (item-item collaborative filtering).

6.3.1 Lọc cộng tác dựa trên người dùng (user-user collaborative filtering)

6.3.1.1 Hàm tương tự (similarity functions)

Công việc quan trọng nhất với lọc cộng tác dựa trên người dùng là xác định được sự giống nhau giữa hai users. Dữ liệu thu thập được duy nhất là ma trận Utility Y , nên

Bảng 6.3: Utility matrix cho users từ u_0 đến u_5 đánh giá items từ i_0 tới i_5

	u_0	u_1	u_2	u_3	u_4	u_5
i_0	5	5	2	1	1	
i_1	3			0		
i_2		5	1		1	2
i_3	1	1	5	4	5	4
i_4	2	2	3			5
i_5	2	0	4	5		
$\bar{u}_j$	2.6	2.6	3	2.5	2.33	3.66

sự giống nhau phải được xác định dựa trên các cột tương ứng với hai users trong ma trận. Xét ví dụ trong Bảng , ma trận Y được xây dựng dựa trên đánh giá của các users từ u_0 đến u_5 với items từ i_0 tới i_5 . Mỗi phần tử trong ma trận này thể hiện số điểm mà mỗi user đánh giá cho item với giá trị cao hơn thể hiện mức độ quan tâm cao hơn. Các phần tử để trống giá trị thì hệ thống sẽ phải điền đầy đủ vào để gợi ý cho users. Đặt mức độ giống nhau của hai users u_i, u_j là $\text{sim}(u_i, u_j)$

Dựa trên quan sát ma trận trong Bảng 6.3, các users u_0, u_1 thích các items i_0, i_1, i_2 và không thích các items i_3, i_4, i_5 . Điều ngược lại thì xảy ra ở các users còn lại. Vì vậy trong trường hợp này, một hàm tương tự tốt cần đảm bảo

$$\text{sim}(u_0, u_1) > \text{sim}(u_0, u_i), \forall i > 1 \quad (6.8)$$

Từ đây, để xác định mức độ quan tâm của u_0 lên i_2 , ta có thể dựa vào hành vi của user u_1 lên item này. Trong trường hợp này, hệ thống đã ghi nhận đánh giá của user u_1 cho sản phẩm này nên hệ thống có thể gợi ý sản phẩm này cho user u_0 .

Để đo được mức độ tương tự của hai users, các vector đặc trưng của người dùng thường được sử dụng, sau đó áp dụng một hàm số tính toán khoảng cách giữa các vector đặc trưng này. Cần lưu ý rằng việc xây dựng vector đặc trưng này khác so với việc xây dựng hồ sơ item (item profile) như trong phần lọc dựa trên nội dung. Vector đặc trưng cho user trong phần này được xây dựng chỉ dựa vào ma trận utility mà không dùng thông tin ngoài như item profile. Với mỗi một user, thông tin duy nhất chúng ta có được là các đánh giá của user đó lên các items được hệ thống ghi nhận, tương ứng với cột của user đó trong ma trận. Tuy nhiên, thường thì một cột dữ liệu sẽ bị mất mát thông tin khá nhiều do các vị trí bị thiếu dữ liệu. Để khắc phục chúng ta có thể dùng một số kỹ thuật để điền đầy vào các vị trí này các giá trị phù hợp để tính toán hàm tương tự. Các giá trị này được điền vào để tính toán giá trị

Bảng 6.4: Utility matrix chuẩn hóa cho users từ u_0 đến u_5 đánh giá items từ i_0 tới i_5

	u_0	u_1	u_2	u_3	u_4	u_5
i_0	2.4	2.4	-1	-1.5	-1.33	0
i_1	0.4	0	0	-2.5	0	0
i_2	0	2.4	-2	0	-1.33	-1.66
i_3	-1.6	-1.6	2	1.5	2.67	0.34
i_4	-0.6	-2.6	0.0	0	0	1.34
i_5	-0.6	-2.6	1	2.5	0	0

hàm tương tự chứ không ảnh hưởng đến giá trị cuối cùng của các vị trí trong utility matrix mà hệ thống đang đi tìm.

Mỗi vị trí trống có thể được thay thế bằng một giá trị nào đó để hạn chế việc sai lệch quá nhiều. Trong phần này có thể sử dụng các kỹ thuật đã trình bày trong Chương 2 về xử lý giá trị bị thiếu. Một trong những cách làm tốt trong trường hợp này là điền đầy các giá trị thiếu bằng giá trị trung bình cộng của các giá trị trên cột tương ứng. Việc này tránh được sự sai lệch với các users quá khó tính hoặc quá dễ tính, tức lúc nào cũng có những items mà user thích hơn so với các items khác.

6.3.1.2 Chuẩn hóa dữ liệu

Hàng cuối cùng trong Bảng 6.3 là giá trị trung bình của các đánh giá cho mỗi user. Giá trị cao tương ứng với các user dễ tính và ngược lại. Khi đó nếu tiếp tục trừ đi mỗi đánh giá trong Bảng giá trị này và thay các giá trị chưa biết bằng 0, ta sẽ được một ma trận utility được chuẩn hóa như trong Bảng 6.4. Tầm quan trọng của việc chuẩn hóa này đến từ những yếu tố sau đây:

- Việc trừ đi giá trị trung bình cộng khiến trong mỗi cột sẽ có giá trị dương và âm. Giá trị dương tương ứng với việc user thích item, giá trị âm tương ứng với việc user không thích item. Những giá trị bằng 0 tương ứng với việc chưa xác định được liệu user có thích item tương ứng hay không
- Về mặt kỹ thuật, số lượng chiều của ma trận rất lớn với hàng triệu users và items, nếu lưu trữ toàn bộ các giá trị này trong một ma trận có khả năng cao không đủ bộ nhớ. Dựa trên quan sát thấy rằng số lượng vị trí có đánh giá là rất ít so với những vị trí trong ma trận không có đánh giá. Có một cách hiệu quả hơn đó là chỉ lưu trữ những đánh giá với vị trí của chúng.

Sau khi đã chuẩn hóa dữ liệu như bên trên, một hàm số tương tự được sử dụng để tính toán mức độ tương tự của các user.

Bảng 6.5: Ma trận độ tương đồng của các users

	u_0	u_1	u_2	u_3	u_4	u_5
u_0	1	0.71792251	-0.6463947	-0.6796738	-0.7534643	-0.2057247
u_1	0.71792251	1	-0.782508	-0.5770733	-0.6210472	-0.7059547
u_2	-0.6463947	-0.782508	1	0.53687549	0.90337195	0.58553168
u_3	-0.6796738	-0.5770733	0.53687549	1	0.4455657	0.05725799
u_4	-0.7534643	-0.6210472	0.90337195	0.4455657	1	0.44158779
u_5	-0.2057247	-0.7059547	0.58553168	0.05725799	0.44158779	1

6.3.1.3 Hàm cosine

Đây là một trong những hàm số được sử dụng nhiều nhất. Công thức của hàm cosine như sau:

$$\text{cosine_similarity}(\mathbf{u}_1, \mathbf{u}_2) = \cos(\mathbf{u}_1, \mathbf{u}_2) = \frac{\mathbf{u}_1^T \mathbf{u}_2}{\|\mathbf{u}_1\|_2 \cdot \|\mathbf{u}_2\|_2} \quad (6.9)$$

Trong đó $\mathbf{u}_{1,2}$ là các vectors tương ứng với user 1,2 đã được chuẩn hóa ở bên trên. Mức độ tương tự của hai vector là một số trong đoạn $[-1,1]$. Giá trị bằng 1 thể hiện hai vector hoàn toàn tương tự nhau. Hàm số $\cos$ của một góc bằng 1 nghĩa là góc giữa hai vectors bằng 0, tức là một vector bằng tích của một số dương với vector còn lại. Giá trị $\cos$ bằng -1 thể hiện hai vector này hoàn toàn ngược nhau. Nghĩa là khi hai users có hành vi hoàn toàn ngược nhau thì độ tương đồng của hai vectors tương ứng là thấp nhất. Việc tính toán giá trị độ tương đồng của các user được thực hiện để xây dựng một ma trận vuông kích thước $n \times n$ với n là số lượng các users trong hệ thống như trong Bảng 6.5. Do tính chất của hàm số $\cos$ ma trận này sẽ là một ma trận đối xứng, các giá trị trên đường chéo sẽ bằng 1. Từ việc quan sát ma trận này, các users tương tự nhau sẽ được gom nhóm lại với nhau để hệ thống gợi ý các item cho các users này tương tự nhau.

Nhìn vào Bảng 6.5 có thể thấy u_0 gần với u_1 (độ giống nhau là dương). Hai người dùng này giống nhau cũng tương đối dễ hiểu vì hai người dùng này có xu hướng đánh giá giống nhau cho các items. Trong khi đó u_2 thì gần các users u_3, u_4 và u_5 và xa users u_0 và u_1 .

6.3.1.4 Dự đoán rating (đánh giá)

Việc xác định mức độ quan tâm của một user lên một item dựa trên các users gần nhất (neighbor users) này rất giống với thuật toán K láng giềng gần nhất (KNN). Khi làm việc với các bài toán cỡ lớn (dữ liệu lớn), thuật toán KNN cũng thường được

sử dụng vì tính toán đơn giản. Việc dự đoán ratings không thể sử dụng trực tiếp KNN mà sẽ cần một số bước trung gian. Tương tự như KNN, trong lọc cộng tác, các giá trị đánh giá bị thiếu được tính toán dự đoán thông qua k users gần nhất. Chỉ quan tâm đến các users đã đánh giá item đang xem xét. Dự đoán đánh giá thường được xác định là trung bình có trọng số của các đánh giá đã chuẩn hóa. Trong KNN trọng số dựa trên khoảng cách giữa các vector (là số không âm). Trong khi đó với lọc cộng tác, trọng số được tính dựa trên điểm giống nhau giữa các users (có thể là số âm). Công thức phổ biến để dự đoán rating của user u cho item i là

$$\hat{y}_{i,u} = \frac{\sum_{u_j \in \mathcal{N}(u,i)} \bar{y}_{i,u_j} \text{sim}(u, u_j)}{\sum_{u_j \in \mathcal{N}(u,i)} |\text{sim}(u, u_j)|} \quad (6.10)$$

(sự khác biệt so với trung bình có trọng số là mẫu số có sử dụng trị tuyệt đối để xử lý các số âm).

trong đó $\mathcal{N}(u, i)$ là tập hợp k users trong các users lân cận (có độ tương đồng cao nhất) của u mà đã đánh giá item i .

Một ví dụ về việc tính normalized rating của u_0 cho i_2 với số nearest neighbors là $k = 2$. Các bước thực hiện là:

- Xác định các users đã đánh giá i_2 , đó là u_1, u_2, u_4, u_5 .
- Xác định độ tương đồng của u_0 với các users này ta nhận được các giá trị 0.72, -0.64, -0.75, -0.2. Hai giá trị ($k=2$) lớn nhất là 0.72 và -0.2 tương ứng với u_1 và u_5 .
- Xác định ratings được chuẩn hóa của u_1 và u_5 cho i_2 , ta thu được hai giá trị lần lượt là 2.4 và -1.66.
- Dự đoán kết quả:

$$\hat{y}_{i_2, u_0} = \frac{0.72 \times 2.4 + (-0.2) \times (-1.66)}{0.72 + |-0.2|} \approx 2.23 \quad (6.11)$$

Việc quy đổi các giá trị ratings đã chuẩn hoá về thang 5 có thể được thực hiện bằng cách cộng các cột của ma trận sau khi dự đoán với giá trị rating trung bình của mỗi user như đã tính. Việc hệ thống quyết định gợi ý items nào cho mỗi user có thể được xác định bằng nhiều cách khác nhau. Có thể sắp xếp items chưa được đánh giá theo thứ tự từ lớn đến bé của các đánh giá được dự đoán thông qua phương pháp bên trên, hoặc chỉ chọn các items có normalized predicted ratings dương - tương ứng với việc user này có nhiều khả năng thích hơn.

6.3.2 Lọc cộng tác dựa trên sản phẩm (item-item collaborative filtering)

Một số hạn chế của User-user CF:

- Trên thực tế, số lượng users luôn lớn hơn số lượng items rất nhiều. Kéo theo đó là ma trận tương đồng là rất lớn với số phần tử phải lưu giữ là hơn 1 nửa của bình phương số lượng users (chú ý rằng ma trận này là đối xứng). Việc này, như đã đề cập ở trên, khiến cho việc lưu trữ ma trận này trong nhiều trường hợp là không khả thi.
- Ma trận Utility thường là rất thưa. Với số lượng users rất lớn so với số lượng items, rất nhiều cột của ma trận này sẽ rất thưa (ít giá trị), tức chỉ có một vài phần tử khác 0. Lý do là users thường không để lại đánh giá của mình khi sử dụng sản phẩm dịch vụ. Cũng chính vì việc này, một khi user đó thay đổi đánh giá hoặc đánh giá thêm items, trung bình cộng các ratings cũng như vector chuẩn hoá tương ứng với user này thay đổi nhiều. Kéo theo đó, việc tính toán ma trận tương đồng, vốn tốn nhiều bộ nhớ và thời gian, cũng cần được thực hiện lại.

Ngược lại, nếu chúng ta tính toán similarity giữa các items rồi recommend những items gần giống với item yêu thích của một user thì sẽ có những lợi ích sau:

- Vì số lượng items thường nhỏ hơn số lượng users, ma trận tương đồng trong trường hợp này cũng nhỏ hơn nhiều, thuận lợi cho việc lưu trữ và tính toán ở các bước sau.
- Vì số lượng phần tử đã biết trong Utility matrix là như nhau nhưng số hàng (items) ít hơn số cột (users), nên trung bình, mỗi hàng của ma trận này sẽ có nhiều phần tử đã biết hơn số phần tử đã biết trong mỗi cột. Việc này cũng dễ hiểu vì mỗi item có thể được đánh giá bởi nhiều users. Kéo theo đó, giá trị trung bình của mỗi hàng ít bị thay đổi hơn khi có thêm một vài đánh giá. Như vậy, việc cập nhật ma trận tương đồng có thể được thực hiện ít thường xuyên hơn.

Cách tiếp cận thứ hai này được gọi là Item-item Collaborative Filtering. Hướng tiếp cận này được sử dụng nhiều trong thực tế hơn. Quy trình dự đoán missing ratings cũng tương tự như trong User-user CF. Về mặt tính toán, Item-item CF có thể nhận được từ User-user CF bằng cách chuyển vị (transpose) ma trận utility, và coi như items đang rate users. Sau khi tính ra kết quả cuối cùng, ta lại chuyển vị một lần nữa để thu được kết quả.

6.4 Lọc cộng tác dựa trên phân rã ma trận (matrix factorization)

6.4.1 Giới thiệu

Nhắc lại rằng trong Content-based Recommendation Systems, mỗi item được mô tả bằng một vector $\mathbf{x}$ được gọi là item profile. Trong phương pháp này, ta cần tìm một vector hệ số $\mathbf{w}$ tương ứng với mỗi user sao cho rating đã biết mà user đó cho item xấp xỉ với:

$$y \approx \mathbf{xw} \quad (6.12)$$

Với cách làm trên, Utility Matrix $\mathbf{Y}$, giả sử đã được điền hết, sẽ xấp xỉ với:

với $\mathbf{M}$, $\mathbf{N}$ lần lượt là số items và số users.

Chú ý rằng, $\mathbf{x}$ được xây dựng dựa trên thông tin mô tả của item và quá trình xây dựng này độc lập với quá trình đi tìm hệ số phù hợp cho mỗi user. Như vậy, việc xây dựng item profile đóng vai trò rất quan trọng và có ảnh hưởng trực tiếp lên hiệu năng của mô hình. Thêm nữa, việc xây dựng từng mô hình riêng lẻ cho mỗi user dẫn đến kết quả chưa thực sự tốt vì không khai thác được đặc điểm của những users gần giống nhau.

Bây giờ, giả sử rằng ta không cần xây dựng từ trước các item profile $\mathbf{x}$ mà vector đặc trưng cho mỗi item này có thể được huấn luyện đồng thời với mô hình của mỗi user (ở đây là 1 vector hệ số). Điều này nghĩa là, biến số trong bài toán tối ưu là cả $\mathbf{X}$ và $\mathbf{W}$; trong đó, $\mathbf{X}$ là ma trận của toàn bộ item profiles, mỗi hàng tương ứng với 1 item, $\mathbf{W}$ là ma trận của toàn bộ user models, mỗi cột tương ứng với 1 user.

Với cách làm này, ta đang cố gắng xấp xỉ Utility Matrix $\mathbf{Y} \in \mathbb{R}^{M \times N}$ bằng tích của hai ma trận $\mathbf{X} \in \mathbb{R}^{M \times K}$ và $\mathbf{W} \in \mathbb{R}^{K \times N}$.

Thông thường, K được chọn là một số nhỏ hơn rất nhiều so với $\mathbf{M}$, $\mathbf{N}$. Khi đó, cả hai ma trận $\mathbf{X}$ và $\mathbf{W}$ đều có hạng (rank) không vượt quá $\mathbf{K}$. Chính vì vậy, phương pháp này còn được gọi là Low-Rank Matrix Factorization. Có một vài điểm lưu ý ở đây:

- Ý tưởng chính đằng sau phân rã ma trận cho hệ khuyến nghị là tồn tại các latent features (tính chất ẩn) mô tả sự liên quan giữa các items và users. Ví dụ với hệ thống gợi ý các bộ phim, tính chất ẩn có thể là hình sự, chính trị, hành động, hài, ...; cũng có thể là một sự kết hợp nào đó của các thể loại này; hoặc cũng có thể là bất cứ điều gì có ý nghĩa. Mỗi item sẽ mang tính chất ẩn ở một mức độ nào đó tương ứng với các hệ số trong vector $\mathbf{x}$ của nó, hệ số càng cao tương ứng với việc mang tính chất đó càng cao. Tương tự, mỗi user cũng sẽ có xu hướng thích những tính chất ẩn nào đó và được mô tả bởi các hệ số trong

vector $\mathbf{w}$ của nó. Hệ số cao tương ứng với việc user thích các bộ phim có tính chất ẩn đó. Giá trị của biểu thức $\mathbf{xw}$ sẽ cao nếu các thành phần tương ứng của $\mathbf{x}$ và $\mathbf{w}$ đều cao. Điều này nghĩa là item mang các tính chất ẩn mà user thích, vậy thì nên gợi ý item này cho user đó.

- Phân rã ma trận được xếp vào lọc cộng tác bởi vì cách đi tìm hàm mất mát. Về cơ bản, để tìm nghiệm của bài toán tối ưu, ta phải lần lượt đi tìm $\mathbf{X}$ và $\mathbf{W}$ khi thành phần còn lại được cố định. Như vậy, mỗi hàng của $\mathbf{X}$ sẽ phụ thuộc vào toàn bộ các cột của $\mathbf{W}$. Ngược lại, mỗi cột của $\mathbf{W}$ lại phụ thuộc vào toàn bộ các hàng của $\mathbf{X}$. Như vậy, có những mối quan hệ ràng buộc chằng chịt giữa các thành phần của hai ma trận trên. Tức chúng ta cần sử dụng thông tin của tất cả để suy ra tất cả. Vậy nên phương pháp này cũng được xếp vào hệ khuyến nghị dựa trên lọc cộng tác.
- Trong các bài toán thực tế, số lượng items $\mathbf{M}$ và số lượng users $\mathbf{N}$ thường rất lớn. Việc tìm ra các mô hình đơn giản giúp dự đoán đánh giá cần được thực hiện một cách nhanh nhất có thể. Lọc cộng tác dựa trên láng giềng gần không yêu cầu việc huấn luyện quá nhiều, nhưng trong quá trình dự đoán (inference), ta cần đi tìm độ tương đồng của user đang xét với toàn bộ các users còn lại rồi suy ra kết quả. Ngược lại, với phương pháp phân rã ma trận, việc huấn luyện có thể phức tạp hơn vì phải lặp đi lặp lại việc tối ưu một ma trận khi cố định ma trận còn lại, nhưng việc dự đoán đơn giản hơn vì ta chỉ cần lấy tích của hai vector $\mathbf{xw}$, mỗi vector có độ dài $\mathbf{K}$ là một số nhỏ hơn nhiều so với $\mathbf{M}, \mathbf{N}$. Vậy nên quá trình dự đoán không yêu cầu khả năng tính toán cao. Việc này khiến nó phù hợp với các mô hình có tập dữ liệu lớn.
- Thêm nữa, việc lưu trữ hai ma trận $\mathbf{X}$ và $\mathbf{W}$ yêu cầu lượng bộ nhớ nhỏ khi so với việc lưu toàn bộ ma trận độ tương đồng trong phương pháp lọc cộng tác dựa trên láng giềng gần nhất. Cụ thể, ta cần bộ nhớ để chứa $\mathbf{K}(\mathbf{M} + \mathbf{N})$ phần tử thay vì lưu $\mathbf{M}^2$ hoặc $\mathbf{N}^2$ của ma trận độ tương đồng.

Bài tập Chương 6

Bài tập 6.1 Hệ thống gợi ý phim Việt Nam dựa trên nội dung:

Thông tin chi tiết về các bộ phim Việt Nam:

<i>Tên phim</i>	<i>Thể loại</i>	<i>Đạo diễn</i>	<i>Diễn viên</i>	<i>Mô tả</i>
<i>Em chưa 18</i>	Hài hước, Tình cảm	Lê Thanh Sơn	Kaity Nguyễn, Kiều Minh Tuấn	Câu chuyện tình yêu tuổi teen.
<i>Cô gái đến từ hôm qua</i>	Tâm lý, Tình cảm	Phan Gia Nhật Linh	Hoàng Yến, Jun Phạm	Cô gái từ tương lai thay đổi quá khứ
<i>Trạng Quỳnh</i>	Hài hước, Dân gian	Đức Thịnh	Trần Quốc Anh, Trần Thành	Cuộc phiêu lưu kỳ bí của Trạng Quỳnh.
<i>Lời nguyện huyết ngải</i>	Kinh dị, Giật gân	Bùi Thạc Chuyên	Yu Dương, Thành Lộc	Cuộc phiêu lưu đầy ám ảnh của ba sinh viên trường Đại học Y

Ma trận Rating từ người dùng:

Người dùng	<i>Em chưa 18</i>	<i>Cô gái đến từ hôm qua</i>	<i>Trạng Quỳnh</i>	<i>Lời nguyện huyết ngải</i>
<i>Người A</i>	5	4	5	4
<i>Người B</i>	4	5	3	5
<i>Người C</i>	3	5	0	3
<i>Người D</i>	5	3	4	2
<i>Người E</i>	4	4	3	5
<i>Người F</i>	?	?	?	?

Yêu cầu cụ thể:

- Xây dựng hệ thống khuyến nghị dựa trên nội dung sử dụng thông tin về thể loại phim, đạo diễn, diễn viên và mô tả của các bộ phim Việt Nam.
- Điền thông tin đánh giá từ người dùng F vào ma trận rating để có thể đưa ra gợi ý phim cho người dùng F.
- Tính toán sự tương đồng giữa các bộ phim dựa trên thông tin nội dung và đề xuất các bộ phim tương tự hoặc có liên quan dựa trên sở thích của người dùng.
- Gợi ý top (K) bộ phim tốt nhất cho người dùng F, với (K) là số lượng bộ phim cần được gợi ý.

Bài tập 6.2 Lọc cộng tác dựa trên người dùng (user-user collaborative filtering):

Bài toán: Hệ thống đề xuất sách cho người dùng dựa trên đánh giá của người dùng khác.

Thông tin sẵn có:

- Ma trận rating: Một bảng dữ liệu lưu trữ xếp hạng của các người dùng cho các sách khác nhau.
- Thông tin người dùng: Các thông tin về sở thích, tuổi, giới tính, v.v. của từng người dùng.

Yêu cầu:

- Xác định sự tương đồng giữa người dùng: Tính toán độ tương tự giữa các người dùng dựa trên xếp hạng mà họ đã đưa ra cho các sách và thông tin cá nhân của họ.
- Dự đoán xếp hạng chưa được xác định: Sử dụng thông tin về độ tương đồng để dự đoán xếp hạng cho một sách cụ thể mà một người dùng chưa đánh giá.

Cụ thể có một số người dùng và ma trận rating của họ cho các cuốn sách như sau:

Ma trận Rating:

	Sách 1	Sách 2	Sách 3	Sách 4	Sách 5
Người A	4	5	-	2	3
Người B	-	5	4	3	-
Người C	4	-	3	-	5
Người D	3	2	5	4	-
Người E	-	3	-	5	4

Thông tin Người dùng:

	Sở thích	Tuổi	Giới tính
Người A	Khoa học	25	Nam
Người B	Trình thám	30	Nữ
Người C	Kinh doanh	28	Nam
Người D	Hài hước	22	Nữ
Người E	Khoa học	27	Nữ

1. Cần dự đoán xếp hạng mà Người A sẽ đánh giá cho Sách 3.
2. Tính toán độ tương tự giữa Người A và các người dùng khác bằng cách sử dụng thông tin về sở thích, tuổi, giới tính và/hoặc các thông tin cá nhân khác.

Bài tập 6.3 **Lọc cộng tác dựa trên sản phẩm (item-item collaborative filtering):**

Bài toán: Hệ thống đề xuất sản phẩm cho người dùng dựa trên đánh giá của sản phẩm tương tự.

Thông tin sẵn có:

- Ma trận rating: Một bảng dữ liệu lưu trữ xếp hạng của các người dùng cho các sản phẩm khác nhau.
- Thông tin sản phẩm: Các thông tin về loại sản phẩm, nhãn hiệu, đặc điểm, v.v. của từng sản phẩm.

Yêu cầu:

- Xác định sự tương đồng giữa các sản phẩm: Tính toán độ tương tự giữa các sản phẩm dựa trên xếp hạng mà người dùng đã đưa ra cho chúng.

- Dự đoán xếp hạng chưa được xác định: Sử dụng thông tin về độ tương đồng để dự đoán xếp hạng cho một sản phẩm cụ thể mà một người dùng chưa đánh giá.

Cụ thể có một số sản phẩm và ma trận rating của người dùng cho các sản phẩm như sau:

Ma trận Rating:

	Sản phẩm1	Sản phẩm2	Sản phẩm3	Sản phẩm4	Sản phẩm5
Người A	5	4	-	2	3
Người B	-	3	5	4	-
Người C	3	2	5	4	-
Người D	4	-	3	-	5
Người E	3	-	5	4	-

Thông tin Sản phẩm:

	Loại	Nhãn hiệu	Đặc điểm
Sản phẩm 1	Điện tử	ABC Company	Màn hình lớn, Pin lâu
Sản phẩm 2	Thời trang	XYZ Brand	Chất liệu cao cấp, Hợp thời trang
Sản phẩm 3	Đồ gia dụng	LMN Corp.	Tiết kiệm năng lượng, Đa chức năng
Sản phẩm 4	Điện tử	PQR Inc.	Camera chất lượng cao, Bộ nhớ lớn
Sản phẩm 5	Thể thao	DEF Co.	Chịu nước, Đệm êm

1. Cần dự đoán xếp hạng mà Người D sẽ đánh giá cho Sản phẩm 2.
2. Tính toán độ tương tự giữa Sản phẩm 4 và các sản phẩm khác bằng cách sử dụng các phương pháp đo lường độ tương đồng

Bài tập 6.4 Các bài tập thảo luận: Lựa chọn các đề tài thảo luận dưới đây về Hệ khuyến nghị:

1. *Xây dựng Hệ thống Khuyến nghị Phim dựa trên Nội Dung:* Sử dụng thông tin về thể loại, diễn viên, đạo diễn và nội dung phim để đề xuất phim tương tự cho người dùng.
2. *Triển khai Lọc Cộng tác dựa trên Người Dùng trong Hệ thống Gợi Ý Phim:* Phân tích lịch sử xem phim và xếp hạng của người dùng để đề xuất phim mới dựa trên sở thích tương đồng.
3. *Phát triển Hệ thống Gợi Ý Sản Phẩm sử dụng Lọc dựa trên Ngữ Cảnh Thời Gian và Vị Trí:* Đề xuất phim hoặc sản phẩm dựa trên thời gian xem, vị trí địa lý và thông tin ngữ cảnh của người dùng.

4. *Áp Dụng Deep Learning trong Hệ thống Khuyến Nghị Phim*: Sử dụng mô hình Deep Learning để cải thiện độ chính xác và hiệu suất của hệ thống khuyến nghị phim.
5. *Xây Dựng Hệ Thống Khuyến Nghị Lai cho Ứng Dụng Mua Sắm Online*: Kết hợp khả năng đề xuất sản phẩm trong nhiều lĩnh vực, từ phim đến thời trang, điện tử, để cung cấp trải nghiệm mua sắm đa dạng.
6. *Làm Việc với Knowledge-Based Recommendations trong Lĩnh Vực Du Lịch*: Xây dựng hệ thống đề xuất điểm đến, tour du lịch và thông tin hữu ích dựa trên sở thích và kiến thức chuyên môn về du lịch.
7. *Lọc Dựa Trên Thói Quen Tiêu Dùng và Hành Vi Người Dùng*: Phát triển mô hình đề xuất phim hoặc sản phẩm dựa trên hành vi mua sắm, thói quen tiêu dùng và xu hướng của người dùng.
8. *Hệ Thống Khuyến Nghị Phim Đa Nền Tảng và Tích Hợp Dữ Liệu Đa Nguồn*: Xây dựng hệ thống đề xuất phim có khả năng hoạt động trên nhiều nền tảng khác nhau (web, di động) và tích hợp dữ liệu từ nhiều nguồn khác nhau.
9. *Lọc Dựa Trên Ngữ Cảnh Thời Gian Thực và Trạng Thái Tâm Trạng Của Người Dùng*: Đề xuất phim dựa trên ngữ cảnh thời gian thực, vị trí và trạng thái tâm trạng của người dùng để cung cấp trải nghiệm tùy chỉnh.
10. *Hệ Thống Khuyến Nghị Sản Phẩm Cho Người Dùng Mới Dựa Trên Thông Tin Mặc Định*: Nghiên cứu cách đề xuất phim hoặc sản phẩm cho người dùng mới bằng cách sử dụng thông tin mặc định hoặc thông tin tổng quát từ cơ sở dữ liệu.

Tài liệu tham khảo

- [1] C. C. Aggarwal. “On the effects of dimensionality reduction on high dimensional similarity search”. In: *Proceedings of the twentieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 2001, pages 256–266.
- [2] C. C. Aggarwal. “On biased reservoir sampling in the presence of stream evolution”. In: *Proceedings of the 32nd international conference on Very large data bases*. 2006, pages 607–618.
- [3] C. C. Aggarwal and S. Parthasarathy. “Mining massively incomplete data sets by conceptual reconstruction”. In: *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 2001, pages 227–232.
- [4] C. C. Aggarwal et al. *Data mining: the textbook*. Volume 1. Springer, 2015.
- [5] M. Berenson et al. “Basic business statistics: Concepts and applications”. In: (2012).
- [6] J. Grus. *Data science from scratch: first principles with python*. O’Reilly Media, 2019.
- [7] D. C. Montgomery and G. C. Runger. *Applied statistics and probability for engineers*. John Wiley & Sons, 2010.
- [8] W. K. Nicholson. *Linear Algebra with Applications*. Lyryx Learning Inc, 2021.
- [9] M. F. Triola. *Elementary statistics using Excel*. Addison-Wesley Publishing Company, 2009.
- [10] J. VanderPlas. *Python data science handbook: Essential tools for working with data*. O’Reilly Media, Inc., 2016.